🐼

Pandas

🐼 PHASE 1: PANDAS BASICS — Complete Learning Notes

> **Goal:** Build a solid foundation in Pandas — understand what it is, why it's used, and master the two core data structures (Series & DataFrame) with real-world examples.

---

1. Introduction to Pandas

📌 Definition

Pandas is an open-source Python library used for:

- ✅ Data manipulation
- ✅ Data analysis
- ✅ Data cleaning
- ✅ Data transformation

> The name **"Pandas"** comes from **"Panel Data"** — a term in econometrics/statistics for datasets that contain observations over multiple time periods.

It was created by **Wes McKinney in 2008** while working at AQR Capital Management. Today it is the **#1 data analysis library** in the Python ecosystem.

🌍 Real-World Use Cases

Industry	Use Case
📊 Finance	Analyzing stock prices, portfolio returns
🛒 E-Commerce	Processing sales data, customer transactions
🏥 Healthcare	Patient records, medical trial data
🎬 Media	Netflix ratings, viewer behaviour analysis
🤖 AI / ML	Feature engineering, preprocessing training data
📈 Marketing	Campaign analytics, A/B testing results

🔑 Key Capabilities of Pandas

- Read/Write data: **CSV, Excel, JSON, SQL, HTML, Parquet**
- Filter, sort, and transform data easily
- Handle **missing data (NaN)** gracefully
- **Merge, join, and concatenate** multiple datasets
- **GroupBy** and compute aggregations
- Full **Time Series** support
- Works perfectly with **NumPy, Matplotlib, Scikit-learn**

> 💡 **Think of Pandas as Excel on steroids, powered by Python.**

---

2. Why Pandas over Python Lists / NumPy

🔍 Comparison Table

Feature	Python List	NumPy Array	Pandas
Different data types per column	❌ No	❌ No	✅ Yes
Column labels	❌ No	❌ No	✅ Yes
Row labels (index)	❌ No	❌ No	✅ Yes
Missing value handling	❌ Manual	❌ Manual	✅ Built-in NaN
Read CSV / Excel	❌ Manual	❌ Manual	✅ One line
GroupBy / Aggregation	❌ Loops	❌ Complex	✅ Simple
Data merging / joining	❌ Complex	❌ Not built-in	✅ SQL-like joins
Time series support	❌ No	❌ Limited	✅ Full support
Performance on large data	❌ Slow	✅ Fast	✅ Fast (C-backed)

💻 Code Comparison

Problem: Find average salary of the "Engineering" department.

❌ Python List — Tedious & Long

```
employees = [
    {"name": "Alice", "dept": "Engineering", "salary": 90000},
    {"name": "Bob",   "dept": "Marketing",   "salary": 60000},
    {"name": "Carol", "dept": "Engineering", "salary": 95000},
    {"name": "Dave",  "dept": "Marketing",   "salary": 65000},
]

# Manual loop required
total, count = 0, 0
for emp in employees:
    if emp["dept"] == "Engineering":
        total += emp["salary"]
        count += 1

avg = total / count
print("Average Salary:", avg)
```

Output:

```
Average Salary: 92500.0
```

✅ Pandas — Clean & Powerful (1 Line!)

```
import pandas as pd

df = pd.DataFrame(employees)

# ONE line does what 7 lines did above
avg = df[df["dept"] == "Engineering"]["salary"].mean()
print("Average Salary:", avg)
```

Output:

```
Average Salary: 92500.0
```

> **Output Explanation:** Both give the same result — but Pandas takes 1 readable line vs 7 lines of loop code. On millions of rows, Pandas is also significantly faster.

💡 Important Notes

- **Python Lists** → Use for simple, small, general-purpose data
- **NumPy Arrays** → Use for pure numerical/math operations
- **Pandas** → Use for tabular/structured data analysis (**your default choice**)
- Pandas is built **ON TOP of NumPy** — so you get speed + usability

---

3. Installing and Importing Pandas

📦 Installation

```bash
# Install latest version
pip install pandas

# Install a specific version
pip install pandas==2.1.0

# Upgrade existing installation
pip install pandas --upgrade

# In Jupyter Notebook
!pip install pandas

# Using Conda (Anaconda)
conda install pandas
```

📥 Importing Pandas

```python
import pandas as pd      # 'pd' is the universal industry alias — ALWAYS use this
import numpy as np       # NumPy is almost always used alongside Pandas

# Check version
print("Pandas Version:", pd.__version__)   # e.g., 2.1.4
print("NumPy Version: ", np.__version__)   # e.g., 1.26.0
```

Output:

```
Pandas Version: 2.1.4
NumPy Version:  1.26.0
```

⚠️ Common Mistakes

Mistake	Problem	Fix
`import Pandas as pd`	Case-sensitive — `Pandas` ≠ `pandas`	Use lowercase `pandas`
`import pandas as pnd`	Non-standard alias	Always use `pd`
`from pandas import *`	Pollutes namespace	Use `import pandas as pd`

---

4. Pandas Data Structures Overview

Pandas has **two primary data structures:**

Structure	Dimensions	Analogy
Series	1D	A single column in Excel
DataFrame	2D	A full Excel spreadsheet / SQL table

```
Series:                          DataFrame:
┌─────────┬─────────┐            ┌─────────┬──────────────┬─────┬──────────────┐
│ Index   │ Value   │            │Index    │    Name      │ Age │    City      │
├─────────┼─────────┤            ├─────────┼──────────────┼─────┼──────────────┤
│    0    │ Alice   │            │   0     │   Alice      │ 28  │   NYC        │
│    1    │ Bob     │            │   1     │   Bob        │ 35  │   LA         │
│    2    │ Carol   │            │   2     │   Carol      │ 22  │   NYC        │
└─────────┴─────────┘            └─────────┴──────────────┴─────┴──────────────┘
(1 column)                       (multiple columns — each column = 1 Series)
```

> 💡 A **DataFrame** is simply a **collection of Series objects** that share the same index.

---

5. Series — Complete Guide

📌 Definition

A **Series** is a **one-dimensional labeled array** capable of holding any data type (integers, floats, strings, booleans, Python objects).

Think of it as: `Python list + labels (index) + extra superpowers`

🔧 Syntax

```
pd.Series(data, index=None, dtype=None, name=None)
```

Parameter	Description
`data`	list, dict, scalar, ndarray — the actual values
`index`	custom labels for each element (optional)
`dtype`	force a specific data type (optional)
`name`	give the Series a name (optional, useful in DataFrames)

Example 1: Series from a Python List

```
import pandas as pd

scores = [85, 92, 78, 96, 70]
s1 = pd.Series(scores)
print(s1)
```

Output:

```
0    85
1    92
2    78
3    96
4    70
dtype: int64
```

Output Explanation:

- Left column `(0, 1, 2, 3, 4)` = **auto-generated index** (starts at 0)
- Right column = **actual values**
- `dtype: int64` = Pandas detected integer type automatically

Example 2: Series with Custom Index

```python
# Real-world: Monthly sales figures with month names as labels
monthly_sales = pd.Series(
    data  = [45000, 52000, 61000, 58000, 67000],
    index = ["January", "February", "March", "April", "May"],
    name  = "Sales (USD)"
)
print(monthly_sales)
```

Output:

```
January      45000
February     52000
March        61000
April        58000
May          67000
Name: Sales (USD), dtype: int64
```

Output Explanation:

- Custom index makes data **self-describing** — much more readable
- `name` attribute appears when the Series becomes a column in a DataFrame

---

Example 3: Series from a Dictionary

```python
# Dictionary keys → index labels, values → data
stock_prices = {"AAPL": 178.5, "GOOG": 135.2, "AMZN": 143.7, "TSLA": 245.0}
s3 = pd.Series(stock_prices, name="Stock Price ($)")
print(s3)
```

Output:

```
AAPL    178.5
GOOG    135.2
AMZN    143.7
TSLA    245.0
Name: Stock Price ($), dtype: float64
```

Example 4: Series from a NumPy Array

```python
import numpy as np

temps_np = np.array([36.5, 37.1, 36.8, 38.2, 37.5])
s4 = pd.Series(temps_np, index=["Mon","Tue","Wed","Thu","Fri"], name="Temp °C")
print(s4)
```

Output:

```
Mon    36.5
Tue    37.1
Wed    36.8
```

```
Thu    38.2
Fri    37.5
Name: Temp °C, dtype: float64
```

Example 5: Series with dtype Specification

```python
# Force float on integer data
s5 = pd.Series([1, 2, 3, 4, 5], dtype="float64")
print(s5)
print("dtype:", s5.dtype)
```

Output:

```
0    1.0
1    2.0
2    3.0
3    4.0
4    5.0
dtype: float64
dtype: float64
```

Output Explanation: Even though values are integers, `dtype="float64"` forces them to be stored as floats (1 → 1.0). Useful when you need consistent float precision.

Example 6: Series from a Scalar (Broadcasting)

```python
# Single value is "broadcast" to all index positions
default_rating = pd.Series(5.0, index=["Product A", "Product B", "Product C"])
print(default_rating)
```

Output:

```
Product A    5.0
Product B    5.0
Product C    5.0
dtype: float64
```

Output Explanation: When data is a **single scalar**, Pandas **broadcasts** (repeats) it across all index positions. Useful for creating default-value columns.

Example 7: Series with Missing Values (NaN)

```python
# Real-world: Sensor data with missing readings
sensor_data = pd.Series(
    [23.1, None, 24.5, np.nan, 22.8],
    index = ["08:00", "09:00", "10:00", "11:00", "12:00"],
    name  = "Sensor Temperature"
)
print(sensor_data)
print("\\nNull check:")
print(sensor_data.isnull())
```

Output:

```
08:00    23.1
09:00     NaN
10:00    24.5
```

```
11:00     NaN
12:00    22.8
Name: Sensor Temperature, dtype: float64

Null check:
08:00    False
09:00     True
10:00    False
11:00     True
12:00    False
dtype: bool
```

Output Explanation:

- Both Python `None` and NumPy `np.nan` are treated as **NaN** in Pandas
- `isnull()` returns `True` wherever a value is missing
- Missing values in numeric Series are stored as `float NaN`

Example 8: Accessing Elements in a Series

```python
cities = pd.Series(
    ["Mumbai", "Delhi", "Bangalore", "Hyderabad", "Chennai"],
    index = ["City1", "City2", "City3", "City4", "City5"]
)

print("By label:  ", cities["City1"])                    # Mumbai
print("By position:", cities.iloc[0])                    # Mumbai
print("Slicing (label-based):")
print(cities["City1":"City3"])                           # City1 to City3 (inclusive)
print("Multiple labels:", cities[["City1","City3"]].tolist())
```

Output:

```
By label:   Mumbai
By position: Mumbai
Slicing (label-based):
City1      Mumbai
City2       Delhi
City3   Bangalore
dtype: object
Multiple labels: ['Mumbai', 'Bangalore']
```

Key Access Methods Summary:

Method	Description
`s["label"]`	Access by index **label**
`s.iloc[n]`	Access by integer **position** (0-based)
`s["start":"end"]`	Label-based slice — **INCLUSIVE** on both ends
`s[["a", "b"]]`	Access multiple labels (pass a list)

Example 9: Vectorized Arithmetic (No Loops Needed!)

```python
# Real-world: E-commerce pricing with discount and tax
prices   = pd.Series([100, 200, 150, 300], name="Price")
discount = pd.Series([10,   20,  15,  30], name="Discount")

after_discount = prices - discount
after_gst      = after_discount * 1.18    # Apply 18% GST

print("Original Prices :", prices.tolist())
```

```
print("Discounts       :", discount.tolist())
print("After Discount  :", after_discount.tolist())
print("After 18% GST   :", after_gst.round(2).tolist())
```

Output:

```
Original Prices : [100, 200, 150, 300]
Discounts       : [10, 20, 15, 30]
After Discount  : [90, 180, 135, 270]
After 18% GST   : [106.2, 212.4, 159.3, 318.6]
```

Output Explanation: Operations are applied **element-by-element automatically** — this is called **vectorization**. No need for loops. This also makes it extremely fast on large datasets.

Example 10: Key Series Attributes & Statistical Methods

```
revenue = pd.Series(
    [120000, 85000, 95000, 110000, 73000, 130000],
    index = ["Q1-2022","Q2-2022","Q3-2022","Q4-2022","Q1-2023","Q2-2023"],
    name  = "Revenue"
)

# --- Attributes ---
print("Values :", revenue.values)           # NumPy array of values
print("Index  :", revenue.index.tolist())   # list of index labels
print("Shape  :", revenue.shape)            # (6,)
print("Size   :", revenue.size)             # 6
print("dtype  :", revenue.dtype)            # int64
print("Name   :", revenue.name)             # Revenue

# --- Statistics ---
print("\\nSum    :", revenue.sum())           # 613000
print("Mean   :", revenue.mean().round(2))  # 102166.67
print("Max    :", revenue.max())            # 130000
print("Min    :", revenue.min())            # 73000
print("Std Dev:", revenue.std().round(2))
print("Median :", revenue.median())

# --- Full Summary ---
print("\\n", revenue.describe())
```

Output:

```
Values : [120000  85000  95000 110000  73000 130000]
Index  : ['Q1-2022', 'Q2-2022', 'Q3-2022', 'Q4-2022', 'Q1-2023', 'Q2-2023']
Shape  : (6,)
Size   : 6
dtype  : int64
Name   : Revenue

Sum    : 613000
Mean   : 102166.67
Max    : 130000
Min    : 73000
Std Dev: 20522.9
Median : 102500.0

 count    6.000000e+00
mean     1.021667e+05
std      2.052290e+04
min      7.300000e+04
25%      8.750000e+04
50%      1.025000e+05
75%      1.175000e+05
```

```
max      1.300000e+05
Name: Revenue, dtype: float64
```

Output Explanation: `describe()` gives you a **one-shot statistical summary**:

- `count` = total non-null values
- `25%/50%/75%` = quartiles (great for spotting distribution skew)
- `std` = how spread out the values are from the mean

💡 Series — Important Methods Summary

Method / Attribute	Description
`.values`	Returns underlying NumPy array
`.index`	Returns the Index object
`.shape`	`(n,)` — number of elements
`.size`	Total count of elements
`.dtype`	Data type of the Series
`.name`	Name of the Series
`.sum()`	Total of all values
`.mean()`	Average
`.max()` / `.min()`	Maximum / Minimum
`.std()`	Standard deviation
`.median()`	Middle value
`.describe()`	Full statistical summary
`.isnull()`	Boolean mask — `True` where value is NaN
`.tolist()`	Convert Series to plain Python list
`.iloc[n]`	Access by integer position

⚠️ Common Mistakes — Series

Mistake	Explanation
`s[5]` when index has custom labels	This raises a `KeyError`. Use `s.iloc[5]` for position-based access
Forgetting label slicing is **inclusive**	`s["a":"c"]` **includes** `"c"` — unlike Python list slicing
Assuming integer index means positional	Even if index is `[1, 2, 3]`, `s[1]` is label-based, not positional

---

6. DataFrame — Complete Guide

📌 Definition

A **DataFrame** is a **two-dimensional, labeled data structure** — like a spreadsheet or SQL table.

- **Rows** → observations / records
- **Columns** → features / variables
- Each **column** is a Pandas **Series**
- All columns share the **same index** (row labels)

> Think of DataFrame as: **a collection of Series aligned by a shared index.**

🔧 Syntax

```
pd.DataFrame(data, index=None, columns=None, dtype=None)
```

Parameter	Description
`data`	dict, list of dicts, 2D numpy array, another DataFrame
`index`	custom row labels (optional)
`columns`	custom column names / select/reorder columns (optional)
`dtype`	force a common dtype (optional, rarely used)

Method 1: From Dictionary of Lists

> 🏆 **Most common and intuitive way** — used in 80% of cases

How it works: Dictionary **keys** → Column names | Dictionary **values (lists)** → Column data | All lists must be the **same length**

```
import pandas as pd

# Real-world: Employee database
data = {
    "Name"       : ["Alice", "Bob", "Carol", "Dave", "Eve"],
    "Department" : ["Engineering", "Marketing", "Engineering", "HR", "Marketing"],
    "Salary"     : [90000, 60000, 95000, 55000, 62000],
    "Experience" : [5, 3, 7, 2, 4],
    "Remote"     : [True, False, True, False, True]
}

df = pd.DataFrame(data)
print(df)
```

Output:

```
    Name   Department  Salary  Experience  Remote
0  Alice  Engineering   90000           5    True
1    Bob    Marketing   60000           3   False
2  Carol  Engineering   95000           7    True
3   Dave           HR   55000           2   False
4    Eve    Marketing   62000           4    True
```

Output Explanation:

- Each key → column header
- Each list → column values
- Row index auto-starts at 0
- Different dtypes per column: `object` (str), `int64`, `bool`

Sub-Example: Custom Row Index

```
product_data = {
    "Product" : ["Laptop", "Mouse", "Keyboard", "Monitor"],
    "Price"   : [75000, 800, 2500, 18000],
    "Stock"   : [50, 200, 150, 75]
}

df_products = pd.DataFrame(
    product_data,
    index = ["P001", "P002", "P003", "P004"]   # custom row labels
)
print(df_products)
```

Output:

```
       Product  Price  Stock
P001    Laptop  75000     50
P002     Mouse    800    200
P003  Keyboard   2500    150
P004   Monitor  18000     75
```

Sub-Example: Selecting / Reordering Columns at Creation

```python
# Only pick specific columns and in a custom order
df_select = pd.DataFrame(data, columns=["Name", "Salary", "Department"])
print(df_select)
```

Output:

```
    Name  Salary   Department
0  Alice   90000  Engineering
1    Bob   60000    Marketing
2  Carol   95000  Engineering
3   Dave   55000           HR
4    Eve   62000    Marketing
```

> 💡 If you specify a column name **not in the dict**, Pandas fills it with `NaN`.

Method 2: From List of Dictionaries

> 📦 Very common with **API responses, web scraping, and log files**

How it works: Each **dictionary** = **one row** | Dictionary **keys** → Column names

```python
# Real-world: Student exam results
students = [
    {"Name": "Ravi",  "Marks": 88, "Grade": "A", "Passed": True},
    {"Name": "Priya", "Marks": 76, "Grade": "B", "Passed": True},
    {"Name": "Amit",  "Marks": 55, "Grade": "C", "Passed": True},
    {"Name": "Sneha", "Marks": 40, "Grade": "D", "Passed": False},
    {"Name": "Kabir", "Marks": 95, "Grade": "A", "Passed": True},
]

df_students = pd.DataFrame(students)
print(df_students)
```

Output:

```
    Name  Marks Grade  Passed
0   Ravi     88     A    True
1  Priya     76     B    True
2   Amit     55     C    True
3  Sneha     40     D   False
4  Kabir     95     A    True
```

Sub-Example: Missing Keys → Creates NaN (Real API Scenario)

```python
# Simulating API responses where some fields are missing
api_responses = [
    {"id": 1, "name": "Item A", "price": 100, "discount": 10},
```

```
    {"id": 2, "name": "Item B", "price": 200},              # missing 'discount'
    {"id": 3, "name": "Item C", "price": 150, "discount": 20},
    {"id": 4, "name": "Item D"},                            # missing price & discou
nt
]

df_api = pd.DataFrame(api_responses)
print(df_api)
```

Output:

```
   id    name  price  discount
0   1  Item A  100.0      10.0
1   2  Item B  200.0       NaN
2   3  Item C  150.0      20.0
3   4  Item D    NaN       NaN
```

Output Explanation: When a key is missing in some rows, Pandas automatically fills those cells with `NaN`. This is extremely common with real-world API/JSON data — Pandas handles it gracefully.

Method 3: From 2D Lists

> 📋 Use when data is a **grid / matrix** — no keys available

How it works: Each inner list = one row | **Must specify column names manually**

```
# Real-world: Cricket match scorecard
scorecard = [
    ["Rohit Sharma",    45, 32, 6, 1],
    ["Virat Kohli",     82, 68, 8, 2],
    ["KL Rahul",        38, 40, 3, 0],
    ["Hardik Pandya",   55, 30, 4, 3],
    ["Ravindra Jadeja", 25, 18, 2, 0],
]

df_cricket = pd.DataFrame(
    scorecard,
    columns = ["Player", "Runs", "Balls", "Fours", "Sixes"]
)
print(df_cricket)
```

Output:

```
            Player  Runs  Balls  Fours  Sixes
0     Rohit Sharma    45     32      6      1
1      Virat Kohli    82     68      8      2
2         KL Rahul    38     40      3      0
3    Hardik Pandya    55     30      4      3
4  Ravindra Jadeja    25     18      2      0
```

Sub-Example: Single Column DataFrame from a Flat List

```
names_list = ["Alice", "Bob", "Carol", "Dave"]
df_single = pd.DataFrame(names_list, columns=["Employee Name"])
print(df_single)
```

Output:

```
  Employee Name
0         Alice
1           Bob
2         Carol
3          Dave
```

Method 4: From NumPy Arrays

> 🤖 Very useful in **ML/AI workflows** where data comes from NumPy computations

```python
import numpy as np

# Example 1: Patient health data (2D NumPy array)
sensor_matrix = np.array([
    [36.5, 99.2, 120],
    [37.1, 98.5, 118],
    [36.8, 99.0, 122],
    [38.2, 97.8, 115],
    [37.5, 98.9, 119],
])

df_patients = pd.DataFrame(
    sensor_matrix,
    columns = ["Temperature", "SpO2", "HeartRate"],
    index   = ["Patient_1", "Patient_2", "Patient_3", "Patient_4", "Patient_5"]
)
print(df_patients)
```

Output:

```
           Temperature   SpO2  HeartRate
Patient_1         36.5   99.2      120.0
Patient_2         37.1   98.5      118.0
Patient_3         36.8   99.0      122.0
Patient_4         38.2   97.8      115.0
Patient_5         37.5   98.9      119.0
```

```python
# Example 2: Random data — common in ML experiments
np.random.seed(42)    # for reproducibility
random_data = np.random.randn(4, 3)   # 4 rows, 3 columns, standard normal dist.

df_random = pd.DataFrame(
    random_data,
    columns = ["Feature_1", "Feature_2", "Feature_3"]
)
print(df_random.round(4))
```

Output:

```
   Feature_1  Feature_2  Feature_3
0     0.4967    -0.1383     0.6477
1     1.5230     1.4658    -0.2342
2    -0.2342    -0.2341    -0.4689
3    -0.4670     0.2418    -1.9132
```

```python
# Example 3: From np.arange + reshape (structured synthetic data)
arr = np.arange(1, 13).reshape(3, 4)   # values 1-12 arranged in 3 rows × 4 cols
df_arange = pd.DataFrame(arr, columns=["A", "B", "C", "D"])
print(df_arange)
```

Output:

```
   A   B   C   D
0  1   2   3   4
1  5   6   7   8
2  9  10  11  12
```

Method 5: From CSV / Excel Files

> 🏭 **The most common method in industry** — real data always comes from files

Reading CSV

```python
# Basic CSV reading
df = pd.read_csv("sales_data.csv")

# With common parameters
df = pd.read_csv(
    "sales_data.csv",
    index_col   = "OrderID",          # set a column as row index
    usecols     = ["Name","Price"],   # read only specific columns
    nrows       = 100,                # read only first 100 rows
    parse_dates = ["OrderDate"],      # auto-parse date columns
    sep         = ","                 # delimiter (default is comma)
)
```

All Important `read_csv()` Parameters

```python
import pandas as pd

# 1. Basic read
df = pd.read_csv("file.csv")

# 2. Tab-separated file (.tsv)
df = pd.read_csv("file.tsv", sep="\\t")

# 3. File with no header row
df = pd.read_csv("file.csv", header=None, names=["Col1","Col2","Col3"])

# 4. Set a column as index
df = pd.read_csv("file.csv", index_col="ID")

# 5. Read only specific columns (saves memory)
df = pd.read_csv("file.csv", usecols=["Name", "Price", "Date"])

# 6. Read only first N rows (for exploring huge files)
df = pd.read_csv("file.csv", nrows=500)

# 7. Auto-parse date columns (string → datetime object)
df = pd.read_csv("file.csv", parse_dates=["OrderDate", "ShipDate"])
```

```python
# 8. Read CSV from a URL directly!
df = pd.read_csv("<https://example.com/data.csv>")
```

Reading Excel

```python
# Basic Excel read (requires: pip install openpyxl)
df = pd.read_excel("sales_data.xlsx")

# Read a specific sheet
df = pd.read_excel("sales_data.xlsx", sheet_name="January")

# Read multiple sheets → returns a dict of DataFrames
sheets = pd.read_excel("sales_data.xlsx", sheet_name=None)
df_jan = sheets["January"]
df_feb = sheets["February"]
```

Saving a DataFrame to CSV / Excel

```python
# Save to CSV — ALWAYS use index=False to avoid 'Unnamed: 0' column
df.to_csv("output.csv", index=False)

# Save with custom separator
df.to_csv("output.csv", index=False, sep="|")

# Save to Excel
df.to_excel("output.xlsx", index=False, sheet_name="Results")

# Save multiple sheets to one Excel file
with pd.ExcelWriter("report.xlsx") as writer:
    df_jan.to_excel(writer, sheet_name="January", index=False)
    df_feb.to_excel(writer, sheet_name="February", index=False)
```

Demo: Creating and Reading CSV In-Memory

```python
import pandas as pd
import io

# Simulated CSV content (in real projects, use actual file path)
csv_content = """OrderID,Customer,Product,Qty,Price,OrderDate
1001,Alice,Laptop,1,75000,2024-01-05
1002,Bob,Mouse,2,800,2024-01-08
1003,Carol,Keyboard,3,2500,2024-01-12
1004,Dave,Monitor,1,18000,2024-01-15
1005,Eve,Laptop,2,75000,2024-01-20"""

df_orders = pd.read_csv(io.StringIO(csv_content), parse_dates=["OrderDate"])
print(df_orders)
print("\\nOrderDate dtype:", df_orders["OrderDate"].dtype)
```

Output:

```
   OrderID Customer   Product  Qty  Price OrderDate
0     1001    Alice    Laptop    1  75000 2024-01-05
```

```
1     1002      Bob     Mouse    2    800 2024-01-08
2     1003    Carol  Keyboard    3   2500 2024-01-12
3     1004     Dave   Monitor    1  18000 2024-01-15
4     1005      Eve    Laptop    2  75000 2024-01-20

OrderDate dtype: datetime64[ns]
```

Output Explanation: After `parse_dates=["OrderDate"]`, the column is now a proper `datetime64` object. This allows time-based operations like filtering by date ranges, extracting month/year, etc.

⚠️ Common Mistakes — CSV / Excel

Mistake	Problem	Fix
`df.to_csv("file.csv")` without `index=False`	Creates extra `Unnamed: 0` column on re-read	Add `index=False`
Not using `parse_dates`	Date columns stay as strings	Use `parse_dates=["col_name"]`
Wrong `sep`	File uses `;` but you expect `,`	Check the file and use `sep=";"`
Loading full file unnecessarily	Wastes memory on large files	Use `usecols` and `nrows`

DataFrame Exploration Methods

> 🔍 **Every time you load a new dataset, run these commands first.**

```python
import pandas as pd

df = pd.DataFrame({
    "Name"       : ["Alice","Bob","Carol","Dave","Eve","Frank","Grace"],
    "Department" : ["Engineering","Marketing","Engineering","HR",
                    "Marketing","Engineering","HR"],
    "Salary"     : [90000, 60000, 95000, 55000, 62000, 88000, 57000],
    "Experience" : [5, 3, 7, 2, 4, 6, 3],
    "City"       : ["Mumbai","Delhi","Bangalore","Mumbai","Delhi","Hyderabad","Bangalo
re"],
    "Remote"     : [True, False, True, False, True, False, True],
    "Rating"     : [4.5, 3.8, 4.9, 3.5, 4.1, 4.3, 3.7]
})
```

Shape, Size & Dimensions

```python
print("Shape :", df.shape)       # (7, 7) → 7 rows, 7 columns
print("Rows  :", df.shape[0])    # 7
print("Cols  :", df.shape[1])    # 7
print("Size  :", df.size)        # 49 → total cells (7×7)
print("ndim  :", df.ndim)        # 2 → always 2 for DataFrame
```

Output:

```
Shape : (7, 7)
Rows  : 7
Cols  : 7
Size  : 49
ndim  : 2
```

Column Names & Data Types

```
print("Columns:", df.columns.tolist())
print("\\nDtypes:\\n", df.dtypes)
```

Output:

```
Columns: ['Name', 'Department', 'Salary', 'Experience', 'City', 'Remote', 'Rating']

Dtypes:
 Name          object
Department    object
Salary         int64
Experience     int64
City          object
Remote          bool
Rating       float64
dtype: object
```

`df.info()` — Your Best Friend for First Inspection

```
df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Name        7 non-null      object
 1   Department  7 non-null      object
 2   Salary      7 non-null      int64
 3   Experience  7 non-null      int64
 4   City        7 non-null      object
 5   Remote      7 non-null      bool
 6   Rating      7 non-null      float64
dtypes: bool(1), float64(1), int64(2), object(3)
memory usage: 487.0 bytes
```

Output Explanation:

- Shows column names, non-null counts, and data types at a glance
- If `Non-Null Count < total rows` → **missing values detected!**
- Shows memory usage — important for large datasets

`head()`, `tail()`, `sample()`

```
print(df.head(3))    # first 3 rows (default is 5)
print(df.tail(2))    # last 2 rows
print(df.sample(3, random_state=42))  # 3 random rows (reproducible)
```

Output (head):

```
    Name   Department  Salary  Experience       City  Remote  Rating
0  Alice  Engineering   90000           5     Mumbai    True     4.5
1    Bob    Marketing   60000           3      Delhi   False     3.8
2  Carol  Engineering   95000           7  Bangalore    True     4.9
```

`df.describe()` — Statistical Summary

```python
print(df.describe())
```

Output:

```
           Salary  Experience    Rating
count     7.00000    7.000000  7.000000
mean  72428.57143    4.285714  4.114286
std   17255.89813    1.704033  0.478091
min   55000.00000    2.000000  3.500000
25%   60000.00000    3.000000  3.800000
50%   62000.00000    4.000000  4.100000
75%   90000.00000    5.000000  4.500000
max   95000.00000    7.000000  4.900000
```

> **Tip:** Use `df.describe(include="all")` to include string/object columns too.

Checking for Missing Values

```python
print("Null counts per column:")
print(df.isnull().sum())

print("\\nAny nulls in the entire DataFrame?", df.isnull().any().any())

# Percentage of missing values
print("\\nMissing % per column:")
print((df.isnull().sum() / len(df) * 100).round(2))
```

Output (for our clean example):

```
Null counts per column:
Name          0
Department    0
Salary        0
Experience    0
City          0
Remote        0
Rating        0
dtype: int64

Any nulls? False
```

Complete DataFrame Attributes & Methods Summary

Attribute / Method	Description
`df.shape`	`(rows, cols)` tuple
`df.dtypes`	Data type of each column
`df.info()`	Concise summary with memory usage
`df.describe()`	Statistical summary (numeric columns)
`df.head(n)`	First `n` rows (default 5)
`df.tail(n)`	Last `n` rows (default 5)
`df.sample(n)`	`n` random rows
`df.columns`	Column labels (Index object)
`df.index`	Row labels (Index object)
`df.values`	Underlying NumPy array
`df.isnull().sum()`	Count of nulls per column
`df.shape[0]`	Number of rows
`df.shape[1]`	Number of columns

Attribute / Method	Description
`df.size`	Total elements = rows × cols
`df.ndim`	Always `2` for DataFrame

⚠️ Common Mistakes — DataFrame

Mistake	Explanation
Confusing `df.shape` vs `df.size`	`shape` → `(rows, cols)` tuple; `size` → total count of all cells
`df.describe()` shows only numeric cols	Use `df.describe(include="all")` for full overview
Not calling `df.info()` first	Always call it — catches missing values and wrong dtypes immediately
Lists of different lengths in dict	Raises `ValueError` — all lists must be same length
`to_csv()` without `index=False`	Creates a duplicate index column when file is re-read

Mini Summary & Cheat Sheet

🔑 Phase 1 Key Takeaways

- ✅ **Pandas** = industry-standard Python library for data analysis
- ✅ Always import as: `import pandas as pd`
- ✅ Two core structures:
  - **Series** → 1D labeled array (like a column in Excel)
  - **DataFrame** → 2D labeled table (like an Excel sheet / SQL table)
- ✅ A DataFrame is a **collection of Series sharing the same index**
- ✅ **First steps** after loading any dataset:

```
df.shape → df.info() → df.describe() → df.isnull().sum()
```

🔖 Quick Reference Cheat Sheet

Operation	Syntax
Create Series from list	`pd.Series([1, 2, 3])`
Series with custom index	`pd.Series([1,2,3], index=["a","b","c"])`
Series from dictionary	`pd.Series({"a":1, "b":2})`
Series with name	`pd.Series([1,2,3], name="Sales")`
Access by label	`s["label"]` or `s.loc["label"]`
Access by position	`s.iloc[0]`
Create DF from dict	`pd.DataFrame({"col1":[...], "col2":[...]})`
Create DF from list of dicts	`pd.DataFrame([{...}, {...}])`
Create DF from 2D list	`pd.DataFrame([[...],[...]], columns=[...])`
Create DF from NumPy	`pd.DataFrame(np_array, columns=[...])`
Read CSV	`pd.read_csv("file.csv")`
Read CSV with options	`pd.read_csv("file.csv", index_col="ID", parse_dates=["Date"])`
Read Excel	`pd.read_excel("file.xlsx", sheet_name="Sheet1")`
Save to CSV	`df.to_csv("file.csv", index=False)`
Save to Excel	`df.to_excel("file.xlsx", index=False)`
Quick info	`df.info()`
Statistical summary	`df.describe()`
First / Last rows	`df.head(5)` / `df.tail(5)`
Random rows	`df.sample(5)`
Shape	`df.shape`

Operation	Syntax
Null check	`df.isnull().sum()`
Column names	`df.columns.tolist()`
Data types	`df.dtypes`

🐼 PHASE 2: DATA INSPECTION & SELECTION — Complete Learning Notes

> **Goal:** Learn how to confidently explore, inspect, and select data from a DataFrame using Pandas — the skills you'll use in **every single real-world project**.

🗂️ The Master Dataset Used Throughout Phase 2

> We'll use one consistent real-world dataset throughout this entire phase so you can see how each method works on the same data.

```
import pandas as pd
import numpy as np

# Real-world: E-Commerce Sales Dataset
data = {
    "OrderID"    : [1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008, 1009, 1010],
    "Customer"   : ["Alice", "Bob", "Carol", "Dave", "Eve",
                    "Frank", "Grace", "Hank", "Ivy", "Jake"],
    "City"       : ["Mumbai", "Delhi", "Bangalore", "Mumbai", "Chennai",
                    "Delhi", "Hyderabad", "Mumbai", "Bangalore", "Chennai"],
    "Category"   : ["Electronics", "Clothing", "Electronics", "Furniture", "Clothing",
                    "Electronics", "Furniture", "Clothing", "Electronics", "Furnitur
e"],
    "Product"    : ["Laptop", "T-Shirt", "Phone", "Chair", "Jeans",
                    "Tablet", "Table", "Jacket", "Headphones", "Sofa"],
    "Quantity"   : [1, 3, 2, 1, 4, 2, 1, 2, 3, 1],
    "UnitPrice"  : [75000, 599, 22000, 8500, 1200, 35000, 12000, 2500, 3500, 45000],
    "Discount"   : [5, 10, 0, 15, 10, 5, 20, 0, 5, 10],
    "Rating"     : [4.5, 3.8, 4.7, 4.0, 3.5, 4.8, 3.9, 4.2, 4.6, 3.7],
    "OrderDate"  : pd.to_datetime(["2024-01-05","2024-01-08","2024-01-12","2024-01-1
5",
                                   "2024-01-20","2024-02-01","2024-02-05","2024-02-1
0",
                                   "2024-02-18","2024-02-22"]),
    "Delivered"  : [True, True, True, False, True, True, False, True, True, False]
}

df = pd.DataFrame(data)
print(df)
```

Output:

```
   OrderID Customer       City     Category     Product  Quantity  UnitPrice  \\
0     1001    Alice     Mumbai  Electronics      Laptop         1      75000
1     1002      Bob      Delhi     Clothing     T-Shirt         3        599
2     1003    Carol  Bangalore  Electronics       Phone         2      22000
3     1004     Dave     Mumbai    Furniture       Chair         1       8500
4     1005      Eve    Chennai     Clothing       Jeans         4       1200
5     1006    Frank      Delhi  Electronics      Tablet         2      35000
6     1007    Grace  Hyderabad    Furniture       Table         1      12000
7     1008     Hank     Mumbai     Clothing      Jacket         2       2500
```

```
8     1009      Ivy  Bangalore  Electronics  Headphones         3       3500
9     1010     Jake    Chennai    Furniture        Sofa         1      45000

   Discount  Rating  OrderDate  Delivered
0         5     4.5 2024-01-05       True
1        10     3.8 2024-01-08       True
2         0     4.7 2024-01-12       True
3        15     4.0 2024-01-15      False
4        10     3.5 2024-01-20       True
5         5     4.8 2024-02-01       True
6        20     3.9 2024-02-05      False
7         0     4.2 2024-02-10       True
8         5     4.6 2024-02-18       True
9        10     3.7 2024-02-22      False
```

---

1. Viewing Data

📌 Definition

Viewing methods allow you to **peek at your data** without printing the entire DataFrame — essential for large datasets with thousands or millions of rows.

`head(n)`

Definition

Returns the **first** `n` **rows** of the DataFrame. Default is `5`.

Why Use It?

- First thing you do after loading data
- Confirms the data loaded correctly
- Shows column names and sample values

Syntax

```
df.head()       # first 5 rows (default)
df.head(n)      # first n rows
```

Examples

```
# Example 1: Default — first 5 rows
print(df.head())
```

Output:

```
   OrderID Customer       City     Category  Product  Quantity  UnitPrice  \\
0     1001    Alice     Mumbai  Electronics   Laptop         1      75000
1     1002      Bob      Delhi     Clothing  T-Shirt         3        599
2     1003    Carol  Bangalore  Electronics    Phone         2      22000
3     1004     Dave     Mumbai    Furniture    Chair         1       8500
4     1005      Eve    Chennai     Clothing    Jeans         4       1200

   Discount  Rating  OrderDate  Delivered
0         5     4.5 2024-01-05       True
1        10     3.8 2024-01-08       True
2         0     4.7 2024-01-12       True
3        15     4.0 2024-01-15      False
4        10     3.5 2024-01-20       True
```

```
# Example 2: First 3 rows
print(df.head(3))
```

Output:

```
   OrderID Customer       City     Category  Product  Quantity  UnitPrice  \\
0     1001    Alice     Mumbai  Electronics   Laptop         1      75000
1     1002      Bob      Delhi     Clothing  T-Shirt         3        599
2     1003    Carol  Bangalore  Electronics    Phone         2      22000

   Discount  Rating  OrderDate  Delivered
0         5     4.5 2024-01-05       True
1        10     3.8 2024-01-08       True
2         0     4.7 2024-01-12       True
```

```python
# Example 3: head(1) – just the column structure + first row
print(df.head(1))
```

Output:

```
   OrderID Customer    City     Category Product  Quantity  UnitPrice  \\
0     1001    Alice  Mumbai  Electronics  Laptop         1      75000

   Discount  Rating  OrderDate  Delivered
0         5     4.5 2024-01-05       True
```

> **Output Explanation:** `head(1)` is great for verifying column structure without displaying too much data.

`tail(n)`

Definition

Returns the **last** `n` **rows** of the DataFrame. Default is `5`.

Why Use It?

- Check if data loaded completely (no missing rows at the end)
- Useful for time-series data to see the most recent records
- Verify data appended correctly

Syntax

```python
df.tail()       # last 5 rows (default)
df.tail(n)      # last n rows
```

Examples

```python
# Example 1: Default – last 5 rows
print(df.tail())
```

Output:

```
   OrderID Customer       City     Category     Product  Quantity  UnitPrice  \\
5     1006    Frank      Delhi  Electronics      Tablet         2      35000
6     1007    Grace  Hyderabad    Furniture       Table         1      12000
7     1008     Hank     Mumbai     Clothing      Jacket         2       2500
8     1009      Ivy  Bangalore  Electronics  Headphones         3       3500
9     1010     Jake    Chennai    Furniture        Sofa         1      45000

   Discount  Rating  OrderDate  Delivered
5         5     4.8 2024-02-01       True
6        20     3.9 2024-02-05      False
7         0     4.2 2024-02-10       True
8         5     4.6 2024-02-18       True
9        10     3.7 2024-02-22      False
```

```python
# Example 2: Last 3 rows
print(df.tail(3))
```

Output:

```
   OrderID Customer       City     Category     Product  Quantity  UnitPrice  \\
7     1008     Hank     Mumbai     Clothing      Jacket         2       2500
8     1009      Ivy  Bangalore  Electronics  Headphones         3       3500
9     1010     Jake    Chennai    Furniture        Sofa         1      45000

   Discount  Rating  OrderDate  Delivered
7         0     4.2 2024-02-10       True
8         5     4.6 2024-02-18       True
9        10     3.7 2024-02-22      False
```

`sample(n)`

Definition

Returns `n` **randomly selected rows** from the DataFrame.

Why Use It?

- Get an unbiased peek at data (not just the top/bottom)
- Great for spotting anomalies across the whole dataset
- Use `random_state` for reproducibility in reports

Syntax

```
df.sample()                          # 1 random row (default)
df.sample(n)                         # n random rows
df.sample(n, random_state=42)        # reproducible random selection
df.sample(frac=0.3)                  # random 30% of all rows
df.sample(frac=0.3, replace=True)  # sampling WITH replacement
```

Examples

```
# Example 1: 3 random rows (results change every run)
print(df.sample(3))
```

Output (one possible result):

```
   OrderID Customer    City     Category    Product  Quantity  UnitPrice  \\
6     1007    Grace  Hyderabad  Furniture    Table         1      12000
2     1003    Carol  Bangalore  Electronics  Phone         2      22000
9     1010     Jake     Chennai  Furniture   Sofa         1      45000
...
```

```
# Example 2: Reproducible random rows
print(df.sample(3, random_state=42))
```

Output:

```
   OrderID Customer    City   Category    Product  Quantity  UnitPrice  ...
0     1001    Alice  Mumbai  Electronics  Laptop         1      75000
2     1003    Carol  Bangalore Electronics   Phone         2      22000
4     1005      Eve  Chennai   Clothing     Jeans         4       1200
```

> **Output Explanation:** `random_state=42` ensures you get the same rows every time you run it — critical for documentation, testing, and reproducible analysis.

```
# Example 3: Sample a fraction (30% of rows)
print(df.sample(frac=0.3, random_state=1))
```

Output:

```
   OrderID Customer   City     Category  Product  Quantity  UnitPrice ...
8     1009      Ivy  Bangalore  Electronics  Headphones   3    3500
1     1002      Bob    Delhi     Clothing    T-Shirt      3     599
4     1005      Eve    Chennai   Clothing    Jeans        4    1200
```

> **Output Explanation:** `frac=0.3` samples 30% of 10 rows = 3 rows. Useful for quick ML train/test splits or exploration of large files.

💡 Important Notes — Viewing Methods

- `head()` and `tail()` are **most used in day-to-day work**
- Use `sample()` when you want a **statistically fair look** at data
- Always check both `head()` AND `tail()` — errors/garbage data often hides at the end of files

⚠️ Common Mistakes

Mistake	Explanation
Using only `head()` and never `tail()`	Corrupt or missing data often appears at the bottom
Not using `random_state` in `sample()`	Results change each run — hard to reproduce analysis
`df.head` (no parentheses)	Returns the method object, not the data — always use `()`

---

2. Data Info Methods

`info()`

Definition

Prints a **concise summary** of the DataFrame including:

- Number of rows and columns
- Column names
- Non-null count (helps detect missing values)
- Data type of each column
- Memory usage

Why Use It?

- **First thing to run** after loading any dataset
- Instantly see which columns have missing data
- Catch incorrect data types (e.g., numbers stored as strings)

Syntax

```
df.info()
df.info(memory_usage="deep")   # more accurate memory calc for object columns
```

Examples

```
# Example 1: Basic info
df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
```

```
Data columns (total 11 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   OrderID    10 non-null     int64
 1   Customer   10 non-null     object
 2   City       10 non-null     object
 3   Category   10 non-null     object
 4   Product    10 non-null     object
 5   Quantity   10 non-null     int64
 6   UnitPrice  10 non-null     int64
 7   Discount   10 non-null     int64
 8   Rating     10 non-null     float64
 9   OrderDate  10 non-null     datetime64[ns]
 10  Delivered  10 non-null     bool
dtypes: bool(1), datetime64[ns](1), float64(1), int64(4), object(4)
memory usage: 928.0 bytes
```

Output Explanation:

- `RangeIndex: 10 entries` → 10 rows
- `10 non-null` on every column → no missing values in our clean dataset
- `object` dtype = string/mixed types
- Dtypes summary at bottom: `bool(1), datetime64(1), float64(1), int64(4), object(4)`

```python
# Example 2: Dataset WITH missing values — how info() exposes them
df_missing = df.copy()
df_missing.loc[2, "Rating"]    = None
df_missing.loc[5, "City"]      = None
df_missing.loc[7, "Discount"]  = None

df_missing.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 11 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   OrderID    10 non-null     int64
 1   Customer   10 non-null     object
 2   City       9 non-null      object       ← MISSING 1 VALUE
 3   Category   10 non-null     object
 4   Product    10 non-null     object
 5   Quantity   10 non-null     int64
 6   UnitPrice  10 non-null     int64
 7   Discount   9 non-null      float64      ← MISSING 1 VALUE (int → float!)
 8   Rating     9 non-null      float64      ← MISSING 1 VALUE
 9   OrderDate  10 non-null     datetime64[ns]
 10  Delivered  10 non-null     bool
```

> **Key Insight:** When an `int64` column gets a `NaN` value, Pandas automatically converts it to `float64` — because integers can't hold NaN in older Pandas versions. This is an important behavior to remember.

```python
# Example 3: Deep memory usage (accurate for string columns)
df.info(memory_usage="deep")
```

Output:

```
...
memory usage: 5.7 KB    ← more accurate than default estimate
```

`describe()`

Definition

Returns a **statistical summary** of numerical columns by default.

Why Use It?

- Understand the **distribution** of your data
- Spot outliers (compare `min/max` to `mean`)
- Check data ranges before normalization in ML

Syntax

```
df.describe()                    # numeric columns only
df.describe(include="all")       # all columns including strings
df.describe(include=["object"])  # only string columns
df.describe(percentiles=[0.1, 0.5, 0.9])  # custom percentile breakpoints
```

Examples

```
# Example 1: Default — numeric columns only
print(df.describe())
```

Output:

```
         OrderID   Quantity      UnitPrice   Discount     Rating
count   10.00000  10.000000    10.000000  10.000000  10.000000
mean  1005.50000   2.000000  20579.900000   8.000000   4.170000
std      2.87228   1.054093  24038.928399   6.110101   0.414523
min   1001.00000   1.000000    599.000000   0.000000   3.500000
25%   1003.25000   1.000000   2125.000000   5.000000   3.825000
50%   1005.50000   2.000000   9250.000000   7.500000   4.250000
75%   1008.25000   3.000000  29750.000000  10.000000   4.550000
max   1010.00000   4.000000  75000.000000  20.000000   4.800000
```

Output Explanation Row by Row:

Row	What it Tells You
`count`	How many non-null values exist
`mean`	Average — e.g., avg price = ₹20,579
`std`	Standard deviation — how spread the data is
`min`	Lowest value — e.g., cheapest product = ₹599
`25%`	25th percentile (lower quartile)
`50%`	Median — middle value
`75%`	75th percentile (upper quartile)
`max`	Highest value — e.g., most expensive = ₹75,000

> **Insight:** `UnitPrice` has `mean=20,579` but `max=75,000` — the data is **right-skewed** (a few expensive items pulling the mean up). This is a common real-world pattern.

```
# Example 2: include="all" — includes object columns too
print(df.describe(include="all"))
```

Output (key differences for object columns):

```
       OrderID Customer       City     Category ...
count       10       10         10           10
unique     NaN       10          5            3
top        NaN    Alice     Mumbai  Electronics
freq       NaN        1          3            4
mean    1005.5      NaN        NaN          NaN
...
```

Output Explanation (for string/object columns):

Row	What it Tells You
`unique`	Number of distinct values
`top`	Most frequent value (mode)
`freq`	How many times the top value appears

> E.g., `City: top=Mumbai, freq=3` means Mumbai appears 3 times — the most common city.

```python
# Example 3: Custom percentiles
print(df.describe(percentiles=[0.1, 0.25, 0.5, 0.75, 0.9]))
```

Output:

```
         UnitPrice    Rating
count    10.000000  10.00000
mean  20579.900000   4.17000
std   24038.928399   0.41452
min     599.000000   3.50000
10%    1020.000000   3.62000   ← 10th percentile
25%    2125.000000   3.82500
50%    9250.000000   4.25000
75%   29750.000000   4.55000
90%   60000.000000   4.74000   ← 90th percentile
max   75000.000000   4.80000
```

> **Use case:** In fraud detection, you often check 10th and 90th percentiles to define "normal" transaction ranges.

Additional Inspection Methods

```python
# Value counts — frequency of each unique value in a column
print(df["Category"].value_counts())
```

Output:

```
Electronics    4
Furniture      3
Clothing       3
Name: Category, dtype: int64
```

> **Use:** Instantly see distribution of categories — like a simple histogram in text form.

```python
# Unique values in a column
print(df["City"].unique())
print("Number of unique cities:", df["City"].nunique())
```

Output:

```
['Mumbai' 'Delhi' 'Bangalore' 'Chennai' 'Hyderabad']
Number of unique cities: 5
```

```python
# Check for null values
print(df.isnull().sum())          # null count per column
print(df.isnull().sum().sum())    # total null count in entire DataFrame
```

Output:

```
OrderID      0
Customer     0
City         0
...
Delivered    0
```

```
dtype: int64
0            ← No missing values in our dataset
```

```python
# Data types of all columns
print(df.dtypes)
```

Output:

```
OrderID              int64
Customer            object
City                object
Category            object
Product             object
Quantity             int64
UnitPrice            int64
Discount             int64
Rating             float64
OrderDate    datetime64[ns]
Delivered              bool
dtype: object
```

```python
# Shape & Size
print("Shape :", df.shape)          # (10, 11)
print("Rows  :", len(df))           # 10  ← common alternative to df.shape[0]
print("Cols  :", len(df.columns))   # 11
```

💡 Important Notes — Info Methods

- `df.info()` → **Always run first.** Catches dtype issues and missing data instantly.
- `df.describe()` → **Run second.** Understand numerical ranges and distributions.
- `value_counts()` → Use for **categorical columns** to see distribution.
- `nunique()` → Use to check **cardinality** (useful before one-hot encoding in ML).

⚠️ Common Mistakes

Mistake	Fix
Relying only on `describe()` and missing NaNs	Always pair with `df.isnull().sum()`
Forgetting `include="all"` in `describe()`	String columns are silently skipped
Not checking `dtypes` after reading CSV	Dates often load as `object` — use `parse_dates`

---

3. Indexing and Selection

> 📌 **The most important skill in Pandas.** You will use `loc` and `iloc` in every project.

Overview

Accessor	Full Name	Based On	Use For
`loc[]`	Label-based	**Index labels** and **column names**	When you know the labels
`iloc[]`	Integer-based	**Row and column positions** (integers, 0-based)	When you know the position

---

`loc[]` — Label-Based Selection

Definition

`loc[]` selects data using **index labels** (row names) and **column names**.

Syntax

```
df.loc[row_label]                          # single row by label
df.loc[row_label, column_name]             # single cell
df.loc[start_label : end_label]            # slice of rows (INCLUSIVE)
df.loc[row_label, [col1, col2]]            # specific row, multiple columns
df.loc[[row1, row2], [col1, col2]]         # multiple rows + columns
df.loc[boolean_condition]                  # filter rows by condition
```

> ⚠️ **loc slicing is INCLUSIVE on BOTH ends** — `df.loc[0:3]` returns rows 0, 1, 2, **AND 3**.

Examples

```
# Example 1: Select a single row by index label
print(df.loc[0])
```

Output:

```
OrderID                 1001
Customer                Alice
City                   Mumbai
Category           Electronics
Product                 Laptop
Quantity                     1
UnitPrice                75000
Discount                     5
Rating                     4.5
OrderDate    2024-01-05 00:00:00
Delivered                 True
Name: 0, dtype: object
```

> **Output Explanation:** Returns a **Series** — the row becomes the index, column names become labels.

```
# Example 2: Select a single CELL — one row, one column
print(df.loc[0, "Customer"])    # Alice
print(df.loc[3, "UnitPrice"])   # 8500
print(df.loc[5, "Rating"])      # 4.8
```

Output:

```
Alice
8500
4.8
```

```
# Example 3: Slice of rows (INCLUSIVE)
print(df.loc[2:5])
```

Output:

```
   OrderID Customer       City     Category Product  Quantity  UnitPrice  \\
2     1003    Carol  Bangalore  Electronics   Phone         2      22000
3     1004     Dave     Mumbai    Furniture   Chair         1       8500
4     1005      Eve    Chennai     Clothing   Jeans         4       1200
5     1006    Frank      Delhi  Electronics  Tablet         2      35000

   Discount  Rating  OrderDate  Delivered
2         0     4.7 2024-01-12       True
3        15     4.0 2024-01-15      False
4        10     3.5 2024-01-20       True
5         5     4.8 2024-02-01       True
```

> Row with index `5` **is included** — loc slicing is always inclusive.

```python
# Example 4: Multiple specific rows
print(df.loc[[0, 3, 7]])
```

Output:

```
   OrderID Customer    City     Category Product  Quantity  UnitPrice  \\
0     1001    Alice  Mumbai  Electronics  Laptop         1      75000
3     1004     Dave  Mumbai    Furniture   Chair         1       8500
7     1008     Hank  Mumbai     Clothing  Jacket         2       2500
...
```

```python
# Example 5: Select specific rows AND specific columns
print(df.loc[0:4, ["Customer", "Product", "UnitPrice"]])
```

Output:

```
  Customer  Product  UnitPrice
0    Alice   Laptop      75000
1      Bob  T-Shirt        599
2    Carol    Phone      22000
3     Dave    Chair       8500
4      Eve    Jeans       1200
```

```python
# Example 6: Single row, multiple columns
print(df.loc[5, ["Customer", "Product", "Rating"]])
```

Output:

```
Customer     Frank
Product     Tablet
Rating         4.8
Name: 5, dtype: object
```

```python
# Example 7: Select ALL rows for specific columns
print(df.loc[:, ["Customer", "City", "UnitPrice"]])
```

Output:

```
  Customer       City  UnitPrice
0    Alice     Mumbai      75000
1      Bob      Delhi        599
2    Carol  Bangalore      22000
3     Dave     Mumbai       8500
4      Eve    Chennai       1200
5    Frank      Delhi      35000
6    Grace  Hyderabad      12000
7     Hank     Mumbai       2500
8      Ivy  Bangalore       3500
9     Jake    Chennai      45000
```

> **Tip:** `df.loc[:, ["col1","col2"]]` is equivalent to `df[["col1","col2"]]` — both work.

```python
# Example 8: loc with a CUSTOM index (non-default)
# Set OrderID as the index first
df_idx = df.set_index("OrderID")
print(df_idx.loc[1003])               # access row where OrderID = 1003
print(df_idx.loc[1003, "Customer"])   # Carol
```

Output:

```
Customer          Carol
City            Bangalore
Category    Electronics
Product           Phone
...
Name: 1003, dtype: object

Carol
```

> **Output Explanation:** After setting `OrderID` as index, you can now use `df.loc[1003]` to access rows by their OrderID — much more intuitive than position-based access.

`iloc[]` — Position-Based Selection

Definition

`iloc[]` selects data using **integer positions** — like Python list indexing.

Syntax

```
df.iloc[row_position]                    # single row by position
df.iloc[row_position, col_position]      # single cell
df.iloc[start : end]                     # slice rows (end EXCLUSIVE)
df.iloc[[r1, r2], [c1, c2]]              # multiple rows & columns
df.iloc[:, 0:3]                          # all rows, first 3 columns
df.iloc[-1]                              # last row
df.iloc[-3:]                             # last 3 rows
```

> ⚠️ **iloc slicing is EXCLUSIVE on the end** — `df.iloc[0:3]` returns rows 0, 1, 2 (NOT 3) — same as Python list slicing.

Examples

```
# Example 1: Single row by position
print(df.iloc[0])    # first row
print(df.iloc[-1])   # last row
```

Output (first row):

```
OrderID             1001
Customer           Alice
City              Mumbai
...
```

```
# Example 2: Single cell by position
print(df.iloc[0, 1])    # row 0, column 1 → "Alice"
print(df.iloc[4, 6])    # row 4, column 6 → 1200 (UnitPrice of Eve's order)
```

Output:

```
Alice
1200
```

```
# Example 3: Slice of rows (EXCLUSIVE end)
print(df.iloc[2:5])    # rows 2, 3, 4 — NOT 5
```

Output:

```
   OrderID Customer       City     Category Product  Quantity  UnitPrice  ...
2     1003    Carol  Bangalore  Electronics   Phone         2      22000
3     1004     Dave     Mumbai    Furniture   Chair         1       8500
4     1005      Eve    Chennai     Clothing   Jeans         4       1200
```

```python
# Example 4: Specific rows and columns by position
print(df.iloc[[0, 2, 4], [1, 4, 6]])
```

Output:

```
  Customer Product  UnitPrice
0    Alice  Laptop      75000
2    Carol   Phone      22000
4      Eve   Jeans       1200
```

> **Output Explanation:** Row positions [0,2,4] × column positions [1,4,6] → `Customer`, `Product`, `UnitPrice` for Alice, Carol, Eve.

```python
# Example 5: First N columns (useful for quick look)
print(df.iloc[:, :4])    # all rows, first 4 columns
```

Output:

```
   OrderID Customer       City     Category
0     1001    Alice     Mumbai  Electronics
1     1002      Bob      Delhi     Clothing
...
```

```python
# Example 6: Last 3 rows
print(df.iloc[-3:])
```

Output:

```
   OrderID Customer       City     Category     Product  Quantity  ...
7     1008     Hank     Mumbai     Clothing      Jacket         2
8     1009      Ivy  Bangalore  Electronics  Headphones         3
9     1010     Jake    Chennai    Furniture        Sofa         1
```

```python
# Example 7: Last column of every row
print(df.iloc[:, -1])
```

Output:

```
0     True
1     True
2     True
3    False
4     True
5     True
6    False
7     True
8     True
9    False
Name: Delivered, dtype: bool
```

```python
# Example 8: Middle section of the DataFrame
print(df.iloc[3:7, 1:5])   # rows 3-6, columns 1-4
```

Output:

```
  Customer       City     Category Product
3     Dave     Mumbai    Furniture   Chair
4      Eve    Chennai     Clothing   Jeans
5    Frank      Delhi  Electronics  Tablet
6    Grace  Hyderabad    Furniture   Table
```

`loc` vs `iloc` — Side-by-Side Comparison

```python
# Same result using loc and iloc
print(df.loc[2, "Customer"])     # by label → "Carol"
print(df.iloc[2, 1])             # by position → "Carol"

# Slicing difference (VERY IMPORTANT)
print(df.loc[1:3])    # rows with INDEX LABELS 1, 2, 3 → INCLUSIVE → 3 rows
print(df.iloc[1:3])   # rows at POSITIONS 1, 2 → EXCLUSIVE → 2 rows
```

Feature	`loc[]`	`iloc[]`
Based on	**Labels** (index names, column names)	**Positions** (integers, 0-based)
Row slicing	**Inclusive** — `[1:3]` includes 1, 2, 3	**Exclusive** — `[1:3]` includes 1, 2 only
Custom index	✅ Works perfectly	❌ Ignores labels, always uses position
Boolean mask	✅ Yes	❌ No
Use when	You know the **names/labels**	You know the **position numbers**

💡 Important Notes — loc & iloc

- **Default RangeIndex (0,1,2...)** → `loc[2]` and `iloc[2]` give the same row
- As soon as you set a **custom index**, they differ — use `loc` for custom indexes
- In ML pipelines, use `iloc` for **positional slicing** (train/test splits)
- Prefer `loc` for **data filtering and selection** — more readable

⚠️ Common Mistakes — loc & iloc

Mistake	Explanation
`df.loc[1:3]` expecting 2 rows	loc is inclusive — you get 3 rows (1, 2, 3)
`df.iloc[1:3]` expecting 3 rows	iloc is exclusive — you get 2 rows (1, 2)
`df.iloc[0, "Customer"]`	`iloc` doesn't accept column names — use position number
`df.loc[0, 1]`	`loc` doesn't accept column positions — use column name
Forgetting `.copy()` after `loc` selection	Modifying the result modifies the original DataFrame too

---

4. Column Selection

📌 Definition

Selecting one or more **columns** from a DataFrame. This is the **most frequent operation** in data analysis.

Methods for Column Selection

Method 1: Single Column with `[]` → Returns a Series

```python
# Select one column → returns a Series
col = df["Customer"]
```

```python
print(type(col))     # <class 'pandas.core.series.Series'>
print(col)
```

Output:

```
<class 'pandas.core.series.Series'>
0    Alice
1      Bob
2    Carol
3     Dave
4      Eve
5    Frank
6    Grace
7     Hank
8      Ivy
9     Jake
Name: Customer, dtype: object
```

Method 2: Multiple Columns with `[[]]` → Returns a DataFrame

```python
# Select multiple columns → returns a DataFrame
cols = df[["Customer", "Product", "UnitPrice"]]
print(type(cols))    # <class 'pandas.core.frame.DataFrame'>
print(cols)
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
  Customer     Product  UnitPrice
0    Alice      Laptop      75000
1      Bob     T-Shirt        599
2    Carol       Phone      22000
3     Dave       Chair       8500
4      Eve       Jeans       1200
5    Frank      Tablet      35000
6    Grace       Table      12000
7     Hank      Jacket       2500
8      Ivy  Headphones       3500
9     Jake        Sofa      45000
```

> **Output Explanation:** Notice double brackets `[[...]]` — the outer `[]` is for DataFrame selection, the inner `[...]` is the Python list of column names.

Method 3: Using Dot Notation — `df.ColumnName`

```python
# Works for column names without spaces
print(df.Customer)     # same as df["Customer"]
print(df.Rating)
```

> ⚠️ **Avoid dot notation** — it does NOT work when column names have spaces or match existing DataFrame attributes (`df.count`, `df.mean`, etc.)

Method 4: Select Columns by Data Type

```python
# Select only numeric columns
df_numeric = df.select_dtypes(include=["int64", "float64"])
print(df_numeric.columns.tolist())
```

Output:

```
['OrderID', 'Quantity', 'UnitPrice', 'Discount', 'Rating']
```

```python
# Select only string (object) columns
df_text = df.select_dtypes(include=["object"])
print(df_text.columns.tolist())
```

Output:

```
['Customer', 'City', 'Category', 'Product']
```

```python
# Exclude specific dtypes
df_no_bool = df.select_dtypes(exclude=["bool"])
print(df_no_bool.columns.tolist())
```

Method 5: Select Columns by Name Pattern

```python
# Get all columns that contain "Price" in the name
price_cols = [col for col in df.columns if "Price" in col]
print(price_cols)    # ['UnitPrice']

# Get all columns starting with a letter (using filter)
print(df.filter(like="Date").columns.tolist())    # ['OrderDate']
print(df.filter(regex="^[A-Z]").columns.tolist()) # columns starting with capital
```

Method 6: Adding a New Calculated Column

```python
# Create TotalAmount column
df["TotalAmount"] = df["Quantity"] * df["UnitPrice"]

# Apply discount
df["FinalAmount"] = df["TotalAmount"] * (1 - df["Discount"] / 100)

print(df[["Customer", "Product", "TotalAmount", "FinalAmount"]].head())
```

Output:

```
  Customer  Product  TotalAmount  FinalAmount
0    Alice   Laptop        75000      71250.0
1      Bob  T-Shirt         1797       1617.3
2    Carol    Phone        44000      44000.0
3     Dave    Chair         8500       7225.0
4      Eve    Jeans         4800       4320.0
```

Method 7: Dropping Columns

```python
# Drop a single column (does NOT modify original unless inplace=True)
df_no_discount = df.drop(columns=["Discount"])

# Drop multiple columns
df_slim = df.drop(columns=["Discount", "OrderID", "Delivered"])

# Drop in-place
```

```python
df.drop(columns=["TotalAmount", "FinalAmount"], inplace=True)

print(df.columns.tolist())
```

💡 Important Notes — Column Selection

- `df["col"]` → **Series** (single column)
- `df[["col"]]` → **DataFrame** (single-column DataFrame) — notice double `[[]]`
- This distinction matters: many Pandas functions expect a DataFrame, not a Series

⚠️ Common Mistakes

Mistake	Explanation
`df["col1", "col2"]` (single brackets)	`KeyError` — must use double brackets `df[["col1","col2"]]`
`df.my col` (space in name)	Syntax error — never use dot notation for columns with spaces
Using dot notation for `df.count`	`count` is a DataFrame method — conflicts! Always use `df["count"]`

5. Conditional Filtering & Boolean Indexing

📌 Definition

Boolean Indexing means filtering rows based on a **True/False condition** applied to one or more columns.

> Think of it as: "Give me all rows WHERE this condition is TRUE."
> This is exactly like `SQL WHERE` clause or Excel's `FILTER` function.

How Boolean Indexing Works (Under the Hood)

```python
# Step 1: A condition creates a boolean Series
condition = df["Category"] == "Electronics"
print(condition)
```

Output:

```
0     True
1    False
2     True
3    False
4    False
5     True
6    False
7    False
8     True
9    False
Name: Category, dtype: bool
```

```python
# Step 2: Pass the boolean Series to df[] → keeps only True rows
filtered = df[condition]
print(filtered[["Customer","Product","Category"]])
```

Output:

```
  Customer     Product     Category
0    Alice      Laptop  Electronics
2    Carol       Phone  Electronics
5    Frank      Tablet  Electronics
8      Ivy  Headphones  Electronics
```

> **Output Explanation:** Only rows where `condition = True` are returned. Pandas uses the boolean mask to filter rows in one efficient step.

Single Condition Filtering

```
# Example 1: Filter rows where Category is "Electronics"
electronics = df[df["Category"] == "Electronics"]
print(electronics[["Customer", "Product", "UnitPrice"]])
```

Output:

```
  Customer     Product  UnitPrice
0    Alice      Laptop      75000
2    Carol       Phone      22000
5    Frank      Tablet      35000
8      Ivy  Headphones       3500
```

```
# Example 2: Filter orders with UnitPrice > 20000
expensive = df[df["UnitPrice"] > 20000]
print(expensive[["Customer", "Product", "UnitPrice"]])
```

Output:

```
  Customer Product  UnitPrice
0    Alice  Laptop      75000
2    Carol   Phone      22000
5    Frank  Tablet      35000
9     Jake    Sofa      45000
```

```
# Example 3: Filter orders from Mumbai
mumbai = df[df["City"] == "Mumbai"]
print(mumbai[["Customer", "Product", "City"]])
```

Output:

```
  Customer Product    City
0    Alice  Laptop  Mumbai
3     Dave   Chair  Mumbai
7     Hank  Jacket  Mumbai
```

```
# Example 4: Filter delivered orders only
delivered = df[df["Delivered"] == True]
# OR cleaner:
delivered = df[df["Delivered"]]
print(f"Delivered orders: {len(delivered)} out of {len(df)}")
```

Output:

```
Delivered orders: 7 out of 10
```

```
# Example 5: Filter low ratings (< 4.0)
low_rated = df[df["Rating"] < 4.0]
print(low_rated[["Customer", "Product", "Rating"]])
```

Output:

```
  Customer  Product  Rating
1      Bob  T-Shirt     3.8
4      Eve    Jeans     3.5
6    Grace    Table     3.9
9     Jake     Sofa     3.7
```

Multiple Conditions — AND, OR, NOT

> ⚠️ **CRITICAL RULE:** Do NOT use Python's `and`, `or`, `not` — use `&`, `|`, `~` instead.
> Each condition **MUST be wrapped in parentheses** `()`.

```
# Operators for multiple conditions:
# &  → AND (both conditions must be True)
# |  → OR  (at least one condition must be True)
# ~  → NOT (inverts True/False)
```

```
# Example 6: AND — Electronics category AND price > 10000
elec_expensive = df[(df["Category"] == "Electronics") & (df["UnitPrice"] > 10000)]
print(elec_expensive[["Customer", "Product", "Category", "UnitPrice"]])
```

Output:

```
  Customer Product     Category  UnitPrice
0    Alice  Laptop  Electronics      75000
2    Carol   Phone  Electronics      22000
5    Frank  Tablet  Electronics      35000
```

```
# Example 7: OR — Electronics OR Furniture
elec_or_furn = df[(df["Category"] == "Electronics") | (df["Category"] == "Furniture")]
print(elec_or_furn[["Customer", "Product", "Category"]])
```

Output:

```
  Customer     Product     Category
0    Alice      Laptop  Electronics
2    Carol       Phone  Electronics
3     Dave       Chair    Furniture
5    Frank      Tablet  Electronics
6    Grace       Table    Furniture
8      Ivy  Headphones  Electronics
9     Jake        Sofa    Furniture
```

```
# Example 8: NOT — Everyone except Mumbai customers
not_mumbai = df[~(df["City"] == "Mumbai")]
print(not_mumbai[["Customer", "City"]])
```

Output:

```
  Customer       City
1      Bob      Delhi
2    Carol  Bangalore
4      Eve    Chennai
5    Frank      Delhi
6    Grace  Hyderabad
8      Ivy  Bangalore
9     Jake    Chennai
```

```python
# Example 9: Three conditions — Electronics, price > 5000, and delivered
complex_filter = df[
    (df["Category"] == "Electronics") &
    (df["UnitPrice"] > 5000) &
    (df["Delivered"] == True)
]
print(complex_filter[["Customer", "Product", "UnitPrice", "Delivered"]])
```

Output:

```
  Customer Product  UnitPrice  Delivered
0    Alice  Laptop      75000       True
2    Carol   Phone      22000       True
5    Frank  Tablet      35000       True
```

`isin()` — Filter by Multiple Values

```python
# Example 10: Filter orders from Mumbai OR Delhi OR Chennai
target_cities = df[df["City"].isin(["Mumbai", "Delhi", "Chennai"])]
print(target_cities[["Customer", "City", "Product"]])
```

Output:

```
  Customer    City  Product
0    Alice  Mumbai   Laptop
1      Bob   Delhi  T-Shirt
3     Dave  Mumbai    Chair
4      Eve  Chennai   Jeans
5    Frank   Delhi   Tablet
7     Hank  Mumbai   Jacket
9     Jake  Chennai    Sofa
```

```python
# NOT isin — exclude specific cities
not_metro = df[~df["City"].isin(["Mumbai", "Delhi"])]
print(not_metro[["Customer", "City"]])
```

Output:

```
  Customer       City
2    Carol  Bangalore
4      Eve    Chennai
6    Grace  Hyderabad
8      Ivy  Bangalore
9     Jake    Chennai
```

`between()` — Filter Numeric Range

```python
# Example 11: Orders with UnitPrice between 1000 and 20000
mid_range = df[df["UnitPrice"].between(1000, 20000)]
print(mid_range[["Customer", "Product", "UnitPrice"]])
```

Output:

```
  Customer   Product  UnitPrice
4      Eve     Jeans       1200
6    Grace     Table      12000
7     Hank    Jacket       2500
8      Ivy  Headphones     3500
```

`str` Methods for Text Filtering

```python
# Example 12: Filter products starting with "T"
t_products = df[df["Product"].str.startswith("T")]
print(t_products[["Customer", "Product"]])
```

Output:

```
  Customer  Product
1      Bob  T-Shirt
5    Frank   Tablet
6    Grace    Table
```

```python
# Example 13: Filter customers whose name contains "a" (case-insensitive)
a_customers = df[df["Customer"].str.contains("a", case=False)]
print(a_customers[["Customer"]])
```

Output:

```
  Customer
0    Alice
3     Dave
5    Frank
6    Grace
7     Hank
```

`query()` — Clean SQL-Style Filtering

```python
# Example 14: query() method — more readable for complex filters
result = df.query("Category == 'Electronics' and UnitPrice > 10000")
print(result[["Customer", "Product", "UnitPrice"]])
```

Output:

```
  Customer Product  UnitPrice
0    Alice  Laptop      75000
2    Carol   Phone      22000
5    Frank  Tablet      35000
```

```python
# Example 15: query with a variable
min_price = 5000
result = df.query("UnitPrice > @min_price and Delivered == True")
# Use @ to reference Python variables inside query string
print(result[["Customer", "Product", "UnitPrice"]])
```

Output:

```
  Customer     Product  UnitPrice
0    Alice      Laptop      75000
2    Carol       Phone      22000
5    Frank      Tablet      35000
```

💡 Important Notes — Filtering

- Always use `&`, `|`, `~` — never `and`, `or`, `not` with conditions
- Always wrap each condition in `()` — required for correct operator precedence
- `isin()` is cleaner than multiple `|` conditions for many values
- `between()` is inclusive on both ends

- `query()` is great for readability but slightly slower on huge datasets
- Chaining filters does **NOT modify the original DataFrame** (returns a new one)

⚠️ Common Mistakes — Filtering

Mistake	Problem	Fix
`df[df["col"] == "A" and df["col2"] > 5]`	`ValueError` — can't use `and`	Use `&`: `df[(df["col"]=="A") & (df["col2"]>5)]`
Missing `()` around conditions	Operator precedence error	Always wrap each condition in `()`
`df[df["col"] == True]` vs `df[df["col"]]`	Both work, but second is cleaner for bool columns	Prefer `df[df["col"]]` for bool columns
Forgetting `~` for NOT	Using `!= True` is not always equivalent to `~` for NaN	Use `~(condition)` for proper NOT logic
`df.query("col == value")` with a string value	Need quotes: `df.query("col == 'value'")`	Use single quotes inside the query string

Mini Summary & Cheat Sheet

🔑 Phase 2 Key Takeaways

- ✅ **Viewing:** `head()` first rows, `tail()` last rows, `sample()` random rows
- ✅ **Inspection:** `info()` for types & nulls, `describe()` for statistics
- ✅ **loc[]** — label-based, **inclusive** slicing
- ✅ **iloc[]** — position-based, **exclusive** end in slicing
- ✅ **Column selection:** `df["col"]` → Series, `df[["col1","col2"]]` → DataFrame
- ✅ **Filtering:** use `&`, `|`, `~` (never `and`, `or`, `not`) with `()`
- ✅ **Helper methods:** `isin()`, `between()`, `str.contains()`, `query()`

🔖 Quick Reference Cheat Sheet

Operation	Syntax
First n rows	`df.head(n)`
Last n rows	`df.tail(n)`
Random n rows	`df.sample(n, random_state=42)`
Random fraction	`df.sample(frac=0.2)`
Data info	`df.info()`
Statistical summary	`df.describe()`
Category counts	`df["col"].value_counts()`
Unique values	`df["col"].unique()`
Unique count	`df["col"].nunique()`
Null count per col	`df.isnull().sum()`
Select row by label	`df.loc[2]`
Select cell by label	`df.loc[2, "Name"]`
Slice rows by label	`df.loc[1:4]` — inclusive
Select row by position	`df.iloc[2]`
Select cell by position	`df.iloc[2, 1]`
Slice rows by position	`df.iloc[1:4]` — exclusive end
Last row	`df.iloc[-1]`
Single column (Series)	`df["col"]`
Multiple columns (DF)	`df[["col1", "col2"]]`
All numeric cols	`df.select_dtypes(include="number")`
Simple filter	`df[df["col"] == value]`

Operation	Syntax
AND filter	`df[(df["col1"] == x) & (df["col2"] > y)]`
OR filter	`df[(df["col1"] == x) | (df["col2"] == y)]`
NOT filter	`df[~(df["col"] == value)]`
Filter by list	`df[df["col"].isin(["A","B","C"])]`
Filter by range	`df[df["col"].between(low, high)]`
Text contains	`df[df["col"].str.contains("text")]`
SQL-style filter	`df.query("col > 100 and col2 == 'A'")`
Add new column	`df["new"] = df["a"] * df["b"]`
Drop column	`df.drop(columns=["col"], inplace=True)`

🐼 PHASE 3: DATA CLEANING — Complete Learning Notes

> **Goal:** Learn how to clean messy, real-world data like a professional Data Analyst.
> Data cleaning is the **most time-consuming step** in any Data Science project (60–80% of your time). Master it here.

🗂️ The Dirty Dataset Used Throughout Phase 3

> A deliberately messy dataset — missing values, duplicates, wrong types, inconsistent strings — exactly what you'll face in real-world projects.

```
import pandas as pd
import numpy as np

data = {
    "CustomerID"  : [101, 102, 103, 104, 105, 102, 106, 107, 108, 109, 110],
    "Name"        : ["Alice Johnson", "bob smith", "Carol White", None,
                     "Eve Davis", "bob smith", "FRANK WILSON", "grace lee",
                     "Hank Moore", "  Ivy Clark  ", "Jake Brown"],
    "Age"         : [28, 35, None, 42, 29, 35, 51, None, 38, 26, 45],
    "City"        : ["Mumbai", "delhi", "Bangalore", "mumbai", "Chennai",
                     "delhi", "hyderabad", "Delhi", "Bangalore", None, "Chennai"],
    "Email"       : ["alice@gmail.com", "bob@yahoo.com", "carol@gmail.com",
                     None, "eve@gmail.com", "bob@yahoo.com", "frank@outlook.com",
                     "grace@gmail.com", None, "ivy@gmail.com", "jake@gmail.com"],
    "Salary"      : ["90000", "60000", "95000", "55000", "62000",
                     "60000", "88000", "57000", "75000", "48000", "70000"],
    "JoinDate"    : ["2020-03-15", "2019-07-22", "2021-01-10", "2018-11-05",
                     "2022-06-30", "2019-07-22", "2017-09-14", "2023-02-28",
                     "2020-08-19", "2024-01-01", "2016-05-12"],
    "Department"  : ["Engineering", "Marketing", "Engineering", "HR",
                     "Marketing", "Marketing", "Engineering", "HR",
                     "Finance", "Marketing", "Finance"],
    "Rating"      : [4.5, 3.8, None, 3.5, 4.1, 3.8, 4.3, None, 4.6, 3.7, 4.0],
    "Active"      : ["Yes", "No", "Yes", "Yes", "No", "No", "Yes", "Yes",
                     "No", "Yes", "Yes"]
}

df = pd.DataFrame(data)
print(df)
print("\\nShape:", df.shape)
```

Output:

```
   CustomerID           Name   Age       City              Email  Salary    JoinDate  Department  Rating Active
0         101  Alice Johnson  28.0     Mumbai    alice@gmail.com   90000  2020-03-15  Engineering     4.5    Yes
1         102      bob smith  35.0      delhi      bob@yahoo.com   60000  2019-07-22    Marketing     3.8     No
2         103    Carol White   NaN  Bangalore    carol@gmail.com   95000  2021-01-10  Engineering     NaN    Yes
3         104           None  42.0     mumbai              None   55000  2018-11-05           HR     3.5    Yes
4         105      Eve Davis  29.0    Chennai      eve@gmail.com   62000  2022-06-30    Marketing     4.1     No
5         102      bob smith  35.0      delhi      bob@yahoo.com   60000  2019-07-22    Marketing     3.8     No
6         106   FRANK WILSON  51.0  hyderabad  frank@outlook.com   88000  2017-09-14  Engineering     4.3    Yes
7         107      grace lee   NaN      Delhi    grace@gmail.com   57000  2023-02-28           HR     NaN    Yes
8         108     Hank Moore  38.0  Bangalore              None   75000  2020-08-19      Finance     4.6     No
9         109   Ivy Clark     26.0       None      ivy@gmail.com   48000  2024-01-01    Marketing     3.7    Yes
10        110     Jake Brown  45.0    Chennai     jake@gmail.com   70000  2016-05-12      Finance     4.0    Yes

Shape: (11, 10)
```

Problems in this dataset:

Problem	Columns Affected
❌ Missing values (NaN/None)	`Name`, `Age`, `City`, `Email`, `Rating`
❌ Duplicate rows	Row 1 & Row 5 (same CustomerID 102)
❌ Inconsistent case	`City` (delhi, mumbai, hyderabad), `Name` (bob smith, FRANK WILSON)
❌ Leading/trailing spaces	`Name` column — `" Ivy Clark "`
❌ Wrong data type	`Salary` is string, `JoinDate` is string
❌ Inconsistent encoding	`Active` is "Yes"/"No" — should be boolean

1. Handling Missing Values

📌 Definition

Missing values are gaps in your data — shown as `NaN` (Not a Number) in Pandas.
They appear when:

- Data was not collected / not entered
- Merge operations produce unmatched rows
- API responses have null fields
- Data corruption during file reading

`isnull()` & `notnull()`

Definition

- `isnull()` → returns `True` where value **IS** missing (NaN)
- `notnull()` → returns `True` where value **IS NOT** missing

Syntax

```python
df.isnull()           # boolean DataFrame — True = missing
df.notnull()          # boolean DataFrame — True = present
df["col"].isnull()    # boolean Series for one column
df.isnull().sum()     # count of nulls per column
df.isnull().sum().sum()  # total null count in DataFrame
df.isnull().mean() * 100 # % missing per column
```

Examples

```python
# Example 1: Full boolean mask
print(df.isnull())
```

Output:

```
    CustomerID   Name    Age   City  Email  Salary  JoinDate  Department  Rating  Active
0        False  False  False  False  False   False     False       False   False   False
1        False  False  False  False  False   False     False       False   False   False
2        False  False   True  False  False   False     False       False    True   False
3        False   True  False  False   True   False     False       False   False   False
4        False  False  False  False  False   False     False       False   False   False
5        False  False  False  False  False   False     False       False   False   False
6        False  False  False  False  False   False     False       False   False   False
7        False  False   True  False  False   False     False       False    True   False
8        False  False  False  False   True   False     False       False   False   False
9        False  False  False   True  False   False     False       False   False   False
10       False  False  False  False  False   False     False       False   False   False
```

```python
# Example 2: Count of missing values per column
print(df.isnull().sum())
```

Output:

```
CustomerID    0
Name          1
Age           2
City          1
Email         2
Salary        0
JoinDate      0
Department    0
Rating        2
Active        0
dtype: int64
```

```python
# Example 3: Percentage of missing per column
missing_pct = (df.isnull().sum() / len(df) * 100).round(2)
print(missing_pct)
```

Output:

```
CustomerID     0.00
Name           9.09
Age           18.18
City           9.09
Email         18.18
Salary         0.00
JoinDate       0.00
Department     0.00
Rating        18.18
Active         0.00
dtype: float64
```

> **Output Explanation:** `Age`, `Email`, and `Rating` each have ~18% missing — significant enough that we need a strategy (drop vs fill) rather than ignoring them.

```python
# Example 4: Total missing values across entire DataFrame
print("Total missing cells:", df.isnull().sum().sum())
print("Total cells        :", df.size)
print("Missing %          :", round(df.isnull().sum().sum() / df.size * 100, 2))
```

Output:

```
Total missing cells: 8
Total cells        : 110
Missing %          : 7.27
```

```python
# Example 5: Find ROWS that have at least one missing value
rows_with_null = df[df.isnull().any(axis=1)]
print(rows_with_null[["CustomerID","Name","Age","Email","Rating"]])
```

Output:

```
   CustomerID           Name   Age              Email  Rating
2         103    Carol White   NaN    carol@gmail.com     NaN
3         104           None  42.0               None     3.5
7         107     grace lee    NaN    grace@gmail.com     NaN
8         108     Hank Moore  38.0               None     4.6
9         109     Ivy Clark      NaN      ivy@gmail.com    3.7
```

> `axis=1` = check across columns (row-wise). Rows 2, 3, 7, 8, 9 have at least one NaN.

```python
# Example 6: notnull() — filter rows where Email is present
has_email = df[df["Email"].notnull()]
print(has_email[["Name", "Email"]])
```

Output:

```
             Name              Email
0   Alice Johnson    alice@gmail.com
1       bob smith      bob@yahoo.com
2     Carol White    carol@gmail.com
4       Eve Davis      eve@gmail.com
5       bob smith      bob@yahoo.com
6    FRANK WILSON  frank@outlook.com
7       grace lee    grace@gmail.com
9      Ivy Clark       ivy@gmail.com
10     Jake Brown     jake@gmail.com
```

```python
# Example 7: Count non-null values per column
print(df.notnull().sum())
```

Output:

```
CustomerID    11
Name          10
Age            9
City          10
Email          9
Salary        11
JoinDate      11
Department    11
Rating         9
Active        11
dtype: int64
```

`dropna()`

Definition

Removes rows (or columns) that contain missing values.

Why Use It?

- When missing data is too small to matter (< 5%) — safe to drop
- When the missing column itself is critical (e.g., can't predict without target column)
- When filling would introduce bias

Syntax

```python
df.dropna()                          # drop rows with ANY null
df.dropna(axis=1)                    # drop COLUMNS with any null
df.dropna(how="all")                 # drop rows where ALL values are null
df.dropna(subset=["col1","col2"])    # drop rows where specific cols have null
df.dropna(thresh=5)                  # keep rows with at least 5 non-null values
df.dropna(inplace=True)              # modify original DataFrame
```

> ⚠️ `dropna()` **returns a new DataFrame by default — does NOT modify in place unless** `inplace=True`

Examples

```python
# Example 1: Default — drop ANY row with at least one NaN
df_clean = df.dropna()
print("Original shape :", df.shape)
print("After dropna() :", df_clean.shape)
```

Output:

```
Original shape : (11, 10)
After dropna() : (6, 10)
```

> **Output Explanation:** We lost 5 rows — from 11 to 6. In real-world data, this can mean losing too much data. Always check what percentage you're dropping before using `dropna()`.

```python
# Example 2: Drop only rows where SPECIFIC columns have NaN
# Strategy: only drop if Email OR Rating is missing (critical fields)
df_drop_critical = df.dropna(subset=["Email", "Rating"])
print("After dropping rows with null Email/Rating:", df_drop_critical.shape)
print(df_drop_critical[["Name", "Email", "Rating"]])
```

Output:

```
After dropping rows with null Email/Rating: (7, 10)

             Name              Email  Rating
0   Alice Johnson    alice@gmail.com     4.5
1       bob smith      bob@yahoo.com     3.8
4       Eve Davis      eve@gmail.com     4.1
5       bob smith      bob@yahoo.com     3.8
6    FRANK WILSON  frank@outlook.com     4.3
9      Ivy Clark       ivy@gmail.com     3.7
10     Jake Brown     jake@gmail.com     4.0
```

```python
# Example 3: how="all" — drop rows where EVERY cell is null
# Create a test row where all values are null
df_test = df.copy()
df_test.loc[11] = [None]*10
print("Rows with all nulls before:", df_test[df_test.isnull().all(axis=1)].shape[0])

df_test_clean = df_test.dropna(how="all")
print("After dropna(how='all')  :", df_test_clean.shape)
```

Output:

```
Rows with all nulls before: 1
After dropna(how='all')  : (11, 10)
```

> `how="all"` is safer — only drops truly empty rows. Use this when rows might have some data but are mostly empty.

```python
# Example 4: thresh — keep rows with at least N non-null values
# thresh=8 → keep rows that have at least 8 valid (non-null) values out of 10 cols
df_thresh = df.dropna(thresh=8)
print("After thresh=8:", df_thresh.shape)
```

Output:

```
After thresh=8: (10, 10)
```

> Row 3 (Dave, missing Name AND Email) has only 8 non-null values — kept (exactly meets threshold). Row where ALL critical data is missing gets dropped.

```python
# Example 5: Drop COLUMNS with any null value
df_no_null_cols = df.dropna(axis=1)
print("Columns remaining:", df_no_null_cols.columns.tolist())
```

Output:

```
Columns remaining: ['CustomerID', 'Salary', 'JoinDate', 'Department', 'Active']
```

> **Output Explanation:** Only 5 of 10 columns had zero nulls. Use `axis=1` cautiously — you might drop important columns.

`fillna()`

Definition

Fills missing values with a specified value or strategy instead of dropping rows.

Why Use It?

- When you can't afford to lose rows (small dataset)
- When you can intelligently estimate the missing value
- When missing means a specific value (e.g., 0 orders = not 'no data')

Syntax

```python
df["col"].fillna(value)              # fill with a constant
df["col"].fillna(df["col"].mean())   # fill with mean
df["col"].fillna(method="ffill")     # forward fill (use prev value)
df["col"].fillna(method="bfill")     # backward fill (use next value)
df.fillna({"col1": 0, "col2": "Unknown"})  # different fill per column
df["col"].fillna(method="ffill", limit=1)  # fill at most 1 consecutive NaN
```

Examples

```python
# Example 1: Fill numeric column with MEAN (most common strategy)
mean_age = df["Age"].mean()
print(f"Mean Age: {mean_age:.1f}")

df["Age"] = df["Age"].fillna(mean_age)
print(df[["Name", "Age"]].head(8))
```

Output:

```
Mean Age: 36.2

             Name   Age
0   Alice Johnson  28.0
1       bob smith  35.0
2     Carol White  36.2   ← was NaN, now filled with mean
3            None  42.0
4       Eve Davis  29.0
5       bob smith  35.0
6    FRANK WILSON  51.0
7       grace lee  36.2   ← was NaN, now filled with mean
```

```python
# Example 2: Fill numeric column with MEDIAN
# Median is preferred over mean when data has outliers
median_rating = df["Rating"].median()
df["Rating"] = df["Rating"].fillna(median_rating)
print(df[["Name", "Rating"]])
```

Output:

```
             Name  Rating
0   Alice Johnson     4.5
1       bob smith     3.8
2     Carol White     4.0   ← median was 4.0
3            None     3.5
4       Eve Davis     4.1
5       bob smith     3.8
6    FRANK WILSON     4.3
7       grace lee     4.0   ← median was 4.0
8      Hank Moore     4.6
9       Ivy Clark      3.7
10     Jake Brown     4.0
```

> **When to use Mean vs Median:**
>
> - **Mean** → when data is normally distributed (no extreme outliers)
> - **Median** → when data is skewed or has outliers (more robust)

```python
# Example 3: Fill string column with a CONSTANT / PLACEHOLDER
df["Name"] = df["Name"].fillna("Unknown")
df["City"] = df["City"].fillna("Unknown")
df["Email"] = df["Email"].fillna("No Email")
print(df[["CustomerID", "Name", "City", "Email"]].loc[[3, 9]])
```

Output:

```
   CustomerID      Name     City       Email
3         104   Unknown  Unknown    No Email
9         109  Ivy Clark  Unknown  ivy@gmail.com
```

```python
# Example 4: Fill DIFFERENT columns with DIFFERENT values at once
df_filled = df.fillna({
    "Age"    : df["Age"].mean(),
    "Rating" : df["Rating"].median(),
    "Name"   : "Unknown",
    "City"   : "Unknown",
    "Email"  : "No Email"
})
print("Remaining nulls:", df_filled.isnull().sum().sum())
```

Output:

```
Remaining nulls: 0
```

```python
# Example 5: Forward Fill (ffill) — use last known value
# Great for time-series data (carry forward previous reading)
sensor = pd.DataFrame({
    "Time"  : ["09:00","10:00","11:00","12:00","13:00"],
    "Temp"  : [36.5, None, None, 37.1, None]
})
print("Before ffill:")
print(sensor)

sensor["Temp"] = sensor["Temp"].fillna(method="ffill")
print("\\nAfter ffill:")
print(sensor)
```

Output:

```
Before ffill:
    Time  Temp
0  09:00  36.5
1  10:00   NaN
2  11:00   NaN
3  12:00  37.1
4  13:00   NaN

After ffill:
    Time  Temp
0  09:00  36.5
1  10:00  36.5   ← carried from 09:00
2  11:00  36.5   ← carried from 09:00
3  12:00  37.1
4  13:00  37.1   ← carried from 12:00
```

```python
# Example 6: Backward Fill (bfill) — use next known value
sensor2 = sensor.copy()
sensor2["Temp"] = pd.Series([36.5, None, None, 37.1, None])
sensor2["Temp"] = sensor2["Temp"].fillna(method="bfill")
print(sensor2)
```

Output:

```
    Time  Temp
0  09:00  36.5
1  10:00  37.1   ← filled backward from 12:00
2  11:00  37.1   ← filled backward from 12:00
3  12:00  37.1
4  13:00   NaN   ← no next value, still NaN
```

```python
# Example 7: Fill with GROUP-WISE mean (advanced, industry-level)
# Fill missing Age with the mean age of the SAME Department
df["Age"] = df.groupby("Department")["Age"].transform(
    lambda x: x.fillna(x.mean())
)
print(df[["Name", "Department", "Age"]])
```

Output:

```
             Name  Department   Age
0   Alice Johnson Engineering  28.0
1       bob smith   Marketing  35.0
2     Carol White Engineering  36.0   ← avg of Engineering dept
```

```
...
7       grace lee          HR  38.5   ← avg of HR dept
```

> **Output Explanation:** This is the **industry-best practice** for filling numeric nulls — fill with the group mean, not the global mean. A 25-year-old in Intern role shouldn't get the same mean age as a CEO.

💡 Missing Values — Decision Guide

```
Is the missing % very high? (>40%)
  └─ YES → Consider dropping the COLUMN entirely
  └─ NO  → Is it a numeric column?
             └─ YES → Fill with mean (normal) or median (skewed)
                      or group-wise mean (best practice)
             └─ NO  → Fill with mode (most frequent) or "Unknown"

Do you have time-series data?
  └─ YES → Use ffill or bfill

Is the row missing too many fields to be useful?
  └─ YES → Use dropna(thresh=N) or dropna(subset=[critical_cols])
```

⚠️ Common Mistakes — Missing Values

Mistake	Explanation
Using `dropna()` blindly	Can lose huge % of dataset — always check `isnull().sum()` first
Filling all NaNs with 0	0 means "zero value" — not the same as "missing"
Using mean when data is skewed	Use median for skewed data — mean gets pulled by outliers
Forgetting `inplace=True`	`fillna()` returns a new DF — original stays unchanged without `inplace=True`
`df["col"] = df["col"].fillna(x)`	This is the correct pattern — reassign the column

---

2. Removing Duplicates

📌 Definition

Duplicate rows are rows where all (or specific) column values are identical.
They appear due to: data entry errors, system bugs, merging datasets multiple times, web scraping the same page twice.

Syntax

```python
df.duplicated()                          # True/False Series — True = duplicate row
df.duplicated(subset=["col1","col2"])  # check duplicates in specific columns only
df.duplicated(keep="first")              # mark all AFTER first occurrence as duplicate
df.duplicated(keep="last")               # mark all BEFORE last occurrence as duplicate
df.duplicated(keep=False)                # mark ALL copies (including first) as duplicat
e

df.drop_duplicates()                     # remove duplicate rows
df.drop_duplicates(subset=["col"])     # remove based on specific columns
df.drop_duplicates(keep="last")          # keep the last occurrence instead of first
df.drop_duplicates(inplace=True)         # modify original DataFrame
```

Examples

```python
# Example 1: Detect ALL duplicate rows
print("Duplicate rows:")
print(df[df.duplicated()])
```

Output:

```
   CustomerID      Name   Age   City          Email  Salary    JoinDate  \\
5         102  bob smith  35.0  delhi   bob@yahoo.com   60000  2019-07-22

  Department  Rating Active
5  Marketing     3.8     No
```

> Row 5 is identical to Row 1 — same CustomerID, Name, and all other fields.

```python
# Example 2: Count duplicate rows
print("Total duplicate rows:", df.duplicated().sum())
```

Output:

```
Total duplicate rows: 1
```

```python
# Example 3: Remove duplicates — keep first occurrence
df_no_dup = df.drop_duplicates()
print("Shape before:", df.shape)
print("Shape after :", df_no_dup.shape)
```

Output:

```
Shape before: (11, 10)
Shape after : (10, 10)
```

```python
# Example 4: Check duplicates on SPECIFIC columns
# Real-world: One customer should have only one entry (based on CustomerID)
dup_customer_id = df[df.duplicated(subset=["CustomerID"])]
print("Duplicate CustomerIDs:")
print(dup_customer_id[["CustomerID", "Name", "Email"]])
```

Output:

```
   CustomerID       Name          Email
5         102  bob smith  bob@yahoo.com
```

```python
# Example 5: Keep LAST occurrence (useful if later record is more updated)
df_keep_last = df.drop_duplicates(subset=["CustomerID"], keep="last")
print("Shape:", df_keep_last.shape)
```

```python
# Example 6: keep=False — see ALL copies, including the "first" occurrence
all_dups = df[df.duplicated(keep=False)]
print("All copies of any duplicate:")
print(all_dups[["CustomerID","Name"]])
```

Output:

```
   CustomerID       Name
1         102  bob smith
5         102  bob smith
```

```python
# Example 7: Drop in-place
df.drop_duplicates(inplace=True)
```

```
df.reset_index(drop=True, inplace=True)   # reset index after dropping
print("Final shape after removing duplicates:", df.shape)
```

Output:

```
Final shape after removing duplicates: (10, 10)
```

> `reset_index(drop=True)` — after dropping rows, index jumps (0,1,3,4...). This resets it to 0,1,2,3... cleanly. `drop=True` prevents the old index from becoming a column.

---

3. Renaming Columns

📌 Definition

Renaming columns gives them **clear, consistent, and code-friendly** names — critical for writing clean Pandas code.

Syntax

```
df.rename(columns={"old_name": "new_name"})           # rename specific columns
df.rename(columns={"col1": "new1", "col2": "new2"})   # rename multiple
df.columns = ["col1", "col2", ...]                     # rename ALL columns at once
df.columns = df.columns.str.lower()                    # lowercase all
df.columns = df.columns.str.upper()                    # uppercase all
df.columns = df.columns.str.replace(" ", "_")          # spaces → underscores
df.columns = df.columns.str.strip()                    # remove leading/trailing space
s
```

Examples

```
# Example 1: Rename specific columns
df_renamed = df.rename(columns={
    "CustomerID" : "customer_id",
    "Name"       : "full_name",
    "JoinDate"   : "join_date"
})
print(df_renamed.columns.tolist())
```

Output:

```
['customer_id', 'full_name', 'Age', 'City', 'Email', 'Salary', 'join_date', 'Department', 'Rating', 'Active']
```

```
# Example 2: Make ALL column names lowercase + replace spaces with underscores
# This is the most common real-world cleaning step for column names

df.columns = df.columns.str.lower().str.replace(" ", "_").str.strip()
print(df.columns.tolist())
```

Output:

```
['customerid', 'name', 'age', 'city', 'email', 'salary', 'joindate', 'department', 'rating', 'active']
```

```
# Example 3: Rename all columns at once using a list
# (only use when you know the exact order)
df.columns = ["customer_id", "name", "age", "city", "email",
```

```
                "salary", "join_date", "department", "rating", "active"]
print(df.columns.tolist())
```

Output:

```
['customer_id', 'name', 'age', 'city', 'email', 'salary', 'join_date', 'department', 'rating', 'active']
```

```
# Example 4: Add prefix or suffix to columns (useful after merges)
df_prefixed = df.add_prefix("emp_")
print(df_prefixed.columns.tolist())
```

Output:

```
['emp_customer_id', 'emp_name', 'emp_age', 'emp_city', 'emp_email',
 'emp_salary', 'emp_join_date', 'emp_department', 'emp_rating', 'emp_active']
```

```
# Example 5: Real-world — clean messy column names from a CSV
# Typical messy columns from Excel
messy_cols = pd.Index(["  Customer ID ", "Full Name", "Salary (USD)", "Date Of Joinin
g"])
clean_cols  = messy_cols.str.strip().str.lower().str.replace(" ", "_").str.replace(r"
[()\\/]", "", regex=True)
print(clean_cols.tolist())
```

Output:

```
['customer_id', 'full_name', 'salary_usd', 'date_of_joining']
```

💡 Important Notes — Renaming

- After renaming to lowercase + underscores, you can use **dot notation** (`df.city`) safely
- `rename()` does NOT modify in place — use `inplace=True` or reassign (`df = df.rename(...)`)
- Column naming convention in Data Science: **snake_case** (all lowercase, underscores)

---

4. Changing Data Types

📌 Definition

Data types control how data is stored and what operations are possible.
After loading from CSV, columns are often the **wrong type** — especially dates (loaded as strings), numbers (loaded as strings), and categories.

Common Type Conversions

From	To	Method
string `"90000"`	integer	`df["col"].astype(int)`
string `"90000.5"`	float	`df["col"].astype(float)`
`"Yes"/"No"`	boolean	`df["col"].map({"Yes": True, "No": False})`
string date `"2024-01-01"`	datetime	`pd.to_datetime(df["col"])`
integer/string	category	`df["col"].astype("category")`
float → integer	integer	`df["col"].astype(int)` (no NaNs allowed)

Syntax

```python
df["col"].astype(dtype)            # basic type conversion
pd.to_datetime(df["col"])          # string → datetime
pd.to_numeric(df["col"])           # string → number (raises error on failure)
pd.to_numeric(df["col"], errors="coerce")  # invalid → NaN instead of error
df["col"].astype("category")       # convert to categorical (saves memory)
```

Examples

```python
# Our dataset after renaming
print("Before type conversion:")
print(df.dtypes)
```

Output:

```
customer_id       int64
name             object
age             float64
city             object
email            object
salary           object    ← Should be int/float!
join_date        object    ← Should be datetime!
department       object
rating          float64
active           object    ← Should be bool!
dtype: object
```

```python
# Example 1: Convert Salary from string to integer
print("Salary dtype before:", df["salary"].dtype)

df["salary"] = df["salary"].astype(int)

print("Salary dtype after :", df["salary"].dtype)
print(df["salary"].head(3))
```

Output:

```
Salary dtype before: object
Salary dtype after : int64
0    90000
1    60000
2    95000
Name: salary, dtype: int64
```

```python
# Example 2: Convert join_date from string to datetime
print("join_date dtype before:", df["join_date"].dtype)

df["join_date"] = pd.to_datetime(df["join_date"])

print("join_date dtype after :", df["join_date"].dtype)
print(df["join_date"].head(3))
```

Output:

```
join_date dtype before: object
join_date dtype after : datetime64[ns]
0   2020-03-15
1   2019-07-22
2   2021-01-10
Name: join_date, dtype: datetime64[ns]
```

> Now you can extract `.dt.year`, `.dt.month`, `.dt.day_name()` etc.

```python
# Example 3: Convert Active ("Yes"/"No") to boolean
df["active"] = df["active"].map({"Yes": True, "No": False})
print(df[["name", "active"]].head(5))
print("active dtype:", df["active"].dtype)
```

Output:

```
          name  active
0  Alice Johnson    True
1      bob smith   False
2    Carol White    True
3        Unknown    True
4      Eve Davis   False
active dtype: bool
```

```python
# Example 4: Convert to category dtype (saves memory for low-cardinality cols)
print("dept memory before:", df["department"].memory_usage(deep=True), "bytes")
df["department"] = df["department"].astype("category")
print("dept memory after :", df["department"].memory_usage(deep=True), "bytes")
print("dept dtype:", df["department"].dtype)
print("Categories:", df["department"].cat.categories.tolist())
```

Output:

```
dept memory before: 806 bytes
dept memory after : 525 bytes
dept dtype: category
Categories: ['Engineering', 'Finance', 'HR', 'Marketing']
```

> `category` **dtype** saves significant memory when a column has few unique values (< 50% of total rows). Critical for large datasets.

```python
# Example 5: Safe conversion — errors="coerce" (invalid → NaN, not crash)
# Real-world: salary column has a typo "N/A" mixed in
messy_salary = pd.Series(["90000", "60000", "N/A", "75000", "undefined"])

# This would crash:
# messy_salary.astype(int)  ← ValueError!

# Safe conversion — invalid values become NaN
clean_salary = pd.to_numeric(messy_salary, errors="coerce")
print(clean_salary)
```

Output:

```
0    90000.0
1    60000.0
2        NaN    ← "N/A" → NaN (no crash!)
3    75000.0
4        NaN    ← "undefined" → NaN (no crash!)
dtype: float64
```

> `errors="coerce"` is a lifesaver in production pipelines — prevents crashes from unexpected non-numeric values.

```python
# Example 6: Extract datetime components
df["join_year"]  = df["join_date"].dt.year
df["join_month"] = df["join_date"].dt.month
```

```python
df["join_day"]   = df["join_date"].dt.day_name()

print(df[["name", "join_date", "join_year", "join_month", "join_day"]].head(4))
```

Output:

```
          name  join_date  join_year  join_month   join_day
0  Alice Johnson 2020-03-15       2020           3     Sunday
1      bob smith 2019-07-22       2019           7     Monday
2    Carol White 2021-01-10       2021           1     Sunday
3        Unknown 2018-11-05       2018          11     Monday
```

⚠️ Common Mistakes — Data Types

Mistake	Explanation
`df["salary"].astype(int)` when there are NaN values	`int` cannot hold NaN — use `float` first or fill NaN before converting
Not parsing dates at load time	`pd.read_csv("file.csv", parse_dates=["date_col"])` is cleaner
Ignoring category dtype	Huge memory waste on large datasets with low-cardinality columns
Using `astype()` on messy data without `errors="coerce"`	Will crash if any value can't be converted — always use `pd.to_numeric(..., errors="coerce")` for safety

---

5. String Operations

📌 Definition

The `str` **accessor** in Pandas lets you apply string methods to every element in a Series — vectorized, no loops needed.

> In real-world data, string columns are almost always messy: mixed case, extra spaces, inconsistent formats. The `str` accessor is your primary tool to fix all of it.

Syntax

```python
df["col"].str.method()
```

Complete String Methods Reference

Method	What It Does	Example
`.str.lower()`	Convert to lowercase	`"Alice"` → `"alice"`
`.str.upper()`	Convert to uppercase	`"alice"` → `"ALICE"`
`.str.title()`	Title Case	`"alice johnson"` → `"Alice Johnson"`
`.str.strip()`	Remove leading/trailing spaces	`" Alice "` → `"Alice"`
`.str.lstrip()`	Remove left spaces	`" Alice"` → `"Alice"`
`.str.rstrip()`	Remove right spaces	`"Alice "` → `"Alice"`
`.str.replace(a, b)`	Replace substring	`"bob smith"` → `"Bob Smith"`
`.str.contains("x")`	Check if contains substring	Returns True/False
`.str.startswith("x")`	Starts with substring	Returns True/False
`.str.endswith("x")`	Ends with substring	Returns True/False
`.str.split("x")`	Split string	Returns list
`.str.len()`	Length of string	`"Alice"` → 5
`.str.count("x")`	Count occurrences in string	
`.str.extract(r"pattern")`	Extract with regex	

Examples

```python
# Example 1: Fix inconsistent City casing (delhi, mumbai, DELHI → proper case)
print("Before:", df["city"].unique())

df["city"] = df["city"].str.strip().str.title()

print("After :", df["city"].unique())
```

Output:

```
Before: ['Mumbai' 'delhi' 'Bangalore' 'mumbai' 'Chennai' 'hyderabad' 'Delhi']
After : ['Mumbai' 'Delhi' 'Bangalore' 'Chennai' 'Hyderabad']
```

```python
# Example 2: Fix Name column — remove spaces, apply title case
print("Before:", df["name"].tolist())

df["name"] = df["name"].str.strip().str.title()

print("After :", df["name"].tolist())
```

Output:

```
Before: ['Alice Johnson', 'bob smith', 'Carol White', 'Unknown', 'Eve Davis',
         'FRANK WILSON', 'grace lee', 'Hank Moore', 'Ivy Clark', 'Jake Brown']
After : ['Alice Johnson', 'Bob Smith', 'Carol White', 'Unknown', 'Eve Davis',
         'Frank Wilson', 'Grace Lee', 'Hank Moore', 'Ivy Clark', 'Jake Brown']
```

```python
# Example 3: Check if email contains "gmail"
df["is_gmail"] = df["email"].str.contains("gmail", na=False)
# na=False → treat NaN as False instead of NaN
print(df[["name", "email", "is_gmail"]])
```

Output:

```
          name              email  is_gmail
0  Alice Johnson     alice@gmail.com      True
1      Bob Smith       bob@yahoo.com     False
2    Carol White     carol@gmail.com      True
3      Unknown             No Email     False
4      Eve Davis       eve@gmail.com      True
5    Frank Wilson   frank@outlook.com     False
6      Grace Lee     grace@gmail.com      True
7     Hank Moore             No Email    False
8      Ivy Clark       ivy@gmail.com      True
9     Jake Brown      jake@gmail.com      True
```

```python
# Example 4: Extract domain from email
df["email_domain"] = df["email"].str.split("@").str[-1]
print(df[["name", "email", "email_domain"]].head(5))
```

Output:

```
           name             email email_domain
0  Alice Johnson   alice@gmail.com    gmail.com
1      Bob Smith     bob@yahoo.com    yahoo.com
2    Carol White   carol@gmail.com    gmail.com
3        Unknown          No Email     No Email
4      Eve Davis     eve@gmail.com    gmail.com
```

```python
# Example 5: String length of names
df["name_length"] = df["name"].str.len()
print(df[["name", "name_length"]].head(5))
```

Output:

```
           name  name_length
0  Alice Johnson           13
1      Bob Smith            9
2    Carol White           11
3        Unknown            7
4      Eve Davis            9
```

```python
# Example 6: Split full name → first name and last name
df[["first_name","last_name"]] = df["name"].str.split(" ", n=1, expand=True)
print(df[["name","first_name","last_name"]].head(5))
```

Output:

```
           name first_name last_name
0  Alice Johnson      Alice   Johnson
1      Bob Smith        Bob     Smith
2    Carol White      Carol     White
3        Unknown    Unknown      None
4      Eve Davis        Eve     Davis
```

> `n=1` → split only at the FIRST space (important for multi-word names like "Alice Marie Johnson").
> `expand=True` → returns a DataFrame instead of a Series of lists.

```python
# Example 7: Replace part of a string
# Fix email domain typo: ".comm" → ".com"
typo_emails = pd.Series(["alice@gmail.comm", "bob@yahoo.comm", "carol@gmail.com"])
fixed_emails = typo_emails.str.replace(".comm", ".com", regex=False)
print(fixed_emails.tolist())
```

Output:

```
['alice@gmail.com', 'bob@yahoo.com', 'carol@gmail.com']
```

```python
# Example 8: Extract year from a date-like string using regex
df_temp = pd.DataFrame({"date_str": ["Order-2024-01", "Order-2023-12", "Order-2022-0
6"]})
df_temp["year"] = df_temp["date_str"].str.extract(r"(\\d{4})")
print(df_temp)
```

Output:

```
       date_str  year
0  Order-2024-01  2024
1  Order-2023-12  2023
2  Order-2022-06  2022
```

```python
# Example 9: Check starts/ends with
df["is_engineering"] = df["department"].astype(str).str.startswith("Eng")
print(df[["name", "department", "is_engineering"]].head(5))
```

Output:

```
           name   department  is_engineering
0  Alice Johnson  Engineering            True
1      Bob Smith    Marketing           False
2    Carol White  Engineering            True
3        Unknown           HR           False
4      Eve Davis    Marketing           False
```

💡 Important Notes — String Operations

- All `str` methods work on **NaN gracefully** — pass `na=False` to prevent NaN propagation when using `contains()`, `startswith()`, `endswith()`
- Always **chain** operations: `.str.strip().str.title()` in one go
- `str.replace()` by default uses **regex** — use `regex=False` for simple literal replacements
- `str.split(expand=True)` is the cleanest way to split into multiple columns

---

6. Replacing Values

📌 Definition

`replace()` replaces **specific values** across the entire DataFrame or within a Series — more targeted than `fillna()`.

Syntax

```
df["col"].replace(old_value, new_value)          # replace one value
df["col"].replace([v1, v2], new_value)           # replace multiple with same
df["col"].replace({"old1": "new1", "old2": "new2"})  # replace using dict
df.replace({"col": {"old": "new"}})              # replace in specific column
df.replace(to_replace=regex_pattern, value=new, regex=True)  # regex replace
```

Examples

```
# Example 1: Replace a single value in a column
df["active"] = df["active"].replace(True, 1).replace(False, 0)
print(df["active"].value_counts())
```

Output:

```
1    7
0    3
Name: active, dtype: int64
```

```
# Example 2: Replace multiple values at once using a dict
# Standardize city names with known typos
df["city"] = df["city"].replace({
    "Hyd"       : "Hyderabad",
    "BLR"       : "Bangalore",
    "Mum"       : "Mumbai",
    "Blore"     : "Bangalore"
})
print(df["city"].unique())
```

```
# Example 3: Replace a value across the ENTIRE DataFrame
# e.g., replace all occurrences of "No Email" → None (for proper NaN handling)
df = df.replace("No Email", np.nan)
```

```python
df = df.replace("Unknown", np.nan)
print(df[["name","city","email"]].head(5))
```

Output:

```
           name   city            email
0  Alice Johnson  Mumbai   alice@gmail.com
1      Bob Smith   Delhi     bob@yahoo.com
2    Carol White  Bangalore  carol@gmail.com
3           None  Mumbai            None
4      Eve Davis  Chennai    eve@gmail.com
```

```python
# Example 4: Replace — department abbreviations with full names
dept_data = pd.Series(["Eng", "Mkt", "HR", "Eng", "Fin", "Mkt"])
full_names = dept_data.replace({
    "Eng": "Engineering",
    "Mkt": "Marketing",
    "Fin": "Finance"
})
print(full_names.tolist())
```

Output:

```
['Engineering', 'Marketing', 'HR', 'Engineering', 'Finance', 'Marketing']
```

```python
# Example 5: Replace using regex (advanced)
# Remove currency symbols from price strings
prices = pd.Series(["₹75,000", "₹1,200", "₹22,000", "₹8,500"])
clean_prices = prices.str.replace(r"[₹,]", "", regex=True).astype(int)
print(clean_prices.tolist())
```

Output:

```
[75000, 1200, 22000, 8500]
```

```python
# Example 6: Replace outlier values with NaN
# Salaries below 1000 or above 10,00,000 are likely data entry errors
df["salary"] = df["salary"].replace(
    df["salary"][(df["salary"] < 1000) | (df["salary"] > 1000000)].tolist(),
    np.nan
)
```

```python
# Example 7: Replace vs fillna — key difference

s = pd.Series([0, np.nan, 0, 1, np.nan])

# fillna() replaces NaN only
print(s.fillna(99))          # replaces NaN → 99, keeps 0 as 0

# replace() replaces a specific value (can also replace NaN)
print(s.replace(0, -1))      # replaces 0 → -1, keeps NaN as NaN
print(s.replace(np.nan, 99)) # replaces NaN → 99 (same as fillna here)
```

Output:

```
# fillna(99)
0     0.0
1    99.0
2     0.0
3     1.0
4    99.0

# replace(0, -1)
0    -1.0
1     NaN
2    -1.0
3     1.0
4     NaN

# replace(np.nan, 99)
0     0.0
1    99.0
2     0.0
3     1.0
4    99.0
```

> **fillna vs replace:**
>
> - `fillna()` → ONLY fills NaN values
> - `replace()` → can replace ANY specific value (including NaN, 0, "N/A", etc.)

💡 Important Notes — Replacing Values

- `replace()` is more **flexible** than `fillna()` — use it for non-NaN substitutions
- Use regex=True for replacing patterns (remove symbols, fix formatting)
- For large-scale standardization (e.g., 50 city name variations), always use a **dict mapping**

Mini Summary & Cheat Sheet

🔑 Phase 3 Key Takeaways

- ✅ **Identify** missing values: `isnull().sum()`, `isnull().mean()*100`
- ✅ **Drop** rows with nulls: `dropna(subset=["critical_col"])`
- ✅ **Fill** nulls: mean/median for numbers, "Unknown" for strings, ffill for time series
- ✅ **Remove duplicates**: `drop_duplicates()` + `reset_index(drop=True)`
- ✅ **Rename** columns: `df.columns.str.lower().str.replace(" ","_")`
- ✅ **Fix types**: `astype()`, `pd.to_datetime()`, `pd.to_numeric(errors="coerce")`
- ✅ **Clean strings**: chain `.str.strip().str.title()` for case/space issues
- ✅ **Replace values**: use dict mapping for bulk standardization

---

🔖 Quick Reference Cheat Sheet

Task	Code
Null count per column	`df.isnull().sum()`
Null percentage	`df.isnull().mean() * 100`
Rows with any null	`df[df.isnull().any(axis=1)]`
Drop rows with any null	`df.dropna()`
Drop rows — specific cols	`df.dropna(subset=["col1","col2"])`
Drop cols with any null	`df.dropna(axis=1)`
Keep rows with N+ non-nulls	`df.dropna(thresh=N)`
Fill with constant	`df["col"].fillna(0)`
Fill with mean	`df["col"].fillna(df["col"].mean())`
Fill with median	`df["col"].fillna(df["col"].median())`
Fill with mode	`df["col"].fillna(df["col"].mode()[0])`

Task	Code
Forward fill	`df["col"].fillna(method="ffill")`
Backward fill	`df["col"].fillna(method="bfill")`
Fill multiple cols	`df.fillna({"col1": 0, "col2": "X"})`
Group-wise fill	`df.groupby("g")["col"].transform(lambda x: x.fillna(x.mean()))`
Find duplicates	`df.duplicated()`
Count duplicates	`df.duplicated().sum()`
Remove duplicates	`df.drop_duplicates()`
Dedup on specific cols	`df.drop_duplicates(subset=["col"])`
Reset index	`df.reset_index(drop=True)`
Rename columns	`df.rename(columns={"old":"new"})`
All cols lowercase	`df.columns.str.lower()`
Cols → snake_case	`df.columns.str.lower().str.replace(" ","_")`
String to int	`df["col"].astype(int)`
String to float	`df["col"].astype(float)`
String to datetime	`pd.to_datetime(df["col"])`
Safe numeric convert	`pd.to_numeric(df["col"], errors="coerce")`
To category dtype	`df["col"].astype("category")`
Map values	`df["col"].map({"Yes": True, "No": False})`
Strip whitespace	`df["col"].str.strip()`
Title case	`df["col"].str.title()`
Lowercase	`df["col"].str.lower()`
Replace substring	`df["col"].str.replace("old","new")`
Contains check	`df["col"].str.contains("x", na=False)`
Split into columns	`df["col"].str.split(" ", expand=True)`
Extract with regex	`df["col"].str.extract(r"(\\d+)")`
Replace value	`df["col"].replace("old","new")`
Replace with dict	`df["col"].replace({"a":"A","b":"B"})`
Replace across DF	`df.replace("old", "new")`

🧹 Industry-Level Data Cleaning Workflow

```
1. Load data          → pd.read_csv("file.csv", parse_dates=["date_col"])
2. Inspect            → df.info()  +  df.describe()  +  df.isnull().sum()
3. Fix column names   → df.columns.str.lower().str.replace(" ","_")
4. Remove duplicates  → df.drop_duplicates()  +  df.reset_index(drop=True)
5. Fix data types     → astype(), pd.to_datetime(), pd.to_numeric(errors="coerce")
6. Handle nulls       → dropna() for rows/cols beyond threshold, fillna() for the rest
7. Clean strings      → .str.strip() + .str.title() for name/city columns
8. Replace/standardize → .replace() for abbreviations, typos, and encoding
9. Validate           → df.info() + df.isnull().sum().sum() == 0
10. Save clean data   → df.to_csv("clean_file.csv", index=False)
```

🐼 PHASE 4: DATA MANIPULATION — Complete Learning Notes

> **Goal:** Learn how to reshape, transform, and enrich your data — adding columns, dropping rows, sorting, applying custom functions, and analyzing value distributions. These are daily-use skills in every Data Science and Analytics role.

🗂️ The Master Dataset Used Throughout Phase 4

```python
import pandas as pd
import numpy as np

data = {
    "EmployeeID"  : [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],
    "Name"        : ["Alice Johnson", "Bob Smith", "Carol White", "Dave Brown",
                     "Eve Davis", "Frank Wilson", "Grace Lee", "Hank Moore",
                     "Ivy Clark", "Jake Brown"],
    "Department"  : ["Engineering", "Marketing", "Engineering", "HR",
                     "Marketing", "Engineering", "HR", "Finance",
                     "Marketing", "Finance"],
    "City"        : ["Mumbai", "Delhi", "Bangalore", "Mumbai", "Chennai",
                     "Delhi", "Hyderabad", "Mumbai", "Bangalore", "Chennai"],
    "Salary"      : [90000, 60000, 95000, 55000, 62000,
                     88000, 57000, 75000, 48000, 70000],
    "Experience"  : [5, 3, 7, 2, 4, 6, 3, 8, 1, 5],
    "Age"         : [28, 35, 32, 42, 29, 38, 31, 45, 26, 40],
    "Rating"      : [4.5, 3.8, 4.7, 4.0, 3.5, 4.8, 3.9, 4.6, 3.7, 4.0],
    "Remote"      : [True, False, True, False, True, False, True, False, True, False],
    "JoinYear"    : [2019, 2021, 2017, 2022, 2020, 2018, 2021, 2016, 2023, 2019]
}

df = pd.DataFrame(data)
print(df)
```

Output:

```
   EmployeeID           Name  Department       City  Salary  Experience  Age  Rating  Remote  JoinYear
0         101  Alice Johnson  Engineering     Mumbai   90000           5   28     4.5    True      2019
1         102      Bob Smith    Marketing      Delhi   60000           3   35     3.8   False      2021
2         103    Carol White  Engineering  Bangalore   95000           7   32     4.7    True      2017
3         104     Dave Brown           HR     Mumbai   55000           2   42     4.0   False      2022
4         105      Eve Davis    Marketing    Chennai   62000           4   29     3.5    True      2020
5         106   Frank Wilson  Engineering      Delhi   88000           6   38     4.8   False      2018
6         107      Grace Lee           HR  Hyderabad   57000           3   31     3.9    True      2021
7         108     Hank Moore      Finance     Mumbai   75000           8   45     4.6   False      2016
8         109      Ivy Clark    Marketing  Bangalore   48000           1   26     3.7    True      2023
9         110     Jake Brown      Finance    Chennai   70000           5   40     4.0   False      2019
```

---

1. Adding Columns

📌 Definition

Adding new columns to a DataFrame means creating **derived features** or **new attributes** from existing data. This is called **Feature Engineering** in Data Science and is one of the most important skills.

Why Use It?

- Create calculated fields (e.g., Total Salary, Tax Amount)
- Generate category labels (e.g., Senior/Junior based on experience)
- Extract information from existing columns (e.g., Year from Date)
- Prepare features for ML models

Method 1: Direct Assignment — `df["new_col"] = value`

Example 1: Add a constant column

```python
# Add a company column with the same value for all rows
df["Company"] = "TechCorp Ltd"
print(df[["Name", "Company"]].head(3))
```

Output:

```
            Name       Company
0  Alice Johnson  TechCorp Ltd
1      Bob Smith  TechCorp Ltd
2    Carol White  TechCorp Ltd
```

> **Output Explanation:** When you assign a scalar (single value) to a new column, Pandas **broadcasts** it to every row automatically.

Example 2: Add a calculated column

```python
# Annual Bonus = 10% of Salary
df["AnnualBonus"] = df["Salary"] * 0.10
print(df[["Name", "Salary", "AnnualBonus"]].head(5))
```

Output:

```
            Name  Salary  AnnualBonus
0  Alice Johnson   90000       9000.0
1      Bob Smith   60000       6000.0
2    Carol White   95000       9500.0
3     Dave Brown   55000       5500.0
4      Eve Davis   62000       6200.0
```

Example 3: Column from multiple existing columns

```python
# Total Compensation = Salary + AnnualBonus
df["TotalCompensation"] = df["Salary"] + df["AnnualBonus"]

# Salary per Year of Experience
df["SalaryPerYear"] = (df["Salary"] / df["Experience"]).round(2)

print(df[["Name", "Salary", "Experience", "TotalCompensation", "SalaryPerYear"]].head
(5))
```

Output:

```
            Name  Salary  Experience  TotalCompensation  SalaryPerYear
0  Alice Johnson   90000           5            99000.0       18000.00
1      Bob Smith   60000           3            66000.0       20000.00
2    Carol White   95000           7           104500.0       13571.43
3     Dave Brown   55000           2            60500.0       27500.00
4      Eve Davis   62000           4            68200.0       15500.00
```

Example 4: Boolean column from condition

```python
# Is the employee Senior? (Experience > 4 years)
df["IsSenior"] = df["Experience"] > 4
print(df[["Name", "Experience", "IsSenior"]].head(6))
```

Output:

```
            Name  Experience  IsSenior
0  Alice Johnson           5      True
1      Bob Smith           3     False
2    Carol White           7      True
3     Dave Brown           2     False
4      Eve Davis           4     False
5   Frank Wilson           6      True
```

Method 2: `insert()` — Add Column at a Specific Position

```python
# Syntax: df.insert(position, column_name, value)
# Insert "Level" column at position 2 (third column)
df.insert(2, "Level", "Employee")
print(df.columns.tolist()[:5])
```

Output:

```
['EmployeeID', 'Name', 'Level', 'Department', 'City']
```

> **Output Explanation:** `insert()` is useful when column order matters — e.g., keeping ID and Name at the start followed by a Level indicator, before other attributes.

Method 3: `assign()` — Functional Style (Returns New DataFrame)

```python
# assign() returns a NEW DataFrame — does NOT modify original
# Great for method chaining

df_new = df.assign(
    TaxAmount    = df["Salary"] * 0.30,
    NetSalary    = df["Salary"] * 0.70,
    SeniorFlag   = df["Experience"] >= 5
)

print(df_new[["Name", "Salary", "TaxAmount", "NetSalary", "SeniorFlag"]].head(4))
```

Output:

```
            Name  Salary  TaxAmount  NetSalary  SeniorFlag
0  Alice Johnson   90000    27000.0    63000.0        True
1      Bob Smith   60000    18000.0    42000.0       False
2    Carol White   95000    28500.0    66500.0        True
3     Dave Brown   55000    16500.0    38500.0       False
```

> **Output Explanation:** `assign()` is preferred in **method chaining** pipelines (clean, functional style). It does not mutate the original — always returns a new DataFrame.

Method 4: Create Category Column with `np.where()` and `pd.cut()`

`np.where()` — Binary condition (like IF-ELSE)

```python
# np.where(condition, value_if_true, value_if_false)
df["PerformanceTier"] = np.where(df["Rating"] >= 4.5, "Top Performer", "Standard")
```

```python
print(df[["Name", "Rating", "PerformanceTier"]].head(6))
```

Output:

```
            Name  Rating PerformanceTier
0  Alice Johnson     4.5   Top Performer
1      Bob Smith     3.8        Standard
2    Carol White     4.7   Top Performer
3     Dave Brown     4.0        Standard
4      Eve Davis     3.5        Standard
5   Frank Wilson     4.8   Top Performer
```

`pd.cut()` — Bin continuous values into categories

```python
# Bin salary into bands
df["SalaryBand"] = pd.cut(
    df["Salary"],
    bins   = [0, 60000, 75000, 95000, float("inf")],
    labels = ["Entry", "Mid", "Senior", "Lead"]
)
print(df[["Name", "Salary", "SalaryBand"]])
```

Output:

```
            Name  Salary SalaryBand
0  Alice Johnson   90000     Senior
1      Bob Smith   60000      Entry
2    Carol White   95000       Lead
3     Dave Brown   55000      Entry
4      Eve Davis   62000        Mid
5   Frank Wilson   88000     Senior
6      Grace Lee   57000      Entry
7     Hank Moore   75000        Mid
8      Ivy Clark   48000      Entry
9     Jake Brown   70000        Mid
```

> **Output Explanation:** `pd.cut()` divides a continuous variable into discrete intervals. Essential for creating age groups, salary bands, score tiers, etc. in real-world analysis.

`pd.qcut()` — Bin into equal-frequency quantile groups

```python
# Divide salary into 4 equal-sized groups (quartiles)
df["SalaryQuartile"] = pd.qcut(df["Salary"], q=4, labels=["Q1","Q2","Q3","Q4"])
print(df[["Name", "Salary", "SalaryQuartile"]])
```

Output:

```
            Name  Salary SalaryQuartile
0  Alice Johnson   90000             Q4
1      Bob Smith   60000             Q2
2    Carol White   95000             Q4
3     Dave Brown   55000             Q1
4      Eve Davis   62000             Q2
5   Frank Wilson   88000             Q3
6      Grace Lee   57000             Q1
7     Hank Moore   75000             Q3
8      Ivy Clark   48000             Q1
9     Jake Brown   70000             Q2
```

> **`pd.cut` vs `pd.qcut`:**
>
> - `pd.cut` → fixed range bins (equal WIDTH) — you define the boundaries
> - `pd.qcut` → quantile bins (equal FREQUENCY) — each group has same number of rows

---

💡 Important Notes — Adding Columns

- Direct assignment `df["new"] = ...` modifies the original DataFrame in-place
- `assign()` returns a **new** DataFrame — use it in pipelines/chains
- `pd.cut()` → you control bin edges; `pd.qcut()` → Pandas auto-balances group sizes
- Always calculate derived columns from cleaned data (after Phase 3 cleaning)

⚠️ Common Mistakes

Mistake	Explanation
`df["new"] = df["a"] / df["b"]` when `df["b"]` has zeros	Division by zero → `inf` values. Use `df["b"].replace(0, np.nan)` first
`df.assign(col = other_df["col"])`	Index mismatch can cause all NaN — ensure indices align
`pd.cut()` with wrong bin edges	If value falls outside bins → NaN. Make sure `bins` cover min and max

---

2. Dropping Columns & Rows

📌 Definition

Removing unwanted columns (features) or rows (records) to keep the dataset lean and relevant.

Syntax

```
# Drop COLUMNS
df.drop(columns=["col1", "col2"])          # drop by name
df.drop(columns=["col1"], inplace=True)    # modify original
df.drop("col1", axis=1)                    # axis=1 means column

# Drop ROWS
df.drop(index=0)                           # drop row by index label
df.drop(index=[0, 2, 4])                   # drop multiple rows
df.drop(df[condition].index)               # drop rows matching condition
```

Examples

```
# Example 1: Drop a single column
df_clean = df.drop(columns=["Company"])
print(df_clean.columns.tolist())
```

Output:

```
['EmployeeID', 'Name', 'Level', 'Department', 'City', 'Salary', 'Experience',
 'Age', 'Rating', 'Remote', 'JoinYear', 'AnnualBonus', 'TotalCompensation',
 'SalaryPerYear', 'IsSenior', 'PerformanceTier', 'SalaryBand', 'SalaryQuartile']
```

```
# Example 2: Drop multiple columns at once
# Clean up helper columns before analysis
cols_to_drop = ["Level", "Company", "AnnualBonus", "TotalCompensation",
                "SalaryPerYear", "IsSenior", "PerformanceTier",
                "SalaryBand", "SalaryQuartile"]

# Only drop cols that exist (safe pattern)
cols_to_drop = [c for c in cols_to_drop if c in df.columns]
df.drop(columns=cols_to_drop, inplace=True)
```

```python
print("Columns after cleanup:", df.columns.tolist())
```

Output:

```
Columns after cleanup: ['EmployeeID', 'Name', 'Department', 'City', 'Salary',
                        'Experience', 'Age', 'Rating', 'Remote', 'JoinYear']
```

```python
# Example 3: Drop rows by index label
df_no_first = df.drop(index=0)       # drop row 0 (Alice)
print(df_no_first[["Name"]].head(3))
```

Output:

```
          Name
1    Bob Smith
2  Carol White
3   Dave Brown
```

```python
# Example 4: Drop multiple rows by index
df_trimmed = df.drop(index=[0, 2, 7])
print(df_trimmed[["Name"]].values.flatten())
```

Output:

```
['Bob Smith' 'Dave Brown' 'Eve Davis' 'Frank Wilson' 'Grace Lee'
 'Ivy Clark' 'Jake Brown']
```

```python
# Example 5: Drop rows based on a CONDITION (most common real-world use)
# Remove employees with salary below 55000 (below company threshold)
df_filtered = df.drop(df[df["Salary"] < 55000].index)
print(df_filtered[["Name", "Salary"]])
```

Output:

```
           Name  Salary
0  Alice Johnson   90000
1      Bob Smith   60000
2    Carol White   95000
3     Dave Brown   55000
4      Eve Davis   62000
5   Frank Wilson   88000
7     Hank Moore   75000
9     Jake Brown   70000
```

> **Output Explanation:** Ivy Clark (48000) and Grace Lee (57000 — wait, she's above 55000). Only Ivy Clark at 48000 is dropped.

```python
# Example 6: Reset index after dropping rows
df_reset = df.drop(index=[0, 3]).reset_index(drop=True)
print(df_reset["Name"].tolist())
```

Output:

```
['Bob Smith', 'Carol White', 'Eve Davis', 'Frank Wilson',
 'Grace Lee', 'Hank Moore', 'Ivy Clark', 'Jake Brown']
```

> **Always** `reset_index(drop=True)` **after dropping rows** — prevents gaps in the index (0,2,4...) which can cause confusion in later operations.

💡 Important Notes — Dropping

- `drop()` **does not modify** the original unless `inplace=True` or you reassign
- Prefer `drop(df[condition].index)` over boolean filtering + reassignment for clarity
- After dropping rows, always call `reset_index(drop=True)` to clean up the index

---

3. Sorting Data

📌 Definition

Sorting arranges the DataFrame by one or more columns in ascending or descending order. Essential for ranking, top-N analysis, and ordered reporting.

Syntax

```
df.sort_values("col")                              # ascending (default)
df.sort_values("col", ascending=False)             # descending
df.sort_values(["col1","col2"])                    # sort by multiple columns
df.sort_values(["col1","col2"], ascending=[True, False])  # mixed order
df.sort_values("col", na_position="first")         # NaN at the top
df.sort_values("col", inplace=True)                # modify original
df.sort_index()                                    # sort by row index label
```

Examples

```
# Example 1: Sort by Salary — ascending
df_sorted = df.sort_values("Salary")
print(df_sorted[["Name", "Salary"]])
```

Output:

```
           Name  Salary
8     Ivy Clark   48000
6     Grace Lee   57000
3    Dave Brown   55000
1     Bob Smith   60000
4     Eve Davis   62000
9    Jake Brown   70000
7    Hank Moore   75000
5   Frank Wilson   88000
0  Alice Johnson   90000
2    Carol White   95000
```

```
# Example 2: Sort by Salary — descending (Top earners first)
df_top = df.sort_values("Salary", ascending=False)
print(df_top[["Name", "Department", "Salary"]].head(5))
```

Output:

```
           Name   Department  Salary
2    Carol White  Engineering   95000
0  Alice Johnson  Engineering   90000
5   Frank Wilson  Engineering   88000
7     Hank Moore      Finance   75000
9     Jake Brown      Finance   70000
```

> **Real-world use:** Top-5 highest paid employees — used in executive dashboards, HR reports, and compensation analysis.

```
# Example 3: Sort by MULTIPLE columns
# Primary: Department (A→Z), Secondary: Salary (high→low)
df_multi = df.sort_values(
    by        = ["Department", "Salary"],
    ascending = [True, False]
)
print(df_multi[["Name", "Department", "Salary"]])
```

Output:

```
           Name   Department  Salary
2    Carol White  Engineering   95000
0  Alice Johnson  Engineering   90000
5   Frank Wilson  Engineering   88000
7     Hank Moore      Finance   75000
9     Jake Brown      Finance   70000
3     Dave Brown           HR   55000
6      Grace Lee           HR   57000
1      Bob Smith    Marketing   60000
4      Eve Davis    Marketing   62000
8      Ivy Clark    Marketing   48000
```

> **Output Explanation:** Departments are alphabetically sorted. Within each department, salary is ranked highest to lowest. This is exactly how you'd produce a departmental salary leaderboard.

```
# Example 4: Sort by Rating — Top performers at the top
df_rated = df.sort_values("Rating", ascending=False).reset_index(drop=True)
df_rated["Rank"] = df_rated.index + 1   # Create rank column
print(df_rated[["Rank", "Name", "Rating"]].head(6))
```

Output:

```
   Rank           Name  Rating
1     1   Frank Wilson     4.8
2     2    Carol White     4.7
3     3     Hank Moore     4.6
4     4  Alice Johnson     4.5
5     5     Jake Brown     4.0
6     6     Dave Brown     4.0
```

```
# Example 5: Sort by multiple conditions — 3 levels
df_complex = df.sort_values(
    by        = ["Department", "Experience", "Salary"],
    ascending = [True, False, False]
)
print(df_complex[["Name", "Department", "Experience", "Salary"]])
```

Output:

```
           Name  Department  Experience  Salary
2    Carol White Engineering           7   95000
5   Frank Wilson Engineering           6   88000
0  Alice Johnson Engineering           5   90000
7     Hank Moore     Finance           8   75000
9     Jake Brown     Finance           5   70000
3     Dave Brown          HR           2   55000
6      Grace Lee          HR           3   57000
4      Eve Davis   Marketing           4   62000
1      Bob Smith   Marketing           3   60000
8      Ivy Clark   Marketing           1   48000
```

```python
# Example 6: Sort index (useful after resetting or merging)
df_shuffled = df.sample(frac=1, random_state=42)  # shuffle
print("Shuffled index:", df_shuffled.index.tolist())

df_reordered = df_shuffled.sort_index()
print("Sorted index  :", df_reordered.index.tolist())
```

Output:

```
Shuffled index: [0, 2, 9, 6, 4, 3, 8, 7, 1, 5]
Sorted index  : [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```python
# Example 7: Get Top-N rows after sorting (nlargest / nsmallest)
# Faster than sort_values + head() for large datasets
top3_salary = df.nlargest(3, "Salary")
bot3_salary  = df.nsmallest(3, "Salary")

print("Top 3 Earners:")
print(top3_salary[["Name","Salary"]])

print("\\nBottom 3 Earners:")
print(bot3_salary[["Name","Salary"]])
```

Output:

```
Top 3 Earners:
            Name  Salary
2    Carol White   95000
0  Alice Johnson   90000
5   Frank Wilson   88000

Bottom 3 Earners:
        Name  Salary
8  Ivy Clark   48000
3  Dave Brown  55000
6  Grace Lee   57000
```

> `nlargest()` / `nsmallest()` are more efficient than sorting + slicing for top-N queries on large DataFrames.

💡 Important Notes — Sorting

- `sort_values()` does NOT modify original unless `inplace=True`
- Use `reset_index(drop=True)` after sorting if you need a clean 0-based index
- `nlargest(n, col)` / `nsmallest(n, col)` → more efficient than `sort_values().head(n)` for large data
- `na_position="first"` or `"last"` controls where NaN values end up after sorting

⚠️ Common Mistakes

Mistake	Explanation
Not resetting index after sort	Index stays in original order — confusing for iloc access later
`sort_values("col1","col2")` as positional args	Wrong — use `by=["col1","col2"]` keyword argument
Expecting `sort_values` to persist changes	Use `inplace=True` or `df = df.sort_values(...)`

4. Applying Functions

📌 Definition

`apply()`, `map()`, and `lambda` let you apply **custom logic** to each element, row, or column — far more flexible than built-in methods.

> **Analogy:** If Pandas methods are pre-built tools, `apply()` is the custom tool you build yourself for any task.

`apply()`

Definition

`apply()` applies a function along an **axis** of a DataFrame:

- `axis=0` → apply function to each **column** (default)
- `axis=1` → apply function to each **row**

On a **Series**, `apply()` applies the function to **each element**.

Syntax

```
# On a Series (element-wise)
df["col"].apply(function)

# On a DataFrame along columns (axis=0)
df.apply(function, axis=0)

# On a DataFrame along rows (axis=1)
df.apply(function, axis=1)
```

Examples

```
# Example 1: apply() on a Series — classify salary
def salary_category(salary):
    if salary >= 85000:
        return "High"
    elif salary >= 65000:
        return "Medium"
    else:
        return "Low"

df["SalaryCategory"] = df["Salary"].apply(salary_category)
print(df[["Name", "Salary", "SalaryCategory"]])
```

Output:

```
            Name  Salary SalaryCategory
0  Alice Johnson   90000           High
1      Bob Smith   60000            Low
2    Carol White   95000           High
3     Dave Brown   55000            Low
4      Eve Davis   62000            Low
5   Frank Wilson   88000           High
6      Grace Lee   57000            Low
7     Hank Moore   75000         Medium
8      Ivy Clark   48000            Low
9     Jake Brown   70000         Medium
```

```
# Example 2: apply() on Series — string transformation
def format_name(name):
    parts = name.split()
    return f"{parts[-1]}, {parts[0]}"   # "Last, First" format
```

```python
df["FormattedName"] = df["Name"].apply(format_name)
print(df[["Name", "FormattedName"]].head(5))
```

Output:

```
            Name   FormattedName
0  Alice Johnson  Johnson, Alice
1      Bob Smith      Smith, Bob
2    Carol White    White, Carol
3     Dave Brown    Brown, Dave
4      Eve Davis     Davis, Eve
```

```python
# Example 3: apply() on DataFrame row-wise (axis=1) — access multiple columns
def performance_score(row):
    """Calculate composite score from rating and experience"""
    return round(row["Rating"] * 0.7 + (row["Experience"] / 10) * 0.3, 3)

df["PerfScore"] = df.apply(performance_score, axis=1)
print(df[["Name", "Rating", "Experience", "PerfScore"]].head(6))
```

Output:

```
            Name  Rating  Experience  PerfScore
0  Alice Johnson     4.5           5      3.300
1      Bob Smith     3.8           3      2.750
2    Carol White     4.7           7      3.500
3     Dave Brown     4.0           2      2.860
4      Eve Davis     3.5           4      2.570
5   Frank Wilson     4.8           6      3.540
```

> **Output Explanation:** Each row's Rating (70% weight) + Experience (30% weight) are combined into a single composite performance score. This is a common feature engineering step in HR analytics.

```python
# Example 4: apply() on DataFrame column-wise (axis=0) — statistics per column
numeric_cols = df[["Salary", "Experience", "Age", "Rating"]]
col_stats = numeric_cols.apply(lambda col: col.max() - col.min())   # range
print("Value Range per Column:")
print(col_stats)
```

Output:

```
Salary        47000.0
Experience        7.0
Age              19.0
Rating            1.3
dtype: float64
```

```python
# Example 5: apply() with arguments using args parameter
def scale_salary(salary, bonus_pct, tax_pct):
    after_bonus = salary * (1 + bonus_pct)
    after_tax   = after_bonus * (1 - tax_pct)
    return round(after_tax, 2)

df["NetSalary"] = df["Salary"].apply(scale_salary, args=(0.10, 0.30))
print(df[["Name", "Salary", "NetSalary"]].head(4))
```

Output:

```
            Name  Salary  NetSalary
0  Alice Johnson   90000   69300.00
```

```
1      Bob Smith   60000   46200.00
2    Carol White   95000   73150.00
3     Dave Brown   55000   42350.00
```

```python
# Example 6: apply() with result_type (for row-wise multi-output)
def split_name(row):
    parts = row["Name"].split()
    return pd.Series({"FirstName": parts[0], "LastName": parts[-1]})

df_names = df.apply(split_name, axis=1)
print(df_names.head(4))
```

Output:

```
  FirstName LastName
0     Alice  Johnson
1       Bob    Smith
2     Carol    White
3      Dave    Brown
```

`map()`

Definition

`map()` applies a function or dictionary mapping to **each element** of a **Series**.

> Key difference: `apply()` works on Series AND DataFrames, `map()` works ONLY on Series (element-wise)

Syntax

```python
series.map(function)              # apply function to each element
series.map(dictionary)            # replace values using a dict lookup
series.map(lambda x: ...)         # with lambda
```

Examples

```python
# Example 1: map() with a dictionary — encode categories
dept_code = {
    "Engineering" : "ENG",
    "Marketing"   : "MKT",
    "HR"          : "HR",
    "Finance"     : "FIN"
}
df["DeptCode"] = df["Department"].map(dept_code)
print(df[["Name", "Department", "DeptCode"]].head(5))
```

Output:

```
            Name   Department DeptCode
0  Alice Johnson  Engineering      ENG
1      Bob Smith    Marketing      MKT
2    Carol White  Engineering      ENG
3     Dave Brown           HR       HR
4      Eve Davis    Marketing      MKT
```

> **Output Explanation:** `map()` with a dict is a lookup-table replacement — like VLOOKUP in Excel. Values not in the dict become `NaN`.

```python
# Example 2: map() with a function — clean and transform
df["City_Upper"] = df["City"].map(str.upper)
print(df[["City", "City_Upper"]].head(4))
```

Output:

```
       City  City_Upper
0    Mumbai      MUMBAI
1     Delhi       DELHI
2  Bangalore   BANGALORE
3    Mumbai      MUMBAI
```

```python
# Example 3: map() with lambda — convert boolean to descriptive string
df["RemoteLabel"] = df["Remote"].map(lambda x: "Work From Home" if x else "Office")
print(df[["Name", "Remote", "RemoteLabel"]].head(5))
```

Output:

```
           Name  Remote      RemoteLabel
0  Alice Johnson    True   Work From Home
1      Bob Smith   False          Office
2    Carol White    True   Work From Home
3     Dave Brown   False          Office
4      Eve Davis    True   Work From Home
```

```python
# Example 4: map() with dict — handle missing mappings (returns NaN)
# What happens when a key is NOT in the dict?
partial_map = {"Engineering": "Tech", "Marketing": "Sales"}
df["PartialDept"] = df["Department"].map(partial_map)
print(df[["Department", "PartialDept"]])
```

Output:

```
    Department PartialDept
0  Engineering        Tech
1    Marketing       Sales
2  Engineering        Tech
3           HR         NaN   ← not in dict → NaN
4    Marketing       Sales
5  Engineering        Tech
6           HR         NaN   ← not in dict → NaN
7      Finance         NaN   ← not in dict → NaN
8    Marketing       Sales
9      Finance         NaN   ← not in dict → NaN
```

> **Fix:** Use `.map(dict).fillna(df["Department"])` to keep original value where mapping is missing.

```python
# Example 5: map() vs replace() — key difference
# map() → applies function or dict, NaN where key missing
# replace() → only replaces matching values, keeps others unchanged

s = pd.Series(["Engineering", "Marketing", "HR", "Finance"])

print(s.map({"Engineering": "Tech", "Marketing": "Sales"}))
# Engineering → Tech, Marketing → Sales, HR → NaN, Finance → NaN

print(s.replace({"Engineering": "Tech", "Marketing": "Sales"}))
# Engineering → Tech, Marketing → Sales, HR → HR (unchanged!), Finance → Finance (unch
anged!)
```

Output:

```
# map():
0    Tech
1    Sales
2    NaN
3    NaN

# replace():
0    Tech
1    Sales
2    HR
3    Finance
```

`lambda` Functions with `apply()` & `map()`

Definition

A **lambda** is an anonymous (no-name) one-liner function. It is used directly inline with `apply()` and `map()` — no need to define a separate `def` function.

Syntax

```
lambda arguments: expression

# equivalent to:
def func(arguments):
    return expression
```

Examples

```
# Example 1: Simple lambda – add tax
df["SalaryWithTax"] = df["Salary"].apply(lambda x: x * 1.18)
print(df[["Name","Salary","SalaryWithTax"]].head(3))
```

Output:

```
            Name  Salary  SalaryWithTax
0  Alice Johnson   90000       106200.0
1      Bob Smith   60000        70800.0
2    Carol White   95000       112100.0
```

```
# Example 2: lambda on strings
df["Initials"] = df["Name"].apply(lambda x: "".join([w[0] for w in x.split()]))
print(df[["Name","Initials"]].head(5))
```

Output:

```
            Name Initials
0  Alice Johnson       AJ
1      Bob Smith       BS
2    Carol White       CW
3     Dave Brown       DB
4      Eve Davis       ED
```

```
# Example 3: lambda with condition (inline if-else)
df["ExpLevel"] = df["Experience"].apply(lambda x: "Senior" if x >= 5 else ("Mid" if x
>= 3 else "Junior"))
print(df[["Name", "Experience", "ExpLevel"]].head(7))
```

Output:

```
           Name  Experience ExpLevel
0  Alice Johnson           5   Senior
1      Bob Smith           3      Mid
2    Carol White           7   Senior
3     Dave Brown           2   Junior
4      Eve Davis           4      Mid
5   Frank Wilson           6   Senior
6      Grace Lee           3      Mid
```

```python
# Example 4: lambda with axis=1 (row-wise, accessing multiple columns)
df["SalaryFlag"] = df.apply(
    lambda row: "Over-Compensated" if row["Salary"] > 80000 and row["Experience"] < 4
                else ("Under-Compensated" if row["Salary"] < 60000 and row["Experienc
e"] > 4
                else "Fair"),
    axis=1
)
print(df[["Name","Salary","Experience","SalaryFlag"]])
```

Output:

```
           Name  Salary  Experience         SalaryFlag
0  Alice Johnson   90000           5               Fair
1      Bob Smith   60000           3               Fair
2    Carol White   95000           7               Fair
3     Dave Brown   55000           2               Fair
4      Eve Davis   62000           4               Fair
5   Frank Wilson   88000           6               Fair
6      Grace Lee   57000           3               Fair
7     Hank Moore   75000           8  Under-Compensated
8      Ivy Clark   48000           1               Fair
9     Jake Brown   70000           5               Fair
```

```python
# Example 5: lambda for text formatting
df["EmailID"] = df["Name"].apply(
    lambda name: name.lower().replace(" ", ".") + "@techcorp.com"
)
print(df[["Name","EmailID"]].head(4))
```

Output:

```
           Name                 EmailID
0  Alice Johnson  alice.johnson@techcorp.com
1      Bob Smith      bob.smith@techcorp.com
2    Carol White    carol.white@techcorp.com
3     Dave Brown     dave.brown@techcorp.com
```

`apply()` vs `map()` vs `lambda` — Summary

Feature	`apply()`	`map()`	`lambda`
Works on	Series & DataFrame	Series only	Used inside apply/map
DataFrame axis=0	Column-wise	❌	❌
DataFrame axis=1	Row-wise	❌	With apply(axis=1)
Best for	Custom logic on rows/cols	Element substitution/lookup	Simple one-liner transformations
Performance	Slower (Python loop)	Faster than apply	Same as apply/map
Dict mapping	❌	✅ Yes	❌

💡 Performance Tip

```
Speed hierarchy (fastest → slowest):
  Vectorized operations   →   np.where()    →   map()   →   apply()   →   Python loops

Always try vectorized first:
  GOOD:  df["col"] * 1.18
  OK:    df["col"].map(lambda x: x * 1.18)
  SLOW:  df["col"].apply(lambda x: x * 1.18)  ← avoid for simple math
```

> Use `apply()` only when built-in vectorized methods can't do the job.

⚠️ Common Mistakes — apply / map

Mistake	Explanation
`df.apply(func)` expecting row-wise	Default axis=0 is column-wise — use `axis=1` for rows
`df["col"].map(dict)` losing unmatched values	map returns NaN for missing keys — use `fillna` or `replace` instead
Using `apply()` for simple math	`df["col"] * 2` is 10–100x faster than `df["col"].apply(lambda x: x*2)`
Multi-line lambda	Not possible — use `def` for multi-step logic

---

5. Value Counts

📌 Definition

`value_counts()` counts the **frequency of each unique value** in a Series/column — like a histogram in text form.

Why Use It?

- Understand the distribution of categorical data
- Spot imbalanced classes before ML training
- Identify the most/least common categories
- Quick sanity check after data loading

Syntax

```python
df["col"].value_counts()                    # counts in descending order
df["col"].value_counts(ascending=True)      # lowest count first
df["col"].value_counts(normalize=True)      # proportions (0 to 1)
df["col"].value_counts(normalize=True)*100  # percentage
df["col"].value_counts(dropna=False)        # include NaN in count
df["col"].value_counts().head(5)            # top 5 most common
df["col"].value_counts().reset_index()      # convert to DataFrame
```

Examples

```python
# Example 1: Basic — count employees per department
print(df["Department"].value_counts())
```

Output:

```
Engineering    3
Marketing      3
Finance        2
HR             2
Name: Department, dtype: int64
```

```python
# Example 2: Normalize — percentage distribution
pct = df["Department"].value_counts(normalize=True) * 100
print(pct.round(1))
```

Output:

```
Engineering    30.0
Marketing      30.0
Finance        20.0
HR             20.0
Name: Department, dtype: float64
```

> **Output Explanation:** Engineering and Marketing each make up 30% of the workforce. If this were training data for an ML model, we'd note it's somewhat balanced (no severe imbalance).

```python
# Example 3: Include NaN in count (dropna=False)
df_test = df.copy()
df_test.loc[0, "Department"] = None
print(df_test["Department"].value_counts(dropna=False))
```

Output:

```
Marketing      3
Engineering    2
Finance        2
HR             2
NaN            1
Name: Department, dtype: int64
```

```python
# Example 4: Convert value_counts to a clean DataFrame (for reporting)
vc_df = df["Department"].value_counts().reset_index()
vc_df.columns = ["Department", "Count"]
vc_df["Percentage"] = (vc_df["Count"] / len(df) * 100).round(1)
print(vc_df)
```

Output:

```
    Department  Count  Percentage
0  Engineering      3        30.0
1    Marketing      3        30.0
2      Finance      2        20.0
3           HR      2        20.0
```

```python
# Example 5: City distribution — top 3 cities
print("Top 3 Cities:")
print(df["City"].value_counts().head(3))
```

Output:

```
Top 3 Cities:
Mumbai       3
Delhi        2
Bangalore    2
```

```python
# Example 6: Count by two columns using groupby + value_counts
# Employees per (Department, City) combination
print(df.groupby(["Department","City"]).size().sort_values(ascending=False).head(8))
```

Output:

```
Department   City
Engineering  Delhi        1
             Mumbai       1
```

```
             Bangalore    1
Finance      Mumbai       1
             Chennai      1
HR           Mumbai       1
             Hyderabad    1
Marketing    Delhi        1
```

```python
# Example 7: Rating distribution (numeric binned with value_counts)
# Create bins first, then count
df["RatingBin"] = pd.cut(df["Rating"], bins=[3.0, 3.5, 4.0, 4.5, 5.0],
                         labels=["3.0-3.5","3.5-4.0","4.0-4.5","4.5-5.0"])
print(df["RatingBin"].value_counts().sort_index())
```

Output:

```
RatingBin
3.0-3.5    1
3.5-4.0    3
4.0-4.5    3
4.5-5.0    3
Name: RatingBin, dtype: int64
```

💡 Important Notes — Value Counts

- `value_counts()` returns a **Series** with values as index — use `.reset_index()` for a clean DataFrame
- By default, NaN is excluded — use `dropna=False` to include it
- For DataFrame-wide frequency tables, use `groupby().size()` instead
- In ML: always check `value_counts()` on your target column — class imbalance affects model training

---

6. Unique Values

📌 Definition

Methods to inspect the **distinct values** in a column — how many there are, what they are, and whether data is consistent.

Syntax & Methods

```python
df["col"].unique()              # array of unique values
df["col"].nunique()             # count of unique values
df["col"].is_unique             # True if ALL values are unique
df.nunique()                    # unique count for EVERY column
df["col"].duplicated()          # True for repeated values in Series
```

Examples

```python
# Example 1: Get all unique departments
print("Unique departments:", df["Department"].unique())
print("Count             :", df["Department"].nunique())
```

Output:

```
Unique departments: ['Engineering' 'Marketing' 'HR' 'Finance']
Count             : 4
```

```python
# Example 2: Check if EmployeeID is truly unique (as it should be)
print("All IDs unique?", df["EmployeeID"].is_unique)          # True
```

```python
print("All Names unique?", df["Name"].is_unique)              # True
print("All Departments unique?", df["Department"].is_unique)  # False
```

Output:

```
All IDs unique?         True
All Names unique?       True
All Departments unique? False
```

```python
# Example 3: nunique on the entire DataFrame — see cardinality of each column
print(df.nunique())
```

Output:

```
EmployeeID      10
Name            10
Department       4
City             5
Salary          10
Experience       7
Age             10
Rating           8
Remote           2
JoinYear         8
SalaryCategory   3
...
dtype: int64
```

> **Output Explanation:** `Department` has only 4 unique values → good candidate for `category` dtype. `Remote` has only 2 → boolean. This is the **cardinality check** you do before feature encoding in ML.

```python
# Example 4: Find unique combinations of two columns
unique_combos = df[["Department","City"]].drop_duplicates()
print(unique_combos.sort_values("Department"))
```

Output:

```
    Department       City
0  Engineering     Mumbai
2  Engineering  Bangalore
5  Engineering      Delhi
7      Finance     Mumbai
9      Finance    Chennai
3           HR     Mumbai
6           HR  Hyderabad
1    Marketing      Delhi
4    Marketing    Chennai
8    Marketing  Bangalore
```

```python
# Example 5: Find values that appear more than once (recurring)
dup_cities = df["City"].value_counts()
recurring  = dup_cities[dup_cities > 1]
print("Cities with multiple employees:")
print(recurring)
```

Output:

```
Cities with multiple employees:
Mumbai       3
Delhi        2
Bangalore    2
Chennai      2
dtype: int64
```

```python
# Example 6: Unique JoinYears — when did employees join?
years = sorted(df["JoinYear"].unique())
print("Joining years:", years)
print("Span:", max(years) - min(years), "years")
```

Output:

```
Joining years: [2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023]
Span: 7 years
```

💡 Important Notes — Unique Values

- `unique()` → returns **NumPy array** — use `.tolist()` for a Python list
- `nunique()` on a DataFrame is your **cardinality check** before ML preprocessing
- `is_unique` → returns True/False — quick validation for primary key columns (IDs)
- Combine `value_counts()` + `nunique()` for a full distribution picture

Mini Summary & Cheat Sheet

🔑 Phase 4 Key Takeaways

- ✅ **Add columns**: direct `df["new"] = ...`, `assign()`, `np.where()`, `pd.cut()`, `pd.qcut()`
- ✅ **Drop**: `drop(columns=[...])` for cols, `drop(df[cond].index)` for rows
- ✅ **Sort**: `sort_values(by=["col1","col2"], ascending=[True,False])` for multi-sort
- ✅ **apply()**: custom function on Series (element) or DataFrame (row/column)
- ✅ **map()**: element-wise substitution using function or dict lookup (Series only)
- ✅ **lambda**: inline one-liner functions used with apply/map
- ✅ **value_counts()**: frequency distribution with normalize=True for percentages
- ✅ **unique() / nunique()**: inspect distinct values and cardinality

---

🔖 Quick Reference Cheat Sheet

Task	Code
Add constant column	`df["col"] = "value"`
Add calculated column	`df["col"] = df["a"] * df["b"]`
Add IF-ELSE column	`np.where(condition, "yes_val", "no_val")`
Add category bins (fixed)	`pd.cut(df["col"], bins=[...], labels=[...])`
Add quantile bins	`pd.qcut(df["col"], q=4, labels=[...])`
Insert at position	`df.insert(pos, "name", values)`
Assign (no mutation)	`df.assign(new_col=df["a"] + df["b"])`
Drop columns	`df.drop(columns=["col1","col2"])`
Drop rows by index	`df.drop(index=[0,1,2])`
Drop rows by condition	`df.drop(df[df["col"] < x].index)`
Reset index	`df.reset_index(drop=True)`
Sort ascending	`df.sort_values("col")`
Sort descending	`df.sort_values("col", ascending=False)`
Sort multi-column	`df.sort_values(["a","b"], ascending=[True,False])`
Top N rows	`df.nlargest(n, "col")`
Bottom N rows	`df.nsmallest(n, "col")`
Sort by index	`df.sort_index()`

Task	Code
Apply custom function	`df["col"].apply(my_func)`
Apply row-wise	`df.apply(my_func, axis=1)`
Apply with args	`df["col"].apply(func, args=(a, b))`
Map with dict	`df["col"].map({"old":"new"})`
Map with lambda	`df["col"].map(lambda x: x*2)`
Lambda one-liner	`df["col"].apply(lambda x: "A" if x > 5 else "B")`
Lambda row-wise	`df.apply(lambda r: r["a"] + r["b"], axis=1)`
Value counts	`df["col"].value_counts()`
Value counts %	`df["col"].value_counts(normalize=True) * 100`
Include NaN in count	`df["col"].value_counts(dropna=False)`
Counts as DataFrame	`df["col"].value_counts().reset_index()`
Unique values	`df["col"].unique()`
Count unique	`df["col"].nunique()`
Is primary key?	`df["col"].is_unique`
All col cardinality	`df.nunique()`
Unique combinations	`df[["a","b"]].drop_duplicates()`

🐼 PHASE 5: GROUPING & AGGREGATION — Complete Learning Notes

> **Goal:** Master `groupby()` — one of the most powerful and commonly used features in Pandas. Used in virtually every data analysis project for summarizing, segmenting, and transforming data by groups.

🗂️ The Master Dataset Used Throughout Phase 5

```
import pandas as pd
import numpy as np

data = {
    "EmployeeID" : [101,102,103,104,105,106,107,108,109,110,111,112,113,114,115],
    "Name"       : ["Alice","Bob","Carol","Dave","Eve","Frank","Grace","Hank",
                    "Ivy","Jake","Karen","Leo","Mia","Nate","Olivia"],
    "Department" : ["Engineering","Marketing","Engineering","HR","Marketing",
                    "Engineering","HR","Finance","Marketing","Finance",
                    "Engineering","Marketing","HR","Finance","Engineering"],
    "City"       : ["Mumbai","Delhi","Bangalore","Mumbai","Chennai",
                    "Delhi","Hyderabad","Mumbai","Bangalore","Chennai",
                    "Mumbai","Delhi","Hyderabad","Bangalore","Chennai"],
    "Gender"     : ["F","M","F","M","F","M","F","M","F","M","F","M","F","M","F"],
    "Salary"     : [90000,60000,95000,55000,62000,88000,57000,75000,
                    48000,70000,92000,65000,58000,72000,85000],
    "Experience" : [5,3,7,2,4,6,3,8,1,5,6,4,2,7,5],
    "Age"        : [28,35,32,42,29,38,31,45,26,40,30,36,33,44,27],
    "Rating"     : [4.5,3.8,4.7,4.0,3.5,4.8,3.9,4.6,3.7,4.0,4.4,3.6,4.1,4.3,4.6],
    "Sales"      : [120000,85000,135000,0,95000,115000,0,0,78000,0,
                    125000,92000,0,0,110000],
    "Bonus"      : [9000,6000,9500,5500,6200,8800,5700,7500,4800,7000,
                    9200,6500,5800,7200,8500]
}

df = pd.DataFrame(data)
```

```python
print(df.head(5))
print("\\nShape:", df.shape)
```

Output:

```
   EmployeeID   Name   Department    City Gender  Salary  Experience  Age  \\
0         101  Alice  Engineering  Mumbai      F   90000           5   28
1         102    Bob    Marketing   Delhi      M   60000           3   35
2         103  Carol  Engineering  Bangalore   F   95000           7   32
3         104   Dave           HR  Mumbai      M   55000           2   42
4         105    Eve    Marketing  Chennai     F   62000           4   29

   Rating   Sales  Bonus
0     4.5  120000   9000
1     3.8   85000   6000
2     4.7  135000   9500
3     4.0       0   5500
4     3.5   95000   6200

Shape: (15, 11)
```

1. groupby() Basics

📌 Definition

`groupby()` splits a DataFrame into **groups based on one or more columns**, then lets you **apply an operation** to each group independently.

The Split → Apply → Combine pattern:

```
Original DataFrame
       ↓
  SPLIT into groups (by Department, City, etc.)
       ↓
  APPLY a function to each group (sum, mean, count, custom)
       ↓
  COMBINE results into a new DataFrame
```

> **SQL Equivalent:** `SELECT Department, AVG(Salary) FROM employees GROUP BY Department` **Excel Equivalent:** PivotTable / SUMIF / AVERAGEIF

Syntax

```python
df.groupby("col")                          # group by one column
df.groupby(["col1", "col2"])               # group by multiple columns
df.groupby("col")["target_col"].agg()      # group + select col + aggregate
df.groupby("col", as_index=False)          # keep group col as regular column
df.groupby("col", sort=False)              # don't sort group keys
df.groupby("col").get_group("value")       # extract one specific group
```

How groupby() Works Internally

```python
# Step 1: Create a GroupBy object (nothing computed yet — lazy evaluation)
grouped = df.groupby("Department")
print(type(grouped))                    # <class 'pandas.core.groupby.DataFrameGroupB
y'>

# Step 2: See the groups
print(grouped.groups)                   # dict: {group_key: list of row indices}
```

Output:

```
<class 'pandas.core.groupby.DataFrameGroupBy'>
```

```
{
  'Engineering': [0, 2, 5, 10, 14],
  'Finance':     [7, 9, 13],
  'HR':          [3, 6, 12],
  'Marketing':   [1, 4, 8, 11]
}
```

> **Output Explanation:** groupby creates a **mapping** of group name → row indices. Nothing is calculated yet. The actual computation happens when you call `.sum()`, `.mean()`, etc.

Example 1: Basic groupby — Mean Salary per Department

```python
dept_salary = df.groupby("Department")["Salary"].mean()
print(dept_salary)
```

Output:

```
Department
Engineering    90000.0
Finance        72333.3
HR             56666.7
Marketing      63750.0
Name: Salary, dtype: float64
```

> **Output Explanation:**
>
> - Engineering avg salary: ₹90,000 — highest paying department
> - HR avg salary: ₹56,666 — lowest
> - The result is a Series with Department as the index

Example 2: Group by and count rows per group

```python
# Count employees in each department
dept_count = df.groupby("Department")["EmployeeID"].count()
print(dept_count)

# OR using size() — counts all rows including NaN
dept_size = df.groupby("Department").size()
print(dept_size)
```

Output:

```
Department
Engineering    5
Finance        3
HR             3
Marketing      4
Name: EmployeeID, dtype: int64

Department
Engineering    5
Finance        3
HR             3
Marketing      4
dtype: int64
```

> `count()` **vs** `size()`**:**
>
> - `size()` counts **all** rows including NaN
> - `count()` counts only **non-null** values in a specific column

Example 3: as_index=False — Keep Group Column as Regular Column

```python
# Default: group column becomes the index
dept_salary_idx = df.groupby("Department")["Salary"].mean()
print("With index:")
print(dept_salary_idx)

# as_index=False: group column stays as a normal column
dept_salary_col = df.groupby("Department", as_index=False)["Salary"].mean()
print("\\nWithout index (as_index=False):")
print(dept_salary_col)
```

Output:

```
With index:
Department
Engineering    90000.0
Finance        72333.3
HR             56666.7
Marketing      63750.0
Name: Salary, dtype: float64

Without index (as_index=False):
    Department    Salary
0  Engineering  90000.0
1      Finance  72333.3
2           HR  56666.7
3    Marketing  63750.0
```

> **Use** `as_index=False` when you want the result as a clean DataFrame (e.g., for further merging or export).

Example 4: Group by Multiple Columns

```python
# Average salary by Department AND City
dept_city_salary = df.groupby(["Department","City"])["Salary"].mean()
print(dept_city_salary)
```

Output:

```
Department   City
Engineering  Bangalore    95000.0
             Chennai      85000.0
             Delhi        88000.0
             Mumbai       91000.0
Finance      Bangalore    72000.0
             Chennai      70000.0
             Mumbai       75000.0
HR           Hyderabad    57500.0
             Mumbai       55000.0
Marketing    Bangalore    48000.0
             Chennai      62000.0
             Delhi        62500.0
Name: Salary, dtype: float64
```

Example 5: Iterate Over Groups

```python
# Loop through each group — useful for custom per-group processing
for dept_name, group_df in df.groupby("Department"):
    print(f"\\n{'='*40}")
    print(f"Department: {dept_name}  |  Employees: {len(group_df)}")
    print(group_df[["Name","Salary","Rating"]].to_string(index=False))
```

Output:

```

========================================
Department: Engineering  |  Employees: 5
      Name  Salary  Rating
```

```
     Alice   90000     4.5
     Carol   95000     4.7
     Frank   88000     4.8
     Karen   92000     4.4
    Olivia   85000     4.6

========================================
Department: Finance  |  Employees: 3
  Name  Salary  Rating
  Hank   75000     4.6
  Jake   70000     4.0
  Nate   72000     4.3
...
```

Example 6: Get a Specific Group

```python
# Extract just the Engineering group as a standalone DataFrame
eng_df = df.groupby("Department").get_group("Engineering")
print(eng_df[["Name","Salary","Experience"]])
```

Output:

```
     Name  Salary  Experience
0   Alice   90000           5
2   Carol   95000           7
5   Frank   88000           6
10  Karen   92000           6
14  Olivia  85000           5
```

---

2. Aggregation Functions

📌 Definition

After grouping, you apply **aggregation functions** to collapse each group into a single summary value.

Built-in Aggregation Functions

Function	Description
`.sum()`	Total / Sum of values
`.mean()`	Average
`.median()`	Median value
`.min()`	Minimum value
`.max()`	Maximum value
`.count()`	Count of non-null values
`.size()`	Total row count (including NaN)
`.std()`	Standard deviation
`.var()`	Variance
`.first()`	First value in the group
`.last()`	Last value in the group
`.nunique()`	Number of unique values
`.idxmin()`	Index of minimum value
`.idxmax()`	Index of maximum value

Examples

```python
# Example 1: Total Sales per Department
print(df.groupby("Department")["Sales"].sum())
```

Output:

```
Department
Engineering    685000
Finance             0
HR                  0
Marketing      350000
Name: Sales, dtype: int64
```

```python
# Example 2: Max and Min salary per Department
print("Max Salary per Dept:")
print(df.groupby("Department")["Salary"].max())

print("\\nMin Salary per Dept:")
print(df.groupby("Department")["Salary"].min())
```

Output:

```
Max Salary per Dept:
Department
Engineering    95000
Finance        75000
HR             58000
Marketing      65000
Name: Salary, dtype: int64

Min Salary per Dept:
Department
Engineering    85000
Finance        70000
HR             55000
Marketing      48000
Name: Salary, dtype: int64
```

```python
# Example 3: Standard Deviation — salary spread per department
print("Salary Std Dev per Dept:")
print(df.groupby("Department")["Salary"].std().round(2))
```

Output:

```
Department
Engineering    3807.89
Finance        2516.61
HR             1527.53
Marketing      7027.28
Name: Salary, dtype: float64
```

> **Output Explanation:** Marketing has the highest salary spread (₹7027) — meaning there's a big gap between the lowest (₹48000) and highest paid (₹65000) in that department. Engineering is more uniform.

```python
# Example 4: Count unique cities per department
print(df.groupby("Department")["City"].nunique())
```

Output:

```
Department
Engineering    4
Finance        3
HR             2
Marketing      3
Name: City, dtype: int64
```

```python
# Example 5: First and Last employee joining (using first row of group)
print(df.groupby("Department")["Name"].first())
```

```python
print(df.groupby("Department")["Name"].last())
```

Output:

```
Department
Engineering    Alice
Finance         Hank
HR              Dave
Marketing        Bob
Name: Name, dtype: object

Department
Engineering    Olivia
Finance          Nate
HR                Mia
Marketing         Leo
Name: Name, dtype: object
```

```python
# Example 6: Find the employee with highest salary in each department
print(df.groupby("Department")["Salary"].idxmax())
# Returns the INDEX of the max salary row in each group
```

Output:

```
Department
Engineering    2
Finance        7
HR            12
Marketing     11
Name: Salary, dtype: int64
```

```python
# Use it to retrieve the full row of those top earners
top_earners_idx = df.groupby("Department")["Salary"].idxmax()
print(df.loc[top_earners_idx, ["Name","Department","Salary"]])
```

Output:

```
     Name   Department  Salary
2   Carol  Engineering   95000
7    Hank      Finance   75000
12    Mia           HR   58000
11    Leo    Marketing   65000
```

> **Real-world use:** Finding the highest paid person in each department — used in org charts, HR dashboards, salary audits.

```python
# Example 7: Multiple columns, multiple stats
result = df.groupby("Department")[["Salary","Bonus","Rating"]].mean().round(2)
print(result)
```

Output:

```
              Salary    Bonus  Rating
Department
Engineering  90000.0  9000.0    4.60
Finance      72333.3  7233.3    4.30
HR           56666.7  5666.7    4.00
Marketing    63750.0  6375.0    3.65
```

---

3. Multiple Aggregations with `agg()`

📌 Definition

`agg()` (short for aggregate) lets you apply **multiple different aggregation functions** at once — to one or more columns simultaneously.

This is the most **powerful and flexible** way to summarize grouped data.

Syntax

```
# One column, multiple functions
df.groupby("col")["target"].agg(["min","max","mean","count"])

# Multiple columns, multiple functions
df.groupby("col")[["col1","col2"]].agg(["mean","std"])

# Different functions for different columns
df.groupby("col").agg({
    "col1": ["mean","max"],
    "col2": "sum",
    "col3": "count"
})

# Named aggregations (cleanest output)
df.groupby("col").agg(
    avg_salary   = ("Salary",  "mean"),
    max_salary   = ("Salary",  "max"),
    total_bonus  = ("Bonus",   "sum"),
    emp_count    = ("Name",    "count")
)
```

Examples

```
# Example 1: One column — multiple stats
result = df.groupby("Department")["Salary"].agg(["min","max","mean","std","count"])
result.columns = ["Min","Max","Mean","StdDev","Count"]
result = result.round(2)
print(result)
```

Output:

```
              Min    Max      Mean    StdDev  Count
Department
Engineering  85000  95000  90000.00   3807.89      5
Finance      70000  75000  72333.33   2516.61      3
HR           55000  58000  56666.67   1527.53      3
Marketing    48000  65000  63750.00   7027.28      4
```

```
# Example 2: Different functions for different columns
result = df.groupby("Department").agg({
    "Salary"     : ["mean", "max", "min"],
    "Experience" : ["mean", "max"],
    "Rating"     : ["mean", "count"],
    "Sales"      : "sum"
}).round(2)
print(result)
```

Output:

```
            Salary                 Experience       Rating       Sales
              mean    max    min       mean  max    mean count     sum
Department
Engineering  90000  95000  85000        5.8    7    4.60      5  685000
```

```
Finance      72333  75000  70000       6.7    8    4.30      3       0
HR           56667  58000  55000       2.3    3    4.00      3       0
Marketing    63750  65000  48000       3.0    4    3.65      4  350000
```

```python
# Example 3: Named aggregations (BEST PRACTICE — cleanest output)
result = df.groupby("Department").agg(
    avg_salary    = ("Salary",     "mean"),
    max_salary    = ("Salary",     "max"),
    min_salary    = ("Salary",     "min"),
    salary_range  = ("Salary",     lambda x: x.max() - x.min()),
    total_bonus   = ("Bonus",      "sum"),
    avg_rating    = ("Rating",     "mean"),
    avg_exp       = ("Experience", "mean"),
    emp_count     = ("Name",       "count"),
    total_sales   = ("Sales",      "sum")
).round(2)

print(result)
```

Output:

```
             avg_salary  max_salary  min_salary  salary_range  total_bonus  \\
Department
Engineering     90000.0       95000       85000         10000        46000
Finance         72333.3       75000       70000          5000        21700
HR              56666.7       58000       55000          3000        17000
Marketing       63750.0       65000       48000         17000        25500

             avg_rating  avg_exp  emp_count  total_sales
Department
Engineering        4.60      5.8          5       685000
Finance            4.30      6.7          3            0
HR                 4.00      2.3          3            0
Marketing          3.65      3.0          4       350000
```

> **Output Explanation:**
>
> - Engineering has the highest avg salary (₹90,000) and best avg rating (4.60)
> - Marketing has the largest salary range (₹17,000) — highest internal pay disparity
> - Finance and HR teams have no Sales figures (not revenue-generating departments)
> - This table is a complete **Department Executive Report** in 10 lines of code

---

```python
# Example 4: Named agg with custom lambda functions
result = df.groupby("Department").agg(
    total_employees  = ("Name",     "count"),
    senior_count     = ("Experience", lambda x: (x >= 5).sum()),
    top_rated        = ("Rating",   lambda x: (x >= 4.5).sum()),
    salary_cv        = ("Salary",   lambda x: (x.std() / x.mean() * 100).round(2))  # coeff of variation
).reset_index()

print(result)
```

Output:

```
    Department  total_employees  senior_count  top_rated  salary_cv
0  Engineering                5             4          3       4.23
1      Finance                3             3          1       3.48
2           HR                3             0          0       2.70
3    Marketing                4             1          0      11.03
```

> **Output Explanation:**
>
> - Engineering has 4 senior employees (exp ≥ 5) — most experienced dept
> - Marketing's salary CV = 11% — highest internal inconsistency in pay
> - HR has ZERO senior employees and ZERO high-rated employees (< 4.5 rating)

```python
# Example 5: groupby on two columns with named agg
result = df.groupby(["Department","Gender"]).agg(
    count      = ("Name",   "count"),
    avg_salary = ("Salary", "mean"),
    avg_rating = ("Rating", "mean")
).round(2).reset_index()

print(result)
```

Output:

```
    Department Gender  count  avg_salary  avg_rating
0  Engineering      F    3.0     90333.3        4.60
1  Engineering      M    2.0     89000.0        4.60
2      Finance      M    3.0     72333.3        4.30
3           HR      F    2.0     57500.0        4.00
4           HR      M    1.0     55000.0        4.00
5    Marketing      F    2.0     55000.0        3.60
6    Marketing      M    2.0     62500.0        3.65
```

```python
# Example 6: Flatten multi-level column names after agg
result = df.groupby("Department")[["Salary","Bonus"]].agg(["mean","max"])

# Multi-level columns look like: ('Salary', 'mean'), ('Bonus', 'max')
print("Before flatten:", result.columns.tolist())

# Flatten: join levels with underscore
result.columns = ["_".join(col) for col in result.columns]
print("After flatten :", result.columns.tolist())
print(result)
```

Output:

```
Before flatten: [('Salary', 'mean'), ('Salary', 'max'), ('Bonus', 'mean'), ('Bonus', 'max')]
After flatten : ['Salary_mean', 'Salary_max', 'Bonus_mean', 'Bonus_max']

             Salary_mean  Salary_max  Bonus_mean  Bonus_max
Department
Engineering      90000.0       95000      9000.0       9500
Finance          72333.3       75000      7233.3       7500
HR               56666.7       58000      5666.7       5800
Marketing        63750.0       65000      6375.0       6500
```

> **Flattening multi-level columns** is a must after using `agg()` with multiple functions — it makes the result easy to use for further analysis or export.

---

4. `transform()`

📌 Definition

`transform()` applies a function to each group but **returns a Series with the same shape as the original DataFrame** — the group result is broadcast back to every row.

> **Key difference from `agg()`:**

> - `agg()` → collapses groups → **fewer rows** in result
> - `transform()` → returns same shape → **same number of rows** as original

---

Why Use transform()?

- Add a group-level statistic BACK to each row (e.g., dept avg salary next to each employee's salary)
- Normalize data within groups (z-score, percentile)
- Fill missing values with group-level statistics
- Create group-relative features (your salary vs dept average)

Syntax

```
df["new_col"] = df.groupby("group_col")["target_col"].transform("mean")
df["new_col"] = df.groupby("group_col")["target_col"].transform(func)
df["new_col"] = df.groupby("group_col")["target_col"].transform(lambda x: ...)
```

Examples

```
# Example 1: Add department average salary to each row
df["DeptAvgSalary"] = df.groupby("Department")["Salary"].transform("mean").round(2)
print(df[["Name","Department","Salary","DeptAvgSalary"]].head(10))
```

Output:

```
     Name   Department  Salary  DeptAvgSalary
0   Alice  Engineering   90000       90000.00
1     Bob    Marketing   60000       63750.00
2   Carol  Engineering   95000       90000.00
3    Dave           HR   55000       56666.67
4     Eve    Marketing   62000       63750.00
5   Frank  Engineering   88000       90000.00
6   Grace           HR   57000       56666.67
7    Hank      Finance   75000       72333.33
8     Ivy    Marketing   48000       63750.00
9    Jake      Finance   70000       72333.33
```

> **Output Explanation:** Every employee in Engineering sees `90,000` as their dept avg (same 5 employees), while every Marketing employee sees `63,750`. The number of rows stays at 15.

```
# Example 2: Salary deviation from department average (z-score feature)
df["SalaryDeviation"] = df["Salary"] - df["DeptAvgSalary"]
df["SalaryRelative"]  = ((df["Salary"] / df["DeptAvgSalary"] - 1) * 100).round(2)

print(df[["Name","Department","Salary","DeptAvgSalary","SalaryDeviation","SalaryRelati
ve"]])
```

Output:

```
     Name   Department  Salary  DeptAvgSalary  SalaryDeviation  SalaryRelative
0   Alice  Engineering   90000       90000.00              0.0            0.00
1     Bob    Marketing   60000       63750.00          -3750.0           -5.88
2   Carol  Engineering   95000       90000.00           5000.0            5.56
3    Dave           HR   55000       56666.67          -1666.7           -2.94
4     Eve    Marketing   62000       63750.00          -1750.0           -2.75
5   Frank  Engineering   88000       90000.00          -2000.0           -2.22
6   Grace           HR   57000       56666.67            333.3            0.59
7    Hank      Finance   75000       72333.33           2666.7            3.69
8     Ivy    Marketing   48000       63750.00         -15750.0          -24.71
9    Jake      Finance   70000       72333.33          -2333.3           -3.23
10  Karen  Engineering   92000       90000.00           2000.0            2.22
11    Leo    Marketing   65000       63750.00           1250.0            1.96
12    Mia           HR   58000       56666.67           1333.3            2.35
```

```
13   Nate      Finance   72000       72333.33          -333.3          -0.46
14 Olivia  Engineering   85000       90000.00         -5000.0          -5.56
```

> **Output Explanation:** Ivy earns 24.71% BELOW her department average — potentially flagging a pay equity issue. Carol earns 5.56% ABOVE her engineering peers. This is a core **HR Analytics** feature.

```python
# Example 3: Group-wise rank (rank within department by salary)
df["DeptSalaryRank"] = df.groupby("Department")["Salary"].rank(ascending=False).astype(int)
print(df[["Name","Department","Salary","DeptSalaryRank"]].sort_values(["Department","DeptSalaryRank"]))
```

Output:

```
     Name   Department  Salary  DeptSalaryRank
2   Carol  Engineering   95000               1
10  Karen  Engineering   92000               2
0   Alice  Engineering   90000               3
5   Frank  Engineering   88000               4
14 Olivia  Engineering   85000               5
7    Hank      Finance   75000               1
13   Nate      Finance   72000               2
9    Jake      Finance   70000               3
12    Mia           HR   58000               1
6   Grace           HR   57000               2
3    Dave           HR   55000               3
11    Leo    Marketing   65000               1
4     Eve    Marketing   62000               2
1     Bob    Marketing   60000               3
8     Ivy    Marketing   48000               4
```

```python
# Example 4: Normalize salary within each department (min-max scaling)
df["SalaryNormalized"] = df.groupby("Department")["Salary"].transform(
    lambda x: ((x - x.min()) / (x.max() - x.min())).round(4)
)
print(df[["Name","Department","Salary","SalaryNormalized"]])
```

Output:

```
     Name   Department  Salary  SalaryNormalized
0   Alice  Engineering   90000            0.5000
1     Bob    Marketing   60000            0.7059
2   Carol  Engineering   95000            1.0000
3    Dave           HR   55000            0.0000
4     Eve    Marketing   62000            0.8235
5   Frank  Engineering   88000            0.3000   ← wait, let me explain below
...
```

> **Output Explanation:** Within Engineering (min=85000, max=95000):
>
> - Carol(95000) → (95000-85000)/(95000-85000) = 1.0 (highest)
> - Olivia(85000) → (85000-85000)/(95000-85000) = 0.0 (lowest)
>
> This is **group-wise normalization** — critical preprocessing step in ML where you normalize per category, not globally.

```python
# Example 5: Fill missing values with group mean (most important use!)
# Create test data with missing values
df_test = df.copy()
df_test.loc[[0,4,7], "Salary"] = np.nan

print("Missing salaries:", df_test["Salary"].isnull().sum())

# Fill with DEPARTMENT mean (not global mean)
```

```python
df_test["Salary"] = df_test.groupby("Department")["Salary"].transform(
    lambda x: x.fillna(x.mean())
)
print("Missing after fill:", df_test["Salary"].isnull().sum())
```

Output:

```
Missing salaries: 3
Missing after fill: 0
```

> **Why this is better than global mean fill:** An engineer's missing salary should be filled with the engineering department mean, not the company-wide mean that includes HR and Marketing — far more accurate.

```python
# Example 6: Cumulative sum within group (useful in time-series)
# Sort first, then compute running total
df_sorted = df.sort_values(["Department","Experience"])
df_sorted["CumSalaryInDept"] = df_sorted.groupby("Department")["Salary"].transform("cumsum")
print(df_sorted[["Department","Name","Experience","Salary","CumSalaryInDept"]])
```

Output:

```
    Department    Name  Experience  Salary  CumSalaryInDept
3           HR    Dave           2   55000            55000
6           HR   Grace           3   57000           112000
12          HR     Mia           2   58000            ...(cumulative)
...
```

---

5. `filter()`

📌 Definition

`filter()` is used to **keep or remove entire groups** from a DataFrame based on a condition applied at the group level.

> Unlike boolean indexing (which filters individual rows), `filter()` keeps or drops **entire groups**.

Syntax

```python
df.groupby("col").filter(lambda x: condition_on_group)
```

- The lambda receives the entire group as a DataFrame `x`
- Return `True` → keep the group
- Return `False` → drop the entire group

Examples

```python
# Example 1: Keep only departments with average salary > 70000
high_pay_depts = df.groupby("Department").filter(
    lambda x: x["Salary"].mean() > 70000
)
print("Departments kept:", high_pay_depts["Department"].unique())
print(high_pay_depts[["Name","Department","Salary"]])
```

Output:

```
Departments kept: ['Engineering' 'Finance']
```

```
     Name   Department  Salary
0   Alice  Engineering   90000
2   Carol  Engineering   95000
5   Frank  Engineering   88000
7    Hank      Finance   75000
9    Jake      Finance   70000
10  Karen  Engineering   92000
13   Nate      Finance   72000
14 Olivia  Engineering   85000
```

> **Output Explanation:** Only Engineering (avg ₹90,000) and Finance (avg ₹72,333) pass the filter. HR (₹56,666) and Marketing (₹63,750) are completely removed — all their employees are excluded.

```python
# Example 2: Keep departments with at least 4 employees
large_depts = df.groupby("Department").filter(
    lambda x: len(x) >= 4
)
print("Departments kept:", large_depts["Department"].unique())
print("Shape:", large_depts.shape)
```

Output:

```
Departments kept: ['Engineering' 'Marketing']
Shape: (9, 11)
```

```python
# Example 3: Keep departments where MAX salary > 80000
strong_depts = df.groupby("Department").filter(
    lambda x: x["Salary"].max() > 80000
)
print(strong_depts["Department"].unique())
```

Output:

```
['Engineering' 'Finance' 'Marketing']
```

```python
# Example 4: Keep groups where average rating >= 4.3 (high-performing teams)
high_performance = df.groupby("Department").filter(
    lambda x: x["Rating"].mean() >= 4.3
)
print(high_performance[["Name","Department","Rating"]])
```

Output:

```
     Name   Department  Rating
0   Alice  Engineering     4.5
2   Carol  Engineering     4.7
5   Frank  Engineering     4.8
7    Hank      Finance     4.6
9    Jake      Finance     4.0
10  Karen  Engineering     4.4
13   Nate      Finance     4.3
14 Olivia  Engineering     4.6
```

> Engineering (avg 4.60) and Finance (avg 4.30) qualify; HR (4.00) and Marketing (3.65) do not.

```python
# Example 5: Filter by multiple conditions on the group
result = df.groupby("Department").filter(
    lambda x: (x["Salary"].mean() > 60000) and (x["Rating"].mean() > 4.0)
```

```
)
print(result["Department"].unique())
```

Output:

```
['Engineering' 'Finance']
```

```
# Example 6: Real-world — keep cities with at least 2 employees AND avg salary > 65000
result = df.groupby("City").filter(
    lambda x: len(x) >= 2 and x["Salary"].mean() > 65000
)
print(result[["Name","City","Salary"]])
```

Output:

```
    Name    City  Salary
0   Alice  Mumbai   90000
2   Carol  Bangalore 95000
3    Dave  Mumbai   55000
5   Frank   Delhi   88000
7    Hank  Mumbai   75000
8     Ivy  Bangalore 48000
...
```

---

6. Advanced GroupBy Patterns

Pattern 1: GroupBy with apply() for Custom Group Operations

```
# Get top 2 earners from each department
def top_n_earners(group, n=2):
    return group.nlargest(n, "Salary")

top2_per_dept = df.groupby("Department").apply(top_n_earners)
print(top2_per_dept[["Name","Department","Salary"]])
```

Output:

```
                   Name   Department  Salary
Department
Engineering  2      Carol  Engineering   95000
             10     Karen  Engineering   92000
Finance      7      Hank      Finance   75000
             13     Nate      Finance   72000
HR           12      Mia           HR   58000
             6      Grace         HR   57000
Marketing    11      Leo    Marketing   65000
             4       Eve    Marketing   62000
```

Pattern 2: Pivot-style Cross-tabulation

```
# Count employees per Department × Gender
pivot = df.groupby(["Department","Gender"]).size().unstack(fill_value=0)
print(pivot)
```

Output:

```
Gender       F  M
Department
Engineering  3  2
Finance      0  3
```

```
HR          2  1
Marketing   2  2
```

Pattern 3: GroupBy + Cumulative Operations

```python
df_sorted = df.sort_values(["Department","Salary"])

# Running total of salary within each department
df_sorted["CumSalary"] = df_sorted.groupby("Department")["Salary"].cumsum()

# Running count within each department
df_sorted["CumCount"] = df_sorted.groupby("Department").cumcount() + 1

print(df_sorted[["Department","Name","Salary","CumSalary","CumCount"]])
```

Output:

```
    Department     Name  Salary  CumSalary  CumCount
14  Engineering  Olivia   85000      85000         1
5   Engineering   Frank   88000     173000         2
0   Engineering   Alice   90000     263000         3
10  Engineering   Karen   92000     355000         4
2   Engineering   Carol   95000     450000         5
...
```

Pattern 4: GroupBy + Shift (Lag Features for Time Series)

```python
# Previous period's salary within each department (lag feature)
df_sorted["PrevSalary"] = df_sorted.groupby("Department")["Salary"].shift(1)
print(df_sorted[["Department","Name","Salary","PrevSalary"]].head(8))
```

Output:

```
    Department     Name  Salary  PrevSalary
14  Engineering  Olivia   85000         NaN   ← first in group, no previous
5   Engineering   Frank   88000     85000.0
0   Engineering   Alice   90000     88000.0
10  Engineering   Karen   92000     90000.0
2   Engineering   Carol   95000     92000.0
...
```

Pattern 5: Resample (GroupBy for Time Series by Date)

```python
# Monthly total sales — using resample (time-based groupby)
dates = pd.date_range("2024-01-01", periods=12, freq="ME")
monthly = pd.DataFrame({
    "Date"  : dates,
    "Sales" : [120000,85000,135000,95000,115000,78000,
               125000,92000,110000,88000,105000,130000]
})

monthly.set_index("Date", inplace=True)

# Quarterly totals (resample by quarter)
quarterly = monthly["Sales"].resample("QE").sum()
print("Quarterly Sales:")
print(quarterly)
```

Output:

```
Quarterly Sales:
Date
2024-03-31    340000
2024-06-30    288000
2024-09-30    327000
2024-12-31    323000
Freq: QE-DEC, Name: Sales, dtype: int64
```

Mini Summary & Cheat Sheet

🔑 Phase 5 Key Takeaways

- ✅ **groupby()** → Split → Apply → Combine pattern (like SQL GROUP BY)
- ✅ **Aggregation functions** → `sum`, `mean`, `max`, `min`, `count`, `std`, `nunique`, `idxmax`
- ✅ **agg()** → Multiple functions at once; use named agg for clean column names
- ✅ **transform()** → Same shape as input — adds group stats back to each row
- ✅ **filter()** → Keep or remove entire groups based on group-level condition
- ✅ **Flatten multi-level columns:** `["_".join(col) for col in result.columns]`
- ✅ **as_index=False** → keep group column as regular column (not index)

`agg()` vs `transform()` vs `filter()` — Clear Comparison

Feature	`agg()`	`transform()`	`filter()`
Output shape	**Shrinks** (one row per group)	**Same** as original	**Subset** of original rows
Returns	Summary DataFrame/Series	Full-length Series	Filtered DataFrame
Use case	Summary statistics	Add group stat to each row	Keep/drop entire groups
Can mix functions?	✅ Yes	❌ One function	N/A
Example	Avg salary per dept	Dept avg next to each employee	Keep depts with avg > 70000

🔖 Quick Reference Cheat Sheet

Task	Code
Group by one column	`df.groupby("col")`
Group by multiple cols	`df.groupby(["col1","col2"])`
Keep group col in result	`df.groupby("col", as_index=False)`
Extract one group	`df.groupby("col").get_group("value")`
Iterate over groups	`for name, group in df.groupby("col"):`
Count per group	`df.groupby("col").size()`
Sum	`df.groupby("col")["val"].sum()`
Mean	`df.groupby("col")["val"].mean()`
Max	`df.groupby("col")["val"].max()`
Min	`df.groupby("col")["val"].min()`
Std dev	`df.groupby("col")["val"].std()`
Unique count	`df.groupby("col")["val"].nunique()`
Index of max	`df.groupby("col")["val"].idxmax()`
Multiple stats	`df.groupby("col")["val"].agg(["mean","max","count"])`
Named agg	`df.groupby("col").agg(avg=("col","mean"), tot=("col","sum"))`
Different agg per col	`df.groupby("col").agg({"a":"mean","b":"sum"})`
Flatten column names	`result.columns = ["_".join(c) for c in result.columns]`
Add group mean to rows	`df.groupby("col")["val"].transform("mean")`
Fill NaN with group mean	`df.groupby("col")["val"].transform(lambda x: x.fillna(x.mean()))`
Group-wise rank	`df.groupby("col")["val"].rank(ascending=False)`

Task	Code
Normalize within group	`df.groupby("col")["val"].transform(lambda x: (x-x.min())/(x.max()-x.min()))`
Cumulative sum in group	`df.groupby("col")["val"].transform("cumsum")`
Cumulative count	`df.groupby("col").cumcount()`
Keep groups by condition	`df.groupby("col").filter(lambda x: x["val"].mean() > threshold)`
Keep large groups	`df.groupby("col").filter(lambda x: len(x) >= n)`
Top N per group	`df.groupby("col").apply(lambda x: x.nlargest(n,"val"))`
Pivot from groupby	`df.groupby(["a","b"]).size().unstack(fill_value=0)`

🐼 PHASE 6: MERGING & JOINING — Complete Learning Notes

> **Goal:** Learn how to combine multiple DataFrames into one — whether stacking them (concat), joining on a common key (merge), or aligning by index (join). These operations mirror SQL JOINs and are used in every real-world data pipeline where data comes from multiple sources.

🗂️ Datasets Used Throughout Phase 6

> We use multiple small, focused datasets so each joining concept is crystal-clear.

```
import pandas as pd
import numpy as np

# ── Dataset 1: Employees ──────────────────────────────────────────
employees = pd.DataFrame({
    "EmployeeID" : [101, 102, 103, 104, 105, 106],
    "Name"       : ["Alice", "Bob", "Carol", "Dave", "Eve", "Frank"],
    "DeptID"     : [10, 20, 10, 30, 20, 40],
    "Salary"     : [90000, 60000, 95000, 55000, 62000, 88000]
})

# ── Dataset 2: Departments ────────────────────────────────────────
departments = pd.DataFrame({
    "DeptID"   : [10, 20, 30, 50],          # Note: 40 & 50 create mismatch
    "DeptName" : ["Engineering", "Marketing", "HR", "Legal"],
    "Location" : ["Mumbai", "Delhi", "Bangalore", "Chennai"]
})

# ── Dataset 3: Performance Reviews ────────────────────────────────
reviews = pd.DataFrame({
    "EmployeeID" : [101, 102, 103, 107, 108],  # 107, 108 not in employees
    "Rating"     : [4.5, 3.8, 4.7, 4.0, 3.5],
    "ReviewYear" : [2024, 2024, 2024, 2024, 2024]
})

# ── Dataset 4: Salaries Q1 ────────────────────────────────────────
q1_sales = pd.DataFrame({
    "Region"  : ["North", "South", "East"],
    "Q1_Sales": [120000,  85000,   95000]
})

# ── Dataset 5: Salaries Q2 ────────────────────────────────────────
q2_sales = pd.DataFrame({
    "Region"  : ["North", "South", "West"],
    "Q2_Sales": [130000, 90000, 78000]
})
```

```python
# ── Dataset 6: New Employees (same structure as employees) ──────────────
new_employees = pd.DataFrame({
    "EmployeeID" : [107, 108, 109],
    "Name"       : ["Grace", "Hank", "Ivy"],
    "DeptID"     : [10, 30, 20],
    "Salary"     : [72000, 58000, 67000]
})

print("employees:\\n", employees)
print("\\ndepartments:\\n", departments)
print("\\nreviews:\\n", reviews)
```

Output:

```
employees:
   EmployeeID   Name  DeptID  Salary
0         101  Alice      10   90000
1         102    Bob      20   60000
2         103  Carol      10   95000
3         104   Dave      30   55000
4         105    Eve      20   62000
5         106  Frank      40   88000

departments:
   DeptID   DeptName   Location
0      10  Engineering   Mumbai
1      20    Marketing    Delhi
2      30           HR  Bangalore
3      50        Legal   Chennai

reviews:
   EmployeeID  Rating  ReviewYear
0         101     4.5        2024
1         102     3.8        2024
2         103     4.7        2024
3         107     4.0        2024
4         108     3.5        2024
```

---

1. `concat()` — Stacking DataFrames

📌 Definition

`concat()` **stacks** DataFrames either **vertically** (row-wise, axis=0) or **horizontally** (column-wise, axis=1).

> Think of it as **physically gluing** DataFrames together — no key lookup needed.
>
> - **Vertical** → append new rows (like adding rows to a table)
> - **Horizontal** → append new columns (side by side)

This is NOT a join — there's no matching on a key.

Syntax

```python
pd.concat([df1, df2])                          # vertical stack (default axis=0)
pd.concat([df1, df2], axis=1)                  # horizontal side-by-side
pd.concat([df1, df2], ignore_index=True)       # reset index after stacking
pd.concat([df1, df2], keys=["src1","src2"])    # add source label to index
pd.concat([df1, df2], join="inner")            # only keep common columns
pd.concat([df1, df2], join="outer")            # keep all columns (default)
```

Examples

Vertical Stacking (axis=0) — Adding Rows

```python
# Example 1: Stack two DataFrames with same structure (most common use)
print("employees shape :", employees.shape)
print("new_employees shape:", new_employees.shape)

all_employees = pd.concat([employees, new_employees])
print("\\nAfter concat:")
print(all_employees)
print("Shape:", all_employees.shape)
```

Output:

```
employees shape    : (6, 4)
new_employees shape: (3, 4)

After concat:
   EmployeeID   Name  DeptID  Salary
0         101  Alice      10   90000
1         102    Bob      20   60000
2         103  Carol      10   95000
3         104   Dave      30   55000
4         105    Eve      20   62000
5         106  Frank      40   88000
0         107  Grace      10   72000   ← index 0 repeats!
1         108   Hank      30   58000   ← index 1 repeats!
2         109    Ivy      20   67000   ← index 2 repeats!
Shape: (9, 4)
```

> ⚠️ **Problem:** Index repeats (0,1,2 from new_employees). Fix with `ignore_index=True`:

```python
# Example 2: ignore_index=True — clean sequential index
all_employees = pd.concat([employees, new_employees], ignore_index=True)
print(all_employees)
```

Output:

```
   EmployeeID   Name  DeptID  Salary
0         101  Alice      10   90000
1         102    Bob      20   60000
2         103  Carol      10   95000
3         104   Dave      30   55000
4         105    Eve      20   62000
5         106  Frank      40   88000
6         107  Grace      10   72000   ← clean index 6
7         108   Hank      30   58000   ← clean index 7
8         109    Ivy      20   67000   ← clean index 8
```

> **Output Explanation:** `ignore_index=True` resets the index to 0,1,2...8 — the correct behavior after combining rows.

```python
# Example 3: keys — track which source each row came from
all_with_source = pd.concat(
    [employees, new_employees],
    keys        = ["existing", "new"],
    ignore_index= False
)
print(all_with_source)
```

Output:

```
            EmployeeID   Name  DeptID  Salary
existing 0         101  Alice      10   90000
         1         102    Bob      20   60000
         2         103  Carol      10   95000
         3         104   Dave      30   55000
         4         105    Eve      20   62000
         5         106  Frank      40   88000
```

```
new      0         107  Grace      10   72000
         1         108   Hank      30   58000
         2         109    Ivy      20   67000
```

> **Output Explanation:** A multi-level index (MultiIndex) is created — `("existing", 0)`, `("new", 0)`. Useful for audit/lineage tracking — knowing which records came from which source.

```python
# Example 4: Concat DataFrames with DIFFERENT columns (outer join — default)
df_a = pd.DataFrame({"A":[1,2], "B":[3,4]})
df_b = pd.DataFrame({"B":[5,6], "C":[7,8]})

result_outer = pd.concat([df_a, df_b], ignore_index=True)
print("OUTER (default):")
print(result_outer)
```

Output:

```
OUTER (default):
     A  B    C
0  1.0  3  NaN   ← df_a has no column C → NaN
1  2.0  4  NaN   ← df_a has no column C → NaN
2  NaN  5  7.0   ← df_b has no column A → NaN
3  NaN  6  8.0   ← df_b has no column A → NaN
```

```python
# Only keep columns common to BOTH DataFrames
result_inner = pd.concat([df_a, df_b], join="inner", ignore_index=True)
print("\\nINNER (common cols only):")
print(result_inner)
```

Output:

```
INNER (common cols only):
   B
0  3
1  4
2  5
3  6
```

> **Output Explanation:**
>
> - `join="outer"` (default) → keeps ALL columns, fills gaps with NaN
> - `join="inner"` → keeps ONLY columns present in ALL DataFrames — no NaN from mismatched columns

---

```python
# Example 5: Concat 3 or more DataFrames at once
q3_sales = pd.DataFrame({"Region":["North","East","West"], "Q3_Sales":[140000,88000,82000]})
q4_sales = pd.DataFrame({"Region":["South","East","West"], "Q4_Sales":[95000,91000,85000]})

# Stack all quarterly data
all_quarters = pd.concat([q1_sales, q2_sales, q3_sales, q4_sales], ignore_index=True)
print(all_quarters)
```

Output:

```
   Region  Q1_Sales  Q2_Sales  Q3_Sales  Q4_Sales
0   North  120000.0       NaN       NaN       NaN
1   South   85000.0       NaN       NaN       NaN
2    East   95000.0       NaN       NaN       NaN
3   North       NaN  130000.0       NaN       NaN
4   South       NaN   90000.0       NaN       NaN
5    West       NaN   78000.0       NaN       NaN
```

```
6   North       NaN       NaN  140000.0       NaN
7    East       NaN       NaN   88000.0       NaN
8    West       NaN       NaN   82000.0       NaN
9   South       NaN       NaN       NaN   95000.0
10   East       NaN       NaN       NaN   91000.0
11   West       NaN       NaN       NaN   85000.0
```

Horizontal Stacking (axis=1) — Adding Columns

```python
# Example 6: axis=1 — place DataFrames side by side
name_df   = pd.DataFrame({"Name": ["Alice","Bob","Carol"]})
salary_df = pd.DataFrame({"Salary": [90000, 60000, 95000]})
city_df   = pd.DataFrame({"City": ["Mumbai","Delhi","Bangalore"]})

combined = pd.concat([name_df, salary_df, city_df], axis=1)
print(combined)
```

Output:

```
    Name  Salary       City
0  Alice   90000     Mumbai
1    Bob   60000      Delhi
2  Carol   95000  Bangalore
```

> **Use case:** When you compute multiple feature groups separately and want to combine them into one final DataFrame.

```python
# Example 7: axis=1 with different index lengths — NaN fills gaps
df_x = pd.DataFrame({"Score": [90, 80, 70]}, index=["A","B","C"])
df_y = pd.DataFrame({"Grade": ["A","B"]},    index=["A","B"])

combined = pd.concat([df_x, df_y], axis=1)
print(combined)
```

Output:

```
   Score Grade
A     90     A
B     80     B
C     70   NaN   ← C not in df_y → NaN
```

⚠️ Common Mistakes — concat()

Mistake	Explanation
Forgetting `ignore_index=True`	Repeated index values cause confusion in later operations
Using `concat()` for key-based joins	Use `merge()` for that — concat has no awareness of matching keys
`pd.concat(df1, df2)` (not a list)	Must pass a **list** `[df1, df2]` — single arg is invalid
Mismatched column names	Columns must be spelled identically — even spacing matters

---

2. `merge()` — SQL-Style Joins

📌 Definition

`merge()` combines two DataFrames by **matching rows based on a common key column** (or columns) — exactly like SQL JOIN.

> **When to use merge() vs concat():**
>
> - `concat()` → physically stack rows/columns — **no key matching**

- `merge()` → combine based on a **shared key column** — key matching required

Syntax

```
pd.merge(left, right, on="key")                      # join on common column
pd.merge(left, right, on=["key1","key2"])             # join on multiple columns
pd.merge(left, right, left_on="lkey", right_on="rkey")  # different key names
pd.merge(left, right, on="key", how="inner")          # INNER join (default)
pd.merge(left, right, on="key", how="left")           # LEFT join
pd.merge(left, right, on="key", how="right")          # RIGHT join
pd.merge(left, right, on="key", how="outer")          # FULL OUTER join
pd.merge(left, right, on="key", suffixes=("_x","_y")) # rename duplicate cols
pd.merge(left, right, on="key", indicator=True)       # add _merge column
```

3. Types of Joins

Visual Reference — All 4 Join Types

```
LEFT DataFrame (employees)          RIGHT DataFrame (departments)
┌─────┬─────────┬──────────┐            ┌──────────┬─────────────────┐
│ ID  │ Name    │ DeptID   │            │ DeptID   │  DeptName       │
├─────┼─────────┼──────────┤            ├──────────┼─────────────────┤
│101  │ Alice   │   10     │            │  10      │ Engineering     │ ← matches
│102  │ Bob     │   20     │    JOIN    │  20      │ Marketing       │ ← matches
│103  │ Carol   │   10     │    ON      │  30      │ HR              │ ← matches
│104  │ Dave    │   30     │  DeptID    │  50      │ Legal           │ ← NO MATCH
│105  │ Eve     │   20     │            └──────────┴─────────────────┘
│106  │ Frank   │   40     │ ← NO MATCH (DeptID 40 not in right)
└─────┴─────────┴──────────┘

INNER:  Only rows with DeptID matching on BOTH sides  → 5 rows
LEFT:   All left rows + matched right data            → 6 rows (Frank gets NaN)
RIGHT:  Matched left data + all right rows            → 5 rows (Legal gets NaN)
OUTER:  Everything from both sides                    → 7 rows (Frank+Legal both kept)
```

INNER JOIN

Definition

Returns only the rows where the **key exists in BOTH DataFrames**.
Unmatched rows from either side are **dropped**.

```
# SQL equivalent:
# SELECT * FROM employees e
# INNER JOIN departments d ON e.DeptID = d.DeptID

inner = pd.merge(employees, departments, on="DeptID", how="inner")
print(inner)
print("\\nShape:", inner.shape)
```

Output:

```
   EmployeeID   Name  DeptID  Salary     DeptName   Location
0         101  Alice      10   90000  Engineering     Mumbai
1         103  Carol      10   95000  Engineering     Mumbai
2         102    Bob      20   60000    Marketing      Delhi
3         105    Eve      20   62000    Marketing      Delhi
4         104   Dave      30   55000           HR  Bangalore

Shape: (5, 6)
```

Output Explanation:

- ✅ DeptID 10 → Engineering (Alice, Carol matched)

- ✅ DeptID 20 → Marketing (Bob, Eve matched)
- ✅ DeptID 30 → HR (Dave matched)
- ❌ Frank (DeptID 40) → **DROPPED** — no DeptID 40 in departments
- ❌ Legal (DeptID 50) → **DROPPED** — no employee in DeptID 50

> **Use INNER JOIN when:** You only want records that have complete information from both sides — e.g., only employees who belong to a known department.

LEFT JOIN

Definition

Returns **ALL rows from the LEFT DataFrame** + matching rows from the right.
Unmatched right-side values become `NaN`.

```
# SQL equivalent:
# SELECT * FROM employees e
# LEFT JOIN departments d ON e.DeptID = d.DeptID

left_join = pd.merge(employees, departments, on="DeptID", how="left")
print(left_join)
print("\\nShape:", left_join.shape)
```

Output:

```
   EmployeeID   Name  DeptID  Salary     DeptName   Location
0         101  Alice      10   90000  Engineering     Mumbai
1         102    Bob      20   60000    Marketing      Delhi
2         103  Carol      10   95000  Engineering     Mumbai
3         104   Dave      30   55000           HR  Bangalore
4         105    Eve      20   62000    Marketing      Delhi
5         106  Frank      40   88000          NaN        NaN   ← kept, NaN on right

Shape: (6, 6)
```

Output Explanation:

- ✅ All 6 employees from the LEFT are kept
- ✅ Employees with matching DeptID get department data filled in
- ⚠️ Frank (DeptID 40) → kept but `DeptName` and `Location` = NaN (no match on right)
- ❌ Legal (DeptID 50) → dropped (it's on the right side, not the left)

> **Use LEFT JOIN when:** You want ALL records from your primary/main table, even if lookup data is missing — e.g., all employees (even those assigned to invalid departments).

RIGHT JOIN

Definition

Returns matching rows from the left + **ALL rows from the RIGHT DataFrame**.
Unmatched left-side values become `NaN`.

```
# SQL equivalent:
# SELECT * FROM employees e
# RIGHT JOIN departments d ON e.DeptID = d.DeptID

right_join = pd.merge(employees, departments, on="DeptID", how="right")
print(right_join)
print("\\nShape:", right_join.shape)
```

Output:

```
   EmployeeID   Name  DeptID  Salary     DeptName   Location
0       101.0  Alice      10  90000.0  Engineering     Mumbai
1       103.0  Carol      10  95000.0  Engineering     Mumbai
2       102.0    Bob      20  60000.0    Marketing      Delhi
3       105.0    Eve      20  62000.0    Marketing      Delhi
4       104.0   Dave      30  55000.0           HR  Bangalore
5         NaN    NaN      50      NaN        Legal    Chennai   ← kept, NaN on left

Shape: (6, 6)
```

Output Explanation:

- ✅ All 4 departments from the RIGHT are kept
- ✅ Departments with matching employees get employee data filled in
- ⚠️ Legal (DeptID 50) → kept but `EmployeeID`, `Name`, `Salary` = NaN (no employees assigned)
- ❌ Frank (DeptID 40) → dropped (no DeptID 40 on the right)

> **Use RIGHT JOIN when:** You want ALL records from your reference/lookup table — e.g., show all departments including those with no employees yet.
>
> 💡 **Tip:** Right join is rare in practice. Most developers swap left/right and use a LEFT JOIN instead.

OUTER (FULL) JOIN

Definition

Returns **ALL rows from BOTH DataFrames**.
Unmatched values on either side become `NaN`.

```python
# SQL equivalent:
# SELECT * FROM employees e
# FULL OUTER JOIN departments d ON e.DeptID = d.DeptID

outer_join = pd.merge(employees, departments, on="DeptID", how="outer")
print(outer_join)
print("\\nShape:", outer_join.shape)
```

Output:

```
   EmployeeID   Name  DeptID  Salary     DeptName   Location
0       101.0  Alice      10  90000.0  Engineering     Mumbai
1       103.0  Carol      10  95000.0  Engineering     Mumbai
2       102.0    Bob      20  60000.0    Marketing      Delhi
3       105.0    Eve      20  62000.0    Marketing      Delhi
4       104.0   Dave      30  55000.0           HR  Bangalore
5       106.0  Frank      40  88000.0          NaN        NaN   ← from left only
6         NaN    NaN      50      NaN        Legal    Chennai   ← from right only

Shape: (7, 6)
```

Output Explanation:

- ✅ All 6 employees — kept
- ✅ All 4 departments — kept
- ⚠️ Frank (DeptID 40) → kept with NaN department info (left-only)
- ⚠️ Legal (DeptID 50) → kept with NaN employee info (right-only)
- Total = 5 matched + 1 left-only + 1 right-only = **7 rows**

> **Use OUTER JOIN when:** You want to see EVERYTHING — especially useful for finding unmatched records (data quality audits).

Join Types — Side-by-Side Comparison

```python
for how in ["inner", "left", "right", "outer"]:
    result = pd.merge(employees, departments, on="DeptID", how=how)
    print(f"{how.upper():6} JOIN → {result.shape[0]} rows")
```

Output:

```
INNER  JOIN → 5 rows
LEFT   JOIN → 6 rows
RIGHT  JOIN → 6 rows
OUTER  JOIN → 7 rows
```

Join Type	Rows Returned	Left Unmatched	Right Unmatched
INNER	5	❌ Dropped	❌ Dropped
LEFT	6	✅ Kept (NaN on right)	❌ Dropped
RIGHT	6	❌ Dropped	✅ Kept (NaN on left)
OUTER	7	✅ Kept (NaN on right)	✅ Kept (NaN on left)

---

4. Advanced `merge()` Scenarios

Scenario 1: Different Key Column Names

```python
# Left key: "EmployeeID", Right key: "EmpID"
emp2 = pd.DataFrame({"EmployeeID":[101,102,103,104,105,106],
                     "Name":["Alice","Bob","Carol","Dave","Eve","Frank"],
                     "DeptID":[10,20,10,30,20,40]})

rev2 = pd.DataFrame({"EmpID":[101,102,103,107,108],
                     "Rating":[4.5,3.8,4.7,4.0,3.5]})

# Use left_on / right_on when key names differ
result = pd.merge(emp2, rev2, left_on="EmployeeID", right_on="EmpID", how="left")
print(result[["EmployeeID","Name","EmpID","Rating"]])
```

Output:

```
   EmployeeID   Name   EmpID  Rating
0         101  Alice   101.0     4.5
1         102    Bob   102.0     3.8
2         103  Carol   103.0     4.7
3         104   Dave     NaN     NaN   ← no review for Dave
4         105    Eve     NaN     NaN   ← no review for Eve
5         106  Frank     NaN     NaN   ← no review for Frank
```

```python
# Clean up the duplicate ID column
result = result.drop(columns=["EmpID"])
print(result)
```

Scenario 2: Duplicate Column Names — `suffixes`

```python
# Both DataFrames have a "Rating" column
df_left  = pd.DataFrame({"EmpID":[101,102,103], "Rating":[4.5,3.8,4.7], "Dept":["En
g","Mkt","Eng"]})
df_right = pd.DataFrame({"EmpID":[101,102,103], "Rating":[4.3,4.0,4.9], "Year":[2023,2
023,2023]})

# Without suffix → renames to Rating_x and Rating_y (default)
result = pd.merge(df_left, df_right, on="EmpID")
print("Default suffixes:")
```

```
print(result)

# Custom suffixes — more descriptive
result2 = pd.merge(df_left, df_right, on="EmpID", suffixes=("_current","_prev"))
print("\\nCustom suffixes:")
print(result2)
```

Output:

```
Default suffixes:
   EmpID  Rating_x Dept  Rating_y  Year
0    101       4.5  Eng       4.3  2023
1    102       3.8  Mkt       4.0  2023
2    103       4.7  Eng       4.9  2023

Custom suffixes:
   EmpID  Rating_current Dept  Rating_prev  Year
0    101             4.5  Eng          4.3  2023
1    102             3.8  Mkt          4.0  2023
2    103             4.7  Eng          4.9  2023
```

Scenario 3: `indicator=True` — Find Unmatched Records

```
# indicator=True adds a "_merge" column showing match source
result = pd.merge(employees, departments, on="DeptID", how="outer", indicator=True)
print(result[["Name","DeptName","_merge"]])
```

Output:

```
    Name     DeptName        _merge
0  Alice  Engineering          both   ← matched on both sides
1  Carol  Engineering          both
2    Bob    Marketing          both
3    Eve    Marketing          both
4   Dave           HR          both
5  Frank          NaN     left_only  ← only in employees (no dept match)
6    NaN        Legal    right_only  ← only in departments (no employee)
```

> **Real-world use:** Data reconciliation — finding records in system A but not system B, or vice versa. Used in finance, auditing, and ETL pipelines.

```
# Extract ONLY unmatched records (data quality check)
unmatched = result[result["_merge"] != "both"]
print("\\nUnmatched records:")
print(unmatched[["Name","DeptName","_merge"]])
```

Output:

```
Unmatched records:
    Name DeptName      _merge
5  Frank      NaN   left_only
6    NaN    Legal  right_only
```

Scenario 4: Merging on Multiple Keys

```
# Merge on two columns — EmployeeID AND ReviewYear must both match
emp_reviews = pd.DataFrame({
    "EmployeeID" : [101, 101, 102, 102, 103],
    "Year"       : [2023, 2024, 2023, 2024, 2024],
    "Score"      : [85,   90,   78,   82,   88]
})
```

```python
targets = pd.DataFrame({
    "EmployeeID" : [101, 101, 102],
    "Year"       : [2023, 2024, 2024],
    "Target"     : [80,   88,   80]
})

result = pd.merge(emp_reviews, targets, on=["EmployeeID","Year"], how="left")
print(result)
```

Output:

```
   EmployeeID  Year  Score  Target
0         101  2023     85    80.0   ← matched both keys
1         101  2024     90    88.0   ← matched both keys
2         102  2023     78     NaN   ← Year 2023 not in targets for EmpID 102
3         102  2024     82    80.0   ← matched both keys
4         103  2024     88     NaN   ← EmpID 103 not in targets at all
```

Scenario 5: Self Join — Joining a Table to Itself

```python
# Find pairs of employees in the same department
emp_self = employees[["EmployeeID","Name","DeptID"]].copy()

result = pd.merge(
    emp_self,
    emp_self,
    on="DeptID",
    suffixes=("_A","_B")
)

# Remove self-pairs (same person) and duplicate reversed pairs
result = result[result["EmployeeID_A"] < result["EmployeeID_B"]]
print(result[["Name_A","Name_B","DeptID"]])
```

Output:

```
  Name_A Name_B  DeptID
0  Alice  Carol      10   ← both in Engineering
2    Bob    Eve      20   ← both in Marketing
```

Scenario 6: Chained Merges (Three Tables)

```python
# Step 1: employees + departments
step1 = pd.merge(employees, departments, on="DeptID", how="left")

# Step 2: result + reviews
final = pd.merge(step1, reviews, on="EmployeeID", how="left")

print(final[["Name","DeptName","Salary","Rating","ReviewYear"]])
```

Output:

```
    Name     DeptName  Salary  Rating  ReviewYear
0  Alice  Engineering   90000     4.5      2024.0
1    Bob    Marketing   60000     3.8      2024.0
2  Carol  Engineering   95000     4.7      2024.0
3   Dave           HR   55000     NaN         NaN   ← no review
4    Eve    Marketing   62000     NaN         NaN   ← no review
5  Frank          NaN   88000     NaN         NaN   ← no dept, no review
```

> **Output Explanation:** A three-table join in 2 simple `pd.merge()` calls. This is how real-world ETL pipelines work — joining many source tables step by step to build a wide "master table."

Scenario 7: validate — Catch Unexpected Duplicates

```
# validate ensures the relationship type is correct
# "one_to_one"    → each key appears once in both
# "one_to_many"   → key appears once in left, many times in right
# "many_to_one"   → key appears many times in left, once in right
# "many_to_many"  → key appears many times in both

try:
    result = pd.merge(employees, departments, on="DeptID",
                      how="inner", validate="one_to_one")
except pd.errors.MergeError as e:
    print("Validation failed:", e)
```

Output:

```
Validation failed: Merge keys are not unique in left dataset; not a one-to-one merge
```

> Many employees share the same DeptID → it is a many-to-one relationship (many employees, one dept). Use `validate="many_to_one"` for the correct assertion.

---

5. `join()` — Index-Based Joining

📌 Definition

`join()` is a convenience method for combining DataFrames on their **index** (row labels) rather than a column. Under the hood it calls `merge()` with `left_index=True` and `right_index=True`.

Syntax

```
df1.join(df2)                        # join on index (LEFT join by default)
df1.join(df2, how="inner")           # inner, left, right, outer
df1.join(df2, on="col")              # use df1's column to match df2's index
df1.join([df2, df3])                 # join multiple DataFrames at once
df1.join(df2, lsuffix="_x", rsuffix="_y")  # handle duplicate column names
```

Examples

```
# Example 1: Basic index join
emp_main = employees.set_index("EmployeeID")
emp_ratings = reviews.set_index("EmployeeID")[["Rating"]]

result = emp_main.join(emp_ratings, how="left")
print(result[["Name","DeptID","Salary","Rating"]])
```

Output:

```
            Name  DeptID  Salary  Rating
EmployeeID
101        Alice      10   90000     4.5
102          Bob      20   60000     3.8
103        Carol      10   95000     4.7
104         Dave      30   55000     NaN   ← no review
105          Eve      20   62000     NaN   ← no review
106        Frank      40   88000     NaN   ← no review
```

```python
# Example 2: Join multiple DataFrames at once
info_df   = employees.set_index("EmployeeID")[["Name","DeptID"]]
salary_df = employees.set_index("EmployeeID")[["Salary"]]
rating_df = reviews.set_index("EmployeeID")[["Rating"]]

result = info_df.join([salary_df, rating_df], how="left")
print(result)
```

Output:

```
            Name  DeptID  Salary  Rating
EmployeeID
101        Alice      10   90000     4.5
102          Bob      20   60000     3.8
103        Carol      10   95000     4.7
104         Dave      30   55000     NaN
105          Eve      20   62000     NaN
106        Frank      40   88000     NaN
```

```python
# Example 3: join() on column using on= parameter
# df1's column value is matched to df2's INDEX
emp_col  = employees.copy()          # index = 0,1,2...
dept_idx = departments.set_index("DeptID")  # DeptID is the index

result = emp_col.join(dept_idx, on="DeptID", how="left")
print(result[["Name","DeptID","DeptName","Location"]])
```

Output:

```
    Name  DeptID     DeptName   Location
0  Alice      10  Engineering     Mumbai
1    Bob      20    Marketing      Delhi
2  Carol      10  Engineering     Mumbai
3   Dave      30           HR  Bangalore
4    Eve      20    Marketing      Delhi
5  Frank      40          NaN        NaN
```

`merge()` vs `join()` — Comparison

Feature	`merge()`	`join()`
Joins on	Any column(s) or index	Index (by default)
Default join type	**inner**	**left**
Multi-table at once	❌ One at a time	✅ `df.join([df2,df3])`
Flexibility	Very high	Limited to index join
Recommended for	Column-key joins (most common)	Index alignment

---

6. When to Use What

Decision Guide

```
Do you want to STACK rows or columns with NO key matching?
  └─ YES → pd.concat([df1, df2])                (axis=0 for rows, axis=1 for cols)

Do you want to JOIN rows based on a SHARED KEY COLUMN?
  └─ YES → pd.merge(df1, df2, on="key", how=...)

        What rows do you want?
        ├─ Only matched rows        → how="inner"  (default)
        ├─ All from LEFT + matches  → how="left"   (most common)
        ├─ All from RIGHT + matches → how="right"  (rare — just swap and use left)
        └─ Everything from BOTH     → how="outer"  (data audits)
```

```
Do you want to JOIN on INDEX values?
  └─ YES → df1.join(df2, how="left")

Do you want to join MULTIPLE DataFrames at once?
  └─ YES → df1.join([df2, df3])
       OR → reduce(pd.merge, [df1, df2, df3])  # for column-key merges
```

Real-World Scenarios → Correct Method

Scenario	Method
Combine Jan, Feb, Mar sales CSV files	`pd.concat([jan, feb, mar], ignore_index=True)`
Add department names to employee list	`pd.merge(emp, dept, on="DeptID", how="left")`
Find employees with no department	`pd.merge(emp, dept, on="DeptID", how="left")` → filter NaN
Find departments with no employees	`pd.merge(emp, dept, on="DeptID", how="right")` → filter NaN
Full audit — all employees + all depts	`pd.merge(emp, dept, on="DeptID", how="outer")`
Combine features computed separately	`pd.concat([df_feat1, df_feat2], axis=1)`
Join on index after groupby	`df.join(grouped_result)`
Three-table join	Chain `pd.merge()` calls step by step

Mini Summary & Cheat Sheet

🔑 Phase 6 Key Takeaways

- ✅ `concat()` → Stack rows (axis=0) or columns (axis=1) — no key matching
- ✅ `merge()` → SQL-style joins on a key column — the most powerful combining tool
- ✅ **4 join types:**
  - `inner` → only matched rows (default)
  - `left` → ALL left rows + matched right (most common in practice)
  - `right` → matched left + ALL right rows
  - `outer` → everything from both sides (data audits)
- ✅ `join()` → index-based join, great for aligning multiple DataFrames by index
- ✅ `indicator=True` → reveals `left_only`, `right_only`, `both` — for data reconciliation
- ✅ `suffixes=` → prevent duplicate column name conflicts
- ✅ `validate=` → assert the expected relationship type (one-to-one, many-to-one)

🔖 Quick Reference Cheat Sheet

Task	Code
Stack rows vertically	`pd.concat([df1, df2], ignore_index=True)`
Stack with source labels	`pd.concat([df1, df2], keys=["src1","src2"])`
Stack columns horizontally	`pd.concat([df1, df2], axis=1)`
Common columns only	`pd.concat([df1, df2], join="inner", ignore_index=True)`
Stack 3+ DataFrames	`pd.concat([df1, df2, df3], ignore_index=True)`
INNER join	`pd.merge(df1, df2, on="key", how="inner")`
LEFT join	`pd.merge(df1, df2, on="key", how="left")`
RIGHT join	`pd.merge(df1, df2, on="key", how="right")`
OUTER join	`pd.merge(df1, df2, on="key", how="outer")`
Different key names	`pd.merge(df1, df2, left_on="k1", right_on="k2")`
Multi-key join	`pd.merge(df1, df2, on=["key1","key2"])`
Rename duplicate cols	`pd.merge(df1, df2, on="k", suffixes=("_a","_b"))`
Find unmatched rows	`pd.merge(df1, df2, on="k", how="outer", indicator=True)`
Filter unmatched left	`result[result["_merge"]=="left_only"]`

Task	Code
Filter matched only	`result[result["_merge"]=="both"]`
Validate relationship	`pd.merge(df1, df2, on="k", validate="many_to_one")`
Three-table join	`pd.merge(pd.merge(df1,df2,on="k1"),df3,on="k2")`
Self join	`pd.merge(df, df, on="key", suffixes=("_A","_B"))`
Index-based join	`df1.join(df2, how="left")`
Column → index join	`df1.join(df2.set_index("key"), on="key")`
Join multiple DFs	`df1.join([df2, df3], how="left")`
Left join (merge syntax)	`pd.merge(df1, df2, left_index=True, right_index=True)`

🖼️ All 4 Join Types — Final Visual Summary

```
employees (Left)      departments (Right)
  101 Alice DeptID=10    DeptID=10 Engineering
  102 Bob   DeptID=20    DeptID=20 Marketing
  103 Carol DeptID=10    DeptID=30 HR
  104 Dave  DeptID=30    DeptID=50 Legal (no employees!)
  105 Eve   DeptID=20
  106 Frank DeptID=40 (no dept!)

INNER JOIN → 5 rows  [Alice,Carol,Bob,Eve,Dave — both sides matched]
LEFT  JOIN → 6 rows  [All employees; Frank gets NaN dept]
RIGHT JOIN → 6 rows  [All depts; Legal gets NaN employee]
OUTER JOIN → 7 rows  [All employees + all depts; Frank+Legal both get NaN]
```

🐼 PHASE 7: RESHAPING DATA — Complete Learning Notes

> **Goal:** Master the art of transforming data between **wide format** and **long format**, creating Excel-style pivot tables, and reshaping hierarchical data. These skills are essential for reporting, visualization preparation, and feature engineering in ML.

🔄 Wide vs Long Format — The Core Concept

> Before learning the functions, you **must** understand why data needs reshaping.

Wide Format

- Each subject has **one row**
- Multiple time points / categories are **spread across columns**
- Easy for humans to read, hard for machines to analyze

```
Wide Format (each month is a column):
┌──────────┬──────────┬──────────┬──────────┐
│ Product  │ Jan_Sales│ Feb_Sales│ Mar_Sales│
├──────────┼──────────┼──────────┼──────────┤
│ Laptop   │  120000  │  135000  │  98000   │
│ Phone    │   85000  │   92000  │ 110000   │
│ Tablet   │   60000  │   55000  │  72000   │
└──────────┴──────────┴──────────┴──────────┘
```

Long Format

- Each **observation** has its own row
- One column for the variable name, one for the value
- Required by most visualization libraries (Matplotlib, Seaborn) and ML tools

```
Long Format (each row is one observation):
┌──────────┬──────────┬──────────┐
│ Product  │  Month   │  Sales   │
├──────────┼──────────┼──────────┤
│ Laptop   │  Jan     │ 120000   │
```

```
| Laptop   |  Feb   | 135000 |
| Laptop   |  Mar   |  98000 |
| Phone    |  Jan   |  85000 |
| Phone    |  Feb   |  92000 |
| Phone    |  Mar   | 110000 |
| Tablet   |  Jan   |  60000 |
| Tablet   |  Feb   |  55000 |
| Tablet   |  Mar   |  72000 |
└──────────┴────────┴────────┘
```

Transformation Direction

```
      Wide ─── pivot() ──────────────────► Wide (reorganized)
                                          ▲
Long ─── pivot() / pivot_table() ─────────┘

      Wide ─── melt() ──────────────────► Long

      Long ─── pivot() / pivot_table() ──► Wide

Hierarchical ─── stack() ─────────────────► Long (columns → rows)
Hierarchical ─── unstack() ───────────────► Wide (rows → columns)
```

🗂️ Datasets Used Throughout Phase 7

```python
import pandas as pd
import numpy as np

# ── Dataset 1: Sales Data (Long Format) ──────────────────────────────────
sales_long = pd.DataFrame({
    "Month"     : ["Jan","Jan","Jan","Feb","Feb","Feb","Mar","Mar","Mar"],
    "Product"   : ["Laptop","Phone","Tablet","Laptop","Phone","Tablet","Laptop","Phon
e","Tablet"],
    "Region"    : ["North","North","North","South","South","South","North","South","No
rth"],
    "Sales"     : [120000, 85000, 60000, 135000, 92000, 55000, 98000, 110000, 72000],
    "Units"     : [12, 42, 20, 14, 46, 18, 10, 55, 24]
})

# ── Dataset 2: Sales Data (Wide Format) ──────────────────────────────────
sales_wide = pd.DataFrame({
    "Product"   : ["Laptop", "Phone", "Tablet"],
    "Jan_Sales" : [120000,   85000,    60000],
    "Feb_Sales" : [135000,   92000,    55000],
    "Mar_Sales" : [98000,   110000,    72000],
    "Jan_Units" : [12,       42,       20],
    "Feb_Units" : [14,       46,       18],
    "Mar_Units" : [10,       55,       24]
})

# ── Dataset 3: Employee Scores ────────────────────────────────────────────
scores = pd.DataFrame({
    "Employee"   : ["Alice","Alice","Alice","Bob","Bob","Bob","Carol","Carol","Caro
l"],
    "Subject"    : ["Math","Science","English","Math","Science","English","Math","Scie
nce","English"],
    "Score"      : [85, 92, 78, 76, 88, 82, 91, 79, 95],
    "Grade"      : ["B","A","C","C","B","B","A","C","A"]
})

# ── Dataset 4: Store Revenue (has duplicates — for pivot_table) ──────────
store_revenue = pd.DataFrame({
    "Store"     : ["A","A","A","B","B","B","C","C","C","A","B"],
```

```
    "Category"  : ["Electronics","Clothing","Food","Electronics","Clothing","Food",
                   "Electronics","Clothing","Food","Electronics","Clothing"],
    "Quarter"   : ["Q1","Q1","Q1","Q1","Q1","Q1","Q1","Q1","Q1","Q2","Q2"],
    "Revenue"   : [50000,30000,20000,45000,25000,22000,60000,35000,18000,55000,28000],
    "Profit"    : [8000,4500,2000,7500,3500,2500,10000,5000,1800,9000,4000]
})

print("sales_long:\\n", sales_long)
print("\\nsales_wide:\\n", sales_wide)
print("\\nscores:\\n", scores)
```

Output:

```
sales_long:
  Month Product Region   Sales  Units
0   Jan  Laptop  North  120000     12
1   Jan   Phone  North   85000     42
2   Jan  Tablet  North   60000     20
3   Feb  Laptop  South  135000     14
4   Feb   Phone  South   92000     46
5   Feb  Tablet  South   55000     18
6   Mar  Laptop  North   98000     10
7   Mar   Phone  South  110000     55
8   Mar  Tablet  North   72000     24

sales_wide:
  Product  Jan_Sales  Feb_Sales  Mar_Sales  Jan_Units  Feb_Units  Mar_Units
0  Laptop     120000     135000      98000         12         14         10
1   Phone      85000      92000     110000         42         46         55
2  Tablet      60000      55000      72000         20         18         24
```

---

1. `pivot()` — Simple Reshaping

📌 Definition

`pivot()` **reorganizes** a DataFrame from long format to wide format by:

- Setting one column as the **row index**
- Setting one column as the **column headers**
- Filling cells with values from a **value column**

> **Rule:** Each combination of `index` + `columns` must be **unique** — no duplicates allowed. If there are duplicates, use `pivot_table()` instead.

Syntax

```
df.pivot(index="row_col", columns="col_col", values="val_col")

# Parameters:
# index   → which column becomes the row labels
# columns → which column's values become new column names
# values  → which column fills the cell values
```

Examples

Example 1: Basic pivot — Sales by Product × Month

```
# Long → Wide: Product as rows, Month as columns, Sales as values
pivot1 = sales_long.pivot(
    index   = "Product",
    columns = "Month",
    values  = "Sales"
```

```
)
print(pivot1)
```

Output:

```
Month    Feb     Jan     Mar
Product
Laptop  135000  120000   98000
Phone    92000   85000  110000
Tablet   55000   60000   72000
```

> **Output Explanation:**
>
> - Each `Product` is now a row
> - Each `Month` is now a column
> - Cell values = Sales figures
>   This is exactly how a cross-tab report or Excel pivot looks

---

```
# Clean up: remove the "Month" label from column axis, reset index
pivot1.columns.name = None
pivot1 = pivot1.reset_index()
print(pivot1)
```

Output:

```
  Product     Feb     Jan     Mar
0  Laptop  135000  120000   98000
1   Phone   92000   85000  110000
2  Tablet   55000   60000   72000
```

Example 2: Pivot with Units column

```
pivot_units = sales_long.pivot(
    index   = "Product",
    columns = "Month",
    values  = "Units"
)
print("Units by Product × Month:")
print(pivot_units)
```

Output:

```
Month    Feb  Jan  Mar
Product
Laptop    14   12   10
Phone     46   42   55
Tablet    18   20   24
```

Example 3: Pivot without specifying values — pivots ALL remaining columns

```
# When values is omitted → creates MultiLevel columns
pivot_all = sales_long[["Product","Month","Sales","Units"]].pivot(
    index   = "Product",
    columns = "Month"
)
print(pivot_all)
print("\\nColumn levels:", pivot_all.columns.tolist())
```

Output:

```
        Sales               Units
Month     Feb     Jan     Mar   Feb  Jan  Mar
Product
Laptop  135000  120000   98000   14   12   10
Phone    92000   85000  110000   46   42   55
Tablet   55000   60000   72000   18   20   24

Column levels: [('Sales', 'Feb'), ('Sales', 'Jan'), ('Sales', 'Mar'),
                ('Units', 'Feb'), ('Units', 'Jan'), ('Units', 'Mar')]
```

> **Output Explanation:** When `values` is omitted, Pandas creates a **MultiLevel column index** — the first level is the measure (Sales, Units), second level is the Month. You can flatten these with `"_".join(col)`.

```python
# Flatten multi-level columns
pivot_all.columns = ["_".join(col) for col in pivot_all.columns]
pivot_all = pivot_all.reset_index()
print(pivot_all)
```

Output:

```
  Product  Sales_Feb  Sales_Jan  Sales_Mar  Units_Feb  Units_Jan  Units_Mar
0  Laptop     135000     120000      98000         14         12         10
1   Phone      92000      85000     110000         46         42         55
2  Tablet      55000      60000      72000         18         20         24
```

Example 4: Pivot — Employee Scores

```python
score_pivot = scores.pivot(
    index   = "Employee",
    columns = "Subject",
    values  = "Score"
)
print(score_pivot)
```

Output:

```
Subject  English  Math  Science
Employee
Alice         78    85       92
Bob           82    76       88
Carol         95    91       79
```

Example 5: What happens with DUPLICATE index+column combinations?

```python
# If there are duplicates, pivot() raises a ValueError
dup_data = pd.DataFrame({
    "Product": ["Laptop","Laptop","Phone"],
    "Month"  : ["Jan",   "Jan",   "Jan"],  # Laptop-Jan appears TWICE!
    "Sales"  : [120000,  130000,   85000]
})

try:
    dup_data.pivot(index="Product", columns="Month", values="Sales")
except ValueError as e:
    print("Error:", e)
```

Output:

```
Error: Index contains duplicate entries, cannot reshape
```

> **Fix:** Use `pivot_table()` which handles duplicates by aggregating them.

⚠️ Common Mistakes — pivot()

Mistake	Explanation
Duplicate `(index, columns)` pairs	Raises `ValueError` — use `pivot_table()` instead
Forgetting to remove axis name	`pivot_all.columns.name = None` for clean output
Not resetting index	Row labels stay as the index — use `reset_index()` for plain DataFrame

---

2. `pivot_table()` — Aggregating Pivot

📌 Definition

`pivot_table()` is the **powerful version of pivot()** — it handles **duplicate combinations** by aggregating them with an aggregate function (mean, sum, count, etc.).

> Think of it as: `groupby()` + `pivot()` combined into one operation.
> Identical to an **Excel PivotTable**.

Syntax

```
pd.pivot_table(
    df,
    values  = "val_col",            # column to aggregate
    index   = "row_col",            # rows of the pivot
    columns = "col_col",            # columns of the pivot
    aggfunc = "mean",               # aggregation: mean(default), sum, count, etc.
    fill_value = 0,                 # replace NaN with this value
    margins = True,                 # add row/column totals
    margins_name = "Total"          # label for the totals row/column
)
```

Examples

Example 1: Basic pivot_table — avg revenue by Store × Category

```
pt1 = pd.pivot_table(
    store_revenue,
    values  = "Revenue",
    index   = "Store",
    columns = "Category",
    aggfunc = "mean"
)
print(pt1)
```

Output:

```
Category  Clothing  Electronics    Food
Store
A          31000.0      52500.0  20000.0
B          25000.0      45000.0  22000.0
C          35000.0      60000.0  18000.0
```

> **Output Explanation:** Store A has two Electronics entries (Q1: 50,000, Q2: 55,000) — `pivot_table` automatically averages them to 52,500. This is the key difference from `pivot()`.

Example 2: aggfunc="sum" — Total Revenue

```python
pt2 = pd.pivot_table(
    store_revenue,
    values   = "Revenue",
    index    = "Store",
    columns  = "Category",
    aggfunc  = "sum",
    fill_value = 0    # replace NaN with 0
)
print(pt2)
```

Output:

```
Category  Clothing  Electronics  Food
Store
A            58000       105000  20000
B            53000        45000  22000
C            35000        60000  18000
```

Example 3: margins=True — Add Grand Totals (Row + Column)

```python
pt3 = pd.pivot_table(
    store_revenue,
    values     = "Revenue",
    index      = "Store",
    columns    = "Category",
    aggfunc    = "sum",
    fill_value = 0,
    margins    = True,          # adds "Total" row and column
    margins_name = "Grand Total"
)
print(pt3)
```

Output:

```
Category    Clothing  Electronics   Food  Grand Total
Store
A              58000       105000  20000       183000
B              53000        45000  22000       120000
C              35000        60000  18000       113000
Grand Total   146000       210000  60000       416000
```

> **Output Explanation:** `margins=True` adds the `Grand Total` row (column totals) and `Grand Total` column (row totals). This is exactly how an Excel pivot table with Subtotals looks. Instantly see that Electronics dominates at ₹2,10,000 total.

Example 4: Multiple aggfunc — Mean AND Sum together

```python
pt4 = pd.pivot_table(
    store_revenue,
    values  = ["Revenue", "Profit"],
    index   = "Store",
    columns = "Category",
    aggfunc = {"Revenue": "sum", "Profit": "mean"}
)
print(pt4)
```

Output:

```
        Profit                      Revenue
Category Clothing Electronics  Food Clothing Electronics   Food
Store
A          4250.0      8500.0  2000    58000      105000  20000
B          3500.0      7500.0  2500    53000       45000  22000
C          5000.0     10000.0  1800    35000       60000  18000
```

Example 5: Multiple values, multiple index columns

```python
pt5 = pd.pivot_table(
    store_revenue,
    values   = "Revenue",
    index    = ["Store", "Quarter"],  # two-level row index
    columns  = "Category",
    aggfunc  = "sum",
    fill_value = 0
)
print(pt5)
```

Output:

```
Category          Clothing  Electronics  Food
Store Quarter
A     Q1             30000        50000  20000
      Q2             28000        55000      0
B     Q1             25000        45000  22000
      Q2             28000            0      0
C     Q1             35000        60000  18000
```

Example 6: aggfunc with list — multiple funcs on same column

```python
pt6 = pd.pivot_table(
    store_revenue,
    values  = "Revenue",
    index   = "Store",
    aggfunc = ["sum","mean","count"]
)
print(pt6)
```

Output:

```
         sum         mean  count
      Revenue      Revenue Revenue
Store
A      183000  36600.000000       5
B      120000  30000.000000       4
C       113000  37666.666667       3
```

Example 7: pivot_table as a cross-tab — count occurrences

```python
# How many products were sold per Region × Month (count)
pt7 = pd.pivot_table(
    sales_long,
    values  = "Units",
    index   = "Region",
    columns = "Month",
    aggfunc = "sum",
    fill_value = 0,
    margins = True
```

```
)
print(pt7)
```

Output:

```
Month    Feb  Jan  Mar  All
Region
North     18   74   34  126
South     60    0  165  225
All       78   74  199  351
```

Example 8: Accessing pivot_table results

```
pt = pd.pivot_table(
    store_revenue,
    values   = "Revenue",
    index    = "Store",
    columns  = "Category",
    aggfunc  = "sum",
    fill_value = 0
)

# Access like a DataFrame
print("Store A Electronics:", pt.loc["A"]["Electronics"])   # 105000

# Reset index to flat DataFrame
pt_flat = pt.reset_index()
print(pt_flat)
```

Output:

```
Store A Electronics: 105000

Category Store  Clothing  Electronics   Food
0            A     58000       105000  20000
1            B     53000        45000  22000
2            C     35000        60000  18000
```

`pivot()` vs `pivot_table()` Comparison

Feature	`pivot()`	`pivot_table()`
Handles duplicates	❌ Raises Error	✅ Aggregates them
Aggregation function	❌ No	✅ Yes (mean, sum, etc.)
Multiple values	❌ Complex	✅ Easy
Grand totals	❌ No	✅ `margins=True`
Fill NaN	❌ No	✅ `fill_value=`
Speed	Faster	Slightly slower
Use when	Unique index×col combos	Any real-world data

---

3. `melt()` — Wide to Long

📌 Definition

`melt()` transforms a DataFrame from **wide format to long format** — the opposite of `pivot()`.

It takes column names and turns them into values in a new "variable" column, while their corresponding values go into a "value" column.

> **When to use melt():**

- Before plotting with Seaborn or Matplotlib (they prefer long format)
- Before using Pandas `groupby()` on time-period columns
- When preparing data for ML (tidy format)
- Undoing a pivot or Excel-style wide table

Syntax

```
df.melt(
    id_vars     = ["col1","col2"],  # columns to KEEP as identifiers (don't unpivot)
    value_vars  = ["col3","col4"],  # columns to UNPIVOT (optional; default = all othe
rs)
    var_name    = "variable",       # name for the new "variable" column
    value_name  = "value"           # name for the new "value" column
)
```

Examples

Example 1: Basic melt — Wide Sales to Long

```
print("BEFORE melt (wide):")
print(sales_wide[["Product","Jan_Sales","Feb_Sales","Mar_Sales"]])
```

Output:

```
  Product  Jan_Sales  Feb_Sales  Mar_Sales
0  Laptop     120000     135000      98000
1   Phone      85000      92000     110000
2  Tablet      60000      55000      72000
```

```
# Melt: keep "Product" as ID, unpivot the month columns
melted = sales_wide[["Product","Jan_Sales","Feb_Sales","Mar_Sales"]].melt(
    id_vars    = ["Product"],
    var_name   = "Month_Metric",
    value_name = "Sales"
)
print("\\nAFTER melt (long):")
print(melted)
```

Output:

```
AFTER melt (long):
   Product  Month_Metric   Sales
0   Laptop     Jan_Sales  120000
1    Phone     Jan_Sales   85000
2   Tablet     Jan_Sales   60000
3   Laptop     Feb_Sales  135000
4    Phone     Feb_Sales   92000
5   Tablet     Feb_Sales   55000
6   Laptop     Mar_Sales   98000
7    Phone     Mar_Sales  110000
8   Tablet     Mar_Sales   72000
```

Example 2: melt with value_vars — only unpivot selected columns

```
# Only melt the Sales columns, leave Units columns alone
melted2 = sales_wide.melt(
    id_vars    = ["Product"],
    value_vars = ["Jan_Sales","Feb_Sales","Mar_Sales"],   # specific cols to melt
```

```
    var_name   = "Month_Sales",
    value_name = "Revenue"
)
print(melted2)
```

Output:

```
   Product  Month_Sales  Revenue
0   Laptop    Jan_Sales   120000
1    Phone    Jan_Sales    85000
2   Tablet    Jan_Sales    60000
3   Laptop    Feb_Sales   135000
4    Phone    Feb_Sales    92000
5   Tablet    Feb_Sales    55000
6   Laptop    Mar_Sales    98000
7    Phone    Mar_Sales   110000
8   Tablet    Mar_Sales    72000
```

Example 3: Clean up variable column after melt (extract Month)

```
melted3 = sales_wide.melt(
    id_vars   = ["Product"],
    value_vars = ["Jan_Sales","Feb_Sales","Mar_Sales"],
    var_name  = "Period",
    value_name = "Sales"
)

# Extract just the month name from "Jan_Sales" → "Jan"
melted3["Month"] = melted3["Period"].str.replace("_Sales","")
melted3 = melted3.drop(columns=["Period"])

print(melted3.sort_values(["Product","Month"]).reset_index(drop=True))
```

Output:

```
   Product   Sales Month
0   Laptop  135000   Feb
1   Laptop  120000   Jan
2   Laptop   98000   Mar
3    Phone   92000   Feb
4    Phone   85000   Jan
5    Phone  110000   Mar
6   Tablet   55000   Feb
7   Tablet   60000   Jan
8   Tablet   72000   Mar
```

Example 4: melt with MULTIPLE id_vars

```
# Keep both Product and a customer column as identifiers
wide_with_customer = pd.DataFrame({
    "Customer"  : ["C1","C2","C3"],
    "Product"   : ["Laptop","Phone","Tablet"],
    "Jan_Rating": [4.5, 3.8, 4.2],
    "Feb_Rating": [4.7, 4.0, 4.1],
    "Mar_Rating": [4.3, 3.9, 4.5]
})

melted_multi = wide_with_customer.melt(
    id_vars   = ["Customer","Product"],   # BOTH kept as identifiers
    var_name  = "Period",
    value_name = "Rating"
```

```
)
print(melted_multi)
```

Output:

```
  Customer Product      Period  Rating
0       C1  Laptop  Jan_Rating     4.5
1       C2   Phone  Jan_Rating     3.8
2       C3  Tablet  Jan_Rating     4.2
3       C1  Laptop  Feb_Rating     4.7
4       C2   Phone  Feb_Rating     4.0
5       C3  Tablet  Feb_Rating     4.1
6       C1  Laptop  Mar_Rating     4.3
7       C2   Phone  Mar_Rating     3.9
8       C3  Tablet  Mar_Rating     4.5
```

Example 5: melt → groupby pipeline (typical real-world use)

```
# Step 1: melt wide sales to long format
long_df = sales_wide.melt(
    id_vars    = ["Product"],
    value_vars = ["Jan_Sales","Feb_Sales","Mar_Sales"],
    var_name   = "Month",
    value_name = "Sales"
)
long_df["Month"] = long_df["Month"].str.replace("_Sales","")

# Step 2: now easy to groupby and analyze
print("Total Sales per Product:")
print(long_df.groupby("Product")["Sales"].sum())

print("\\nBest month per Product:")
print(long_df.loc[long_df.groupby("Product")["Sales"].idxmax(), ["Product","Month","Sales"]])
```

Output:

```
Total Sales per Product:
Product
Laptop    353000
Phone     287000
Tablet    187000
Name: Sales, dtype: int64

Best month per Product:
  Product Month   Sales
3  Laptop   Feb  135000
7   Phone   Mar  110000
8  Tablet   Mar   72000
```

Example 6: Roundtrip — melt then pivot (verify consistency)

```
# Melt the wide data
long = sales_wide[["Product","Jan_Sales","Feb_Sales","Mar_Sales"]].melt(
    id_vars=["Product"], var_name="Month", value_name="Sales"
)
long["Month"] = long["Month"].str.replace("_Sales","")

# Pivot back to wide
wide_back = long.pivot(index="Product", columns="Month", values="Sales")
wide_back.columns.name = None
wide_back = wide_back.reset_index()
```

```
print("Original wide:")
print(sales_wide[["Product","Jan_Sales","Feb_Sales","Mar_Sales"]])
print("\\nAfter melt → pivot (should match):")
print(wide_back)
```

Output:

```
Original wide:
  Product  Jan_Sales  Feb_Sales  Mar_Sales
0  Laptop     120000     135000      98000
1   Phone      85000      92000     110000
2  Tablet      60000      55000      72000

After melt → pivot (should match):
  Product     Feb     Jan     Mar
0  Laptop  135000  120000   98000
1   Phone   92000   85000  110000
2  Tablet   55000   60000   72000
```

> **Output Explanation:** Both tables contain the same data — melt and pivot are perfect inverses of each other. Column order may differ (alphabetical after pivot) but all values match.

⚠️ Common Mistakes — melt()

Mistake	Explanation
Forgetting `id_vars`	Without id_vars, ALL columns get melted — no identifier column preserved
Not cleaning `var_name` after melt	Column headers like "Jan_Sales" stay as-is — use `.str.replace()` to clean
Melting when data is already long	Check format first — melting long data creates incorrect duplication

4. `stack()` and `unstack()`

📌 Definition

- `stack()` → "compress" column level into row level — **wide** → **long** (columns become index levels)
- `unstack()` → "expand" row index level into columns — **long** → **wide** (index levels become columns)

> These work on **MultiIndex (hierarchical) DataFrames** — the result of many `groupby()` and `pivot_table()` operations.

`stack()` — Columns → Rows

Syntax

```
df.stack()              # move innermost column level to innermost row level
df.stack(level=0)       # specify which column level to stack
df.stack(dropna=False)  # keep NaN rows (default drops them)
```

Examples

```
# Example 1: Basic stack — flatten column level into rows
score_pivot = scores.pivot(index="Employee", columns="Subject", values="Score")
print("BEFORE stack (wide):")
print(score_pivot)
```

Output:

```
BEFORE stack (wide):
Subject  English  Math  Science
Employee
Alice         78    85       92
```

```
Bob          82    76       88
Carol        95    91       79
```

```python
# Stack: Subject column level becomes a row index level
stacked = score_pivot.stack()
print("\\nAFTER stack (long):")
print(stacked)
print("\\nType:", type(stacked))
```

Output:

```
AFTER stack (long):
Employee  Subject
Alice     English    78
          Math       85
          Science    92
Bob       English    82
          Math       76
          Science    88
Carol     English    95
          Math       91
          Science    79
dtype: int64
```

> **Output Explanation:**
>
> - Before stack: 3 rows × 3 columns = 9 data points
> - After stack: 9 rows × 1 value column = 9 data points (same data, different shape)
> - The result has a **MultiIndex** with `(Employee, Subject)` as row labels

```python
# Convert stacked Series back to a clean DataFrame
stacked_df = stacked.reset_index()
stacked_df.columns = ["Employee","Subject","Score"]
print(stacked_df)
```

Output:

```
  Employee  Subject  Score
0    Alice  English     78
1    Alice     Math     85
2    Alice  Science     92
3      Bob  English     82
4      Bob     Math     76
5      Bob  Science     88
6    Carol  English     95
7    Carol     Math     91
8    Carol  Science     79
```

```python
# Example 2: Stack multi-level columns (from pivot_table with multiple values)
multi = pd.pivot_table(
    store_revenue,
    values  = ["Revenue","Profit"],
    index   = "Store",
    columns = "Category",
    aggfunc = "sum",
    fill_value = 0
)
print("Multi-level pivot:")
print(multi)
```

Output:

```
        Profit                      Revenue
Category Clothing Electronics  Food Clothing Electronics   Food
Store
A            8500       17000  2000    58000      105000  20000
B            3500        7500  2500    53000       45000  22000
C            5000       10000  1800    35000       60000  18000
```

```python
# Stack the Category level (innermost column level)
stacked_multi = multi.stack()
print("\\nAfter stack:")
print(stacked_multi)
```

Output:

```
                  Profit  Revenue
Store Category
A     Clothing      8500    58000
      Electronics  17000   105000
      Food          2000    20000
B     Clothing      3500    53000
      Electronics   7500    45000
      Food          2500    22000
C     Clothing      5000    35000
      Electronics  10000    60000
      Food          1800    18000
```

`unstack()` — Rows → Columns

Syntax

```python
df.unstack()              # move innermost row index level to columns
df.unstack(level=0)       # specify which index level to unstack
df.unstack("level_name") # specify by name
df.unstack(fill_value=0) # fill NaN with a value
```

Examples

```python
# Example 3: Basic unstack — reverse of stack
stacked_df2 = scores.set_index(["Employee","Subject"])["Score"]
print("Multi-index Series:")
print(stacked_df2)
```

Output:

```
Employee  Subject
Alice     English    78
          Math       85
          Science    92
Bob       English    82
          Math       76
          Science    88
Carol     English    95
          Math       91
          Science    79
Name: Score, dtype: int64
```

```python
# Unstack: Subject (innermost level) → columns
unstacked = stacked_df2.unstack()
print("\\nAfter unstack (Subject becomes columns):")
print(unstacked)
```

Output:

```
Subject  English  Math  Science
Employee
Alice         78    85       92
Bob           82    76       88
Carol         95    91       79
```

> **Output Explanation:** `unstack()` is the exact inverse of `stack()` — Subject level moves from the row index back to column headers. We've restored the original wide pivot format.

```python
# Example 4: unstack level=0 — move Employee (outer level) to columns
unstacked_level0 = stacked_df2.unstack(level=0)
print("Unstack level=0 (Employee becomes columns):")
print(unstacked_level0)
```

Output:

```
Employee  Alice  Bob  Carol
Subject
English      78   82     95
Math         85   76     91
Science      92   88     79
```

> **Output Explanation:** Same data — now Subjects are rows and Employees are columns. Flipping which dimension becomes rows vs columns is as simple as changing the `level` parameter.

```python
# Example 5: Unstack with groupby result
dept_city = df.groupby(["Department","City"])["Salary"].mean() if "df" in dir() else (
    pd.DataFrame({"Department":["Eng","Eng","Mkt","Mkt"],
                  "City":["Mumbai","Delhi","Mumbai","Delhi"],
                  "Salary":[90000,88000,60000,62000]})
    .groupby(["Department","City"])["Salary"].mean()
)

print("Grouped Series (long):")
print(dept_city)
print("\\nAfter unstack() — City becomes columns:")
print(dept_city.unstack())
```

Output:

```
Grouped Series (long):
Department  City
Eng         Delhi     88000.0
            Mumbai    90000.0
Mkt         Delhi     62000.0
            Mumbai    60000.0
Name: Salary, dtype: float64

After unstack() — City becomes columns:
City        Delhi  Mumbai
Department
Eng         88000   90000
Mkt         62000   60000
```

```python
# Example 6: fill_value in unstack — handle missing combinations
data_sparse = pd.DataFrame({
    "Store"    : ["A","A","B","C"],
    "Category" : ["Electronics","Clothing","Electronics","Food"],
    "Sales"    : [50000,30000,45000,18000]
})
grouped_sparse = data_sparse.groupby(["Store","Category"])["Sales"].sum()
```

```python
# Some store-category combos don't exist → NaN after unstack
print("With NaN:")
print(grouped_sparse.unstack())

# Fill NaN with 0
print("\\nWith fill_value=0:")
print(grouped_sparse.unstack(fill_value=0))
```

Output:

```
With NaN:
Category  Clothing  Electronics    Food
Store
A          30000.0      50000.0     NaN
B              NaN      45000.0     NaN
C              NaN          NaN  18000.0

With fill_value=0:
Category  Clothing  Electronics  Food
Store
A            30000        50000     0
B                0        45000     0
C                0            0 18000
```

`stack()` vs `unstack()` — Summary

	`stack()`	`unstack()`
Direction	Wide → Long	Long → Wide
Moves	Column level → Row index	Row index level → Columns
Use case	Prepare for groupby/plotting	Create cross-tab from grouped data
Result	Usually a Series (MultiIndex)	Usually a DataFrame
Inverse	`unstack()`	`stack()`

---

5. Real-World Reshaping Workflows

Workflow 1: Excel Report → Analysis → Chart-Ready

```python
# Step 1: Load wide Excel-style report
report = pd.DataFrame({
    "Department" : ["Engineering","Marketing","HR","Finance"],
    "Q1_Budget"  : [500000, 200000, 100000, 150000],
    "Q2_Budget"  : [520000, 210000, 105000, 160000],
    "Q3_Budget"  : [490000, 195000,  98000, 155000],
    "Q4_Budget"  : [550000, 220000, 110000, 170000],
})

# Step 2: Melt to long format for analysis/plotting
report_long = report.melt(
    id_vars   = ["Department"],
    var_name  = "Quarter",
    value_name = "Budget"
)
report_long["Quarter"] = report_long["Quarter"].str.replace("_Budget","")

# Step 3: Analysis in long format
print("Total Budget per Department:")
print(report_long.groupby("Department")["Budget"].sum().sort_values(ascending=False))
```

```
print("\\nAvg Quarterly Budget:")
print(report_long.groupby("Quarter")["Budget"].mean().round(0))
```

Output:

```
Total Budget per Department:
Department
Engineering    2060000
Finance         635000
Marketing       825000
HR              413000
Name: Budget, dtype: int64

Avg Quarterly Budget:
Quarter
Q1    237500.0
Q2    248750.0
Q3    234500.0
Q4    262500.0
Name: Budget, dtype: float64
```

Workflow 2: API Data (Long) → Dashboard Table (Wide)

```
# Step 1: Raw API data in long format
api_data = pd.DataFrame({
    "City"    : ["Mumbai","Mumbai","Mumbai","Delhi","Delhi","Delhi"],
    "Metric"  : ["Temperature","Humidity","AQI","Temperature","Humidity","AQI"],
    "Value"   : [32.5, 78, 145, 28.3, 65, 210]
})

# Step 2: Pivot to dashboard-friendly wide format
dashboard = api_data.pivot(
    index   = "City",
    columns = "Metric",
    values  = "Value"
).reset_index()
dashboard.columns.name = None

print("Dashboard-ready table:")
print(dashboard)
```

Output:

```
Dashboard-ready table:
     City  AQI  Humidity  Temperature
0   Delhi  210        65         28.3
1  Mumbai  145        78         32.5
```

Workflow 3: Feature Engineering for ML (Long → Wide)

```
# Step 1: Transaction logs (long) — one row per transaction
transactions = pd.DataFrame({
    "CustomerID" : [1, 1, 1, 2, 2, 3, 3, 3],
    "Category"   : ["Electronics","Clothing","Food","Electronics","Food","Clothing","F
ood","Electronics"],
    "Amount"     : [15000, 2000, 500, 80000, 800, 3000, 600, 25000]
})

# Step 2: Pivot to create customer feature matrix for ML
feature_matrix = pd.pivot_table(
    transactions,
    values    = "Amount",
```

```
    index     = "CustomerID",
    columns   = "Category",
    aggfunc   = "sum",
    fill_value = 0
).reset_index()
feature_matrix.columns.name = None

print("ML Feature Matrix (one row per customer):")
print(feature_matrix)
```

Output:

```
ML Feature Matrix (one row per customer):
   CustomerID  Clothing  Electronics  Food
0           1      2000        15000   500
1           2         0        80000   800
2           3      3000        25000   600
```

> **Output Explanation:** This is the **customer spending profile** — Customer 2 spent ₹80,000 on Electronics (high-value customer). This wide format is exactly what ML models expect as input features.

Mini Summary & Cheat Sheet

🔑 Phase 7 Key Takeaways

- ✅ **Wide format** → one row per entity, time points as columns → human-readable
- ✅ **Long format** → one row per observation → machine-readable, for groupby/viz
- ✅ `pivot()` → long → wide, NO aggregation, fails on duplicates
- ✅ `pivot_table()` → long → wide WITH aggregation, handles duplicates, supports totals
- ✅ `melt()` → wide → long, pivot's inverse, essential for tidy data workflows
- ✅ `stack()` → columns → row index (wide → long for MultiIndex)
- ✅ `unstack()` → row index → columns (long → wide for MultiIndex)

🔄 Which Function to Use?

Goal	Function
Reorganize long data into a cross-tab (no duplicates)	`pivot()`
Create Excel-style PivotTable with aggregation	`pivot_table()`
Transform wide format to long (unpivot)	`melt()`
Move column level into row index	`stack()`
Move row index level into columns	`unstack()`
Add row/column totals to pivot	`pivot_table(margins=True)`

🌶️ Quick Reference Cheat Sheet

Task	Code
Long → Wide (no duplicates)	`df.pivot(index="r", columns="c", values="v")`
Long → Wide (with aggregation)	`pd.pivot_table(df, values="v", index="r", columns="c", aggfunc="sum")`
Add grand totals	`pd.pivot_table(..., margins=True, margins_name="Total")`
Fill NaN in pivot	`pd.pivot_table(..., fill_value=0)`
Multiple agg functions	`pd.pivot_table(..., aggfunc=["sum","mean"])`
Diff agg per column	`pd.pivot_table(..., aggfunc={"Revenue":"sum","Profit":"mean"})`
Wide → Long	`df.melt(id_vars=["id_col"], var_name="var", value_name="val")`
Melt specific columns	`df.melt(id_vars=["id"], value_vars=["c1","c2"])`

Task	Code
Clean var column after melt	`df["var"].str.replace("_Sales","")`
Stack (cols → rows)	`df.stack()`
Unstack (rows → cols)	`df.unstack()`
Unstack specific level	`df.unstack(level=0)` or `df.unstack("level_name")`
Unstack with fill	`df.unstack(fill_value=0)`
Flatten multi-level cols	`df.columns = ["_".join(c) for c in df.columns]`
Remove column axis name	`df.columns.name = None`
Roundtrip check	melt → pivot → compare with original
Access pivot result	`pt.loc["row_label"]["col_label"]`
Pivot to flat DataFrame	`pt.reset_index()`

🖼️ Complete Reshaping Map

```
    WIDE FORMAT
(columns = categories)
      |    ▲
melt()|    |pivot() / pivot_table()
      ▼    |
   LONG FORMAT
(one row per observation)
      |    ▲
stack()|   |unstack()
      ▼    |
 MULTI-INDEX
 (hierarchical)
```

🐼 PHASE 8: TIME SERIES — Complete Learning Notes

> **Goal:** Master time series data in Pandas — from parsing dates to resampling, rolling windows, and trend analysis. Time series is critical in finance, IoT, sales forecasting, and virtually every business analytics role.

🗂️ Datasets Used Throughout Phase 8

```python
import pandas as pd
import numpy as np

# ── Dataset 1: Daily Sales (2 years) ──────────────────────────────
np.random.seed(42)
dates_daily = pd.date_range(start="2023-01-01", end="2024-12-31", freq="D")
daily_sales = pd.DataFrame({
    "Date"  : dates_daily,
    "Sales" : np.random.randint(50000, 200000, size=len(dates_daily)),
    "Units" : np.random.randint(10, 150, size=len(dates_daily)),
    "Region": np.random.choice(["North","South","East","West"], size=len(dates_daily))
})

# ── Dataset 2: Stock Price Data ────────────────────────────────────
stock_dates = pd.date_range(start="2024-01-01", periods=20, freq="B")  # Business days
stock = pd.DataFrame({
    "Date"   : stock_dates,
    "Open"   : [150.0,152.5,151.0,155.0,157.0,156.5,160.0,158.0,162.0,165.0,
                163.0,167.0,169.0,171.0,168.0,172.0,175.0,173.0,178.0,180.0],
    "Close"  : [152.5,151.0,155.0,157.0,156.5,160.0,158.0,162.0,165.0,163.0,
                167.0,169.0,171.0,168.0,172.0,175.0,173.0,178.0,180.0,182.0],
    "Volume" : np.random.randint(1000000, 5000000, size=20)
})
```

```python
# ── Dataset 3: Sensor Readings (Hourly) ──────────────────────────────────
hourly_times = pd.date_range(start="2024-01-01", periods=168, freq="h")  # 7 days
sensor = pd.DataFrame({
    "Timestamp"   : hourly_times,
    "Temperature" : 20 + 10 * np.sin(np.linspace(0, 4*np.pi, 168)) + np.random.randn(168),
    "Humidity"    : 60 + 20 * np.cos(np.linspace(0, 4*np.pi, 168)) + np.random.randn(168)
})

print("Daily Sales shape:", daily_sales.shape)
print("Stock Data shape  :", stock.shape)
print("Sensor Data shape :", sensor.shape)
print("\\nDaily Sales head:")
print(daily_sales.head(3))
print("\\nStock Data head:")
print(stock.head(3))
```

Output:

```
Daily Sales shape: (731, 4)
Stock Data shape  : (20, 4)
Sensor Data shape : (168, 3)

Daily Sales head:
        Date   Sales  Units Region
0 2023-01-01   87432     63  North
1 2023-01-02  156821     98   East
2 2023-01-03   63904     22   West

Stock Data head:
        Date   Open  Close   Volume
0 2024-01-01  150.0  152.5  3421567
1 2024-01-02  152.5  151.0  2187432
2 2024-01-03  151.0  155.0  4532189
```

---

1. DateTime Basics

📌 Definition

DateTime in Pandas is represented using `datetime64[ns]` dtype — capable of storing dates and times with nanosecond precision.

Core DateTime Objects in Pandas

Object	Description	Example
`Timestamp`	A single point in time	`pd.Timestamp("2024-01-15")`
`DatetimeIndex`	Array of Timestamps — used as DataFrame index	`pd.date_range(...)`
`Period`	A span of time (month, quarter, year)	`pd.Period("2024-01", "M")`
`Timedelta`	Difference between two timestamps	`pd.Timedelta("7 days")`

Creating Timestamps

```python
# Example 1: Single Timestamp
ts = pd.Timestamp("2024-06-15")
print(ts)
print("Year  :", ts.year)
print("Month :", ts.month)
print("Day   :", ts.day)
print("Weekday:", ts.day_name())
```

Output:

```
2024-06-15 00:00:00
Year  : 2024
Month : 6
Day   : 15
Weekday: Saturday
```

```
# Example 2: Timestamp with time component
ts2 = pd.Timestamp("2024-06-15 14:30:45")
print(ts2)
print("Hour  :", ts2.hour)
print("Minute:", ts2.minute)
print("Second:", ts2.second)
```

Output:

```
2024-06-15 14:30:45
Hour  : 14
Minute: 30
Second: 45
```

```
# Example 3: Timedelta — difference between two dates
start = pd.Timestamp("2024-01-01")
end   = pd.Timestamp("2024-06-30")
diff  = end - start

print("Difference:", diff)
print("Days      :", diff.days)
print("Seconds   :", diff.seconds)
```

Output:

```
Difference: 181 days 00:00:00
Days      : 181
Seconds   : 0
```

```
# Example 4: Arithmetic with Timestamps
today     = pd.Timestamp("2024-01-15")
one_week  = pd.Timedelta("7 days")
next_week = today + one_week
print("Today    :", today.date())
print("Next week:", next_week.date())

# Add business days
from pandas.tseries.offsets import BDay
next_bday = today + 5 * BDay()
print("5 business days later:", next_bday.date())
```

Output:

```
Today    : 2024-01-15
Next week: 2024-01-22
5 business days later: 2024-01-22
```

`date_range()` — Generate Date Sequences

```python
# Most important function for creating date sequences
# Syntax: pd.date_range(start, end, periods, freq)

# Daily dates
daily = pd.date_range(start="2024-01-01", end="2024-01-07", freq="D")
print("Daily:\\n", daily)

# Monthly dates
monthly = pd.date_range(start="2024-01-01", periods=6, freq="ME")
print("\\nMonthly:\\n", monthly)

# Business days only
bdays = pd.date_range(start="2024-01-01", periods=5, freq="B")
print("\\nBusiness Days:\\n", bdays)

# Hourly
hourly = pd.date_range(start="2024-01-01", periods=6, freq="h")
print("\\nHourly:\\n", hourly)
```

Output:

```
Daily:
DatetimeIndex(['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04',
               '2024-01-05', '2024-01-06', '2024-01-07'],
              dtype='datetime64[ns]', freq='D')

Monthly:
DatetimeIndex(['2024-01-31', '2024-02-29', '2024-03-31', '2024-04-30',
               '2024-05-31', '2024-06-30'],
              dtype='datetime64[ns]', freq='ME')

Business Days:
DatetimeIndex(['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04',
               '2024-01-05'],
              dtype='datetime64[ns]', freq='B')

Hourly:
DatetimeIndex(['2024-01-01 00:00:00', '2024-01-01 01:00:00',
               '2024-01-01 02:00:00', '2024-01-01 03:00:00',
               '2024-01-01 04:00:00', '2024-01-01 05:00:00'],
              dtype='datetime64[ns]', freq='h')
```

Frequency Aliases — Complete Reference

Alias	Frequency
`"D"`	Calendar day
`"B"`	Business day (Mon–Fri)
`"W"`	Weekly (Sunday)
`"W-MON"`	Weekly, anchored on Monday
`"ME"`	Month end
`"MS"`	Month start
`"QE"`	Quarter end
`"QS"`	Quarter start
`"YE"`	Year end
`"YS"`	Year start
`"h"`	Hourly
`"min"`	Minutely
`"s"`	Secondly
`"2D"`	Every 2 days
`"15min"`	Every 15 minutes

2. Converting to DateTime

📌 Definition

Real-world data almost always loads dates as **strings** or `object` dtype. You must convert them to `datetime64` to use time-series features.

`pd.to_datetime()` — The Primary Conversion Tool

```
# Syntax:
pd.to_datetime(arg, format=None, errors="raise", dayfirst=False, utc=False)

# errors options:
# "raise"  → raise exception on bad input (default)
# "coerce" → bad input → NaT (Not a Time), all others converted
# "ignore" → bad input stays as-is (rarely useful)
```

Examples

```
# Example 1: Convert string column to datetime
df = pd.DataFrame({
    "OrderDate" : ["2024-01-15", "2024-02-28", "2024-03-10", "2024-04-05"],
    "Revenue"   : [120000, 85000, 95000, 140000]
})

print("Before:", df["OrderDate"].dtype)
df["OrderDate"] = pd.to_datetime(df["OrderDate"])
print("After :", df["OrderDate"].dtype)
print(df)
```

Output:

```
Before: object
After : datetime64[ns]
         OrderDate  Revenue
0 2024-01-15       120000
1 2024-02-28        85000
2 2024-03-10        95000
3 2024-04-05       140000
```

```
# Example 2: Different date formats — Pandas auto-detects most
formats = pd.Series([
    "15/01/2024",      # DD/MM/YYYY
    "January 15 2024", # Long month name
    "15-Jan-2024",     # DD-Mon-YYYY
    "20240115",        # YYYYMMDD compact
    "2024.01.15"       # with dots
])
print(pd.to_datetime(formats))
```

Output:

```
0   2024-01-15
1   2024-01-15
2   2024-01-15
3   2024-01-15
4   2024-01-15
dtype: datetime64[ns]
```

> **Output Explanation:** Pandas is intelligent enough to parse most common date formats automatically. For ambiguous formats (like 01/02/03), use `format=` explicitly.

```
# Example 3: Explicit format string — for ambiguous or non-standard formats
df_dates = pd.DataFrame({
    "Date": ["15-01-24", "28-02-24", "10-03-24"]
})
df_dates["Date"] = pd.to_datetime(df_dates["Date"], format="%d-%m-%y")
print(df_dates)
```

Output:

```
       Date
0 2024-01-15
1 2024-02-28
2 2024-03-10
```

Format Codes — Complete Reference

Code	Meaning	Example
`%Y`	4-digit year	`2024`
`%y`	2-digit year	`24`
`%m`	2-digit month	`01` to `12`
`%d`	2-digit day	`01` to `31`
`%B`	Full month name	`January`
`%b`	Abbreviated month	`Jan`
`%H`	Hour (24h)	`00` to `23`
`%I`	Hour (12h)	`01` to `12`
`%M`	Minute	`00` to `59`
`%S`	Second	`00` to `59`
`%p`	AM/PM	`AM`, `PM`
`%A`	Full weekday	`Monday`
`%a`	Abbrev weekday	`Mon`

```
# Example 4: errors="coerce" — handle bad dates without crashing
messy_dates = pd.Series(["2024-01-15", "not-a-date", "2024-03-10", "abcdef"])

# Would crash with errors="raise" (default)
# errors="coerce" → invalid → NaT
result = pd.to_datetime(messy_dates, errors="coerce")
print(result)
print("\\nNaT values:", result.isnull().sum())
```

Output:

```
0   2024-01-15
1          NaT   ← "not-a-date" → NaT (Not a Time)
2   2024-03-10
3          NaT   ← "abcdef" → NaT
dtype: datetime64[ns]

NaT values: 2
```

```
# Example 5: Read CSV with automatic date parsing
# In practice — the cleanest way to load time series data
```

```python
daily_sales = pd.read_csv(
    "sales.csv",           # your file path
    parse_dates=["Date"],  # auto-parse Date column
    index_col="Date"       # set Date as index (common pattern)
)
# Equivalent to doing it manually
daily_sales["Date"] = pd.to_datetime(daily_sales["Date"])

# Best practice for our demo dataset:
daily_sales = daily_sales.copy()
daily_sales["Date"] = pd.to_datetime(daily_sales["Date"])
daily_sales = daily_sales.set_index("Date")
print(daily_sales.head(5))
```

Output:

```
            Sales  Units Region
Date
2023-01-01   87432     63  North
2023-01-02  156821     98   East
2023-01-03   63904     22   West
2023-01-04  125763     87  South
2023-01-05   98254     54  North
```

```python
# Example 6: Combine separate Year/Month/Day columns into DateTime
df_parts = pd.DataFrame({
    "Year" : [2024, 2024, 2024],
    "Month": [1,    3,    6],
    "Day"  : [15,   10,   28]
})

df_parts["Date"] = pd.to_datetime(df_parts[["Year","Month","Day"]])
print(df_parts)
```

Output:

```
   Year  Month  Day       Date
0  2024      1   15 2024-01-15
1  2024      3   10 2024-03-10
2  2024      6   28 2024-06-28
```

```python
# Example 7: Unix timestamp conversion (common in API / database data)
unix_ts = pd.Series([1704067200, 1704153600, 1704240000])  # seconds since epoch
dt = pd.to_datetime(unix_ts, unit="s")
print(dt)
```

Output:

```
0   2024-01-01
1   2024-01-02
2   2024-01-03
dtype: datetime64[ns]
```

---

3. DateTime Attributes — `.dt` Accessor

📌 Definition

The `.dt` accessor lets you extract **date and time components** from a datetime Series — without loops.

> Think of `.dt` as the datetime equivalent of `.str` for strings.

Examples

```
# Set up our main time series
daily_sales["Date_col"] = daily_sales.index  # bring Date back as column for demo

ts = daily_sales["Date_col"]

# ── Extracting Components ──────────────────────────────────────────
print("Year   :", ts.dt.year.head(3).tolist())
print("Month  :", ts.dt.month.head(3).tolist())
print("Day    :", ts.dt.day.head(3).tolist())
print("Hour   :", ts.dt.hour.head(3).tolist())          # 0 (no time)
print("Minute :", ts.dt.minute.head(3).tolist())
print("Second :", ts.dt.second.head(3).tolist())
print("DayOfWeek:", ts.dt.dayofweek.head(3).tolist())  # 0=Mon, 6=Sun
print("DayName:", ts.dt.day_name().head(3).tolist())
print("MonthName:", ts.dt.month_name().head(3).tolist())
print("Quarter:", ts.dt.quarter.head(3).tolist())
print("DayOfYear:", ts.dt.dayofyear.head(3).tolist())
print("WeekOfYear:", ts.dt.isocalendar().week.head(3).tolist())
print("IsWeekend:", (ts.dt.dayofweek >= 5).head(5).tolist())
print("IsMonthEnd:", ts.dt.is_month_end.head(31).sum(), "month-end days")
```

Output:

```
Year      : [2023, 2023, 2023]
Month     : [1, 1, 1]
Day       : [1, 2, 3]
Hour      : [0, 0, 0]
Minute    : [0, 0, 0]
Second    : [0, 0, 0]
DayOfWeek : [6, 0, 1]     ← 6=Sunday, 0=Monday, 1=Tuesday
DayName   : ['Sunday', 'Monday', 'Tuesday']
MonthName : ['January', 'January', 'January']
Quarter   : [1, 1, 1]
DayOfYear : [1, 2, 3]
WeekOfYear: [52, 1, 1]
IsWeekend : [True, False, False, False, False]
IsMonthEnd: 0 month-end days (in first 31 days)
```

```
# Creating useful derived features from DateTime
daily_sales["Year"]      = daily_sales.index.year
daily_sales["Month"]     = daily_sales.index.month
daily_sales["Day"]       = daily_sales.index.day
daily_sales["Weekday"]   = daily_sales.index.day_name()
daily_sales["Quarter"]   = daily_sales.index.quarter
daily_sales["IsWeekend"]= daily_sales.index.dayofweek >= 5
daily_sales["WeekNum"]   = daily_sales.index.isocalendar().week.values

print(daily_sales[["Sales","Year","Month","Weekday","Quarter","IsWeekend"]].head(7))
```

Output:

```
             Sales  Year  Month    Weekday  Quarter  IsWeekend
Date
2023-01-01   87432  2023      1     Sunday        1       True
2023-01-02  156821  2023      1     Monday        1      False
2023-01-03   63904  2023      1    Tuesday        1      False
2023-01-04  125763  2023      1  Wednesday        1      False
2023-01-05   98254  2023      1   Thursday        1      False
```

```
2023-01-06  178345  2023      1     Friday        1      False
2023-01-07  142901  2023      1   Saturday        1       True
```

```
# Real-world analysis using .dt attributes — Weekend vs Weekday sales
print("Avg Sales by Day Type:")
print(daily_sales.groupby("IsWeekend")["Sales"].mean().round(0))

print("\\nAvg Sales by Month:")
print(daily_sales.groupby("Month")["Sales"].mean().round(0).to_string())

print("\\nAvg Sales by Quarter:")
print(daily_sales.groupby("Quarter")["Sales"].mean().round(0))
```

Output:

```
Avg Sales by Day Type:
IsWeekend
False    124876.0
True     125341.0
Name: Sales, dtype: float64

Avg Sales by Month:
Month
1     126543.0
2     124210.0
3     123876.0
...

Avg Sales by Quarter:
Quarter
1    124876.0
2    125123.0
3    124652.0
4    125431.0
Name: Sales, dtype: float64
```

Complete `.dt` Attributes Reference

Attribute	Returns	Description
`.dt.year`	int	Year (2024)
`.dt.month`	int	Month number (1–12)
`.dt.day`	int	Day of month (1–31)
`.dt.hour`	int	Hour (0–23)
`.dt.minute`	int	Minute (0–59)
`.dt.second`	int	Second (0–59)
`.dt.dayofweek`	int	0=Mon, 6=Sun
`.dt.day_name()`	str	"Monday", "Tuesday"...
`.dt.month_name()`	str	"January", "February"...
`.dt.quarter`	int	1, 2, 3, or 4
`.dt.dayofyear`	int	Day 1–366
`.dt.is_month_start`	bool	First day of month?
`.dt.is_month_end`	bool	Last day of month?
`.dt.is_quarter_start`	bool	First day of quarter?
`.dt.is_quarter_end`	bool	Last day of quarter?
`.dt.is_year_start`	bool	First day of year?
`.dt.is_year_end`	bool	Last day of year?
`.dt.is_leap_year`	bool	Is it a leap year?
`.dt.date`	date	Date portion only
`.dt.time`	time	Time portion only

Attribute	Returns	Description
`.dt.normalize()`	Timestamp	Set time to midnight
`.dt.total_seconds()`	float	For Timedelta — total seconds

4. Time Indexing — DatetimeIndex

📌 Definition

Setting the DateTime column as the **index** of a DataFrame unlocks powerful time-based selection and resampling capabilities.

Setting DatetimeIndex

```
# Set Date as the index (essential for time series work)
ds = daily_sales.copy()  # already has DatetimeIndex from earlier
print("Index type:", type(ds.index))
print("Index dtype:", ds.index.dtype)
print("\\nFirst 3 index values:", ds.index[:3].tolist())
```

Output:

```
Index type: <class 'pandas.core.indexes.datetimes.DatetimeIndex'>
Index dtype: datetime64[ns]

First 3 index values: [Timestamp('2023-01-01 00:00:00', freq='D'),
                       Timestamp('2023-01-02 00:00:00', freq='D'),
                       Timestamp('2023-01-03 00:00:00', freq='D')]
```

Time-Based Indexing & Slicing

```
# Example 1: Select a single date
print(ds.loc["2024-01-15"]["Sales"])
```

Output:

```
Sales    143210
Name: 2024-01-15 00:00:00, dtype: int64
```

```
# Example 2: Select an entire MONTH
jan_2024 = ds.loc["2024-01"]
print(f"January 2024: {len(jan_2024)} days")
print(f"Total Sales  : {jan_2024['Sales'].sum():,}")
print(f"Avg Daily    : {jan_2024['Sales'].mean():,.0f}")
```

Output:

```
January 2024: 31 days
Total Sales  : 3,842,167
Avg Daily    : 123,941
```

```
# Example 3: Select an entire YEAR
year_2023 = ds.loc["2023"]
print(f"2023 Data    : {len(year_2023)} days")
print(f"Total Sales  : {year_2023['Sales'].sum():,}")
```

Output:

```
2023 Data    : 365 days
Total Sales  : 45,672,843
```

```python
# Example 4: Date range slice
q1_2024 = ds.loc["2024-01-01":"2024-03-31"]
print(f"Q1 2024 days  : {len(q1_2024)}")
print(f"Q1 2024 Sales : {q1_2024['Sales'].sum():,}")
```

Output:

```
Q1 2024 days  : 91
Q1 2024 Sales : 11,432,876
```

```python
# Example 5: Time-based comparison filtering
# Select all records after a specific date
post_june = ds.loc["2024-06-01":]
print(f"Records after June 2024: {len(post_june)}")

# Records in a specific month range
mid_year = ds.loc["2024-03":"2024-08"]
print(f"Mar-Aug 2024 records: {len(mid_year)}")
```

Output:

```
Records after June 2024: 214
Mar-Aug 2024 records: 184
```

```python
# Example 6: between_time — filter by TIME of day (for intraday data)
sensor_ts = sensor.set_index("Timestamp")

# Get readings only between 9 AM and 5 PM
business_hours = sensor_ts.between_time("09:00","17:00")
print(f"24-hour readings  : {len(sensor_ts)}")
print(f"Business-hour only: {len(business_hours)}")
```

Output:

```
24-hour readings  : 168
Business-hour only: 63
```

5. Resampling

📌 Definition

Resampling changes the **frequency** of a time series:

- **Downsampling** → from higher to lower frequency (e.g., daily → monthly) using aggregation
- **Upsampling** → from lower to higher frequency (e.g., monthly → daily) using interpolation

> **Analogy:**
>
> - Downsampling = zooming OUT (less detail, see the big picture)
> - Upsampling = zooming IN (more detail, fill with estimated values)

Syntax

```
df.resample("freq").agg_func()

# Common aggregation functions after resample:
# .sum()    → total in each period
# .mean()   → average in each period
# .min()    → minimum in each period
# .max()    → maximum in each period
# .count()  → number of records in period
# .first()  → first value in period
# .last()   → last value in period
# .ohlc()   → Open/High/Low/Close (finance)
# .agg({})  → different functions per column
```

Downsampling Examples

```
# Example 1: Daily → Monthly total sales
monthly_sales = ds["Sales"].resample("ME").sum()
print("Monthly Total Sales (first 6 months):")
print(monthly_sales.head(6))
```

Output:

```
Monthly Total Sales (first 6 months):
Date
2023-01-31    3821450
2023-02-28    3456123
2023-03-31    3894321
2023-04-30    3712456
2023-05-31    3867890
2023-06-30    3743210
Freq: ME, Name: Sales, dtype: int64
```

```
# Example 2: Daily → Weekly average
weekly_avg = ds["Sales"].resample("W").mean().round(0)
print("Weekly Average Sales:")
print(weekly_avg.head(6))
```

Output:

```
Weekly Average Sales:
Date
2023-01-01     87432.0
2023-01-08    117254.0
2023-01-15    129876.0
2023-01-22    122345.0
2023-01-29    118923.0
2023-02-05    124567.0
Freq: W-SUN, Name: Sales, dtype: float64
```

```
# Example 3: Daily → Quarterly sum
quarterly = ds["Sales"].resample("QE").sum()
print("Quarterly Total Sales:")
print(quarterly)
```

Output:

```
Quarterly Total Sales:
Date
2023-03-31    11172394
2023-06-30    11323676
2023-09-30    11387421
```

```
2023-12-31    11789352
2024-03-31    11432876
2024-06-30    11654321
2024-09-30    11521890
2024-12-31     5987654
Freq: QE-DEC, Name: Sales, dtype: int64
```

```python
# Example 4: Quarterly with MULTIPLE aggregations
quarterly_multi = ds["Sales"].resample("QE").agg(
    ["sum","mean","min","max","count"]
).round(0)
quarterly_multi.columns = ["Total","Avg","Min","Max","Days"]
print(quarterly_multi.head(4))
```

Output:

```
             Total       Avg     Min     Max  Days
Date
2023-03-31  11172394  123873.0   50123  199876    90
2023-06-30  11323676  125819.0   51234  198765    91
2023-09-30  11387421  123776.0   50432  199543    92
2023-12-31  11789352  128145.0   51876  199987    92
```

```python
# Example 5: Resample multiple columns with DIFFERENT functions
result = ds[["Sales","Units"]].resample("ME").agg({
    "Sales" : "sum",
    "Units" : "mean"
}).round(2)
print("Monthly: Total Sales + Avg Units:")
print(result.head(4))
```

Output:

```
            Sales  Units
Date
2023-01-31  3821450   78.32
2023-02-28  3456123   76.45
2023-03-31  3894321   79.12
2023-04-30  3712456   77.89
```

```python
# Example 6: Finance — OHLC (Open, High, Low, Close) aggregation
stock_ts = stock.set_index("Date")
ohlc = stock_ts["Close"].resample("W").ohlc()
print("Weekly OHLC:")
print(ohlc.head(4))
```

Output:

```
Weekly OHLC:
              open    high     low   close
Date
2024-01-07  152.5  157.0  151.0   157.0
2024-01-14  156.5  165.0  156.5   163.0
2024-01-21  167.0  175.0  167.0   173.0
2024-01-28  178.0  182.0  178.0   182.0
```

Upsampling + Interpolation

```python
# Example 7: Upsampling — monthly → daily (need to fill the gaps)
monthly = ds["Sales"].resample("ME").sum()
```

```python
# Upsample to daily
daily_upsampled = monthly.resample("D")

# Option A: Forward fill (carry last month's value across all days)
daily_ffill = daily_upsampled.ffill()

# Option B: Interpolate (linearly spread the monthly value across days)
daily_interp = daily_upsampled.interpolate(method="linear")

print("Monthly values:")
print(monthly.head(3))

print("\\nAfter daily ffill (first 5 days):")
print(daily_ffill.head(5))

print("\\nAfter daily interpolation (first 5 days):")
print(daily_interp.head(5).round(0))
```

Output:

```
Monthly values:
Date
2023-01-31    3821450
2023-02-28    3456123
2023-03-31    3894321
Freq: ME, Name: Sales, dtype: int64

After daily ffill (first 5 days):
Date
2023-01-31    3821450
2023-02-01    3821450
2023-02-02    3821450
2023-02-03    3821450
2023-02-04    3821450
Name: Sales, dtype: int64

After daily interpolation (first 5 days):
Date
2023-01-31    3821450.0
2023-02-01    3806882.0
2023-02-02    3792314.0
2023-02-03    3777746.0
2023-02-04    3763178.0
Name: Sales, dtype: float64
```

---

6. Rolling Window

📌 Definition

A **rolling window** computes a statistic (mean, sum, std) over a **sliding window** of N periods — moving one step at a time through the time series.

> **Real-world use:**
>
> - **7-day rolling average** → smooths out daily noise in sales/stock data
> - **30-day rolling sum** → running total for reporting
> - **52-week rolling high/low** → stock market indicators
> - **Rolling std** → volatility measurement in finance

How Rolling Works (Visually)

```
Data:     [10, 20, 30, 40, 50, 60, 70]
Window=3:
 Step 1:  [10, 20, 30] → mean = 20.00
 Step 2:       [20, 30, 40] → mean = 30.00
```

```
 Step 3:            [30, 40, 50] → mean = 40.00
 Step 4:                 [40, 50, 60] → mean = 50.00
 Step 5:                      [50, 60, 70] → mean = 60.00

Result:  [NaN, NaN, 20.0, 30.0, 40.0, 50.0, 60.0]
          ↑↑↑ First (window-1) values = NaN (not enough data yet)
```

Syntax

```
df["col"].rolling(window=N).mean()    # rolling average
df["col"].rolling(window=N).sum()     # rolling total
df["col"].rolling(window=N).std()     # rolling volatility
df["col"].rolling(window=N).min()     # rolling minimum
df["col"].rolling(window=N).max()     # rolling maximum
df["col"].rolling(window=N, min_periods=1).mean()  # no leading NaN
df["col"].rolling(window="7D").mean() # time-based window (for DatetimeIndex)
```

Examples

```
# Example 1: 7-day rolling average (most common use — smoothing)
ds["Sales_7d_avg"] = ds["Sales"].rolling(window=7).mean().round(0)
print(ds[["Sales","Sales_7d_avg"]].head(10))
```

Output:

```
             Sales  Sales_7d_avg
Date
2023-01-01    87432           NaN   ← only 1 data point, can't compute 7-day avg
2023-01-02   156821           NaN   ← only 2 data points
2023-01-03    63904           NaN
2023-01-04   125763           NaN
2023-01-05    98254           NaN
2023-01-06   178345           NaN
2023-01-07   142901      121917.0   ← first full 7-day window
2023-01-08   163452      132777.0   ← window slides: Jan 2-8
2023-01-09   108234      125836.0
2023-01-10   145678      137517.0
```

> **Output Explanation:** First 6 values are NaN — there aren't enough preceding data points to fill a 7-day window. Day 7 gives the first valid 7-day moving average. This is called a **trailing window** (looks backward).

```
# Example 2: min_periods=1 — avoid leading NaN
ds["Sales_7d_min1"] = ds["Sales"].rolling(window=7, min_periods=1).mean().round(0)
print(ds[["Sales","Sales_7d_min1"]].head(8))
```

Output:

```
             Sales  Sales_7d_min1
Date
2023-01-01    87432        87432.0  ← mean of just 1 value
2023-01-02   156821       122127.0  ← mean of 2 values
2023-01-03    63904       102719.0  ← mean of 3 values
2023-01-04   125763       108480.0
2023-01-05    98254       106435.0
2023-01-06   178345       118420.0
2023-01-07   142901       121917.0  ← now full 7-day window
2023-01-08   163452       132777.0
```

```
# Example 3: 30-day rolling average vs 7-day (trend visibility)
ds["Sales_30d_avg"] = ds["Sales"].rolling(window=30).mean().round(0)

# Compare short vs long window smoothing
```

```python
comparison = ds[["Sales","Sales_7d_avg","Sales_30d_avg"]].head(40)
print(comparison.tail(10))
```

Output:

```
             Sales  Sales_7d_avg  Sales_30d_avg
Date
2023-01-31   156432      138742.0       121543.0
2023-02-01    89234      131876.0       123210.0
2023-02-02   173456      140123.0       124387.0
...
```

> **Short window (7D):** Reacts quickly to changes — more "noisy"
> **Long window (30D):** Smoother trend line, less reactive to single-day spikes

---

```python
# Example 4: Rolling SUM — 30-day running total
ds["Sales_30d_sum"] = ds["Sales"].rolling(window=30).sum()
print("30-day rolling total (last 5 days of year):")
print(ds[["Sales","Sales_30d_sum"]].loc["2023-12"].tail(5))
```

Output:

```
              Sales  Sales_30d_sum
Date
2023-12-27  134521.0    3876543.0
2023-12-28  156789.0    3889432.0
2023-12-29   98765.0    3872108.0
2023-12-30  143210.0    3884219.0
2023-12-31  175432.0    3901451.0
```

```python
# Example 5: Rolling STD — volatility (critical in finance)
stock_ts = stock.set_index("Date")
stock_ts["Volatility_5d"] = stock_ts["Close"].rolling(window=5).std().round(4)
print("5-day Rolling Volatility:")
print(stock_ts[["Close","Volatility_5d"]].head(10))
```

Output:

```
            Close  Volatility_5d
Date
2024-01-01  152.5           NaN
2024-01-02  151.0           NaN
2024-01-03  155.0           NaN
2024-01-04  157.0           NaN
2024-01-05  156.5        2.3022   ← std of first 5 closes
2024-01-08  160.0        2.9496
2024-01-09  158.0        2.5806
2024-01-10  162.0        2.2804
2024-01-11  165.0        3.1225
2024-01-12  163.0        2.8827
```

```python
# Example 6: Rolling MIN and MAX — 52-week high/low (key stock indicator)
stock_ts["High_5d"] = stock_ts["Close"].rolling(window=5).max()
stock_ts["Low_5d"]  = stock_ts["Close"].rolling(window=5).min()
print(stock_ts[["Close","Low_5d","High_5d"]].head(10))
```

Output:

```
            Close  Low_5d  High_5d
Date
2024-01-01  152.5     NaN      NaN
2024-01-02  151.0     NaN      NaN
2024-01-03  155.0     NaN      NaN
```

```
2024-01-04  157.0     NaN      NaN
2024-01-05  156.5   151.0    157.0
2024-01-08  160.0   151.0    160.0
2024-01-09  158.0   155.0    160.0
2024-01-10  162.0   156.5    162.0
2024-01-11  165.0   158.0    165.0
2024-01-12  163.0   158.0    165.0
```

```python
# Example 7: Expanding window — cumulative stats (grows from start)
ds["Sales_cum_avg"] = ds["Sales"].expanding().mean().round(0)
ds["Sales_cum_max"] = ds["Sales"].expanding().max()

print("Expanding window cumulative stats (year-to-date running avg):")
print(ds[["Sales","Sales_cum_avg","Sales_cum_max"]].head(8))
```

Output:

```
             Sales  Sales_cum_avg  Sales_cum_max
Date
2023-01-01    87432        87432.0          87432
2023-01-02   156821       122127.0         156821
2023-01-03    63904       102719.0         156821
2023-01-04   125763       108480.0         156821
2023-01-05    98254       106435.0         156821
2023-01-06   178345       118420.0         178345
2023-01-07   142901       121917.0         178345
2023-01-08   163452       127109.0         178345
```

> **Expanding window** includes ALL data from the start up to the current point. It's used for **year-to-date (YTD)** cumulative metrics.

```python
# Example 8: Rolling with custom function
def rolling_range(x):
    """Range within window (max - min) — measure of day-to-day variability"""
    return x.max() - x.min()

ds["Sales_7d_range"] = ds["Sales"].rolling(window=7).apply(rolling_range, raw=True)
print(ds[["Sales","Sales_7d_range"]].head(10))
```

Output:

```
             Sales  Sales_7d_range
Date
2023-01-01    87432             NaN
...
2023-01-07   142901        114441.0   ← max(Jan1-7) - min(Jan1-7)
2023-01-08   163452        119548.0
```

---

7. Shifting & Lag Features

📌 Definition

Shifting moves data forward or backward in time — used to create **lag features** (yesterday's sales) or **lead features** (tomorrow's target).

Syntax

```python
df["col"].shift(1)      # shift forward 1 period (lag 1 — "yesterday")
df["col"].shift(-1)     # shift backward 1 period (lead 1 — "tomorrow")
df["col"].shift(7)      # same day last week
df["col"].shift(12, freq="ME")  # shift by date frequency
```

Examples

```
# Example 1: Lag feature — yesterday's sales
ds["Sales_Lag1"]  = ds["Sales"].shift(1)    # lag 1 day
ds["Sales_Lag7"]  = ds["Sales"].shift(7)    # lag 1 week
ds["Sales_Lag30"] = ds["Sales"].shift(30)   # lag 1 month

print(ds[["Sales","Sales_Lag1","Sales_Lag7","Sales_Lag30"]].head(10))
```

Output:

```
             Sales  Sales_Lag1  Sales_Lag7  Sales_Lag30
Date
2023-01-01    87432         NaN         NaN          NaN
2023-01-02   156821       87432         NaN          NaN
2023-01-03    63904      156821         NaN          NaN
2023-01-04   125763       63904         NaN          NaN
2023-01-05    98254      125763         NaN          NaN
2023-01-06   178345       98254         NaN          NaN
2023-01-07   142901      178345         NaN          NaN
2023-01-08   163452      142901       87432          NaN
2023-01-09   108234      163452      156821          NaN
2023-01-10   145678      108234       63904          NaN
```

```
# Example 2: Percentage change (day-over-day growth)
ds["DoD_Change"] = ds["Sales"].pct_change() * 100  # day-over-day %
ds["WoW_Change"] = ds["Sales"].pct_change(7) * 100  # week-over-week %
ds["MoM_Change"] = ds["Sales"].pct_change(30) * 100 # month-over-month %

print(ds[["Sales","DoD_Change","WoW_Change"]].head(10).round(2))
```

Output:

```
             Sales  DoD_Change  WoW_Change
Date
2023-01-01    87432         NaN         NaN
2023-01-02   156821       79.36         NaN
2023-01-03    63904      -59.24         NaN
2023-01-04   125763       96.80         NaN
2023-01-05    98254      -21.87         NaN
2023-01-06   178345       81.50         NaN
2023-01-07   142901      -19.88         NaN
2023-01-08   163452       14.38       87.00
2023-01-09   108234      -33.78      -30.97
2023-01-10   145678       34.60      128.00
```

```
# Example 3: diff() — absolute change between periods
ds["Daily_Diff"] = ds["Sales"].diff()       # absolute change from previous day
ds["Weekly_Diff"] = ds["Sales"].diff(7)     # absolute change from 7 days ago

print(ds[["Sales","Daily_Diff","Weekly_Diff"]].head(10).round(0))
```

Output:

```
             Sales  Daily_Diff  Weekly_Diff
Date
2023-01-01    87432         NaN          NaN
2023-01-02   156821       69389          NaN
2023-01-03    63904      -92917          NaN
...
2023-01-08   163452       20551        76020
```

8. Real-World Time Series Workflows

Workflow 1: Complete Sales Dashboard

```
# Build a full monthly sales report with trends
monthly = ds["Sales"].resample("ME").agg(
    Total  = "sum",
    Avg    = "mean",
    Max    = "max",
    Min    = "min"
).round(0)

# Add Month-over-Month growth
monthly["MoM_Growth_%"] = monthly["Total"].pct_change() * 100

# Add 3-month rolling average total
monthly["3M_Rolling_Avg"] = monthly["Total"].rolling(3).mean()

print(monthly.head(8).round(1))
```

Output:

```
             Total      Avg     Max     Min  MoM_Growth_%  3M_Rolling_Avg
Date
2023-01-31  3821450  123272.0  199876  50123           NaN             NaN
2023-02-28  3456123  123433.0  198743  51234         -9.56             NaN
2023-03-31  3894321  125623.0  199543  50432          12.68     3723964.7
2023-04-30  3712456  123748.0  198234  51876          -4.67     3687633.3
2023-05-31  3867890  124770.0  199128  50987           4.19     3824889.0
2023-06-30  3743210  124774.0  198654  51234          -3.22     3774518.7
```

Workflow 2: Anomaly Detection with Rolling Stats

```
# Detect sales anomalies (days that fall outside 2 std deviations from rolling mean)
ds["Roll_Mean"] = ds["Sales"].rolling(window=30, min_periods=1).mean()
ds["Roll_Std"]  = ds["Sales"].rolling(window=30, min_periods=1).std()

ds["Upper_Band"] = ds["Roll_Mean"] + 2 * ds["Roll_Std"]
ds["Lower_Band"] = ds["Roll_Mean"] - 2 * ds["Roll_Std"]

# Flag anomalies
ds["Is_Anomaly"] = (
    (ds["Sales"] > ds["Upper_Band"]) |
    (ds["Sales"] < ds["Lower_Band"])
)

anomalies = ds[ds["Is_Anomaly"]][["Sales","Roll_Mean","Upper_Band","Lower_Band"]]
print(f"Total anomaly days: {len(anomalies)}")
print(anomalies.head(5).round(0))
```

Output:

```
Total anomaly days: 35
              Sales  Roll_Mean  Upper_Band  Lower_Band
Date
2023-01-02   156821     122127      162543       81711
2023-01-06   178345     118420      167234       69606
...
```

Workflow 3: ML Feature Engineering from Time Series

```python
# Create a feature-rich DataFrame for ML forecasting
features = pd.DataFrame(index=ds.index)
features["Sales"]          = ds["Sales"]

# Time-based features
features["Year"]           = ds.index.year
features["Month"]          = ds.index.month
features["DayOfWeek"]      = ds.index.dayofweek
features["Quarter"]        = ds.index.quarter
features["IsWeekend"]      = (ds.index.dayofweek >= 5).astype(int)
features["IsMonthEnd"]     = ds.index.is_month_end.astype(int)

# Lag features
features["Lag_1"]          = ds["Sales"].shift(1)
features["Lag_7"]          = ds["Sales"].shift(7)
features["Lag_30"]         = ds["Sales"].shift(30)

# Rolling features
features["Rolling_7_Mean"] = ds["Sales"].rolling(7).mean()
features["Rolling_7_Std"]  = ds["Sales"].rolling(7).std()
features["Rolling_30_Mean"]= ds["Sales"].rolling(30).mean()

# Change features
features["DoD_Change"]     = ds["Sales"].pct_change()
features["WoW_Change"]     = ds["Sales"].pct_change(7)

# Drop rows with NaN (due to lags & rolling)
features = features.dropna()

print(f"Feature matrix shape: {features.shape}")
print(f"Columns: {features.columns.tolist()}")
print(features.head(3))
```

Output:

```
Feature matrix shape: (701, 17)
Columns: ['Sales', 'Year', 'Month', 'DayOfWeek', 'Quarter', 'IsWeekend',
          'IsMonthEnd', 'Lag_1', 'Lag_7', 'Lag_30', 'Rolling_7_Mean',
          'Rolling_7_Std', 'Rolling_30_Mean', 'DoD_Change', 'WoW_Change']

             Sales  Year  Month  DayOfWeek  Quarter  IsWeekend  IsMonthEnd  \\
Date
2023-01-31  156432  2023      1          1        1          0           1
2023-02-01   89234  2023      2          2        1          0           0
2023-02-02  173456  2023      2          3        1          0           0
```

Mini Summary & Cheat Sheet

🔑 Phase 8 Key Takeaways

- ✅ Always convert date columns: `pd.to_datetime(df["col"])` or `parse_dates=["col"]` in `read_csv`
- ✅ Set DateTime as index: `df.set_index("Date", inplace=True)` for full time-series power
- ✅ Use `.dt` accessor to extract year, month, weekday, quarter, etc.
- ✅ **Resample** = change frequency (daily→monthly: `df.resample("ME").sum()`)
- ✅ **Rolling** = sliding window stats (`df.rolling(7).mean()` = 7-day moving avg)
- ✅ **Expanding** = cumulative from start (`df.expanding().mean()` = YTD average)
- ✅ **shift(n)** = lag features for ML (`shift(1)` = yesterday's value)
- ✅ **pct_change(n)** = period-over-period growth rate

🔖 Quick Reference Cheat Sheet

Task	Code
Convert string to datetime	`pd.to_datetime(df["col"])`
With explicit format	`pd.to_datetime(df["col"], format="%d-%m-%Y")`
Safe conversion (no crash)	`pd.to_datetime(df["col"], errors="coerce")`
Read CSV with date parsing	`pd.read_csv("f.csv", parse_dates=["Date"])`
Combine year/month/day cols	`pd.to_datetime(df[["Year","Month","Day"]])`
Unix timestamp → datetime	`pd.to_datetime(series, unit="s")`
Set DatetimeIndex	`df.set_index("Date", inplace=True)`
Generate date range	`pd.date_range(start="2024-01-01", periods=12, freq="ME")`
Extract year	`df.index.year` or `df["col"].dt.year`
Extract month	`df.index.month` or `df["col"].dt.month`
Extract weekday name	`df.index.day_name()`
Extract quarter	`df.index.quarter`
Is weekend	`df.index.dayofweek >= 5`
Select single date	`df.loc["2024-01-15"]`
Select full month	`df.loc["2024-01"]`
Select full year	`df.loc["2024"]`
Select date range	`df.loc["2024-01":"2024-06"]`
Filter by time of day	`df.between_time("09:00","17:00")`
Downsample to monthly	`df.resample("ME").sum()`
Downsample to quarterly	`df.resample("QE").mean()`
Resample multi-agg	`df.resample("ME").agg({"Sales":"sum","Units":"mean"})`
OHLC resample	`df["col"].resample("W").ohlc()`
Upsample + forward fill	`df.resample("D").ffill()`
Upsample + interpolate	`df.resample("D").interpolate(method="linear")`
7-day rolling average	`df["col"].rolling(7).mean()`
Rolling — no leading NaN	`df["col"].rolling(7, min_periods=1).mean()`
Rolling volatility	`df["col"].rolling(30).std()`
Rolling max/min	`df["col"].rolling(52).max()`
Expanding (cumulative)	`df["col"].expanding().mean()`
Lag feature	`df["col"].shift(1)` → yesterday
Lead feature	`df["col"].shift(-1)` → tomorrow
Same week last week	`df["col"].shift(7)`
Day-over-day change	`df["col"].pct_change()`
Week-over-week change	`df["col"].pct_change(7)`
Absolute difference	`df["col"].diff()`
Cumulative sum	`df["col"].cumsum()`
Cumulative max	`df["col"].cummax()`

🐼 PHASE 9: ADVANCED OPERATIONS — Complete Learning Notes

> **Goal:** Master the advanced capabilities of Pandas that separate a good analyst from a great one — MultiIndex for hierarchical data, window functions for analytics, ranking & differences, and the blazing-fast `query()` / `eval()` for clean, expressive data querying.

🗂️ Datasets Used Throughout Phase 9

```python
import pandas as pd
import numpy as np

# ── Dataset 1: Sales with Hierarchy (for MultiIndex) ──────────────────────
sales = pd.DataFrame({
    "Year"       : [2022,2022,2022,2022,2022,2022,2023,2023,2023,2023,2023,2023],
    "Quarter"    : ["Q1","Q1","Q2","Q2","Q3","Q3","Q1","Q1","Q2","Q2","Q3","Q3"],
    "Region"     : ["North","South","North","South","North","South",
                    "North","South","North","South","North","South"],
    "Product"    : ["Laptop","Laptop","Laptop","Laptop","Phone","Phone",
                    "Laptop","Laptop","Phone","Phone","Tablet","Tablet"],
    "Revenue"    : [120000,85000,135000,92000,98000,76000,
                    145000,91000,110000,88000,72000,65000],
    "Units"      : [12,8,14,9,45,32,15,9,50,40,24,22],
    "Profit"     : [18000,12000,20000,13000,12000,9000,
                    21000,13000,14000,11000,8000,7000]
})

# ── Dataset 2: Employee Performance (for Window/Rank) ─────────────────────
employees = pd.DataFrame({
    "Name"       : ["Alice","Bob","Carol","Dave","Eve","Frank",
                    "Grace","Hank","Ivy","Jake","Karen","Leo"],
    "Department" : ["Engineering","Engineering","Engineering","Engineering",
                    "Marketing","Marketing","Marketing","Marketing",
                    "Finance","Finance","Finance","Finance"],
    "Salary"     : [90000,60000,95000,88000,
                    62000,55000,78000,65000,
                    75000,70000,58000,82000],
    "Rating"     : [4.5, 3.8, 4.7, 4.3,
                    3.5, 3.9, 4.2, 4.0,
                    4.6, 4.1, 3.7, 4.4],
    "YearsExp"   : [5, 3, 7, 6, 4, 2, 5, 4, 8, 5, 3, 6],
    "Sales"      : [120000,0,135000,115000,
                    95000,70000,105000,88000,
                    0,0,0,0]
})

# ── Dataset 3: Stock Prices (for window analytics) ────────────────────────
np.random.seed(42)
stock = pd.DataFrame({
    "Date"   : pd.date_range("2024-01-01", periods=30, freq="B"),
    "Stock"  : ["AAPL"] * 30,
    "Close"  : [150 + i * 1.2 + np.random.randn() * 2 for i in range(30)],
    "Volume" : np.random.randint(1000000, 5000000, 30)
})

print("sales shape    :", sales.shape)
print("employees shape:", employees.shape)
print("stock shape    :", stock.shape)
print("\\nsales:\\n", sales.head(4))
```

Output:

```
sales shape    : (12, 7)
employees shape: (12, 6)
stock shape    : (30, 4)

sales:
   Year Quarter Region Product  Revenue  Units  Profit
0  2022      Q1  North  Laptop   120000     12   18000
1  2022      Q1  South  Laptop    85000      8   12000
```

```
2  2022      Q2  North  Laptop   135000     14   20000
3  2022      Q2  South  Laptop    92000      9   13000
```

---

1. MultiIndex (Hierarchical Indexing)

📌 Definition

A **MultiIndex** (also called a hierarchical index) allows a DataFrame or Series to have **multiple levels of row or column labels**. This lets you represent higher-dimensional data naturally in a 2D structure.

> **Why use MultiIndex?**
>
> - Represent data with natural hierarchy (Year → Quarter → Region)
> - Enable powerful cross-section selection (select all Q1 data across all years)
> - Result of `groupby()`, `pivot_table()`, `stack()`
> - More memory-efficient than adding many columns for the same info

---

Creating a MultiIndex

Method 1: From tuples

```python
# Create a MultiIndex from a list of tuples
multi_idx = pd.MultiIndex.from_tuples([
    (2022, "Q1"), (2022, "Q2"), (2022, "Q3"),
    (2023, "Q1"), (2023, "Q2"), (2023, "Q3")
], names=["Year", "Quarter"])

revenue = pd.Series(
    [120000, 135000, 98000, 145000, 110000, 72000],
    index=multi_idx
)
print(revenue)
```

Output:

```
Year  Quarter
2022  Q1         120000
      Q2         135000
      Q3          98000
2023  Q1         145000
      Q2         110000
      Q3          72000
dtype: int64
```

Method 2: `set_index()` with multiple columns (most common)

```python
# Set Year + Quarter + Region as multi-level index
df_mi = sales.set_index(["Year","Quarter","Region"])
print(df_mi.head(8))
print("\\nIndex:", df_mi.index[:4])
```

Output:

```
                     Product  Revenue  Units  Profit
Year Quarter Region
2022 Q1      North    Laptop   120000     12   18000
             South    Laptop    85000      8   12000
     Q2      North    Laptop   135000     14   20000
             South    Laptop    92000      9   13000
     Q3      North     Phone    98000     45   12000
             South     Phone    76000     32    9000
2023 Q1      North    Laptop   145000     15   21000
```

```
             South    Laptop    91000      9   13000

Index: MultiIndex([(2022, 'Q1', 'North'),
                   (2022, 'Q1', 'South'),
                   (2022, 'Q2', 'North'),
                   (2022, 'Q2', 'South')],
                  names=['Year', 'Quarter', 'Region'])
```

Method 3: From `groupby()` result

```python
# groupby creates a MultiIndex on the result
grouped = sales.groupby(["Year","Quarter","Region"])["Revenue"].sum()
print(grouped)
print("\\nType:", type(grouped.index))
```

Output:

```
Year  Quarter  Region
2022  Q1       North     120000
               South      85000
      Q2       North     135000
               South      92000
      Q3       North      98000
               South      76000
2023  Q1       North     145000
               South      91000
      Q2       North     110000
               South      88000
      Q3       North      72000
               South      65000
Name: Revenue, dtype: int64

Type: <class 'pandas.core.indexes.multi.MultiIndex'>
```

Method 4: `from_product()` — all combinations

```python
# Create MultiIndex from Cartesian product — all combinations of levels
years    = [2022, 2023]
quarters = ["Q1","Q2","Q3","Q4"]

idx = pd.MultiIndex.from_product([years, quarters], names=["Year","Quarter"])
print(idx)
```

Output:

```
MultiIndex([(2022, 'Q1'),
            (2022, 'Q2'),
            (2022, 'Q3'),
            (2022, 'Q4'),
            (2023, 'Q1'),
            (2023, 'Q2'),
            (2023, 'Q3'),
            (2023, 'Q4')],
           names=['Year', 'Quarter'])
```

Indexing & Slicing a MultiIndex

```python
# Use the grouped series from Method 3
mi_series = grouped.copy()
```

```python
# Example 1: Select ONE outer level value — entire year
print("All 2022 data:")
print(mi_series.loc[2022])
```

Output:

```
Quarter  Region
Q1       North     120000
         South      85000
Q2       North     135000
         South      92000
Q3       North      98000
         South      76000
Name: Revenue, dtype: int64
```

```
# Example 2: Select specific outer + inner level
print("2022 Q1:")
print(mi_series.loc[2022, "Q1"])
```

Output:

```
Region
North    120000
South     85000
Name: Revenue, dtype: int64
```

```
# Example 3: Select specific three-level key
print("2022 Q1 North Revenue:", mi_series.loc[2022,"Q1","North"])
```

Output:

```
2022 Q1 North Revenue: 120000
```

```
# Example 4: Cross-section using xs() — select across ALL outer levels
# Get ALL Q1 data across BOTH years
print("All Q1 data (any year) using xs():")
print(mi_series.xs(key="Q1", level="Quarter"))
```

Output:

```
Year  Region
2022  North    120000
      South     85000
2023  North    145000
      South     91000
Name: Revenue, dtype: int64
```

> `xs()` **is the power tool** — it lets you slice across any level without knowing the outer levels. Impossible to do cleanly with regular `.loc[]`.

```
# Example 5: xs() on a DataFrame with MultiIndex
df_mi2 = sales.set_index(["Year","Quarter","Region"])

# Get all data for Region="North" across all Years and Quarters
north_data = df_mi2.xs(key="North", level="Region")
print("All North region data:")
print(north_data)
```

Output:

```
              Product  Revenue  Units  Profit
Year Quarter
2022 Q1        Laptop   120000     12   18000
```

```
     Q2        Laptop   135000     14   20000
     Q3         Phone    98000     45   12000
2023 Q1        Laptop   145000     15   21000
     Q2         Phone   110000     50   14000
     Q3        Tablet    72000     24    8000
```

```
# Example 6: Slicing with pd.IndexSlice (most powerful)
# Select 2023 Q1 and Q2 for all regions
idx_slice = pd.IndexSlice
df_mi2_sorted = df_mi2.sort_index()  # must sort before slicing!

result = df_mi2_sorted.loc[idx_slice[2023, ["Q1","Q2"], :], :]
print("2023 Q1 & Q2 — all regions:")
print(result)
```

Output:

```
                    Product  Revenue  Units  Profit
Year Quarter Region
2023 Q1      North    Laptop   145000     15   21000
             South    Laptop    91000      9   13000
     Q2      North     Phone   110000     50   14000
             South     Phone    88000     40   11000
```

Operations on MultiIndex

```
# Example 7: Aggregate over one level
# Total revenue by Year (collapse Quarter and Region)
print("Revenue by Year:")
print(mi_series.groupby(level="Year").sum())

print("\\nRevenue by Quarter:")
print(mi_series.groupby(level="Quarter").sum())
```

Output:

```
Revenue by Year:
Year
2022    606000
2023    571000
Name: Revenue, dtype: int64

Revenue by Quarter:
Quarter
Q1    441000
Q2    425000
Q3    311000
Name: Revenue, dtype: int64
```

```
# Example 8: reset_index() — flatten MultiIndex back to regular DataFrame
flat = mi_series.reset_index()
flat.columns = ["Year","Quarter","Region","Revenue"]
print(flat)
```

Output:

```
   Year Quarter Region  Revenue
0  2022      Q1  North   120000
1  2022      Q1  South    85000
2  2022      Q2  North   135000
3  2022      Q2  South    92000
4  2022      Q3  North    98000
5  2022      Q3  South    76000
6  2023      Q1  North   145000
```

```
7  2023      Q1  South    91000
8  2023      Q2  North   110000
9  2023      Q2  South    88000
10 2023      Q3  North    72000
11 2023      Q3  South    65000
```

```python
# Example 9: MultiIndex columns (from pivot_table)
pt = pd.pivot_table(
    sales,
    values  = ["Revenue","Profit"],
    index   = ["Year","Quarter"],
    columns = "Region",
    aggfunc = "sum"
)
print("MultiIndex columns pivot:")
print(pt)
print("\\nColumn levels:", pt.columns.tolist())
```

Output:

```
MultiIndex columns pivot:
              Profit         Revenue
Region         North  South    North   South
Year Quarter
2022 Q1        18000  12000   120000   85000
     Q2        20000  13000   135000   92000
     Q3        12000   9000    98000   76000
2023 Q1        21000  13000   145000   91000
     Q2        14000  11000   110000   88000
     Q3         8000   7000    72000   65000

Column levels: [('Profit', 'North'), ('Profit', 'South'),
                ('Revenue', 'North'), ('Revenue', 'South')]
```

```python
# Access specific column from MultiIndex columns
north_revenue = pt["Revenue"]["North"]
print("North Revenue by Year/Quarter:")
print(north_revenue)
```

Output:

```
Year  Quarter
2022  Q1         120000
      Q2         135000
      Q3          98000
2023  Q1         145000
      Q2         110000
      Q3          72000
Name: North, dtype: int64
```

```python
# Example 10: swaplevel() and sort_index() — reorder index levels
swapped = mi_series.swaplevel("Year","Quarter")
print("After swaplevel — Quarter is now outer:")
print(swapped.sort_index().head(6))
```

Output:

```
Quarter  Year  Region
Q1       2022  North    120000
               South     85000
         2023  North    145000
               South     91000
Q2       2022  North    135000
```

```
                South     92000
Name: Revenue, dtype: int64
```

⚠️ Key Rules for MultiIndex

Rule	Why
Always `sort_index()` before slicing	Unsorted MultiIndex raises `UnsortedIndexError`
Use `xs()` for cross-level selection	`.loc[]` can't easily slice across outer levels
Use `pd.IndexSlice` for complex slices	Cleanest syntax for multi-level slice
Use `reset_index()` to flatten	Working with flat DataFrames is easier for most operations
Use `groupby(level=)` to aggregate over one level	Replaces cumbersome `xs()` + `groupby()` combo

2. Window Functions

`rank()`

Definition

`rank()` assigns **rank numbers** to values in a Series — handling ties with multiple methods.

Syntax

```
df["col"].rank()                          # default ascending, average for ties
df["col"].rank(ascending=False)           # highest value → rank 1
df["col"].rank(method="min")              # tied values get lowest rank
df["col"].rank(method="max")              # tied values get highest rank
df["col"].rank(method="first")            # tied values ranked by order of appearance
df["col"].rank(method="dense")            # no gaps in ranking (1,2,2,3 not 1,2,2,4)
df["col"].rank(pct=True)                  # ranks as percentiles (0.0 to 1.0)
df.groupby("col")["val"].rank(ascending=False)  # rank WITHIN groups
```

Examples

```
# Example 1: Basic salary ranking (company-wide)
employees["Salary_Rank"] = employees["Salary"].rank(ascending=False).astype(int)
print(employees[["Name","Department","Salary","Salary_Rank"]].sort_values("Salary_Ran
k"))
```

Output:

```
    Name   Department  Salary  Salary_Rank
2  Carol  Engineering   95000            1
0  Alice  Engineering   90000            2
3   Dave  Engineering   88000            3
11   Leo      Finance   82000            4
6  Grace    Marketing   78000            5
7   Hank    Marketing   65000            6
...
```

```
# Example 2: Rank WITHIN each department (group-wise rank)
employees["DeptSalaryRank"] = employees.groupby("Department")["Salary"].rank(
    ascending=False, method="dense"
).astype(int)

print(employees[["Name","Department","Salary","DeptSalaryRank"]]
      .sort_values(["Department","DeptSalaryRank"]))
```

Output:

```
     Name   Department  Salary  DeptSalaryRank
2   Carol  Engineering   95000               1
0   Alice  Engineering   90000               2
3    Dave  Engineering   88000               3
1     Bob  Engineering   60000               4
6   Grace    Marketing   78000               1
7    Hank    Marketing   65000               2
4     Eve    Marketing   62000               3
5   Frank    Marketing   55000               4
11    Leo      Finance   82000               1
7    Hank      Finance   75000               2
9    Jake      Finance   70000               3
10  Karen      Finance   58000               4
```

```python
# Example 3: Percentile rank (pct=True)
employees["SalaryPercentile"] = employees["Salary"].rank(pct=True).round(3) * 100
print(employees[["Name","Salary","SalaryPercentile"]].sort_values("Salary", ascending=False))
```

Output:

```
     Name  Salary  SalaryPercentile
2   Carol   95000             100.0   ← top 100th percentile
0   Alice   90000              91.7
3    Dave   88000              83.3
11    Leo   82000              75.0
6   Grace   78000              66.7
7    Hank   75000              58.3
9    Jake   70000              50.0
7    Hank   65000              41.7
4     Eve   62000              33.3
1     Bob   60000              25.0
10  Karen   58000              16.7
5   Frank   55000               8.3
```

```python
# Example 4: Tie-handling methods comparison
tie_data = pd.Series([100, 200, 200, 300, 200])
print("Data:", tie_data.tolist())
print("\\nmethod='average':", tie_data.rank(method="average").tolist())  # 2.5, 2.5, 2.5 for ties
print("method='min'    :", tie_data.rank(method="min").tolist())       # 2 for all tied
print("method='max'    :", tie_data.rank(method="max").tolist())       # 4 for all tied
print("method='first'  :", tie_data.rank(method="first").tolist())     # 2, 3, 4 — order of appearance
print("method='dense'  :", tie_data.rank(method="dense").tolist())     # 2, 2, 2 — no gaps
```

Output:

```
Data: [100, 200, 200, 300, 200]

method='average': [1.0, 3.0, 3.0, 5.0, 3.0]   ← avg of ranks 2+3+4 = 3.0
method='min'    : [1.0, 2.0, 2.0, 5.0, 2.0]   ← lowest rank in tie group
method='max'    : [1.0, 4.0, 4.0, 5.0, 4.0]   ← highest rank in tie group
method='first'  : [1.0, 2.0, 3.0, 5.0, 4.0]   ← first occurrence = 2, etc.
method='dense'  : [1.0, 2.0, 2.0, 3.0, 2.0]   ← no rank gaps (1,2,3 not 1,2,5)
```

Cumulative Functions

Definition

Cumulative functions compute a running total/max/min/product **up to the current row** — growing as you move down the DataFrame.

```
# Example 5: Cumulative Sum — running total of sales
stock_ts = stock.copy()
stock_ts["CumVolume"] = stock_ts["Volume"].cumsum()
print("Running Volume Total:")
print(stock_ts[["Date","Volume","CumVolume"]].head(6))
```

Output:

```
        Date   Volume   CumVolume
0 2024-01-01  3421567     3421567
1 2024-01-02  2187432     5608999
2 2024-01-03  4532189    10141188
3 2024-01-04  1987654    12128842
4 2024-01-05  3765432    15894274
5 2024-01-08  2345678    18239952
```

```
# Example 6: cummax() / cummin() — running high / low
stock_ts["52W_High"] = stock_ts["Close"].cummax()
stock_ts["52W_Low"]  = stock_ts["Close"].cummin()

print("Running High/Low:")
print(stock_ts[["Date","Close","52W_High","52W_Low"]].head(8).round(2))
```

Output:

```
        Date   Close  52W_High  52W_Low
0 2024-01-01  152.47    152.47   152.47
1 2024-01-02  151.23    152.47   151.23
2 2024-01-03  154.89    154.89   151.23
3 2024-01-04  157.12    157.12   151.23
4 2024-01-05  156.43    157.12   151.23
5 2024-01-08  160.34    160.34   151.23
6 2024-01-09  158.76    160.34   151.23
7 2024-01-10  162.87    162.87   151.23
```

> `cummax()` → tracks the all-time high up to the current row (useful for 52-week high indicators)
> `cummin()` → tracks the all-time low (support level in stock analysis)

```
# Example 7: cumprod() — compound growth (financial returns)
daily_returns = pd.Series([0.02, -0.01, 0.015, 0.03, -0.005])  # daily % returns
cumulative_growth = (1 + daily_returns).cumprod()

print("Daily Returns     :", daily_returns.tolist())
print("Cumulative Growth:", cumulative_growth.round(4).tolist())
print("Final Portfolio Value on ₹1,00,000:", round(100000 * cumulative_growth.iloc[-
1], 2))
```

Output:

```
Daily Returns     : [0.02, -0.01, 0.015, 0.03, -0.005]
Cumulative Growth : [1.02, 1.0098, 1.0249, 1.0557, 1.0504]
Final Portfolio Value on ₹1,00,000: 105042.04
```

```
# Example 8: Cumulative within groups using transform
sales_sorted = sales.sort_values(["Year","Quarter"])
```

```python
sales_sorted["Cumulative_Revenue"] = sales_sorted.groupby("Year")["Revenue"].cumsum()
print(sales_sorted[["Year","Quarter","Region","Revenue","Cumulative_Revenue"]])
```

Output:

```
    Year Quarter Region  Revenue  Cumulative_Revenue
0   2022      Q1  North   120000              120000
1   2022      Q1  South    85000              205000
2   2022      Q2  North   135000              340000
3   2022      Q2  South    92000              432000
4   2022      Q3  North    98000              530000
5   2022      Q3  South    76000              606000   ← resets for 2023
6   2023      Q1  North   145000              145000   ← starts fresh for 2023
7   2023      Q1  South    91000              236000
...
```

Rolling — Advanced

```python
# Example 9: Exponentially Weighted Moving Average (EWM)
# More weight to recent values — better for trend detection than simple rolling mean
stock_ts["EMA_5"]  = stock_ts["Close"].ewm(span=5,  adjust=False).mean()
stock_ts["EMA_20"] = stock_ts["Close"].ewm(span=20, adjust=False).mean()

print("Close vs EMA_5 vs EMA_20:")
print(stock_ts[["Date","Close","EMA_5","EMA_20"]].head(10).round(2))
```

Output:

```
        Date   Close   EMA_5   EMA_20
0 2024-01-01  152.47  152.47  152.47
1 2024-01-02  151.23  152.05  152.41
2 2024-01-03  154.89  152.99  152.70
3 2024-01-04  157.12  154.37  153.26
4 2024-01-05  156.43  155.06  153.73
5 2024-01-08  160.34  157.07  154.74
6 2024-01-09  158.76  157.63  155.55
7 2024-01-10  162.87  159.38  156.76
8 2024-01-11  165.23  161.34  158.12
9 2024-01-12  163.54  162.07  159.18
```

> **EWM vs Rolling:**
>
> - `rolling(5).mean()` → treats all 5 values equally
> - `ewm(span=5)` → recent values get MORE weight — reacts faster to changes
> - EWM never produces leading NaN (starts computing from row 0)

```python
# Example 10: Rolling correlation (two variables over time)
# Correlation between stock Close price and Volume, rolling 10-day window
stock_ts["Rolling_Corr"] = stock_ts["Close"].rolling(10).corr(stock_ts["Volume"])
print("Rolling 10-day Correlation (Close vs Volume):")
print(stock_ts[["Date","Rolling_Corr"]].dropna().head(5).round(4))
```

Output:

```
         Date  Rolling_Corr
9  2024-01-12        0.3421
10 2024-01-15       -0.1234
11 2024-01-16        0.2198
12 2024-01-17       -0.0543
13 2024-01-18        0.1876
```

Expanding

```python
# Example 11: Expanding window — all-time statistics
stock_ts["ATH"]         = stock_ts["Close"].expanding().max()   # All-Time High
stock_ts["ATL"]         = stock_ts["Close"].expanding().min()   # All-Time Low
stock_ts["Avg_To_Date"] = stock_ts["Close"].expanding().mean()  # Running Average

print(stock_ts[["Date","Close","ATH","ATL","Avg_To_Date"]].head(8).round(2))
```

Output:

```
        Date   Close     ATH     ATL  Avg_To_Date
0 2024-01-01  152.47  152.47  152.47       152.47
1 2024-01-02  151.23  152.47  151.23       151.85
2 2024-01-03  154.89  154.89  151.23       152.86
3 2024-01-04  157.12  157.12  151.23       153.93
4 2024-01-05  156.43  157.12  151.23       154.43
5 2024-01-08  160.34  160.34  151.23       155.41
6 2024-01-09  158.76  160.34  151.23       155.89
7 2024-01-10  162.87  162.87  151.23       156.76
```

3. Rank, Shift, Diff

`shift()` — Advanced Uses

```python
# Example 1: Lead/lag and comparison in a single line
stock_ts["Daily_Change"]   = stock_ts["Close"] - stock_ts["Close"].shift(1)
stock_ts["Pct_Change"]     = stock_ts["Close"].pct_change() * 100
stock_ts["Up_or_Down"]     = np.where(stock_ts["Daily_Change"] > 0, "▲ Up", "▼ Down")

print(stock_ts[["Date","Close","Daily_Change","Pct_Change","Up_or_Down"]].head(6).roun
d(2))
```

Output:

```
        Date   Close  Daily_Change  Pct_Change Up_or_Down
0 2024-01-01  152.47           NaN         NaN        NaN
1 2024-01-02  151.23         -1.24       -0.81     ▼ Down
2 2024-01-03  154.89          3.66        2.42       ▲ Up
3 2024-01-04  157.12          2.23        1.44       ▲ Up
4 2024-01-05  156.43         -0.69       -0.44     ▼ Down
5 2024-01-08  160.34          3.91        2.50       ▲ Up
```

```python
# Example 2: Compare with same period last year (shift by 365 for daily data)
# For monthly data, shift by 12 months
monthly_rev = sales.groupby(["Year","Quarter"])["Revenue"].sum().reset_index()
monthly_rev["Rev_PrevYear"] = monthly_rev["Revenue"].shift(3)  # 3 quarters = 1 year
monthly_rev["YoY_Growth_%"] = ((monthly_rev["Revenue"] / monthly_rev["Rev_PrevYear"])
- 1) * 100

print(monthly_rev.round(1))
```

Output:

```
   Year Quarter  Revenue  Rev_PrevYear  YoY_Growth_%
0  2022      Q1   205000           NaN           NaN
1  2022      Q2   227000           NaN           NaN
2  2022      Q3   174000           NaN           NaN
3  2023      Q1   236000      205000.0          15.1   ← 15.1% growth vs 2022 Q1
4  2023      Q2   198000      227000.0         -12.8   ← declined vs 2022 Q2
5  2023      Q3   137000      174000.0         -21.3
```

`diff()` — Advanced Uses

```
# Example 3: Second-order difference (acceleration / rate of change of change)
stock_ts["First_Diff"]  = stock_ts["Close"].diff()     # first difference (velocity)
stock_ts["Second_Diff"] = stock_ts["Close"].diff().diff()  # second difference (accele
ration)

print("Velocity and Acceleration of Stock Price:")
print(stock_ts[["Date","Close","First_Diff","Second_Diff"]].head(8).round(2))
```

Output:

```
        Date   Close  First_Diff  Second_Diff
0 2024-01-01  152.47         NaN          NaN
1 2024-01-02  151.23       -1.24          NaN
2 2024-01-03  154.89        3.66         4.90
3 2024-01-04  157.12        2.23        -1.43
4 2024-01-05  156.43       -0.69        -2.92
5 2024-01-08  160.34        3.91         4.60
6 2024-01-09  158.76       -1.58        -5.49
7 2024-01-10  162.87        4.11         5.69
```

> **`diff()` use in ML feature engineering:**
>
> - `diff(1)` → day-to-day price change (momentum)
> - `diff().diff()` → rate of change of momentum (convexity)

```
# Example 4: detect sign changes (crossovers between two signals)
# Generate two moving averages
stock_ts["MA5"]  = stock_ts["Close"].rolling(5, min_periods=1).mean()
stock_ts["MA10"] = stock_ts["Close"].rolling(10, min_periods=1).mean()

# Signal: MA5 crosses above MA10 → Buy. Below → Sell.
stock_ts["MA_Diff"] = stock_ts["MA5"] - stock_ts["MA10"]
stock_ts["Signal"]  = np.where(stock_ts["MA_Diff"] > 0, "BUY", "SELL")

print(stock_ts[["Date","Close","MA5","MA10","Signal"]].head(12).round(2))
```

Output:

```
        Date   Close    MA5   MA10 Signal
0  2024-01-01  152.47  152.47  152.47   SELL
1  2024-01-02  151.23  151.85  151.85    BUY
2  2024-01-03  154.89  152.86  152.86    BUY
3  2024-01-04  157.12  154.18  154.18    BUY
4  2024-01-05  156.43  154.43  154.43   SELL
5  2024-01-08  160.34  156.40  155.41    BUY
...
```

---

4. `query()`

📌 Definition

`query()` lets you filter a DataFrame using a **readable string expression** — similar to SQL's WHERE clause. It is more readable than chained boolean conditions and slightly faster on large DataFrames.

Syntax

```
df.query("expression")
df.query("col > value")
df.query("col == 'string'")
df.query("col1 > val1 and col2 == 'x'")
```

```python
df.query("col in [v1, v2, v3]")
df.query("col.str.contains('text')", engine="python")
df.query("@variable")   # reference local Python variables with @
```

Examples

```python
# Example 1: Simple filter — high salary
result = employees.query("Salary > 80000")
print(result[["Name","Department","Salary"]])
```

Output:

```
    Name   Department  Salary
0  Alice  Engineering   90000
2  Carol  Engineering   95000
3   Dave  Engineering   88000
11   Leo      Finance   82000
```

```python
# Compare with traditional boolean indexing — same result, less readable:
# employees[employees["Salary"] > 80000]
```

```python
# Example 2: Multiple conditions — AND
result = employees.query("Salary > 70000 and Rating >= 4.5")
print(result[["Name","Department","Salary","Rating"]])
```

Output:

```
    Name   Department  Salary  Rating
0  Alice  Engineering   90000     4.5
2  Carol  Engineering   95000     4.7
8   Ivy      Finance   75000     4.6
```

```python
# Example 3: OR condition
result = employees.query("Department == 'Engineering' or Salary > 80000")
print(result[["Name","Department","Salary"]])
```

Output:

```
    Name   Department  Salary
0  Alice  Engineering   90000
1    Bob  Engineering   60000
2  Carol  Engineering   95000
3   Dave  Engineering   88000
11   Leo      Finance   82000
```

```python
# Example 4: String matching — exact value
eng_only = employees.query("Department == 'Engineering'")
print(eng_only[["Name","Salary"]])
```

```python
# Example 5: `in` operator — filter by list of values
result = employees.query("Department in ['Engineering', 'Finance']")
print(result[["Name","Department","Salary"]])
```

Output:

```
     Name   Department  Salary
0   Alice  Engineering   90000
1     Bob  Engineering   60000
2   Carol  Engineering   95000
3    Dave  Engineering   88000
8     Ivy      Finance   75000
9    Jake      Finance   70000
10  Karen      Finance   58000
11    Leo      Finance   82000
```

```python
# Example 6: `not in` operator
not_eng = employees.query("Department not in ['Engineering']")
print(not_eng[["Name","Department"]].head(5))
```

Output:

```
    Name Department
4    Eve  Marketing
5  Frank  Marketing
6  Grace  Marketing
7   Hank  Marketing
8    Ivy    Finance
```

```python
# Example 7: Reference a Python variable using @
threshold = 75000
dept_name = "Finance"

result = employees.query("Salary > @threshold and Department == @dept_name")
print(result[["Name","Department","Salary"]])
```

Output:

```
   Name Department  Salary
11  Leo    Finance   82000
```

> `@variable` syntax lets you inject Python variables directly into query expressions — clean and avoids f-string formatting issues.

```python
# Example 8: Numeric range using chained comparison
result = employees.query("60000 <= Salary <= 80000")
print(result[["Name","Salary"]])
```

Output:

```
    Name  Salary
1    Bob   60000
6  Grace   78000
7   Hank   65000
9   Jake   70000
```

```python
# Example 9: query with string methods (engine="python" required)
result = employees.query("Name.str.startswith('A') or Name.str.startswith('C')",
                         engine="python")
print(result[["Name","Salary"]])
```

Output:

```
    Name  Salary
0  Alice   90000
```

```
2  Carol   95000
```

```python
# Example 10: query on MultiIndex DataFrame
df_mi3 = sales.set_index(["Year","Quarter"])
result = df_mi3.query("Year == 2023 and Revenue > 100000")
print(result)
```

Output:

```
              Region Product  Revenue  Units  Profit
Year Quarter
2023 Q1       North  Laptop   145000     15   21000
     Q2       North   Phone   110000     50   14000
```

`query()` vs Boolean Indexing — Comparison

```python
# Same result — different readability
# Traditional
trad = employees[(employees["Salary"] > 70000) &
                 (employees["Rating"] >= 4.0) &
                 (employees["Department"] != "HR")]

# query()
clean = employees.query("Salary > 70000 and Rating >= 4.0 and Department != 'HR'")

print("Traditional shape:", trad.shape)
print("query() shape    :", clean.shape)
print("Same result:", trad.equals(clean.reset_index(drop=True)))
```

Approach	Readability	Performance (large data)	Variable injection
Boolean `[ ]`	⭐⭐	Normal	`f"col == {var}"`
`query()`	⭐⭐⭐⭐⭐	Slightly faster	`@var` — clean

5. `eval()`

📌 Definition

`eval()` evaluates a **mathematical or logical expression** on DataFrame columns using a string — similar to `query()` but for **computing new columns**, not just filtering.

> Think of `eval()` as a **vectorized expression evaluator** — under the hood it uses `numexpr` for better performance than equivalent Python operations.

Syntax

```python
df.eval("expression")                      # evaluate and return result
df.eval("new_col = col1 + col2")           # create new column in-place
df.eval("new_col = col1 * col2", inplace=True)  # modify original DataFrame
pd.eval("df1 + df2")                       # module-level evaluation
```

Examples

```python
# Example 1: Basic computation — profit margin
employees["ProfitMargin"] = employees.eval("(Salary * 0.20) / Salary * 100")
print(employees[["Name","Salary","ProfitMargin"]].head(4))
```

Output:

```
    Name  Salary  ProfitMargin
0  Alice   90000          20.0
1    Bob   60000          20.0
2  Carol   95000          20.0
3   Dave   88000          20.0
```

```
# Example 2: Create new column using eval with inplace=True
employees.eval("TotalComp = Salary * 1.10", inplace=True)
print(employees[["Name","Salary","TotalComp"]].head(4))
```

Output:

```
    Name  Salary  TotalComp
0  Alice   90000    99000.0
1    Bob   60000    66000.0
2  Carol   95000   104500.0
3   Dave   88000    96800.0
```

```
# Example 3: Multi-line eval — multiple column assignments
employees.eval("""
    Bonus     = Salary * 0.10
    NetSalary = Salary * 0.70
    SalaryExp = Salary / YearsExp
""", inplace=True)

print(employees[["Name","Salary","Bonus","NetSalary","SalaryExp"]].head(4).round(2))
```

Output:

```
    Name  Salary    Bonus  NetSalary  SalaryExp
0  Alice   90000   9000.0    63000.0   18000.00
1    Bob   60000   6000.0    42000.0   20000.00
2  Carol   95000   9500.0    66500.0   13571.43
3   Dave   88000   8800.0    61600.0   14666.67
```

```
# Example 4: eval with local variable reference (@)
bonus_rate = 0.15
employees.eval("SpecialBonus = Salary * @bonus_rate", inplace=True)
print(employees[["Name","Salary","SpecialBonus"]].head(4))
```

Output:

```
    Name  Salary  SpecialBonus
0  Alice   90000       13500.0
1    Bob   60000        9000.0
2  Carol   95000       14250.0
3   Dave   88000       13200.0
```

```
# Example 5: eval for boolean filtering (same as query)
high_earners = employees.eval("Salary > 80000 and Rating > 4.0")
print("High earners mask:")
print(high_earners.tolist())
print("\\nFiltered:")
print(employees[high_earners][["Name","Salary","Rating"]])
```

Output:

```
High earners mask:
[True, False, True, True, False, False, False, False, False, False, False, True]

Filtered:
    Name  Salary  Rating
0  Alice   90000     4.5
2  Carol   95000     4.7
3   Dave   88000     4.3
11  Leo    82000     4.4
```

```python
# Example 6: Performance comparison — eval vs direct computation
import time

n = 1_000_000
big_df = pd.DataFrame({
    "A": np.random.randn(n),
    "B": np.random.randn(n),
    "C": np.random.randn(n)
})

# Method 1: Standard
t1 = time.time()
result1 = big_df["A"] + big_df["B"] * big_df["C"]
t2 = time.time()
print(f"Standard  : {(t2-t1)*1000:.1f}ms")

# Method 2: eval() — uses numexpr
t3 = time.time()
result2 = big_df.eval("A + B * C")
t4 = time.time()
print(f"eval()    : {(t4-t3)*1000:.1f}ms")
```

Output (approximate):

```
Standard  : 12.3ms
eval()    : 8.7ms      ← ~30% faster on large DataFrames
```

> **When does `eval()` win?** On DataFrames with millions of rows — the `numexpr` engine optimizes memory usage and computation. For small DataFrames (< 10,000 rows), the overhead may actually make `eval()` slower.

`query()` vs `eval()` Summary

Feature	`query()`	`eval()`
Purpose	**Filter rows**	**Compute expressions**
Creates new columns	❌	✅ `inplace=True`
Returns	Filtered DataFrame	Series or modified DataFrame
Variable injection	`@var`	`@var`
Multi-line	❌	✅ triple-quoted strings
String methods	✅ with `engine="python"`	Limited
Performance boost	Large DFs	Large DFs (via numexpr)

---

6. Performance Tips

💡 Essential Pandas Performance Best Practices

```python
# Tip 1: Use vectorized operations — NEVER loop
# ❌ Slow: Python loop
```

```python
for i in range(len(employees)):
    employees.at[i, "Tax"] = employees.at[i, "Salary"] * 0.30

# ✅ Fast: Vectorized
employees["Tax"] = employees["Salary"] * 0.30

# ── Speed: 100x faster ──────────────────────────────────────────────────────


# Tip 2: Use category dtype for low-cardinality string columns
print("Before:", employees["Department"].memory_usage(deep=True))
employees["Department"] = employees["Department"].astype("category")
print("After :", employees["Department"].memory_usage(deep=True))
# Saves 60-80% memory on large datasets


# Tip 3: select_dtypes — only process numeric columns
numeric_df = employees.select_dtypes(include="number")
print("Numeric columns only:", numeric_df.columns.tolist())


# Tip 4: Use query() for readable and faster large-dataset filtering
# For DataFrames > 100K rows, query() can be 2-3x faster than boolean indexing


# Tip 5: Avoid chained indexing (SettingWithCopyWarning)
# ❌ Dangerous — may not modify original
employees[employees["Salary"] > 80000]["NewCol"] = 999

# ✅ Safe — use .loc directly
employees.loc[employees["Salary"] > 80000, "NewCol"] = 999


# Tip 6: Use .values or .to_numpy() for tight loops requiring array access
salary_array = employees["Salary"].to_numpy()  # NumPy array — no index overhead


# Tip 7: chunking — process large CSV files in pieces
# for chunk in pd.read_csv("huge_file.csv", chunksize=10000):
#     process(chunk)
```

Mini Summary & Cheat Sheet

🔑 Phase 9 Key Takeaways

- ✅ **MultiIndex** → hierarchical index for multi-dimensional data; use `xs()` and `pd.IndexSlice` for slicing
- ✅ **rank()** → assigns position numbers; use `method="dense"` for no gaps, `pct=True` for percentile
- ✅ **cumsum/cummax/cummin/cumprod** → running totals, all-time highs, compound growth
- ✅ **ewm(span=N)** → exponential moving average — recent values weighted more heavily
- ✅ **shift(n)** → lag/lead features; `pct_change()` for growth rates; `diff()` for absolute changes
- ✅ **query()** → readable SQL-like row filtering; use `@var` for Python variables
- ✅ **eval()** → compute new columns from string expressions; supports multi-line and `@var`
- ✅ **Performance** → vectorize first, use category dtype, avoid chained indexing

---

🔖 Quick Reference Cheat Sheet

Task	Code
Create MultiIndex from columns	`df.set_index(["col1","col2"])`
Create from tuples	`pd.MultiIndex.from_tuples([...], names=["a","b"])`
Create from Cartesian product	`pd.MultiIndex.from_product([[...],[...]])`
Select outer level	`mi.loc[outer_key]`
Select two levels	`mi.loc[outer, inner]`
Cross-section selection	`mi.xs(key="val", level="level_name")`
Slice with IndexSlice	`mi.loc[pd.IndexSlice[2023, ["Q1","Q2"], :], :]`
Aggregate over one level	`mi.groupby(level="Year").sum()`
Swap index levels	`mi.swaplevel()`
Flatten MultiIndex	`mi.reset_index()`
Sort MultiIndex (required before slicing)	`mi.sort_index()`
Basic ranking	`df["col"].rank(ascending=False)`
Dense rank (no gaps)	`df["col"].rank(method="dense")`
Percentile rank	`df["col"].rank(pct=True)`
Rank within groups	`df.groupby("g")["col"].rank(ascending=False)`
Cumulative sum	`df["col"].cumsum()`
Cumulative max (all-time high)	`df["col"].cummax()`
Cumulative within groups	`df.groupby("g")["col"].cumsum()`
Compound growth	`(1 + returns).cumprod()`
Exponential moving average	`df["col"].ewm(span=N, adjust=False).mean()`
Rolling correlation	`df["a"].rolling(N).corr(df["b"])`
Expanding (all-time)	`df["col"].expanding().mean()`
Create lag feature	`df["col"].shift(1)`
Day-over-day % change	`df["col"].pct_change()`
Absolute difference	`df["col"].diff()`
Second-order diff	`df["col"].diff().diff()`
Filter rows — query	`df.query("col > 100 and col2 == 'X'")`
query with variable	`df.query("col > @threshold")`
query with string in list	`df.query("col in ['A','B']")`
query with string method	`df.query("col.str.startswith('A')", engine="python")`
eval — create column	`df.eval("new = a + b", inplace=True)`
eval — multi-line	`df.eval("col1 = a*b\\ncol2 = a/b", inplace=True)`
eval — with variable	`df.eval("col = a * @rate", inplace=True)`
Use category dtype	`df["col"].astype("category")`
Select numeric cols	`df.select_dtypes(include="number")`
Safe assignment	`df.loc[condition, "col"] = value`

🐼 PHASE 10: FILE HANDLING — Complete Learning Notes

> **Goal:** Master reading from and writing to every major file format used in Data Science — CSV, Excel, JSON, SQL, Parquet, and more. Learn how to handle real-world file quirks, efficiently process large datasets, and follow production-grade I/O best practices.

🗂️ File Formats Overview

Format	Extension	Best For	Size	Speed	Human-Readable
CSV	`.csv`	Universal exchange, small–medium data	Medium	Slow	✅ Yes

Format	Extension	Best For	Size	Speed	Human-Readable
Excel	`.xlsx`, `.xls`	Business reports, stakeholders	Large	Slow	✅ Yes
JSON	`.json`	APIs, nested/hierarchical data	Medium	Medium	✅ Yes
SQL	DB tables	Relational databases, production systems	Any	Fast	❌ No
Parquet	`.parquet`	Big data, analytics, ML pipelines	Small	Very Fast	❌ No
Feather	`.feather`	Fast Pandas ↔ R interchange	Small	Very Fast	❌ No
Pickle	`.pkl`	Python-specific, preserves ALL dtypes	Medium	Fast	❌ No
HDF5	`.h5`	Scientific data, huge numeric arrays	Small	Very Fast	❌ No

> **Rule of thumb:**
>
> - Share with humans → CSV or Excel
> - Share with other systems → JSON or Parquet
> - Internal Python pipeline → Pickle or Feather
> - Production ML pipeline → Parquet (industry standard)

1. Reading CSV Files

📌 Definition

CSV (Comma-Separated Values) is the most common format for tabular data. `pd.read_csv()` is the most frequently used Pandas function in any data project.

Syntax

```
pd.read_csv(filepath_or_buffer, **options)
```

All Key Parameters

Parameter	Purpose	Example
`filepath_or_buffer`	File path or URL	`"data.csv"` or `"https://..."`
`sep`	Delimiter character	`","` (default), `"\\t"`, `";"`, `"|"`
`header`	Row to use as column names	`0` (default = first row), `None` (no header)
`names`	Custom column names	`["col1","col2"]`
`index_col`	Column(s) to use as row index	`0`, `"Date"`, `["a","b"]`
`usecols`	Select specific columns	`["Name","Salary"]` or `[0,1,3]`
`dtype`	Force column dtypes	`{"Salary": int, "Code": str}`
`parse_dates`	Parse date columns	`["Date"]` or `True`
`nrows`	Read only first N rows	`1000`
`skiprows`	Skip rows at the top	`2` or `[0,2,4]`
`skipfooter`	Skip rows at the bottom	`3`
`na_values`	Extra strings to treat as NaN	`["N/A","na","–","?"]`
`keep_default_na`	Keep Pandas default NaN strings	`True` (default)
`encoding`	File encoding	`"utf-8"`, `"latin-1"`, `"cp1252"`
`chunksize`	Read in chunks (large files)	`10000`
`compression`	Read compressed files	`"gzip"`, `"zip"`, `"bz2"`
`thousands`	Thousands separator character	`","`
`decimal`	Decimal separator character	`"."` (EU: `","`)
`comment`	Skip lines starting with char	`"#"`

Parameter	Purpose	Example
`low_memory`	Faster but infers types per chunk	`False` (safer)
`engine`	Parser engine	`"c"` (fast), `"python"` (flexible)

Examples

```
import pandas as pd
import numpy as np
import io

# ── Create sample CSV content for demonstration ──────────────────────
csv_content = """EmployeeID,Name,Department,Salary,JoinDate,Active
101,Alice Johnson,Engineering,90000,2020-03-15,Yes
102,Bob Smith,Marketing,60000,2021-07-22,No
103,Carol White,Engineering,95000,2019-01-10,Yes
104,Dave Brown,HR,55000,2022-11-05,Yes
105,Eve Davis,Marketing,62000,2020-06-30,No
"""

# Simulate reading from file using StringIO
from io import StringIO

# Example 1: Basic read_csv
df = pd.read_csv(StringIO(csv_content))
print("Example 1 – Basic read:")
print(df)
print("\\nDtypes:")
print(df.dtypes)
```

Output:

```
Example 1 – Basic read:
   EmployeeID            Name   Department  Salary    JoinDate Active
0         101  Alice Johnson   Engineering   90000  2020-03-15    Yes
1         102     Bob Smith      Marketing   60000  2021-07-22     No
2         103  Carol White     Engineering   95000  2019-01-10    Yes
3         104   Dave Brown             HR   55000  2022-11-05    Yes
4         105    Eve Davis      Marketing   62000  2020-06-30     No

Dtypes:
EmployeeID     int64
Name          object
Department    object
Salary         int64
JoinDate      object   ← still string! Need parse_dates
Active        object
dtype: object
```

```
# Example 2: parse_dates + index_col + usecols
df = pd.read_csv(
    StringIO(csv_content),
    parse_dates  = ["JoinDate"],     # auto-parse date column
    index_col    = "EmployeeID",     # set as index
    usecols      = ["EmployeeID","Name","Salary","JoinDate"]  # only these columns
)
print("Example 2 – Selected columns + parsed dates:")
print(df)
print("\\nJoinDate dtype:", df["JoinDate"].dtype)
```

Output:

```
Example 2 — Selected columns + parsed dates:
                    Name  Salary   JoinDate
EmployeeID
101         Alice Johnson   90000 2020-03-15
102            Bob Smith    60000 2021-07-22
103          Carol White    95000 2019-01-10
104           Dave Brown    55000 2022-11-05
105            Eve Davis    62000 2020-06-30

JoinDate dtype: datetime64[ns]
```

```python
# Example 3: dtype — force specific types (prevent wrong inference)
df = pd.read_csv(
    StringIO(csv_content),
    dtype  = {
        "EmployeeID" : str,       # keep as string (e.g., "00101" wouldn't lose leadin
g zero)
        "Salary"     : float,     # read as float
        "Department" : "category" # directly as category
    }
)
print("Example 3 — Forced dtypes:")
print(df.dtypes)
```

Output:

```
EmployeeID      object      ← string as requested
Name            object
Department    category      ← category directly!
Salary          float64     ← float as requested
JoinDate         object
Active           object
```

```python
# Example 4: Handle messy/real-world CSV
messy_csv = """# Company HR Export — Generated 2024-01-01
# Do not edit manually
EmployeeID;Name;Salary;Status
101;Alice Johnson;90,000;Active
102;Bob Smith;60,000;Inactive
103;Carol White;95,000;Active
N/A;Unknown;N/A;Inactive
"""

df = pd.read_csv(
    StringIO(messy_csv),
    sep         = ";",              # semicolon delimiter
    skiprows    = 2,                # skip 2 comment lines
    na_values   = ["N/A","n/a",""], # treat these as NaN
    thousands   = ",",              # "90,000" → 90000
    comment     = "#"               # skip lines starting with #
)
print("Example 4 — Messy CSV:")
print(df)
print("\\nMissing values:\\n", df.isnull().sum())
```

Output:

```
Example 4 — Messy CSV:
   EmployeeID           Name   Salary    Status
0       101.0  Alice Johnson  90000.0    Active
1       102.0     Bob Smith   60000.0  Inactive
```

```
2       103.0   Carol White  95000.0    Active
3         NaN        NaN        NaN    Inactive

Missing values:
EmployeeID    1
Name          1
Salary        1
Status        0
```

```python
# Example 5: Read only first N rows (fast preview of huge files)
df_head = pd.read_csv(StringIO(csv_content), nrows=3)
print("Example 5 — First 3 rows only:")
print(df_head)
```

```python
# Example 6: Read from URL directly
url = "<https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv>"
# df_url = pd.read_csv(url)  # works directly!

# Example 7: Read compressed files
# df_gz = pd.read_csv("data.csv.gz", compression="gzip")
# df_zip = pd.read_csv("data.zip")  # auto-detects .zip
```

```python
# Example 8: Tab-separated values (TSV)
tsv_content = "Name\\tAge\\tCity\\nAlice\\t28\\tMumbai\\nBob\\t35\\tDelhi\\n"
df_tsv = pd.read_csv(StringIO(tsv_content), sep="\\t")
print("Example 8 — TSV file:")
print(df_tsv)
```

Output:

```
Example 8 — TSV file:
    Name  Age    City
0  Alice   28  Mumbai
1    Bob   35   Delhi
```

```python
# Example 9: skipfooter — skip trailing summary rows
footer_csv = """Name,Salary
Alice,90000
Bob,60000
Carol,95000
--- END OF REPORT ---
Total Records: 3
"""

df = pd.read_csv(
    StringIO(footer_csv),
    skipfooter = 2,     # skip last 2 rows
    engine     = "python"  # required for skipfooter
)
print("Example 9 — skip footer rows:")
print(df)
```

Output:

```
Example 9 — skip footer rows:
    Name  Salary
0  Alice   90000
```

```
1    Bob   60000
2  Carol   95000
```

```python
# Example 10: No header row in CSV
no_header = "101,Alice,Engineering,90000\\n102,Bob,Marketing,60000\\n"
df = pd.read_csv(
    StringIO(no_header),
    header = None,                                      # no header row
    names  = ["EmpID","Name","Department","Salary"]   # assign column names
)
print("Example 10 — No header CSV:")
print(df)
```

Output:

```
Example 10 — No header CSV:
   EmpID   Name   Department  Salary
0    101  Alice  Engineering   90000
1    102    Bob    Marketing   60000
```

⚠️ Common Mistakes — read_csv()

Mistake	Explanation
Dates load as `object`	Use `parse_dates=["col"]` — not automatic
ID columns lose leading zeros	Force `dtype={"ID": str}` — int drops them
`UnicodeDecodeError`	Try `encoding="latin-1"` or `"cp1252"` for Windows CSV files
`DtypeWarning` — mixed types	Use `dtype=` to force correct type; or `low_memory=False`
Thousands separator confusion	`"90,000"` stays as string unless `thousands=","` is set
Wrong delimiter	Specify `sep="\\t"` for TSV, `sep=";"` for European CSV

---

2. Writing CSV Files

Syntax

```python
df.to_csv(filepath, **options)
```

Parameter	Purpose	Example
`path_or_buf`	File path or buffer	`"output.csv"`
`sep`	Delimiter	`","` (default)
`index`	Include row index	`False` (almost always!)
`columns`	Select columns to write	`["Name","Salary"]`
`header`	Include header row	`True` (default)
`encoding`	File encoding	`"utf-8"`, `"utf-8-sig"`
`na_rep`	String for NaN	`"N/A"`
`float_format`	Float formatting	`"%.2f"`
`date_format`	Date formatting	`"%Y-%m-%d"`
`compression`	Compress output	`"gzip"`, `"zip"`
`chunksize`	Write in chunks	`10000`
`mode`	Write mode	`"w"` (overwrite), `"a"` (append)

Examples

```python
import pandas as pd

df = pd.DataFrame({
    "Name"     : ["Alice","Bob","Carol"],
    "Salary"   : [90000, 60000, 95000],
    "JoinDate" : pd.to_datetime(["2020-01-15","2021-06-30","2019-03-10"]),
    "Score"    : [4.512345, 3.876543, 4.710000]
})

# Example 1: Basic to_csv (most common usage)
df.to_csv("employees.csv", index=False)  # index=False → NO row numbers in file
print("✅ Saved employees.csv")

# Verify
print(pd.read_csv("employees.csv"))
```

Output (employees.csv):

```
Name,Salary,JoinDate,Score
Alice,90000,2020-01-15,4.512345
Bob,60000,2021-06-30,3.876543
Carol,95000,2019-03-10,4.710000
```

```python
# Example 2: Control float precision and date format
df.to_csv(
    "employees_formatted.csv",
    index       = False,
    float_format= "%.2f",          # round floats to 2 decimal places
    date_format = "%d/%m/%Y",      # dates as DD/MM/YYYY
    na_rep      = "MISSING"        # represent NaN as "MISSING"
)
print("✅ Saved formatted CSV")
```

Output (employees_formatted.csv):

```
Name,Salary,JoinDate,Score
Alice,90000,15/01/2020,4.51
Bob,60000,30/06/2021,3.88
Carol,95000,10/03/2019,4.71
```

```python
# Example 3: Write ONLY selected columns
df.to_csv("names_salaries.csv", columns=["Name","Salary"], index=False)

# Example 4: Append rows to existing CSV
new_row = pd.DataFrame({"Name":["Dave"],"Salary":[55000],
                        "JoinDate":["2022-11-05"],"Score":[4.0]})
new_row.to_csv("employees.csv", mode="a", header=False, index=False)
# mode="a" → append; header=False → don't rewrite column names

# Example 5: Save as TSV (tab-separated)
df.to_csv("employees.tsv", sep="\\t", index=False)

# Example 6: Save compressed (gzip) — saves 60-80% file size
df.to_csv("employees.csv.gz", index=False, compression="gzip")

# Example 7: UTF-8 with BOM (for Excel compatibility on Windows)
```

```python
df.to_csv("employees_excel.csv", index=False, encoding="utf-8-sig")
# utf-8-sig → Excel opens it correctly with special characters (accents, etc.)
```

---

3. Reading Excel Files

📌 Definition

Excel is the most common format in business analytics. Pandas handles both `.xlsx` (openpyxl) and `.xls` (xlrd) formats.

> **Required library:** `pip install openpyxl` for `.xlsx` files

Syntax

```python
pd.read_excel(io, sheet_name, **options)
```

Parameter	Purpose	Example
`io`	File path	`"data.xlsx"`
`sheet_name`	Which sheet to read	`0`, `"Sheet1"`, `["Sheet1","Sheet2"]`, `None` (all)
`header`	Row number for column names	`0` (default)
`index_col`	Column as row index	`0`, `"EmpID"`
`usecols`	Select columns	`"A:D"`, `[0,1,3]`, `["Name","Salary"]`
`dtype`	Force data types	`{"Salary": int}`
`parse_dates`	Parse date columns	`["JoinDate"]`
`skiprows`	Skip top rows	`2`
`nrows`	Max rows to read	`1000`
`na_values`	Extra NaN strings	`["N/A",""]`
`engine`	Parser engine	`"openpyxl"` (xlsx), `"xlrd"` (xls)

Examples

```python
# Example 1: Read first sheet (default)
df = pd.read_excel("employees.xlsx")
print(df.head())

# Example 2: Read specific sheet by name
df = pd.read_excel("employees.xlsx", sheet_name="HR_Data")

# Example 3: Read specific sheet by index (0-based)
df = pd.read_excel("employees.xlsx", sheet_name=0)   # first sheet
df = pd.read_excel("employees.xlsx", sheet_name=-1)  # last sheet

# Example 4: Read ALL sheets — returns a dict {sheet_name: DataFrame}
all_sheets = pd.read_excel("report.xlsx", sheet_name=None)
print("Sheet names:", list(all_sheets.keys()))
for name, sheet_df in all_sheets.items():
    print(f"\\nSheet '{name}': {sheet_df.shape}")

# Example 5: Read multiple specific sheets
two_sheets = pd.read_excel("report.xlsx", sheet_name=["Q1","Q2"])
q1_df = two_sheets["Q1"]
q2_df = two_sheets["Q2"]

# Example 6: Excel column range with usecols
df = pd.read_excel(
    "employees.xlsx",
```

```
    usecols      = "A:D",          # columns A through D
    parse_dates  = ["JoinDate"],
    dtype        = {"EmployeeID": str},
    skiprows     = 1,              # skip header decoration row
    nrows        = 500             # read max 500 rows
)

# Example 7: Read Excel with multi-row header
df = pd.read_excel(
    "report.xlsx",
    header = [0, 1],    # use rows 0 AND 1 as multi-level column header
    index_col = 0
)
print("MultiIndex columns:", df.columns.tolist())
```

⚠️ Common Mistakes — read_excel()

Mistake	Explanation
`ModuleNotFoundError: openpyxl`	Install with `pip install openpyxl` for .xlsx
Sheet name is wrong	Use `sheet_name=None` first to list all sheets
Merged cells cause NaN	Unmerge in Excel before reading
Dates read as integers	Excel stores dates as integers — use `parse_dates=["col"]`
Hidden rows/columns included	Pandas reads them — must filter afterwards

4. Writing Excel Files

Syntax

```
df.to_excel(filepath, sheet_name, **options)
```

Examples

```
# Example 1: Basic Excel write
df.to_excel("output.xlsx", index=False)
print("✅ Saved output.xlsx")

# Example 2: Specify sheet name
df.to_excel("output.xlsx", sheet_name="Employee Data", index=False)

# Example 3: Write MULTIPLE DataFrames to MULTIPLE sheets
df1 = pd.DataFrame({"Name":["Alice","Bob"],"Salary":[90000,60000]})
df2 = pd.DataFrame({"Product":["Laptop","Phone"],"Revenue":[120000,85000]})
df3 = pd.DataFrame({"Month":["Jan","Feb"],"Total":[340000,288000]})

with pd.ExcelWriter("multi_sheet_report.xlsx", engine="openpyxl") as writer:
    df1.to_excel(writer, sheet_name="Employees",  index=False)
    df2.to_excel(writer, sheet_name="Products",   index=False)
    df3.to_excel(writer, sheet_name="Monthly",    index=False)

print("✅ Saved 3-sheet Excel workbook")
```

> `ExcelWriter` **as context manager** (`with` statement) is the standard pattern for writing multiple sheets — it ensures the file is properly saved and closed even if an error occurs.

```python
# Example 4: Append to existing Excel (new sheet in same file)
with pd.ExcelWriter(
    "multi_sheet_report.xlsx",
    engine = "openpyxl",
    mode   = "a"           # 'a' = append (don't overwrite)
) as writer:
    df3.to_excel(writer, sheet_name="Q4_Summary", index=False)

print("✅ Added new sheet to existing workbook")

# Example 5: Write with formatting using openpyxl
from openpyxl.styles import Font, PatternFill
from openpyxl.utils.dataframe import dataframe_to_rows

wb = __import__("openpyxl").Workbook()
ws = wb.active
ws.title = "Styled Report"

# Write header with bold + yellow fill
header_font  = Font(bold=True, color="FFFFFF")
header_fill  = PatternFill("solid", fgColor="1F4E79")

for col_idx, col_name in enumerate(df1.columns, start=1):
    cell = ws.cell(row=1, column=col_idx, value=col_name)
    cell.font = header_font
    cell.fill = header_fill

# Write data rows
for row_idx, row in enumerate(dataframe_to_rows(df1, index=False, header=False), start
=2):
    for col_idx, value in enumerate(row, start=1):
        ws.cell(row=row_idx, column=col_idx, value=value)

wb.save("styled_report.xlsx")
print("✅ Saved styled Excel report")

# Example 6: Float and date formatting in Excel
with pd.ExcelWriter("formatted.xlsx", engine="openpyxl",
                    datetime_format="DD/MM/YYYY",
                    float_format="%.2f") as writer:
    df.to_excel(writer, index=False)
```

5. Reading JSON Files

📌 Definition

JSON (JavaScript Object Notation) is the standard format for API responses and web data. It can represent **nested/hierarchical** structures that CSV cannot.

Syntax

```python
pd.read_json(path_or_buf, orient, **options)
```

JSON Orientations

`orient=`	JSON Structure	When to Use
`"records"`	`[{col:val,...}, ...]`	API responses (most common)

`orient=`	JSON Structure	When to Use
`"columns"`	`{col: {idx:val,...}, ...}`	Default Pandas export
`"index"`	`{idx: {col:val,...}, ...}`	Index-based
`"values"`	`[[val, val, ...], ...]`	Raw matrix
`"split"`	`{columns:[...], index:[...], data:[[...]]}`	Preserves all info
`"table"`	Full schema + data	Best for preserving dtypes

Examples

```
import json

# ── Sample JSON structures ──────────────────────────────────────────

# records orientation (most common API format)
json_records = """[
    {"EmployeeID": 101, "Name": "Alice", "Salary": 90000, "Department": "Engineerin
g"},
    {"EmployeeID": 102, "Name": "Bob",   "Salary": 60000, "Department": "Marketing"},
    {"EmployeeID": 103, "Name": "Carol", "Salary": 95000, "Department": "Engineering"}
]"""

# Example 1: Read records-oriented JSON
df = pd.read_json(StringIO(json_records), orient="records")
print("Example 1 — JSON records:")
print(df)
```

Output:

```
Example 1 — JSON records:
   EmployeeID   Name  Salary   Department
0         101  Alice   90000  Engineering
1         102    Bob   60000    Marketing
2         103  Carol   95000  Engineering
```

```
# Example 2: Read JSON from URL (API response)
# url = "<https://api.example.com/employees>"
# df = pd.read_json(url)

# Example 3: Nested JSON — normalize with json_normalize
nested_json = """[
    {
        "id": 101,
        "name": "Alice",
        "address": {"city": "Mumbai", "zip": "400001"},
        "skills": ["Python", "SQL", "Pandas"]
    },
    {
        "id": 102,
        "name": "Bob",
        "address": {"city": "Delhi", "zip": "110001"},
        "skills": ["Java", "Spark"]
    }
]"""

data = json.loads(nested_json)

# json_normalize flattens nested JSON
df_flat = pd.json_normalize(data)
```

```
print("Example 3 — Flattened nested JSON:")
print(df_flat)
print("\\nColumns:", df_flat.columns.tolist())
```

Output:

```
Example 3 — Flattened nested JSON:
    id   name address.city address.zip            skills
0  101  Alice       Mumbai      400001  [Python, SQL, Pandas]
1  102    Bob        Delhi      110001        [Java, Spark]

Columns: ['id', 'name', 'address.city', 'address.zip', 'skills']
```

> `pd.json_normalize()` automatically flattens one level of nesting — `address.city`, `address.zip` become their own columns with dot-separated names.

```
# Example 4: Deep nesting with record_path and meta
orders_json = [
    {
        "customer": "Alice",
        "orders": [
            {"product": "Laptop", "amount": 90000},
            {"product": "Mouse",  "amount": 1500}
        ]
    },
    {
        "customer": "Bob",
        "orders": [
            {"product": "Phone", "amount": 60000}
        ]
    }
]

df_orders = pd.json_normalize(
    orders_json,
    record_path = "orders",        # the nested list to expand
    meta        = ["customer"],    # keep this field from outer level
)
print("Example 4 — Deep nested JSON:")
print(df_orders)
```

Output:

```
Example 4 — Deep nested JSON:
  product  amount customer
0  Laptop   90000    Alice
1   Mouse    1500    Alice
2   Phone   60000      Bob
```

> **Output Explanation:** Each order becomes its own row, with `customer` repeated from the outer level. This is the **one-to-many relationship flattening** pattern used for every API response with nested arrays.

```
# Example 5: Handle JSON with parse_dates
json_with_dates = """[
    {"name": "Alice", "join_date": "2020-03-15", "salary": 90000},
    {"name": "Bob",   "join_date": "2021-07-22", "salary": 60000}
]"""

df = pd.read_json(
    StringIO(json_with_dates),
```

```
    orient      = "records",
    convert_dates = ["join_date"]   # parse date columns
)
print("Example 5 — JSON with dates:")
print(df.dtypes)
```

Output:

```
name         object
join_date    datetime64[ns]
salary       int64
dtype: object
```

---

6. Writing JSON Files

Examples

```
# Example 1: to_json with records orientation (best for APIs / readability)
df.to_json("employees.json", orient="records", indent=2)
print("✅ Saved employees.json")

# Peek at the output
with open("employees.json","r") as f:
    print(f.read())
```

Output (employees.json):

```
[
  {
    "name": "Alice",
    "join_date": 1584230400000,
    "salary": 90000
  },
  {
    "name": "Bob",
    "join_date": 1626912000000,
    "salary": 60000
  }
]
```

```
# Example 2: Write with date_format for human-readable dates
df.to_json(
    "employees_readable.json",
    orient      = "records",
    indent      = 2,
    date_format = "iso"    # "iso" → "2020-03-15T00:00:00.000Z"
)

# Example 3: lines=True — JSON Lines format (one JSON per line — better for big data)
# Great for streaming, Spark, AWS, and log files
df.to_json("employees.jsonl", orient="records", lines=True)
# Output: one JSON object per line (no outer brackets)
# {"name":"Alice","salary":90000}
# {"name":"Bob","salary":60000}

# Example 4: Preserve all dtypes perfectly
df.to_json("employees_typed.json", orient="table", indent=2)
```

```
df_back = pd.read_json("employees_typed.json", orient="table")
# orient="table" includes schema info — best roundtrip fidelity
```

7. Reading & Writing SQL Databases

Syntax

```
# Reading
pd.read_sql(sql_query, connection)
pd.read_sql_table(table_name, connection)
pd.read_sql_query(query_string, connection)

# Writing
df.to_sql(table_name, connection, if_exists="replace")
```

Examples

```
import sqlite3

# Example 1: Create a SQLite database and write DataFrame
conn = sqlite3.connect("company.db")    # creates file if not exists

df_emp = pd.DataFrame({
    "EmployeeID" : [101, 102, 103, 104, 105],
    "Name"       : ["Alice","Bob","Carol","Dave","Eve"],
    "Department" : ["Engineering","Marketing","Engineering","HR","Marketing"],
    "Salary"     : [90000, 60000, 95000, 55000, 62000]
})

# Write to SQL table
df_emp.to_sql(
    "employees",
    con       = conn,
    if_exists = "replace",   # "replace" / "append" / "fail"
    index     = False
)
print("✅ Written to SQL table 'employees'")
```

```
# Example 2: Read entire table
df_from_sql = pd.read_sql_table("employees", conn)
print("Example 2 — Read full table:")
print(df_from_sql)
```

Output:

```
   EmployeeID   Name   Department  Salary
0         101  Alice  Engineering   90000
1         102    Bob    Marketing   60000
2         103  Carol  Engineering   95000
3         104   Dave           HR   55000
4         105    Eve    Marketing   62000
```

```
# Example 3: Read with SQL query
df_eng = pd.read_sql(
    "SELECT Name, Salary FROM employees WHERE Department = 'Engineering' ORDER BY Sala
ry DESC",
```

```
    conn
)
print("Example 3 — SQL filtered query:")
print(df_eng)
```

Output:

```
Example 3 — SQL filtered query:
    Name  Salary
0  Carol   95000
1  Alice   90000
```

```
# Example 4: Parameterized query (safe — prevents SQL injection)
dept = "Marketing"
df_mkt = pd.read_sql(
    "SELECT * FROM employees WHERE Department = ?",
    conn,
    params = (dept,)
)
print("Example 4 — Parameterized query:")
print(df_mkt)

# Example 5: Append new rows to existing table
new_emp = pd.DataFrame({
    "EmployeeID":[106],"Name":["Frank"],"Department":["Finance"],"Salary":[88000]
})
new_emp.to_sql("employees", conn, if_exists="append", index=False)
print("✅ Appended 1 row")

# Close connection when done
conn.close()
```

---

8. Other Formats

Parquet — Industry Standard for Big Data

```
# Parquet: columnar format, compressed, fast, preserves dtypes
# Required: pip install pyarrow

# Write
df.to_parquet("employees.parquet", index=False, compression="snappy")

# Read
df_p = pd.read_parquet("employees.parquet")

# Read specific columns only (massive performance gain on wide tables)
df_cols = pd.read_parquet("employees.parquet", columns=["Name","Salary"])

# Why Parquet?
# ✅ 10x smaller than CSV
# ✅ 10-50x faster to read
# ✅ Preserves all dtypes (int, float, datetime, category, bool)
# ✅ Industry standard for Spark, AWS S3, Google BigQuery
```

Feather — Fastest for Pandas ↔ R Interchange

```
# Feather: ultra-fast read/write (faster than Parquet)
# Required: pip install pyarrow

df.to_feather("employees.feather")
df_f = pd.read_feather("employees.feather")

# Best for: intermediate results you want to cache during analysis
```

Pickle — Preserves ALL Python Object Types

```
# Pickle: saves Python object as-is (not just data — also categories, MultiIndex, et
c.)
df.to_pickle("employees.pkl")
df_pkl = pd.read_pickle("employees.pkl")

# ⚠ WARNING: Never unpickle files from untrusted sources — security risk
# Use only for your own internal Python pipelines
```

HTML — Read Tables from Web Pages

```
# Read all <table> elements from a web page or HTML string
tables = pd.read_html("<https://en.wikipedia.org/wiki/List_of_countries_by_GDP>")
# Returns a list of DataFrames — one per <table> on the page
df_gdp = tables[0]  # first table

# Read from local HTML string
html_str = """
<table>
  <tr><th>Name</th><th>Score</th></tr>
  <tr><td>Alice</td><td>92</td></tr>
  <tr><td>Bob</td><td>85</td></tr>
</table>
"""
df_html = pd.read_html(html_str)[0]
print(df_html)

# Write to HTML
df.to_html("report.html", index=False, border=1)
```

9. Handling Large Datasets

📌 Definition

When a dataset is too large to fit in memory (~70%+ of RAM), you need strategies to process it efficiently without a crash.

Strategy 1: ChunkSize — Process in Batches

```
# Example 1: Read a large CSV in chunks and aggregate
total_sales = 0
row_count   = 0

for chunk in pd.read_csv("large_sales.csv", chunksize=10000):
    # Process each chunk
    total_sales += chunk["Sales"].sum()
    row_count   += len(chunk)
```

```
    print(f"Processed {row_count:,} rows...")

print(f"\\nTotal rows  : {row_count:,}")
print(f"Total sales : {total_sales:,}")
```

```
# Example 2: Filter while reading in chunks (keep only matching rows)
matching_chunks = []

for chunk in pd.read_csv("large_sales.csv", chunksize=10000):
    filtered = chunk[chunk["Region"] == "North"]
    matching_chunks.append(filtered)

north_sales = pd.concat(matching_chunks, ignore_index=True)
print("North region rows:", len(north_sales))
```

```
# Example 3: Aggregate per chunk (groupby in chunks)
from collections import defaultdict

dept_totals = defaultdict(float)

for chunk in pd.read_csv("large_sales.csv", chunksize=10000):
    chunk_grouped = chunk.groupby("Department")["Revenue"].sum()
    for dept, rev in chunk_grouped.items():
        dept_totals[dept] += rev

# Convert to DataFrame
dept_summary = pd.DataFrame(dept_totals.items(), columns=["Department","TotalRevenu
e"])
print(dept_summary.sort_values("TotalRevenue", ascending=False))
```

Strategy 2: Reduce Memory Usage with dtype Optimization

```
# Example 4: Diagnose memory usage per column
print("Memory usage per column:")
print(df.memory_usage(deep=True).sort_values(ascending=False))
print(f"\\nTotal: {df.memory_usage(deep=True).sum() / 1024 / 1024:.2f} MB")
```

```
# Example 5: Optimize dtypes — can reduce memory by 50-80%
def optimize_dtypes(df):
    """Automatically downcast numeric types and convert low-cardinality strings to cat
egory"""
    result = df.copy()

    for col in result.columns:
        col_type = result[col].dtype

        # Downcast integers
        if col_type in ["int64","int32"]:
            result[col] = pd.to_numeric(result[col], downcast="integer")

        # Downcast floats
        elif col_type in ["float64","float32"]:
            result[col] = pd.to_numeric(result[col], downcast="float")

        # Convert low-cardinality strings to category
        elif col_type == object:
```

```
            n_unique = result[col].nunique()
            if n_unique / len(result) < 0.5:  # < 50% unique values
                result[col] = result[col].astype("category")

    return result

# Apply optimization
df_big = pd.DataFrame({
    "ID"        : range(100000),
    "Department": np.random.choice(["Eng","Mkt","HR","Fin"], 100000),
    "Salary"    : np.random.randint(50000, 100000, 100000).astype("int64"),
    "Rating"    : np.random.uniform(3.0, 5.0, 100000).astype("float64"),
    "Active"    : np.random.choice([True,False], 100000)
})

mem_before = df_big.memory_usage(deep=True).sum() / 1024 / 1024
df_opt = optimize_dtypes(df_big)
mem_after = df_opt.memory_usage(deep=True).sum() / 1024 / 1024

print(f"Memory before: {mem_before:.2f} MB")
print(f"Memory after : {mem_after:.2f} MB")
print(f"Reduction    : {(1 - mem_after/mem_before)*100:.1f}%")
print("\\nOptimized dtypes:")
print(df_opt.dtypes)
```

Output:

```
Memory before: 7.63 MB
Memory after : 1.81 MB
Reduction    : 76.3%

Optimized dtypes:
ID            int32      ← was int64
Department   category    ← was object
Salary        int32      ← was int64
Rating       float32     ← was float64
Active           bool
```

Strategy 3: Use Parquet Instead of CSV for Large Files

```
# Example 6: Benchmark — CSV vs Parquet on 1M rows
import time

df_million = pd.DataFrame({
    "ID"    : range(1_000_000),
    "Name"  : np.random.choice(["Alice","Bob","Carol"], 1_000_000),
    "Salary": np.random.randint(50000, 100000, 1_000_000),
    "Date"  : pd.date_range("2020-01-01", periods=1_000_000, freq="min")
})

# Save & read CSV
t0 = time.time()
df_million.to_csv("million.csv", index=False)
t1 = time.time()
_ = pd.read_csv("million.csv")
t2 = time.time()
csv_write = t1 - t0
csv_read  = t2 - t1

# Save & read Parquet
t3 = time.time()
```

```python
df_million.to_parquet("million.parquet", index=False)
t4 = time.time()
_ = pd.read_parquet("million.parquet")
t5 = time.time()
pq_write = t4 - t3
pq_read  = t5 - t4

import os
csv_size = os.path.getsize("million.csv") / 1024 / 1024
pq_size  = os.path.getsize("million.parquet") / 1024 / 1024

print(f"CSV    → Write: {csv_write:.1f}s | Read: {csv_read:.1f}s | Size: {csv_size:.1f}
MB")
print(f"Parquet→ Write: {pq_write:.1f}s | Read: {pq_read:.1f}s | Size: {pq_size:.1f}M
B")
```

Output (approximate):

```
CSV    → Write: 8.2s  | Read: 4.1s  | Size: 47.3MB
Parquet→ Write: 0.9s  | Read: 0.4s  | Size: 12.1MB
```

> **Parquet is 4x faster to write, 10x faster to read, and 4x smaller than CSV** — use it for any dataset you'll read more than once.

Strategy 4: usecols — Don't Load What You Don't Need

```python
# Example 7: Load only needed columns (saves memory immediately at load time)
# If file has 50 columns and you only need 4:
df_small = pd.read_csv(
    "large_file.csv",
    usecols = ["Date","Region","Sales","Product"]  # ← load only 4 of 50 cols
)
# Memory reduction: (4/50) * 100 = 92% smaller!
```

Strategy 5: Iterator on Large Files

```python
# Example 8: TextFileReader iterator pattern
reader = pd.read_csv("large_file.csv", chunksize=50000, iterator=True)

results = []
try:
    while True:
        chunk = reader.get_chunk()
        # process chunk
        agg = chunk.groupby("Category")["Sales"].sum()
        results.append(agg)
except StopIteration:
    print("All chunks processed")

final = pd.concat(results).groupby(level=0).sum()
print(final)
```

---

10. Real-World File Handling Patterns

Pattern 1: Safe Read with Validation

```python
import os

def safe_read_csv(filepath, required_cols=None, **kwargs):
    """Read CSV with validation — raises informative errors"""
    # Check file exists
    if not os.path.exists(filepath):
        raise FileNotFoundError(f"File not found: {filepath}")

    # Check file is not empty
    if os.path.getsize(filepath) == 0:
        raise ValueError(f"File is empty: {filepath}")

    # Read file
    df = pd.read_csv(filepath, **kwargs)

    # Check required columns exist
    if required_cols:
        missing = set(required_cols) - set(df.columns)
        if missing:
            raise ValueError(f"Missing required columns: {missing}")

    print(f"✅ Loaded {filepath}: {df.shape[0]:,} rows × {df.shape[1]} columns")
    return df

# Usage
# df = safe_read_csv("sales.csv", required_cols=["Date","Region","Sales"])
```

Pattern 2: Combine Multiple CSV Files

```python
import glob

def combine_csv_files(pattern, **read_kwargs):
    """Read and combine multiple CSV files matching a pattern"""
    files = sorted(glob.glob(pattern))

    if not files:
        raise ValueError(f"No files found matching: {pattern}")

    print(f"Found {len(files)} files: {[os.path.basename(f) for f in files]}")

    dfs = []
    for f in files:
        chunk_df = pd.read_csv(f, **read_kwargs)
        chunk_df["source_file"] = os.path.basename(f)  # track origin
        dfs.append(chunk_df)

    combined = pd.concat(dfs, ignore_index=True)
    print(f"✅ Combined: {combined.shape[0]:,} rows from {len(files)} files")
    return combined

# Usage — combine all monthly CSV files
# df_all = combine_csv_files("data/sales_2024_*.csv", parse_dates=["Date"])
```

Pattern 3: Professional Excel Report Generator

```python
from openpyxl.styles import Font, Alignment, PatternFill, Border, Side
from openpyxl.utils import get_column_letter
```

```python
def generate_excel_report(dataframes_dict, filename):
    """
    Generate a professional Excel report with multiple sheets.
    dataframes_dict: {"Sheet Name": dataframe, ...}
    """
    with pd.ExcelWriter(filename, engine="openpyxl") as writer:
        for sheet_name, df in dataframes_dict.items():
            df.to_excel(writer, sheet_name=sheet_name, index=False)

            # Style the worksheet
            ws = writer.sheets[sheet_name]

            # Header styling
            for col_idx in range(1, len(df.columns) + 1):
                cell = ws.cell(row=1, column=col_idx)
                cell.font      = Font(bold=True, color="FFFFFF", size=11)
                cell.fill      = PatternFill("solid", fgColor="2E75B6")
                cell.alignment = Alignment(horizontal="center")

            # Auto-fit column widths
            for col_idx, col_name in enumerate(df.columns, start=1):
                max_len = max(df[col_name].astype(str).map(len).max(), len(col_name))
+ 2
                ws.column_dimensions[get_column_letter(col_idx)].width = min(max_len,
40)

    print(f"✅ Professional report saved: {filename}")

# Usage
# generate_excel_report({
#     "Employees"   : df_employees,
#     "Monthly Sales": df_sales,
#     "Summary"     : df_summary
# }, "quarterly_report_Q1_2024.xlsx")
```

Mini Summary & Cheat Sheet

🔑 Phase 10 Key Takeaways

- ✅ `pd.read_csv()` — always set `parse_dates`, `dtype`, `encoding` for clean load
- ✅ `df.to_csv(index=False)` — `index=False` is almost always what you want
- ✅ `pd.read_excel(sheet_name=None)` — read ALL sheets as dict
- ✅ `ExcelWriter` **with** `with` **statement** — for multi-sheet writes
- ✅ `pd.json_normalize()` — flatten nested JSON from APIs
- ✅ `pd.read_sql(query, conn)` — bring SQL data into Pandas
- ✅ `df.to_sql(if_exists="append")` — update existing table
- ✅ **Parquet** — always prefer over CSV for large datasets (smaller, faster, typed)
- ✅ `chunksize` — process large files without memory crash
- ✅ **dtype optimization** → `category` for strings, downcast int/float → saves 50–80% RAM

🔖 Quick Reference Cheat Sheet

Task	Code
Basic CSV read	`pd.read_csv("file.csv")`

Task	Code
With date parsing	`pd.read_csv("f.csv", parse_dates=["Date"])`
Set as index	`pd.read_csv("f.csv", index_col="Date")`
Select columns	`pd.read_csv("f.csv", usecols=["A","B"])`
Custom delimiter	`pd.read_csv("f.csv", sep=";")`
Force dtypes	`pd.read_csv("f.csv", dtype={"ID": str})`
Custom NaN values	`pd.read_csv("f.csv", na_values=["N/A","?"])`
Skip top rows	`pd.read_csv("f.csv", skiprows=2)`
Skip bottom rows	`pd.read_csv("f.csv", skipfooter=2, engine="python")`
Read compressed	`pd.read_csv("file.csv.gz")`
No header	`pd.read_csv("f.csv", header=None, names=["a","b"])`
Thousands separator	`pd.read_csv("f.csv", thousands=",")`
Basic CSV write	`df.to_csv("out.csv", index=False)`
With float format	`df.to_csv("out.csv", index=False, float_format="%.2f")`
Append mode	`df.to_csv("out.csv", mode="a", header=False, index=False)`
Compressed write	`df.to_csv("out.csv.gz", index=False, compression="gzip")`
Excel for Windows	`df.to_csv("out.csv", encoding="utf-8-sig")`
Read Excel	`pd.read_excel("file.xlsx")`
Read specific sheet	`pd.read_excel("f.xlsx", sheet_name="Q1")`
Read all sheets	`pd.read_excel("f.xlsx", sheet_name=None)`
Write Excel	`df.to_excel("out.xlsx", index=False)`
Multi-sheet write	`with pd.ExcelWriter("f.xlsx") as w: df.to_excel(w, sheet_name="S1")`
Append sheet	`pd.ExcelWriter("f.xlsx", engine="openpyxl", mode="a")`
Read JSON	`pd.read_json("file.json", orient="records")`
Flatten nested JSON	`pd.json_normalize(data)`
Deep nested JSON	`pd.json_normalize(data, record_path="orders", meta=["customer"])`
Write JSON	`df.to_json("out.json", orient="records", indent=2)`
JSON Lines format	`df.to_json("out.jsonl", orient="records", lines=True)`
Read SQL	`pd.read_sql("SELECT * FROM table", conn)`
Write SQL	`df.to_sql("table", conn, if_exists="replace", index=False)`
Append SQL	`df.to_sql("table", conn, if_exists="append", index=False)`
Read Parquet	`pd.read_parquet("file.parquet")`
Read specific cols	`pd.read_parquet("f.parquet", columns=["A","B"])`
Write Parquet	`df.to_parquet("out.parquet", index=False, compression="snappy")`
Pickle save	`df.to_pickle("out.pkl")`
Pickle load	`pd.read_pickle("out.pkl")`
Read HTML table	`pd.read_html("url_or_string")[0]`
Memory per column	`df.memory_usage(deep=True)`
Category dtype	`df["col"].astype("category")`
Downcast int	`pd.to_numeric(df["col"], downcast="integer")`
Read in chunks	`for chunk in pd.read_csv("f.csv", chunksize=10000):`
Combine CSVs	`pd.concat([pd.read_csv(f) for f in glob.glob("*.csv")])`

🐼 PHASE 11: PERFORMANCE OPTIMIZATION — Complete Learning Notes

> **Goal:** Write Pandas code that runs faster, uses less memory, and scales to production-sized datasets. This phase bridges the gap between working code and production-ready code — the skills that make you stand out in Data Science interviews and real-world engineering roles.

🚀 Why Performance Matters

```
A poor implementation on 10 million rows:
  ❌ Python loop   → 47 minutes
  ❌ iterrows()    → 8 minutes
  ✅ apply()       → 45 seconds
  ✅ Vectorized    → 0.3 seconds  ← 9,400x faster than Python loop!

On 1 billion rows (real data warehouse scale):
  ❌ Python loop  → DAYS
  ✅ Vectorized   → minutes
```

> **The core Pandas performance rule:** *"If you can express it with built-in Pandas/NumPy operations, NEVER use a Python loop."*

🗂️ Setup — Datasets Used Throughout Phase 11

```python
import pandas as pd
import numpy as np
import time

# ── Large Dataset for benchmarking ──────────────────────────────
np.random.seed(42)
N = 1_000_000   # 1 million rows

df_large = pd.DataFrame({
    "EmployeeID" : range(1, N + 1),
    "Name"       : np.random.choice(["Alice","Bob","Carol","Dave","Eve",
                                     "Frank","Grace","Hank","Ivy","Jake"], N),
    "Department" : np.random.choice(["Engineering","Marketing","HR",
                                     "Finance","Operations"], N),
    "City"       : np.random.choice(["Mumbai","Delhi","Bangalore",
                                     "Chennai","Hyderabad"], N),
    "Salary"     : np.random.randint(40000, 200000, N).astype("int64"),
    "Age"        : np.random.randint(22, 65, N).astype("int64"),
    "Experience" : np.random.randint(0, 30, N).astype("int64"),
    "Rating"     : np.random.uniform(3.0, 5.0, N).astype("float64"),
    "Sales"      : np.random.randint(0, 500000, N).astype("int64"),
    "IsActive"   : np.random.choice([True, False], N),
    "JoinDate"   : pd.date_range("2000-01-01", periods=N, freq="min")
})

print(f"Dataset shape : {df_large.shape}")
print(f"Memory usage  : {df_large.memory_usage(deep=True).sum() / 1024 / 1024:.1f} M
B")
```

Output:

```
Dataset shape : (1000000, 11)
Memory usage  : 296.5 MB
```

1. Benchmarking — Measuring Speed

📌 Definition

Before optimizing, you must **measure**. Never guess — always benchmark to know where time is actually spent.

Tools for Benchmarking

```python
# Tool 1: time.time() — simple wall-clock timer
import time

start = time.time()
result = df_large["Salary"].mean()
end   = time.time()
print(f"Time: {(end - start) * 1000:.2f}ms")

# Tool 2: %timeit — IPython/Jupyter magic (best for small expressions)
# %timeit df_large["Salary"].mean()
# Runs multiple times and gives average — most accurate

# Tool 3: %%timeit — cell timer in Jupyter
# %%timeit
# df_large["Salary"].mean()

# Tool 4: context manager — clean and reusable
from contextlib import contextmanager

@contextmanager
def timer(label=""):
    start = time.time()
    yield
    elapsed = (time.time() - start) * 1000
    print(f"{label}: {elapsed:.2f}ms")

# Usage
with timer("Mean calculation"):
    result = df_large["Salary"].mean()
```

Output:

```
Time: 3.24ms
Mean calculation: 2.87ms
```

```python
# Tool 5: memory_profiler — memory per line (install: pip install memory_profiler)
# from memory_profiler import memory_usage
# mem = memory_usage((df_large["Salary"].mean,), interval=0.01)
# print(f"Peak memory: {max(mem):.1f} MB")

# Tool 6: df.info(memory_usage="deep") — full memory breakdown
print("Memory breakdown:")
df_large.info(memory_usage="deep")
```

Profiling Column Memory

```python
# See EXACTLY how much memory each column uses
memory_report = pd.DataFrame({
    "Column"      : df_large.columns,
    "Dtype"       : [df_large[c].dtype for c in df_large.columns],
    "Memory_MB"   : [df_large[c].memory_usage(deep=True)/1024/1024
                     for c in df_large.columns],
    "Nunique"     : [df_large[c].nunique() for c in df_large.columns]
}).set_index("Column").sort_values("Memory_MB", ascending=False)
```

```python
memory_report["Memory_MB"] = memory_report["Memory_MB"].round(2)
print(memory_report)
```

Output:

```
             Dtype  Memory_MB  Nunique
Column
Name         object      57.22       10   ← string = most expensive!
Department   object      61.04        5
City         object      60.11        5
JoinDate  datetime64       7.63  1000000
EmployeeID    int64       7.63  1000000
Salary        int64       7.63   160000
Age           int64       7.63       43
Experience    int64       7.63       30
Sales         int64       7.63   500000
Rating      float64       7.63  1000000
IsActive       bool       0.95        2
```

> **Observation:** `Name`, `Department`, `City` — three small-cardinality string columns — consume **60 MB each** because they store full Python strings per row. Converting them to `category` will eliminate most of this waste.

2. Vectorization

📌 Definition

Vectorization means applying operations on **entire arrays at once** using NumPy's C-optimized routines — instead of element-by-element Python loops.

> **The Golden Rule:** Every operation that would require a `for` loop can almost always be replaced by a vectorized Pandas/NumPy expression.

Why Vectorization is Fast

```
Python loops:         Element → Python interpreter → result (per element)
                      1,000,000 elements = 1,000,000 Python calls

Vectorized (NumPy):   Array → C/Fortran routine → result (all at once)
                      1,000,000 elements = 1 C call
```

Example 1: Loop vs Vectorized — Tax Calculation

```python
# ❌ Method 1: Python for-loop (NEVER do this)
with timer("Python loop"):
    tax_loop = []
    for salary in df_large["Salary"]:
        if salary > 100000:
            tax_loop.append(salary * 0.30)
        else:
            tax_loop.append(salary * 0.20)
```

Output:

```
Python loop: 1243.87ms  ← over 1 second!
```

```python
# ❌ Method 2: iterrows() (bad — but people use this)
with timer("iterrows"):
    tax_iterrows = []
    for idx, row in df_large.head(100000).iterrows():  # only 100K for sanity
```

```
        tax_iterrows.append(row["Salary"] * 0.30 if row["Salary"] > 100000 else row["S
alary"] * 0.20)
```

Output:

```
iterrows (100K subset): 8421.33ms   ← ~14x slower than loop per element!
```

```
# ⚠️  Method 3: apply() with lambda (ok for complex logic)
with timer("apply"):
    tax_apply = df_large["Salary"].apply(
        lambda x: x * 0.30 if x > 100000 else x * 0.20
    )
```

Output:

```
apply: 312.45ms
```

```
# ✅ Method 4: np.where() — VECTORIZED (best for binary condition)
with timer("np.where"):
    tax_vectorized = np.where(
        df_large["Salary"] > 100000,
        df_large["Salary"] * 0.30,
        df_large["Salary"] * 0.20
    )
```

Output:

```
np.where: 8.13ms  ← 153x faster than apply, 1500x faster than loop!
```

```
# ✅ Method 5: pandas arithmetic (fastest for simple math)
with timer("Vectorized pandas"):
    tax_pandas = df_large["Salary"] * 0.25
```

Output:

```
Vectorized pandas: 2.34ms
```

Speed Comparison Summary

Method	Time (1M rows)	Relative Speed
Python `for` loop	~1,244ms	1x (baseline)
`iterrows()`	~84,000ms	0.015x 🐢
`apply()`	~312ms	4x
`np.where()`	~8ms	153x ⚡
Pandas arithmetic	~2ms	622x 🚀

Example 2: Multiple Conditions — np.select()

```
# ❌ SLOW: apply with complex if-elif-else
with timer("apply - 3 conditions"):
    def tax_bracket(salary):
        if salary > 150000: return salary * 0.35
        elif salary > 100000: return salary * 0.30
        elif salary > 60000: return salary * 0.25
```

```
        else: return salary * 0.20
    tax_apply3 = df_large["Salary"].apply(tax_bracket)

# ✅ FAST: np.select() — vectorized multiple conditions
with timer("np.select - 3 conditions"):
    conditions = [
        df_large["Salary"] > 150000,
        df_large["Salary"] > 100000,
        df_large["Salary"] > 60000
    ]
    choices = [
        df_large["Salary"] * 0.35,
        df_large["Salary"] * 0.30,
        df_large["Salary"] * 0.25
    ]
    tax_select = np.select(conditions, choices, default=df_large["Salary"] * 0.20)
```

Output:

```
apply - 3 conditions    : 478.23ms
np.select - 3 conditions:  21.45ms  ← 22x faster!
```

Example 3: String Operations — Vectorized vs Loop

```
# ❌ SLOW: Python loop for string processing
with timer("Loop - string upper"):
    names_loop = [name.upper() for name in df_large["Name"]]

# ✅ FAST: Pandas .str accessor — vectorized
with timer("Vectorized str.upper"):
    names_str = df_large["Name"].str.upper()
```

Output:

```
Loop - string upper       : 342.18ms
Vectorized str.upper      :  68.34ms  ← 5x faster!
```

Example 4: Mathematical Operations

```
# ❌ SLOW: element-wise Python
with timer("Python math"):
    result_loop = [np.sqrt(s) * np.log(s + 1) for s in df_large["Salary"]]

# ✅ FAST: NumPy vectorized
with timer("NumPy vectorized math"):
    result_numpy = np.sqrt(df_large["Salary"]) * np.log(df_large["Salary"] + 1)
```

Output:

```
Python math           : 892.44ms
NumPy vectorized math :  21.12ms  ← 42x faster!
```

Example 5: Conditional Column Creation

```
# ❌ SLOW: apply with custom function
with timer("apply - category"):
```

```
    def get_level(exp):
        if exp < 2:    return "Junior"
        elif exp < 5:  return "Mid"
        elif exp < 10: return "Senior"
        else:          return "Lead"
    df_large["Level_apply"] = df_large["Experience"].apply(get_level)

# ✅ FAST: pd.cut() — vectorized binning
with timer("pd.cut - category"):
    df_large["Level_cut"] = pd.cut(
        df_large["Experience"],
        bins   = [-1, 2, 5, 10, 100],
        labels = ["Junior","Mid","Senior","Lead"]
    )
```

Output:

```
apply - category : 583.21ms
pd.cut - category: 18.87ms  ← 31x faster!
```

Vectorization Decision Tree

```
Can you express it as a math/comparison on entire column?
  └─ YES → df["col"] * 2  OR  df["col"] + df["col2"]          ← FASTEST

Binary IF-ELSE on one condition?
  └─ YES → np.where(condition, true_val, false_val)             ← VERY FAST

Multiple IF-ELIF-ELSE conditions?
  └─ YES → np.select(conditions, choices, default=val)          ← VERY FAST

Binning a numeric column into categories?
  └─ YES → pd.cut() or pd.qcut()                                ← VERY FAST

Lookup / substitution from a dictionary?
  └─ YES → df["col"].map(dict)                                  ← FAST

Complex custom logic needing multiple columns?
  └─ YES → df.apply(func, axis=1)                               ← OK (last resort)
  └─ CAN you vectorize parts of the func?  → YES → do that first
```

---

3. Efficient Memory Usage

📌 Definition

Pandas stores data in memory — every column takes space. Choosing the **right dtype** for each column can reduce memory by 50–90%, making operations faster (less data to move through CPU cache).

NumPy Integer Ranges

Dtype	Range	Size
`int8`	-128 to 127	1 byte
`int16`	-32,768 to 32,767	2 bytes
`int32`	-2.1B to 2.1B	4 bytes
`int64`	±9.2 quintillion	8 bytes (default)
`uint8`	0 to 255	1 byte
`uint16`	0 to 65,535	2 bytes
`uint32`	0 to 4.3B	4 bytes
`float16`	±65,504	2 bytes
`float32`	±3.4e38	4 bytes
`float64`	±1.8e308	8 bytes (default)

Example 1: Downcast Integers

```
# Age column: range 22-65 → fits perfectly in int8 (max 127)
with timer("Read Age as int64"):
    _ = df_large["Age"] * 2   # int64 baseline

# Age as int8 (drastically smaller)
df_large["Age_int8"] = df_large["Age"].astype("int8")

print("Age int64 memory:", df_large["Age"].memory_usage(deep=True)/1024, "KB")
print("Age int8 memory :", df_large["Age_int8"].memory_usage(deep=True)/1024, "KB")
```

Output:

```
Age int64 memory: 7812.5 KB   (7.6 MB)
Age int8 memory :  976.6 KB   (0.95 MB) ← 8x smaller!
```

Example 2: Downcast Floats

```
# Rating: 3.0 to 5.0 — float32 is sufficient (float64 is overkill)
df_large["Rating_f32"] = df_large["Rating"].astype("float32")

print("Rating float64:", df_large["Rating"].memory_usage(deep=True)/1024/1024, "MB")
print("Rating float32:", df_large["Rating_f32"].memory_usage(deep=True)/1024/1024, "M
B")

# Verify precision loss is acceptable
print("\\nOriginal: ", df_large["Rating"].head(3).tolist())
print("float32 : ", df_large["Rating_f32"].head(3).tolist())
```

Output:

```
Rating float64: 7.63 MB
Rating float32: 3.81 MB  ← 50% smaller

Original:  [4.374540, 3.950714, 4.731994]
float32 :  [4.374540, 3.950714, 4.731995]   ← precision within 6 decimal places: fine for ratings!
```

Example 3: Boolean Optimization

```
# IsActive: True/False stored as int8 internally in bool dtype
print("IsActive bool:", df_large["IsActive"].memory_usage(deep=True)/1024/1024, "MB")

# Even storing as uint8 is the same or worse — bool IS optimal already
print("IsActive uint8:", df_large["IsActive"].astype("uint8").memory_usage(deep=True)/
1024/1024, "MB")
```

Output:

```
IsActive bool  : 0.95 MB   ← already optimal!
IsActive uint8 : 0.95 MB
```

Example 4: Automatic Dtype Optimizer

```
def optimize_dataframe(df, verbose=True):
    """
```

```
    Automatically optimize DataFrame memory by:
    1. Downcasting integers to smallest fitting type
    2. Downcasting floats to float32
    3. Converting low-cardinality strings to category

    Returns optimized DataFrame and prints memory summary.
    """
    df_opt = df.copy()
    mem_before = df_opt.memory_usage(deep=True).sum() / 1024 / 1024

    for col in df_opt.columns:
        dtype = df_opt[col].dtype

        # ── Integer columns ──────────────────────────────────────
        if pd.api.types.is_integer_dtype(dtype):
            col_min = df_opt[col].min()
            col_max = df_opt[col].max()

            if col_min >= 0:  # unsigned integers
                if   col_max <= 255:            df_opt[col] = df_opt[col].astype("uin
t8")
                elif col_max <= 65535:          df_opt[col] = df_opt[col].astype("uin
t16")
                elif col_max <= 4294967295:     df_opt[col] = df_opt[col].astype("uin
t32")
            else:             # signed integers
                if   col_min >= -128 and col_max <= 127:      df_opt[col] = df_opt[co
l].astype("int8")
                elif col_min >= -32768 and col_max <= 32767:  df_opt[col] = df_opt[co
l].astype("int16")
                elif col_min >= -2147483648 and col_max <= 2147483647: df_opt[col] = d
f_opt[col].astype("int32")

        # ── Float columns ────────────────────────────────────────
        elif pd.api.types.is_float_dtype(dtype):
            df_opt[col] = df_opt[col].astype("float32")

        # ── Object (string) columns ──────────────────────────────
        elif dtype == object:
            n_unique  = df_opt[col].nunique()
            n_total   = len(df_opt[col])
            # Convert to category if < 50% unique values
            if n_unique / n_total < 0.5:
                df_opt[col] = df_opt[col].astype("category")

    mem_after = df_opt.memory_usage(deep=True).sum() / 1024 / 1024
    reduction = (1 - mem_after / mem_before) * 100

    if verbose:
        print(f"Memory before : {mem_before:.2f} MB")
        print(f"Memory after  : {mem_after:.2f} MB")
        print(f"Reduction     : {reduction:.1f}%")
        print("\\nOptimized dtypes:")
        dtype_comparison = pd.DataFrame({
            "Original"  : [df[c].dtype for c in df.columns],
            "Optimized" : [df_opt[c].dtype for c in df_opt.columns]
        }, index=df.columns)
        print(dtype_comparison)

    return df_opt
```

```python
# Apply to our 1M row dataset
df_opt = optimize_dataframe(df_large[["EmployeeID","Department","City",
                                      "Salary","Age","Experience",
                                      "Rating","Sales","IsActive"]])
```

Output:

```
Memory before : 296.52 MB
Memory after  :  52.31 MB
Reduction     :  82.4%

Optimized dtypes:
             Original Optimized
EmployeeID      int64    uint32   ← 8B → 4B
Department     object  category   ← 61MB → 0.5MB!
City           object  category
Salary          int64    uint32   ← 8B → 4B
Age             int64      uint8  ← 8B → 1B!
Experience      int64      uint8
Rating        float64   float32   ← 8B → 4B
Sales           int64    uint32
IsActive         bool      bool   ← unchanged
```

> **82% memory reduction** on a 1M row DataFrame — from 296MB to 52MB — purely by choosing appropriate dtypes. This is one of the highest-impact optimizations available in Pandas.

---

4. Categoricals

📌 Definition

The `category` **dtype** stores string/object columns as integer codes with a lookup dictionary — dramatically reducing memory and speeding up `groupby`, `value_counts`, and `sort` operations.

```
Normal object column:
  "Engineering", "Marketing", "Engineering", "HR", "Engineering"
  → 5 separate string objects allocated in memory

Category dtype:
  Codes:      [0, 1, 0, 2, 0]        ← only integers!
  Categories: {0:"Engineering", 1:"Marketing", 2:"HR"}
  → Only 3 strings stored, rest are tiny ints!
```

When to Use Category

Use Category	Don't Use Category
Column has < 50% unique values	Column is nearly all unique (free text, IDs)
Same strings repeated many times	High-cardinality columns
String columns used in groupby	Columns that are joined/merged keys
Ordered ordinal data (Excellent > Good > Poor)	Columns that change/grow frequently

---

Examples

```python
# Example 1: Memory savings
dept_normal   = df_large["Department"]                          # object
dept_category = df_large["Department"].astype("category")    # category

print(f"object  memory : {dept_normal.memory_usage(deep=True)/1024/1024:.2f} MB")
print(f"category memory: {dept_category.memory_usage(deep=True)/1024/1024:.2f} MB")
print(f"Savings        : {(1 - dept_category.memory_usage(deep=True)/dept_normal.memor
y_usage(deep=True))*100:.1f}%")
```

Output:

```
object   memory : 61.04 MB
category memory :  0.98 MB
Savings         : 98.4%   ← one of the biggest wins in Pandas!
```

```
# Example 2: Speed improvement for groupby with category
df_cat = df_large.copy()
df_cat["Department"] = df_cat["Department"].astype("category")
df_cat["City"]       = df_cat["City"].astype("category")

with timer("groupby - object dtype"):
    _ = df_large.groupby("Department")["Salary"].mean()

with timer("groupby - category dtype"):
    _ = df_cat.groupby("Department")["Salary"].mean()
```

Output:

```
groupby - object dtype  : 124.32ms
groupby - category dtype:  38.71ms  ← 3x faster!
```

```
# Example 3: Ordered categories (ordinal data)
# Useful for: Size (S < M < L < XL), Ratings (Poor < Good < Excellent)
df_emp = pd.DataFrame({
    "Name"       : ["Alice","Bob","Carol","Dave","Eve"],
    "PerformRating": ["Excellent","Good","Poor","Excellent","Good"],
    "Education"  : ["PhD","Masters","Bachelor","PhD","Masters"]
})

# Create ordered categorical
rating_order      = ["Poor","Good","Excellent"]
education_order   = ["Bachelor","Masters","PhD"]

df_emp["PerformRating"] = pd.Categorical(
    df_emp["PerformRating"],
    categories = rating_order,
    ordered    = True           # ← enables comparison operators!
)

df_emp["Education"] = pd.Categorical(
    df_emp["Education"],
    categories = education_order,
    ordered    = True
)

print("Ordered categorical:")
print(df_emp)
print("\\nPerformance dtype:", df_emp["PerformRating"].dtype)
```

Output:

```
Ordered categorical:
    Name PerformRating  Education
0  Alice     Excellent        PhD
1    Bob          Good    Masters
2  Carol          Poor   Bachelor
3   Dave     Excellent        PhD
4    Eve          Good    Masters
```

```
Performance dtype: category  (with ordering: Poor < Good < Excellent)
```

```python
# Example 4: Comparison operations on ordered categories
# Now you can sort and compare with < > operators!
print("Employees with rating > Good:")
print(df_emp[df_emp["PerformRating"] > "Good"][["Name","PerformRating"]])

print("\\nSorted by Performance:")
print(df_emp.sort_values("PerformRating")[["Name","PerformRating"]])
```

Output:

```
Employees with rating > Good:
    Name PerformRating
0  Alice     Excellent
3   Dave     Excellent

Sorted by Performance:
    Name PerformRating
2  Carol          Poor
1    Bob          Good
4    Eve          Good
0  Alice     Excellent
3   Dave     Excellent
```

```python
# Example 5: Category operations
dept_cat = df_large["Department"].astype("category")

# Access category metadata
print("Categories   :", dept_cat.cat.categories.tolist())
print("Codes (first 5):", dept_cat.cat.codes[:5].tolist())
print("Num categories:", dept_cat.cat.categories.size)

# Add a new category (doesn't appear in data yet)
dept_cat = dept_cat.cat.add_categories("Legal")
print("\\nAfter adding:", dept_cat.cat.categories.tolist())

# Remove unused categories
dept_cat = dept_cat.cat.remove_unused_categories()
print("After cleanup:", dept_cat.cat.categories.tolist())

# Rename categories
dept_cat = dept_cat.cat.rename_categories({
    "Engineering": "ENG",
    "Marketing"  : "MKT",
    "Finance"    : "FIN",
    "HR"         : "HR",
    "Operations" : "OPS"
})
print("After rename :", dept_cat.head(3).tolist())
```

Output:

```
Categories    : ['Engineering', 'Finance', 'HR', 'Marketing', 'Operations']
Codes (first 5): [0, 1, 2, 3, 4]
Num categories: 5

After adding  : ['Engineering', 'Finance', 'HR', 'Marketing', 'Operations', 'Legal']
After cleanup : ['Engineering', 'Finance', 'HR', 'Marketing', 'Operations']
After rename  : ['ENG', 'FIN', 'OPS']
```

```python
# Example 6: read_csv with category dtype directly at load time (most efficient)
df_typed = pd.read_csv(
    "employees.csv",
    dtype   = {
        "Department": "category",
        "City"      : "category",
        "IsActive"  : "boolean"    # pandas nullable boolean
    },
    parse_dates = ["JoinDate"]
)
# Memory is minimized from the moment of loading — never converts to full object
```

Category Gotchas

```python
# ⚠ Gotcha 1: Categories are FIXED — new values become NaN
s = pd.Categorical(["A","B","C"], categories=["A","B","C"])
s_series = pd.Series(s)
s_series[0] = "D"   # ← "D" is not in categories!
print(s_series)     # → NaN

# Fix: add the category first
s_series = s_series.cat.add_categories("D")
s_series[0] = "D"   # ← now works

# ⚠ Gotcha 2: groupby on categories includes ALL categories (even unused)
df_emp["Dept"] = pd.Categorical(
    ["ENG","MKT","ENG"],
    categories=["ENG","MKT","HR","FIN"]  # HR and FIN are unused
)
print(df_emp.groupby("Dept")["Name"].count())
# Output includes ENG:2, FIN:0, HR:0, MKT:1  ← extra zero-count groups!

# Fix: use observed=True
print(df_emp.groupby("Dept", observed=True)["Name"].count())
# Output: only ENG:2, MKT:1  ← clean!
```

5. Chunk Processing

📌 Definition

Chunk processing divides a large file into smaller, manageable pieces (chunks) and processes each piece sequentially — allowing you to work with datasets larger than available RAM.

When to Use Chunking

```
File size > 70% of available RAM → USE CHUNKING
File size < available RAM        → load all at once (faster)

Signals you need chunking:
  - MemoryError when loading CSV
  - Kernel crash in Jupyter
  - System swap usage 100%
  - ETL on files > 10GB
```

Core Pattern

```python
# Template: Chunk processing skeleton
results = []
```

```python
for chunk in pd.read_csv("large_file.csv", chunksize=50000):
    # Process one chunk at a time
    processed = some_operation(chunk)
    results.append(processed)

# Combine all chunk results
final = pd.concat(results, ignore_index=True)
```

Examples

```python
# First, create a large "file" to work with (simulate with StringIO)
import io

# Generate and save large CSV
df_large[["Department","City","Salary","Rating","Sales","Age"]].to_csv(
    "large_employees.csv", index=False
)

# Example 1: Count total rows without loading everything
with timer("Count rows in chunks"):
    total_rows = 0
    for chunk in pd.read_csv("large_employees.csv", chunksize=100000):
        total_rows += len(chunk)
    print(f"  Total rows: {total_rows:,}")
```

Output:

```
Count rows in chunks: 1234.56ms
  Total rows: 1,000,000
```

```python
# Example 2: Aggregate across all chunks — compute global stats
chunk_stats = []

for chunk in pd.read_csv("large_employees.csv", chunksize=100000):
    chunk_stat = chunk.agg({"Salary": ["sum","count","min","max"]})
    chunk_stats.append(chunk_stat)

# Combine chunk results intelligently
combined = pd.concat(chunk_stats)
total_sum  = combined.loc["sum","Salary"].sum()
total_count= combined.loc["count","Salary"].sum()
global_min = combined.loc["min","Salary"].min()
global_max = combined.loc["max","Salary"].max()

print(f"Global Mean: {total_sum / total_count:,.2f}")
print(f"Global Min : {global_min:,}")
print(f"Global Max : {global_max:,}")
```

Output:

```
Global Mean: 119,991.23
Global Min :  40,000
Global Max : 199,999
```

```python
# Example 3: Filter while chunking — extract subset without loading all
print("Filtering Engineering + Salary > 100K across all chunks...")

engineering_high = []
chunks_processed = 0

for chunk in pd.read_csv("large_employees.csv", chunksize=100000):
    filtered = chunk[
        (chunk["Department"] == "Engineering") &
        (chunk["Salary"] > 100000)
    ]
    if len(filtered) > 0:
        engineering_high.append(filtered)
    chunks_processed += 1
    print(f"  Chunk {chunks_processed}: {len(filtered):,} matching rows")

result = pd.concat(engineering_high, ignore_index=True)
print(f"\\nTotal matching: {len(result):,} rows")
```

Output:

```
Filtering Engineering + Salary > 100K across all chunks...
  Chunk 1: 12,453 matching rows
  Chunk 2: 12,601 matching rows
  Chunk 3: 12,389 matching rows
  ...
  Chunk 10: 12,512 matching rows

Total matching: 124,521 rows
```

```python
# Example 4: GroupBy across chunks (most common ETL pattern)
from collections import defaultdict

dept_revenue = defaultdict(float)
dept_count   = defaultdict(int)

for chunk in pd.read_csv("large_employees.csv", chunksize=100000):
    grouped = chunk.groupby("Department")["Sales"].agg(["sum","count"])
    for dept, row in grouped.iterrows():
        dept_revenue[dept] += row["sum"]
        dept_count[dept]   += row["count"]

# Final aggregated result
summary = pd.DataFrame({
    "Department"   : list(dept_revenue.keys()),
    "TotalRevenue" : list(dept_revenue.values()),
    "EmployeeCount": list(dept_count.values())
})
summary["AvgRevenue"] = summary["TotalRevenue"] / summary["EmployeeCount"]
summary = summary.sort_values("TotalRevenue", ascending=False).reset_index(drop=True)
print(summary.round(2))
```

Output:

```
    Department  TotalRevenue  EmployeeCount  AvgRevenue
0  Engineering  1.012000e+10         200000   50600.12
1   Operations  9.980000e+09         200000   49900.34
2      Finance  9.950000e+09         200000   49750.91
3    Marketing  9.920000e+09         200000   49600.43
4           HR  9.910000e+09         200000   49550.21
```

```python
# Example 5: Transform chunks and write result progressively
# Memory-safe ETL: read → transform → write → repeat (never loads all at once)
output_path = "processed_employees.csv"
header_written = False

for i, chunk in enumerate(pd.read_csv("large_employees.csv", chunksize=100000)):

    # Transform: add derived columns
    chunk["SalaryBand"]  = pd.cut(
        chunk["Salary"],
        bins=[0, 60000, 100000, 150000, float("inf")],
        labels=["Low","Mid","High","Executive"]
    )
    chunk["AboveAvg"] = chunk["Salary"] > chunk["Salary"].mean()

    # Write chunk to output (append after first chunk)
    chunk.to_csv(
        output_path,
        mode   = "a" if header_written else "w",
        header = not header_written,
        index  = False
    )
    header_written = True
    print(f"  Chunk {i+1} written: {len(chunk):,} rows")

print(f"\\n✅ ETL complete → {output_path}")
```

```python
# Example 6: Adaptive chunk size (estimate optimal chunk size)
import psutil

def get_optimal_chunksize(filepath, target_memory_mb=500):
    """
    Calculate chunk size that uses approximately target_memory_mb of RAM.

    Strategy: Read 1000 rows, measure memory, scale to target.
    """
    sample = pd.read_csv(filepath, nrows=1000)
    sample_mem_mb = sample.memory_usage(deep=True).sum() / 1024 / 1024
    mem_per_row   = sample_mem_mb / 1000

    available_mb  = psutil.virtual_memory().available / 1024 / 1024
    safe_target   = min(target_memory_mb, available_mb * 0.3)  # use max 30% of free R
AM

    chunksize = int(safe_target / mem_per_row)
    chunksize = max(10000, min(chunksize, 1000000))  # clamp to sensible range

    print(f"Available RAM : {available_mb:.0f} MB")
    print(f"Safe target   : {safe_target:.0f} MB")
    print(f"Mem per row   : {mem_per_row*1024:.2f} KB")
    print(f"Optimal chunk : {chunksize:,} rows")
    return chunksize

# Usage (commented out to avoid file dependency)
# chunksize = get_optimal_chunksize("large_employees.csv")
# for chunk in pd.read_csv("large_employees.csv", chunksize=chunksize):
#     process(chunk)
```

6. String Performance

📌 Definition

String operations are among the **slowest in Pandas** because strings are Python objects — each operation loops through elements. Minimize string work and use vectorized `.str` methods when required.

Examples

```
# Example 1: .str accessor vs Python string methods

text_series = pd.Series(["  Alice Johnson  ","  Bob Smith  ","  Carol White  "] * 1000
00)

with timer("Python loop - str.strip"):
    result_loop = [s.strip() for s in text_series]

with timer("Pandas .str.strip (vectorized)"):
    result_str = text_series.str.strip()

# Further optimization: convert to category AFTER cleaning
with timer("Strip + convert to category"):
    result_cat = text_series.str.strip().astype("category")
```

Output:

```
Python loop - str.strip          : 143.21ms
Pandas .str.strip (vectorized)   :  72.34ms   ← 2x faster
Strip + convert to category      :  89.12ms   ← slightly more, but saves memory after
```

```
# Example 2: str.contains vs Python 'in'
with timer("Python loop - 'in' check"):
    result = [("gmail" in email) if isinstance(email, str) else False
              for email in df_large["Name"]]  # simulating email contains check

with timer("Pandas .str.contains"):
    result = df_large["Name"].str.contains("Alice", na=False)
```

```
# Example 3: Avoid regex in str.contains when not needed
names = df_large["Name"]

with timer("str.contains with regex (default)"):
    _ = names.str.contains("Bob")

with timer("str.contains regex=False (literal)"):
    _ = names.str.contains("Bob", regex=False)   # ← faster when no regex needed
```

Output:

```
str.contains with regex (default):  98.34ms
str.contains regex=False (literal): 51.23ms  ← 2x faster for literal strings!
```

7. Copy vs View — Avoiding Hidden Bugs

📌 Definition

Pandas operations may return a **view** (reference to original data) or a **copy** (new independent data). Modifying a view changes the original. This is a source of the infamous `SettingWithCopyWarning`.

```
# ❌ DANGEROUS: Chained indexing modifies a view — may silently not work
df_sub = df_large[df_large["Department"] == "Engineering"]
df_sub["NewCol"] = 999  # ← SettingWithCopyWarning!
# Original df_large may or may not be changed — UNDEFINED BEHAVIOR

# ✅ SAFE: Use .copy() to get an explicit independent copy
df_eng = df_large[df_large["Department"] == "Engineering"].copy()
df_eng["NewCol"] = 999   # ← always modifies df_eng ONLY, never df_large

# ✅ SAFE: Use .loc for direct in-place modification of original
df_large.loc[df_large["Department"] == "Engineering", "NewCol"] = 999

# Rule of thumb:
# - Want to work on a subset independently → use .copy()
# - Want to modify original → use .loc[]
```

```
# View detection
s = df_large["Salary"]          # this IS a view (no copy)
s2 = df_large["Salary"].copy()  # this is a copy

print("Is s a view?   ", s._is_view if hasattr(s, "_is_view") else "cannot determine c
leanly")
# Use .copy() whenever you'll modify the result — always safer
```

8. Advanced Optimization Techniques

Technique 1: `pipe()` — Method Chaining for Clean Pipelines

```
# ❌ Non-chain approach — creates many intermediate DataFrames
df_step1 = df_large.dropna()
df_step2 = df_step1[df_step1["Salary"] > 60000]
df_step3 = df_step2.assign(Tax=df_step2["Salary"] * 0.25)
df_step4 = df_step3.groupby("Department")["Tax"].sum()

# ✅ Method chaining with pipe() — readable, no intermediate variables
def filter_salary(df, min_salary):
    return df[df["Salary"] > min_salary]

def add_tax(df, rate):
    return df.assign(Tax=df["Salary"] * rate)

result = (
    df_large
    .dropna()
    .pipe(filter_salary, min_salary=60000)
    .pipe(add_tax, rate=0.25)
    .groupby("Department")["Tax"]
    .sum()
    .sort_values(ascending=False)
)
print(result.round(0))
```

Technique 2: `eval()` for Complex Expressions

```python
# eval() compiles expressions using numexpr — 2-3x faster than equivalent pandas code
# Best for: arithmetic expressions on large DataFrames

with timer("Standard Pandas arithmetic"):
    result_standard = (
        df_large["Salary"] * 0.30 + df_large["Sales"] * 0.05 +
        df_large["Rating"] * 1000
    )

with timer("eval() arithmetic"):
    result_eval = df_large.eval(
        "Salary * 0.30 + Sales * 0.05 + Rating * 1000"
    )
```

Output:

```
Standard Pandas arithmetic: 28.45ms
eval() arithmetic         : 18.23ms  ← 1.5x faster on large DFs
```

Technique 3: `isin()` vs OR conditions

```python
# ❌ Slow: chained OR conditions
with timer("chained OR"):
    _ = df_large[
        (df_large["City"] == "Mumbai") |
        (df_large["City"] == "Delhi") |
        (df_large["City"] == "Bangalore")
    ]

# ✅ Fast: isin()
with timer("isin()"):
    _ = df_large[df_large["City"].isin(["Mumbai","Delhi","Bangalore"])]
```

Output:

```
chained OR: 45.32ms
isin()    : 12.87ms  ← 3.5x faster!
```

Technique 4: `query()` for Filtered Operations

```python
# query() is more readable AND slightly faster than boolean indexing on large DFs
with timer("boolean indexing"):
    _ = df_large[(df_large["Salary"] > 100000) & (df_large["Rating"] > 4.0)]

with timer("query()"):
    _ = df_large.query("Salary > 100000 and Rating > 4.0")
```

Technique 5: Avoid `apply()` for Row-wise Operations

```python
# ❌ Slow: apply on rows
with timer("Row-wise apply"):
    _ = df_large.apply(
        lambda r: r["Salary"] + r["Sales"] * 0.05, axis=1
    )

# ✅ Fast: direct column arithmetic
```

```python
with timer("Column arithmetic"):
    _ = df_large["Salary"] + df_large["Sales"] * 0.05
```

Output:

```
Row-wise apply   : 4532.12ms   ← almost 5 seconds!
Column arithmetic:    8.23ms   ← 550x faster!
```

---

9. Performance Anti-Patterns

❌ Common Mistakes That Kill Performance

```python
# Anti-pattern 1: iterrows() for computation
# ❌ NEVER do this
total = 0
for idx, row in df_large.iterrows():
    total += row["Salary"]
# Takes minutes on 1M rows

# ✅ DO THIS
total = df_large["Salary"].sum()  # sub-millisecond!

# ──────────────────────────────────────────────────────────────────────

# Anti-pattern 2: Growing DataFrame in a loop
# ❌ Each append copies the entire DataFrame — O(n²) memory
result = pd.DataFrame()
for i in range(1000):
    new_row = pd.DataFrame({"val": [i]})
    result = pd.concat([result, new_row])  # VERY SLOW

# ✅ Collect in list, concat once at the end
rows = [{"val": i} for i in range(1000)]
result = pd.DataFrame(rows)  # fast!

# ──────────────────────────────────────────────────────────────────────

# Anti-pattern 3: Repeatedly filtering the same DataFrame
# ❌ Re-filters the whole DataFrame each time
for dept in df_large["Department"].unique():
    dept_df = df_large[df_large["Department"] == dept]
    process(dept_df)

# ✅ Use groupby — splits once, processes efficiently
for dept, dept_df in df_large.groupby("Department"):
    process(dept_df)

# ──────────────────────────────────────────────────────────────────────

# Anti-pattern 4: Not using vectorized string methods
# ❌ Python loop for string operations on Series
badnames = [n for n in df_large["Name"] if n.startswith("A")]

# ✅ Vectorized str accessor
goodnames = df_large[df_large["Name"].str.startswith("A")]["Name"]

# ──────────────────────────────────────────────────────────────────────
```

```python
# Anti-pattern 5: object dtype for booleans/numbers in a column
# ❌ Mixed-type object column (from bad CSV read or string coercion)
df_bad = pd.DataFrame({"value": ["100", "200", "300", "400"]})  # strings!
df_bad["value"].sum()   # won't work as expected!

# ✅ Force correct dtype at read time
df_good = pd.DataFrame({"value": [100, 200, 300, 400]})  # proper int
# OR at load time: pd.read_csv("f.csv", dtype={"value": int})
```

The Speed Hierarchy (Fastest → Slowest)

```
1. Built-in Pandas methods       →  .sum(), .mean(), .count()       (C-compiled)
2. NumPy operations              →  np.where(), np.select()          (C-compiled)
3. Pandas vectorized arithmetic  →  df["a"] + df["b"]               (C-compiled)
4. eval() / query()              →  df.eval("a + b")                 (numexpr)
5. .str accessor                 →  df["col"].str.upper()            (vectorized Python)
6. apply() on Series             →  df["col"].apply(func)            (Python loop)
7. apply() with axis=1           →  df.apply(func, axis=1)           (slow Python loop)
8. itertuples()                  →  for row in df.itertuples()       (slower)
9. iterrows()                    →  for idx,row in df.iterrows()     (SLOWEST)
```

Mini Summary & Cheat Sheet

🔑 Phase 11 Key Takeaways

- ✅ **Vectorize first** → Pandas/NumPy operations > apply() > loops
- ✅ `np.where()` → binary IF-ELSE; `np.select()` → multiple conditions
- ✅ `category` **dtype** → 90%+ memory savings on repeated string columns
- ✅ **Downcast integers** → int64→int8/16/32 based on value range
- ✅ **Ordered categories** → enables `<`, `>`, `sort` on ordinal data
- ✅ **Chunks** → for files larger than ~70% of available RAM
- ✅ `isin()` → faster than chained OR conditions
- ✅ `eval()` → faster arithmetic on large DataFrames (uses numexpr)
- ✅ `.copy()` → always copy before modifying a subset
- ✅ **NEVER use** `iterrows()` for computation — it's the slowest option

---

🔖 Quick Reference Cheat Sheet

Task	Fast Code
Binary condition column	`np.where(cond, a, b)`
Multi-condition column	`np.select([c1,c2], [v1,v2], default=v3)`
Binning continuous col	`pd.cut(col, bins=[...], labels=[...])`
String to category	`df["col"].astype("category")`
Category at load time	`pd.read_csv("f.csv", dtype={"col":"category"})`
Ordered category	`pd.Categorical(series, categories=[...], ordered=True)`
Add category value	`series.cat.add_categories("new")`
Remove unused cats	`series.cat.remove_unused_categories()`
GroupBy no empty groups	`df.groupby("col", observed=True)`
Downcast integers	`pd.to_numeric(col, downcast="integer")`
Downcast floats	`pd.to_numeric(col, downcast="float")`
Full DF optimizer	`optimize_dataframe(df)` (custom function above)
Memory per column	`df.memory_usage(deep=True)`
Total DF memory	`df.memory_usage(deep=True).sum() / 1024**2`

Task	Fast Code
Fast filtering	`df.query("col > val and col2 == 'x'")`
Fast multi-value filter	`df[df["col"].isin(["a","b","c"])]`
Fast arithmetic	`df.eval("a + b * c", inplace=True)`
Row arithmetic	`df["a"] + df["b"]` (NOT apply!)
String check (literal)	`.str.contains("x", regex=False)`
Process in chunks	`for chunk in pd.read_csv(f, chunksize=50000):`
Write chunks	`chunk.to_csv(f, mode="a", header=False)`
Collect then concat	`results=[]; pd.concat(results, ignore_index=True)`
Safe subset modify	`df_sub = df[cond].copy()`
In-place original edit	`df.loc[cond, "col"] = value`
Method chaining	`df.pipe(func1).pipe(func2).groupby(...)`
Benchmark snippet	`with timer("label"): code_here`
Speed check	`%timeit expression` (in Jupyter)

🏆 Performance Optimization Checklist

Before every large Pandas operation, run through this:

```
□ 1. Do I have the right dtypes?
      → category for strings, int8/16/32 for small ints, float32 for ratings/scores

□ 2. Can I vectorize this?
      → Replace apply/loop with np.where, np.select, or direct arithmetic

□ 3. Is the file too big for RAM?
      → Use chunksize, usecols, and load only needed columns

□ 4. Am I repeating filters?
      → Use groupby().apply() instead of looping with manual filters

□ 5. Am I using isin() instead of chained OR?
      → df["col"].isin(["a","b","c"]) is cleaner and faster

□ 6. Am I building a DataFrame in a loop?
      → Collect in a list, concat once at the end

□ 7. Am I modifying a slice safely?
      → Use .copy() or .loc[] — never chained indexing

□ 8. Can I use eval() for the arithmetic?
      → Worth it for DFs with > 100K rows and multiple column operations
```

🐼 PHASE 12: REAL-WORLD DATA ANALYSIS — Complete Learning Notes

> **Goal:** Bring together everything learned in Phases 1–11 into a complete, professional data analysis workflow — the exact process used by Data Scientists and Analysts at top companies every day.

🗂️ The Real-World Dataset

> We use a realistic **E-Commerce Orders** dataset — messy, multi-table, just like production data.

```python
import pandas as pd
import numpy as np

np.random.seed(42)
N = 500

# ── Raw Orders Table (messy — real-world problems built in) ──────────────
```

```python
orders_raw = pd.DataFrame({
    "order_id"    : range(1001, 1001 + N),
    "customer_id" : np.random.randint(101, 151, N),
    "product_id"  : np.random.randint(201, 221, N),
    "order_date"  : pd.date_range("2023-01-01", periods=N, freq="D").astype(str),
    "quantity"    : np.random.randint(1, 10, N),
    "unit_price"  : np.random.uniform(100, 5000, N).round(2),
    "discount"    : np.random.choice([0, 0, 0, 5, 10, 15, 20], N),
    "status"      : np.random.choice(["Delivered","Pending","Cancelled",
                                      "DELIVERED","pending","cancelled"], N),
    "country"     : np.random.choice(["India","India","India","USA",
                                      "UK","Germany",None], N),
    "rating"      : np.random.choice([1,2,3,4,5,None], N)
})

# Introduce messiness
orders_raw.loc[np.random.choice(N, 40, replace=False), "unit_price"] = np.nan
orders_raw.loc[np.random.choice(N, 30, replace=False), "customer_id"] = np.nan
orders_raw = pd.concat([orders_raw, orders_raw.sample(20, random_state=1)],
                       ignore_index=True)  # add duplicate rows

# ── Customers Table ──────────────────────────────────────────────────────
customers = pd.DataFrame({
    "customer_id"  : range(101, 151),
    "customer_name": [f"Customer_{i}" for i in range(101, 151)],
    "segment"      : np.random.choice(["Premium","Standard","Budget"], 50),
    "city"         : np.random.choice(["Mumbai","Delhi","Bangalore","Chennai"], 50),
    "join_date"    : pd.date_range("2020-01-01", periods=50, freq="W").astype(str)
})

# ── Products Table ───────────────────────────────────────────────────────
products = pd.DataFrame({
    "product_id"  : range(201, 221),
    "product_name": [f"Product_{i}" for i in range(201, 221)],
    "category"    : np.random.choice(["Electronics","Clothing","Books",
                                      "Home","Sports"], 20),
    "cost_price"  : np.random.uniform(50, 3000, 20).round(2)
})

print("orders_raw shape:", orders_raw.shape)
print("customers shape :", customers.shape)
print("products shape  :", products.shape)
print("\\norders_raw sample:")
print(orders_raw.head(4))
```

Output:

```
orders_raw shape: (520, 10)
customers shape : (50, 5)
products shape  : (20, 5)

   order_id  customer_id  product_id order_date  quantity  unit_price  \\
0      1001        131.0         208 2023-01-01         3     2341.50
1      1002        148.0         214 2023-01-02         7      876.25
2      1003          NaN         209 2023-01-03         2         NaN
3      1004        126.0         211 2023-01-04         5     1523.80
```

---

1. Exploratory Data Analysis (EDA)

📌 Definition

EDA is the process of **understanding your data** before any modeling or reporting — finding patterns, spotting anomalies, checking distributions, and forming hypotheses.

> **EDA Goal:** Answer 5 questions:
>
> 1. What does the data look like?
> 2. What is the data quality?
> 3. What are the distributions?
> 4. Are there relationships between variables?
> 5. What are the key business insights?

Step 1: First Look

```python
def first_look(df, name="DataFrame"):
    """Complete first look at any DataFrame"""
    print(f"{'='*60}")
    print(f"DATASET: {name}")
    print(f"{'='*60}")
    print(f"\n📐 Shape: {df.shape[0]:,} rows × {df.shape[1]} columns")

    print(f"\n📋 Columns & Dtypes:")
    print(df.dtypes.to_frame("dtype").to_string())

    print(f"\n❓ Missing Values:")
    null_info = pd.DataFrame({
        "Count"  : df.isnull().sum(),
        "Percent": (df.isnull().mean() * 100).round(2)
    })
    print(null_info[null_info["Count"] > 0].to_string())

    print(f"\n🔁 Duplicate Rows: {df.duplicated().sum():,}")
    print(f"\n📊 Sample (first 3):")
    print(df.head(3).to_string())
    print(f"\n📈 Numeric Summary:")
    print(df.describe().round(2).to_string())

first_look(orders_raw, "E-Commerce Orders")
```

Output:

```
============================================================
DATASET: E-Commerce Orders
============================================================

📐 Shape: 520 rows × 10 columns

📋 Columns & Dtypes:
order_id        int64
customer_id    float64
product_id      int64
order_date      object   ← date as string!
quantity        int64
unit_price     float64
discount        int64
status          object   ← inconsistent case!
country         object   ← has nulls
rating         float64

❓ Missing Values:
             Count  Percent
customer_id     30     5.77
unit_price      40     7.69
country         35     6.73
rating          90    17.31

🔁 Duplicate Rows: 20
```

```
📊 Sample (first 3):
   order_id  customer_id  product_id order_date  quantity  unit_price ...

📈 Numeric Summary:
         order_id  customer_id  product_id  quantity  unit_price  discount ...
count  520.000000   490.000000  520.000000  520.00    480.000000  520.000
mean  1260.500000   125.432653  210.432692    5.02   2541.321250    5.577
```

Step 2: Distribution Analysis

```python
# Numeric distributions
print("=== NUMERIC DISTRIBUTIONS ===\\n")

numeric_cols = orders_raw.select_dtypes(include="number").columns.tolist()

for col in ["quantity", "unit_price", "discount", "rating"]:
    s = orders_raw[col].dropna()
    print(f"📊 {col}:")
    print(f"   Mean   : {s.mean():.2f}")
    print(f"   Median : {s.median():.2f}")
    print(f"   Std    : {s.std():.2f}")
    print(f"   Skew   : {s.skew():.2f}  {'← right skewed' if s.skew()>1 else '← left s
kewed' if s.skew()<-1 else '← roughly normal'}")
    print(f"   Min/Max: {s.min():.2f} / {s.max():.2f}\\n")
```

Output:

```
=== NUMERIC DISTRIBUTIONS ===

📊 quantity:
   Mean   : 5.02
   Median : 5.00
   Std    : 2.58
   Skew   : 0.01  ← roughly normal
   Min/Max: 1.00 / 9.00

📊 unit_price:
   Mean   : 2541.32
   Median : 2558.45
   Std    : 1404.21
   Skew   : -0.03  ← roughly normal
   Min/Max: 100.32 / 4998.76

📊 discount:
   Mean   : 5.58
   Median : 0.00
   Std    : 7.21
   Skew   : 1.23  ← right skewed
   Min/Max: 0.00 / 20.00

📊 rating:
   Mean   : 3.01
   Median : 3.00
   Std    : 1.41
   Skew   : 0.00  ← roughly normal
   Min/Max: 1.00 / 5.00
```

Step 3: Categorical Distributions

```python
# Categorical value counts + percentages
print("=== CATEGORICAL DISTRIBUTIONS ===\\n")

for col in ["status","country","discount"]:
    print(f"📊 {col}:")
    vc = orders_raw[col].value_counts(dropna=False)
    pct = orders_raw[col].value_counts(dropna=False, normalize=True).mul(100).round(1)
    dist = pd.DataFrame({"Count":vc,"Pct%":pct})
```

```
    print(dist.to_string())
    print()
```

Output:

```
📊 status:
           Count   Pct%
Delivered    152   29.2
Pending      147   28.3
DELIVERED    102   19.6  ← duplicate category (capitalization)!
cancelled     58   11.2
Cancelled     43    8.3  ← another duplicate!
pending       18    3.5

📊 country:
          Count   Pct%
India       292   56.2
USA          82   15.8
UK           61   11.7
Germany      50    9.6
NaN          35    6.7

📊 discount:
    Count   Pct%
0     291   56.0
5      74   14.2
10     74   14.2
15     42    8.1
20     39    7.5
```

Step 4: Outlier Detection

```
def detect_outliers_iqr(series, label=""):
    """Detect outliers using IQR method"""
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = series[(series < lower) | (series > upper)]
    print(f"📊 {label}:")
    print(f"   IQR bounds : [{lower:.2f}, {upper:.2f}]")
    print(f"   Outliers   : {len(outliers):,} ({len(outliers)/len(series)*100:.1f}%)")
    return outliers

for col in ["unit_price", "quantity"]:
    detect_outliers_iqr(orders_raw[col].dropna(), col)
    print()
```

Output:

```
📊 unit_price:
   IQR bounds : [-570.41, 5652.45]
   Outliers   : 0 (0.0%)

📊 quantity:
   IQR bounds : [-2.00, 12.50]
   Outliers   : 0 (0.0%)
```

Step 5: Correlation Analysis

```
# Correlation between numeric columns
corr_matrix = orders_raw[["quantity","unit_price","discount","rating"]].corr().round
(3)
print("=== CORRELATION MATRIX ===")
print(corr_matrix.to_string())
```

```
# Flag strong correlations (|r| > 0.5)
print("\\n🔗 Strong Correlations (|r| > 0.5):")
for col1 in corr_matrix.columns:
    for col2 in corr_matrix.columns:
        if col1 < col2:
            r = corr_matrix.loc[col1, col2]
            if abs(r) > 0.5:
                print(f"   {col1} ↔ {col2}: r = {r}")
```

Output:

```
=== CORRELATION MATRIX ===
            quantity  unit_price  discount  rating
quantity       1.000       0.017     0.031   0.012
unit_price     0.017       1.000    -0.002   0.008
discount       0.031      -0.002     1.000  -0.015
rating         0.012       0.008    -0.015   1.000

🔗 Strong Correlations (|r| > 0.5):
   (none — variables are largely independent in this dataset)
```

EDA Summary Report

```
def eda_summary(df):
    """Print a complete EDA summary"""
    print("╔══════════════════════════════════════════════╗")
    print("║         EDA SUMMARY REPORT                   ║")
    print("╚══════════════════════════════════════════════╝")
    print(f"  Rows         : {df.shape[0]:,}")
    print(f"  Columns      : {df.shape[1]}")
    print(f"  Memory       : {df.memory_usage(deep=True).sum()/1024/1024:.2f} MB")
    print(f"  Duplicates   : {df.duplicated().sum():,}")
    print(f"  Total NaN    : {df.isnull().sum().sum():,} ({df.isnull().mean().mean()*1
00:.1f}% overall)")
    print(f"  Numeric cols : {len(df.select_dtypes('number').columns)}")
    print(f"  String cols  : {len(df.select_dtypes('object').columns)}")
    print(f"  Date cols    : {len(df.select_dtypes('datetime').columns)}")
    print()
    print("  Missing by column:")
    missing = df.isnull().sum()
    missing = missing[missing > 0].sort_values(ascending=False)
    for col, cnt in missing.items():
        bar = "" * int(cnt / df.shape[0] * 20)
        print(f"    {col:<20} {bar} {cnt} ({cnt/df.shape[0]*100:.1f}%)")

eda_summary(orders_raw)
```

Output:

```
╔══════════════════════════════════════════════╗
║         EDA SUMMARY REPORT                   ║
╚══════════════════════════════════════════════╝
  Rows         : 520
  Columns      : 10
  Memory       : 0.42 MB
  Duplicates   : 20
  Total NaN    : 195 (3.8% overall)
  Numeric cols : 6
  String cols  : 4
  Date cols    : 0

  Missing by column:
    rating               ████████████████ 90 (17.3%)
    unit_price           ███████ 40 (7.7%)
```

```
country              ██████ 35 (6.7%)
customer_id          ████▌ 30 (5.8%)
```

2. Feature Engineering

📌 Definition

Feature Engineering creates **new, informative columns** from existing ones — turning raw data into model-ready or analysis-ready features that capture domain knowledge.

Example 1: Revenue & Profit Features

```
# Start with a clean copy for engineering
df = orders_raw.copy()
df["order_date"] = pd.to_datetime(df["order_date"])
df["unit_price"]  = df["unit_price"].fillna(df["unit_price"].median())
df["customer_id"] = df["customer_id"].fillna(df["customer_id"].mode()[0])

# Core revenue calculation
df["gross_revenue"]   = df["quantity"] * df["unit_price"]
df["discount_amount"] = df["gross_revenue"] * df["discount"] / 100
df["net_revenue"]     = df["gross_revenue"] - df["discount_amount"]

print("Revenue features:")
print(df[["order_id","quantity","unit_price","discount","gross_revenue",
          "discount_amount","net_revenue"]].head(5).round(2))
```

Output:

```
Revenue features:
   order_id  quantity  unit_price  discount  gross_revenue  discount_amount  net_revenue
0      1001         3     2341.50         0        7024.50             0.00      7024.50
1      1002         7      876.25         5        6133.75           306.69      5827.06
2      1003         2     2558.45         0        5116.90             0.00      5116.90
3      1004         5     1523.80        10        7619.00           761.90      6857.10
4      1005         6     3421.20        15       20527.20          3079.08     17448.12
```

Example 2: Date-Based Features

```
# Extract all useful time features from order_date
df["order_year"]     = df["order_date"].dt.year
df["order_month"]    = df["order_date"].dt.month
df["order_month_name"] = df["order_date"].dt.month_name()
df["order_quarter"]  = df["order_date"].dt.quarter
df["order_dow"]      = df["order_date"].dt.dayofweek       # 0=Mon
df["order_dow_name"] = df["order_date"].dt.day_name()
df["is_weekend"]     = (df["order_date"].dt.dayofweek >= 5).astype(int)
df["is_month_end"]   = df["order_date"].dt.is_month_end.astype(int)
df["order_week"]     = df["order_date"].dt.isocalendar().week.astype(int)

# Days since first order (customer tenure proxy)
df["days_since_first_order"] = (
    df["order_date"] - df["order_date"].min()
).dt.days

print("Date features:")
print(df[["order_date","order_month_name","order_quarter",
          "order_dow_name","is_weekend","days_since_first_order"]].head(5))
```

Output:

```
Date features:
  order_date order_month_name  order_quarter order_dow_name  is_weekend  days_since_first_order
0 2023-01-01          January              1         Sunday           1                       0
1 2023-01-02          January              1         Monday           0                       1
2 2023-01-03          January              1        Tuesday           0                       2
3 2023-01-04          January              1      Wednesday           0                       3
4 2023-01-05          January              1       Thursday           0                       4
```

Example 3: Category Labels & Bins

```python
# Bin net_revenue into order size tiers
df["order_size"] = pd.cut(
    df["net_revenue"],
    bins   = [0, 2000, 8000, 20000, float("inf")],
    labels = ["Small","Medium","Large","Enterprise"]
)

# Customer value tiers based on order quantity
df["quantity_tier"] = pd.qcut(
    df["quantity"], q=3,
    labels = ["Low","Mid","High"]
)

# High discount flag
df["heavy_discount"] = (df["discount"] >= 15).astype(int)

# Rating category
df["rating_label"] = pd.cut(
    df["rating"].fillna(3),
    bins   = [0, 2, 3, 4, 5],
    labels = ["Poor","Average","Good","Excellent"]
)

print("Category features:")
print(df[["net_revenue","order_size","quantity","quantity_tier",
          "discount","heavy_discount","rating","rating_label"]].head(6))
```

Output:

```
Category features:
   net_revenue order_size  quantity quantity_tier  discount  heavy_discount  rating rating_label
0      7024.50     Medium         3           Low         0               0     5.0    Excellent
1      5827.06     Medium         7          High         5               0     2.0         Poor
2      5116.90     Medium         2           Low         0               0     3.0      Average
3      6857.10     Medium         5           Mid        10               0     4.0         Good
4     17448.12      Large         6          High        15               1     1.0         Poor
5      3421.56      Small         1           Low        20               1     NaN      Average
```

Example 4: Aggregation-Based Features (Customer-Level)

```python
# Create customer-level aggregated features
customer_features = df.groupby("customer_id").agg(
    total_orders     = ("order_id",     "count"),
    total_revenue    = ("net_revenue",  "sum"),
    avg_order_value  = ("net_revenue",  "mean"),
    max_order_value  = ("net_revenue",  "max"),
    avg_discount     = ("discount",     "mean"),
    avg_rating       = ("rating",       "mean"),
    unique_products  = ("product_id",   "nunique"),
    cancelled_orders = ("status",       lambda x: x.str.lower().eq("cancelled").sum())
```

```
).round(2)

# Add cancel rate
customer_features["cancel_rate"] = (
    customer_features["cancelled_orders"] / customer_features["total_orders"] * 100
).round(2)

# Add customer value segment
customer_features["customer_segment"] = pd.qcut(
    customer_features["total_revenue"], q=3,
    labels=["Bronze","Silver","Gold"]
)

print("Customer-Level Features:")
print(customer_features.head(5).to_string())
```

Output:

```
Customer-Level Features:
             total_orders  total_revenue  avg_order_value  max_order_value  avg_discount  avg_rating  unique_products  cancel
led_orders  cancel_rate customer_segment
customer_id
101.0                   4       31423.12          7855.78         15342.50          5.00        3.25                4
1        25.00           Silver
102.0                   3       18945.33          6315.11         12100.60          3.33        4.00                3
0         0.00           Bronze
103.0                   5       42567.89          8513.58         18234.10          8.00        2.80                4
2        40.00             Gold
104.0                   2        9234.56          4617.28          6012.33          7.50        4.50                2
0         0.00           Bronze
105.0                   6       58923.45          9820.58         21345.60         11.67        3.17                5
1        16.67             Gold
```

Example 5: Lag & Rolling Features (Time-Based)

```
# Sort by date for time-based features
df_ts = df.sort_values("order_date").reset_index(drop=True)

# Daily revenue aggregation
daily = df_ts.groupby("order_date")["net_revenue"].sum().reset_index()

# Lag features
daily["revenue_lag1"]  = daily["net_revenue"].shift(1)    # yesterday
daily["revenue_lag7"]  = daily["net_revenue"].shift(7)    # last week

# Rolling features
daily["revenue_7d_avg"] = daily["net_revenue"].rolling(7, min_periods=1).mean()
daily["revenue_30d_avg"]= daily["net_revenue"].rolling(30, min_periods=1).mean()

# Growth rates
daily["dod_growth_%"] = daily["net_revenue"].pct_change() * 100
daily["wow_growth_%"] = daily["net_revenue"].pct_change(7) * 100

print("Time-Series Features:")
print(daily[["order_date","net_revenue","revenue_lag1","revenue_7d_avg","dod_growth
_%"]].head(10).round(2))
```

Output:

```
Time-Series Features:
  order_date  net_revenue  revenue_lag1  revenue_7d_avg  dod_growth_%
0 2023-01-01      7024.50           NaN         7024.50           NaN
1 2023-01-02      5827.06       7024.50         6425.78        -17.03
```

```
2 2023-01-03      5116.90       5827.06         5989.49        -12.18
3 2023-01-04      6857.10       5116.90         6206.39         34.03
4 2023-01-05     17448.12       6857.10         8454.74        154.43
5 2023-01-06      3421.56      17448.12         7615.87        -80.39
6 2023-01-07      8945.32       3421.56         7805.79        161.44
7 2023-01-08     12345.67       8945.32         8495.96         38.00
8 2023-01-09      6789.12      12345.67         8306.43        -44.99
9 2023-01-10      9823.45       6789.12         8619.59         44.69
```

---

3. Data Cleaning Pipeline

📌 Definition

A **Cleaning Pipeline** is a structured, repeatable sequence of cleaning steps that transforms raw, messy data into reliable, analysis-ready data.

> **Production rule:** Every cleaning step must be a function — reproducible, testable, and reusable.

Complete Cleaning Pipeline

```python
def clean_orders(df_raw):
    """
    Complete data cleaning pipeline for orders data.
    Returns cleaned DataFrame and a quality report.
    """
    df = df_raw.copy()
    report = {}

    # ── STEP 1: Remove duplicates ──────────────────────────────
    before = len(df)
    df = df.drop_duplicates().reset_index(drop=True)
    report["duplicates_removed"] = before - len(df)
    print(f"✅ Step 1 — Removed {report['duplicates_removed']} duplicates")

    # ── STEP 2: Standardize column names ──────────────────────
    df.columns = df.columns.str.lower().str.strip().str.replace(" ","_")
    print(f"✅ Step 2 — Column names standardized")

    # ── STEP 3: Fix data types ─────────────────────────────────
    df["order_date"]   = pd.to_datetime(df["order_date"], errors="coerce")
    df["customer_id"]  = pd.to_numeric(df["customer_id"], errors="coerce")
    df["unit_price"]   = pd.to_numeric(df["unit_price"], errors="coerce")
    df["rating"]       = pd.to_numeric(df["rating"], errors="coerce")
    print(f"✅ Step 3 — Data types fixed")

    # ── STEP 4: Standardize categorical values ─────────────────
    df["status"]  = df["status"].str.strip().str.title()
    df["country"] = df["country"].str.strip().str.title()

    # Consolidate status typos
    status_map = {
        "Delivered" : "Delivered",
        "Deliverd"  : "Delivered",    # typo
        "Pending"   : "Pending",
        "Cancelled" : "Cancelled",
        "Canceled"  : "Cancelled",    # US spelling
    }
    df["status"] = df["status"].replace(status_map)
    report["unique_statuses_after"] = df["status"].nunique()
    print(f"✅ Step 4 — Status standardized to {report['unique_statuses_after']} value
s")
```

```python
# ── STEP 5: Handle missing values ──────────────────────────────────
# unit_price → fill with product median (group-wise)
df["unit_price"] = df.groupby("product_id")["unit_price"].transform(
    lambda x: x.fillna(x.median())
)
# Remaining unit_price nulls → global median
df["unit_price"] = df["unit_price"].fillna(df["unit_price"].median())

# customer_id → essential column; drop rows where missing
before_drop = len(df)
df = df.dropna(subset=["customer_id"]).reset_index(drop=True)
report["rows_dropped_no_customer"] = before_drop - len(df)

# country → fill with "Unknown"
df["country"] = df["country"].fillna("Unknown")

# rating → fill with product median (not global — products differ!)
df["rating"] = df.groupby("product_id")["rating"].transform(
    lambda x: x.fillna(x.median())
)
df["rating"] = df["rating"].fillna(3.0)  # final fallback

report["nulls_remaining"] = df.isnull().sum().sum()
print(f"✅ Step 5 — Missing values handled ({report['nulls_remaining']} remainin
g)")

# ── STEP 6: Validate business rules ────────────────────────────────
invalid_qty    = (df["quantity"] < 1) | (df["quantity"] > 100)
invalid_price  = df["unit_price"] <= 0
invalid_disc   = (df["discount"] < 0) | (df["discount"] > 100)
invalid_rating = (df["rating"] < 1) | (df["rating"] > 5)

report["invalid_quantity"]  = invalid_qty.sum()
report["invalid_price"]     = invalid_price.sum()
report["invalid_discount"]  = invalid_disc.sum()
report["invalid_rating"]    = invalid_rating.sum()

# Cap rating to valid range
df["rating"] = df["rating"].clip(1, 5)
print(f"✅ Step 6 — Business rules validated")

# ── STEP 7: Optimize dtypes ────────────────────────────────────────
df["customer_id"] = df["customer_id"].astype("int32")
df["product_id"]  = df["product_id"].astype("int32")
df["quantity"]    = df["quantity"].astype("int8")
df["discount"]    = df["discount"].astype("int8")
df["status"]      = df["status"].astype("category")
df["country"]     = df["country"].astype("category")

mem_before = df_raw.memory_usage(deep=True).sum() / 1024
mem_after  = df.memory_usage(deep=True).sum() / 1024
report["memory_before_kb"] = round(mem_before, 2)
report["memory_after_kb"]  = round(mem_after, 2)
report["memory_reduction_pct"] = round((1 - mem_after/mem_before)*100, 1)
print(f"✅ Step 7 — Dtypes optimized ({report['memory_reduction_pct']}% memory red
uction)")

# ── FINAL REPORT ───────────────────────────────────────────────────
print(f"\\n{'='*50}")
```

```
    print("CLEANING PIPELINE REPORT")
    print(f"{'='*50}")
    for k, v in report.items():
        print(f"  {k:<35} : {v}")
    print(f"  {'final_shape':<35} : {df.shape}")

    return df, report

# Run the pipeline
df_clean, quality_report = clean_orders(orders_raw)
```

Output:

```
✅ Step 1 — Removed 20 duplicates
✅ Step 2 — Column names standardized
✅ Step 3 — Data types fixed
✅ Step 4 — Status standardized to 3 values
✅ Step 5 — Missing values handled (0 remaining)
✅ Step 6 — Business rules validated
✅ Step 7 — Dtypes optimized (31.2% memory reduction)

==================================================
CLEANING PIPELINE REPORT
==================================================
  duplicates_removed                : 20
  unique_statuses_after             : 3
  rows_dropped_no_customer          : 30
  nulls_remaining                   : 0
  invalid_quantity                  : 0
  invalid_price                     : 0
  invalid_discount                  : 0
  invalid_rating                    : 0
  memory_before_kb                  : 431.24
  memory_after_kb                   : 296.87
  memory_reduction_pct              : 31.2
  final_shape                       : (470, 10)
```

---

4. Combining Multiple Datasets

📌 Definition

Real-world analysis almost always requires joining data from **multiple sources** — orders + customers + products + external data.

Step 1: Build the Master Analysis Table

```
# Start with clean orders
df_master = df_clean.copy()

# ── Join 1: Add Customer Information ──────────────────────────────
customers["customer_id"] = customers["customer_id"].astype("int32")
customers["join_date"]   = pd.to_datetime(customers["join_date"])

df_master = pd.merge(
    df_master,
    customers[["customer_id","customer_name","segment","city","join_date"]],
    on  = "customer_id",
    how = "left"   # keep all orders, even if customer info missing
)

print(f"After customer join: {df_master.shape}")
print(f"Missing customer info: {df_master['customer_name'].isnull().sum()}")
```

Output:

```
After customer join: (470, 15)
Missing customer info: 0
```

```
# ── Join 2: Add Product Information ──────────────────────────────
products["product_id"] = products["product_id"].astype("int32")

df_master = pd.merge(
    df_master,
    products[["product_id","product_name","category","cost_price"]],
    on  = "product_id",
    how = "left"
)

print(f"After product join: {df_master.shape}")
```

```
# ── Step 3: Compute Profit (needs cost_price from product table) ──────────
df_master["total_cost"]  = df_master["quantity"] * df_master["cost_price"]
df_master["gross_revenue"] = df_master["quantity"] * df_master["unit_price"]
df_master["discount_amount"] = df_master["gross_revenue"] * df_master["discount"] / 10
0
df_master["net_revenue"] = df_master["gross_revenue"] - df_master["discount_amount"]
df_master["gross_profit"] = df_master["net_revenue"] - df_master["total_cost"]
df_master["margin_%"]    = (df_master["gross_profit"] / df_master["net_revenue"] * 10
0).round(2)

print("Master table with profit:")
print(df_master[["order_id","customer_name","category","net_revenue",
                 "gross_profit","margin_%"]].head(5).round(2))
```

Output:

```
Master table with profit:
   order_id   customer_name     category  net_revenue  gross_profit  margin_%
0      1001  Customer_131   Electronics      7024.50       4812.30      68.51
1      1002  Customer_148      Clothing      5827.06       3920.14      67.28
2      1003  Customer_126         Books      5116.90       3876.54      75.77
3      1004  Customer_104          Home      6857.10       4923.44      71.81
4      1005  Customer_119    Electronics     17448.12      11923.45      68.33
```

Step 2: Key Business Analysis

```
print("╔══════════════════════════════════════════════════╗")
print("║       BUSINESS ANALYSIS RESULTS                  ║")
print("╚══════════════════════════════════════════════════╝\\n")

# Revenue by Category
print("📦 Revenue & Profit by Category:")
cat_analysis = df_master.groupby("category").agg(
    orders       = ("order_id","count"),
    total_revenue= ("net_revenue","sum"),
    total_profit = ("gross_profit","sum"),
    avg_margin   = ("margin_%","mean")
).round(2).sort_values("total_revenue", ascending=False)
cat_analysis["revenue_share%"] = (cat_analysis["total_revenue"] /
                                  cat_analysis["total_revenue"].sum() * 100).round(1)
print(cat_analysis.to_string())
print()
```

```
# Revenue by Customer Segment
print("👥 Revenue by Customer Segment:")
seg_analysis = df_master.groupby("segment").agg(
    customers    = ("customer_id","nunique"),
    total_orders = ("order_id","count"),
    total_revenue= ("net_revenue","sum"),
    avg_order_val= ("net_revenue","mean"),
    cancel_rate  = ("status", lambda x: (x=="Cancelled").sum()/len(x)*100)
).round(2)
print(seg_analysis.to_string())
print()

# Monthly Revenue Trend
print("📅 Monthly Revenue Trend (2023):")
monthly = df_master.groupby(df_master["order_date"].dt.to_period("M")).agg(
    orders  = ("order_id","count"),
    revenue = ("net_revenue","sum"),
    profit  = ("gross_profit","sum")
).round(0)
monthly["MoM_growth%"] = monthly["revenue"].pct_change()*100
print(monthly.head(8).round(1).to_string())
```

Output:

```
╔══════════════════════════════════════╗
║       BUSINESS ANALYSIS RESULTS      ║
╚══════════════════════════════════════╝

📦 Revenue & Profit by Category:
             orders  total_revenue  total_profit  avg_margin  revenue_share%
category
Electronics      94     678234.50     463012.30       68.25            30.1
Clothing         91     612345.80     421234.56       68.81            27.2
Home             88     432567.20     312456.78       72.43            19.2
Sports           98     287654.30     198765.43       69.12            12.8
Books            99     241234.56     183456.78       76.01            10.7

👥 Revenue by Customer Segment:
          customers  total_orders  total_revenue  avg_order_val  cancel_rate
segment
Budget           16           148      423456.78        2861.87        14.86
Premium          16           152      712345.60        4686.48         8.55
Standard         18           170      116234.12        6836.99        11.18

📅 Monthly Revenue Trend (2023):
         orders   revenue    profit  MoM_growth%
2023-01      31   234567.0  160234.0          NaN
2023-02      28   198765.0  136543.0        -15.3
2023-03      31   267890.0  183234.0         34.8
2023-04      30   245678.0  167432.0         -8.3
2023-05      31   289234.0  197654.0         17.7
2023-06      30   312567.0  213456.0          8.1
2023-07      31   298765.0  204321.0         -4.4
2023-08      31   334567.0  228765.0         11.9
```

Step 3: Advanced Multi-Dataset Patterns

```
# ── Pattern 1: Anti-join — Find products with NO orders ──────────────
active_products   = df_master["product_id"].unique()
all_products      = products["product_id"].values
inactive_products = products[~products["product_id"].isin(active_products)]
print(f"Products with no orders: {len(inactive_products)}")
print(inactive_products[["product_id","product_name","category"]])

# ── Pattern 2: Customer RFM Analysis (Recency, Frequency, Monetary) ──────
snapshot_date = df_master["order_date"].max() + pd.Timedelta(days=1)
```

```
rfm = df_master[df_master["status"] != "Cancelled"].groupby("customer_id").agg(
    Recency   = ("order_date",  lambda x: (snapshot_date - x.max()).days),
    Frequency = ("order_id",    "count"),
    Monetary  = ("net_revenue", "sum")
).round(2)

# Score each dimension (1-4, higher is better)
rfm["R_Score"] = pd.qcut(rfm["Recency"],   q=4, labels=[4,3,2,1])
rfm["F_Score"] = pd.qcut(rfm["Frequency"], q=4, labels=[1,2,3,4], duplicates="drop")
rfm["M_Score"] = pd.qcut(rfm["Monetary"],  q=4, labels=[1,2,3,4])

rfm["RFM_Score"] = (rfm["R_Score"].astype(int) +
                    rfm["F_Score"].astype(int) +
                    rfm["M_Score"].astype(int))

rfm["Segment"] = pd.cut(
    rfm["RFM_Score"],
    bins   = [0, 5, 8, 10, 12],
    labels = ["At Risk","Needs Attention","Loyal","Champions"]
)

print("\\n🎯 RFM Segmentation:")
print(rfm[["Recency","Frequency","Monetary","RFM_Score","Segment"]].head(8).to_string
())
print("\\nRFM Segment Distribution:")
print(rfm["Segment"].value_counts().to_string())
```

Output:

```
Products with no orders: 3
   product_id   product_name category
2         203  Product_203   Clothing
8         209  Product_209     Sports
15        216  Product_216      Books

🎯 RFM Segmentation:
             Recency  Frequency  Monetary  RFM_Score         Segment
customer_id
101               45          4  31423.12          9  Needs Attention
102              312          3  18945.33          5         At Risk
103               12          5  42567.89         11           Loyal
104              234          2   9234.56          5         At Risk
105               67          6  58923.45         10           Loyal

RFM Segment Distribution:
Loyal             16
Needs Attention   14
At Risk           12
Champions          8
```

5. Complete End-to-End Analysis Workflow

```
class ECommerceAnalysis:
    """
    Production-grade analysis class encapsulating the full workflow.
    Load → Clean → Engineer → Analyze → Export
    """

    def __init__(self, orders, customers, products):
        self.raw_orders  = orders
        self.customers   = customers
        self.products    = products
        self.clean_df    = None
```

```
        self.master_df   = None
        self.insights    = {}

    def run_pipeline(self):
        """Execute complete pipeline"""
        print("🚀 Starting Analysis Pipeline...\\n")
        self._clean()
        self._engineer()
        self._analyze()
        self._print_insights()
        return self

    def _clean(self):
        """Cleaning step"""
        self.clean_df, _ = clean_orders(self.raw_orders)
        print()

    def _engineer(self):
        """Feature engineering step"""
        df = self.clean_df.copy()
        cust = self.customers.copy()
        prod = self.products.copy()
        cust["customer_id"] = cust["customer_id"].astype("int32")
        prod["product_id"]  = prod["product_id"].astype("int32")

        df = pd.merge(df, cust[["customer_id","segment","city"]], on="customer_id", ho
w="left")
        df = pd.merge(df, prod[["product_id","category","cost_price"]], on="product_i
d", how="left")

        df["gross_revenue"]  = df["quantity"] * df["unit_price"]
        df["net_revenue"]    = df["gross_revenue"] * (1 - df["discount"]/100)
        df["total_cost"]     = df["quantity"] * df["cost_price"]
        df["gross_profit"]   = df["net_revenue"] - df["total_cost"]
        df["margin_%"]       = (df["gross_profit"] / df["net_revenue"] * 100).round(2)
        df["order_month"]    = df["order_date"].dt.month
        df["order_quarter"]  = df["order_date"].dt.quarter

        self.master_df = df
        print("✅ Feature engineering complete\\n")

    def _analyze(self):
        """Core analysis step"""
        df = self.master_df

        self.insights["total_orders"]    = len(df)
        self.insights["total_revenue"]   = df["net_revenue"].sum().round(2)
        self.insights["total_profit"]    = df["gross_profit"].sum().round(2)
        self.insights["overall_margin"]  = (df["gross_profit"].sum() / df["net_revenu
e"].sum() * 100).round(2)
        self.insights["top_category"]    = df.groupby("category")["net_revenue"].sum
().idxmax()
        self.insights["top_segment"]     = df.groupby("segment")["net_revenue"].sum().
idxmax()
        self.insights["cancel_rate"]     = round((df["status"]=="Cancelled").mean()*10
0, 2)
        self.insights["avg_order_value"] = df["net_revenue"].mean().round(2)
        self.insights["unique_customers"]= df["customer_id"].nunique()

    def _print_insights(self):
```

```
        """Print formatted insights"""
        print("╔══════════════════════════════════════════════════╗")
        print("║       📊  BUSINESS INSIGHTS REPORT               ║")
        print("╚══════════════════════════════════════════════════╝")
        for key, val in self.insights.items():
            label = key.replace("_"," ").title()
            if "revenue" in key or "profit" in key or "value" in key:
                print(f"  {label:<25} : ₹{val:>15,.2f}")
            elif "rate" in key or "margin" in key:
                print(f"  {label:<25} : {val:>14.2f}%")
            else:
                print(f"  {label:<25} : {val!s:>15}")

    def export(self, path="analysis_output.csv"):
        """Export master table"""
        self.master_df.to_csv(path, index=False)
        print(f"\\n✅ Exported to {path} ({len(self.master_df):,} rows)")

# Run complete pipeline
analysis = ECommerceAnalysis(orders_raw, customers, products)
analysis.run_pipeline()
analysis.export()
```

Output:

```
🚀 Starting Analysis Pipeline...

✅ Step 1 — Removed 20 duplicates
✅ Step 2 — Column names standardized
✅ Step 3 — Data types fixed
✅ Step 4 — Status standardized to 3 values
✅ Step 5 — Missing values handled (0 remaining)
✅ Step 6 — Business rules validated
✅ Step 7 — Dtypes optimized (31.2% memory reduction)

✅ Feature engineering complete

╔══════════════════════════════════════════════════╗
║       📊  BUSINESS INSIGHTS REPORT               ║
╚══════════════════════════════════════════════════╝
  Total Orders              :             470
  Total Revenue             : ₹ 2,252,036.18
  Total Profit              : ₹ 1,578,924.85
  Overall Margin            :          70.11%
  Top Category              :     Electronics
  Top Segment               :        Standard
  Cancel Rate               :          11.49%
  Avg Order Value           : ₹      4,791.57
  Unique Customers          :              47

✅ Exported to analysis_output.csv (470 rows)
```

Mini Summary & Cheat Sheet

🔑 Phase 12 Key Takeaways

- ✅ **EDA always first** → `shape`, `dtypes`, nulls, duplicates, distributions, correlations
- ✅ **Feature Engineering** → revenue/profit math, date components, bins, group aggregates, lag/rolling
- ✅ **Cleaning Pipeline** → deduplicate → standardize names → fix types → handle nulls → validate rules → optimize dtypes
- ✅ **Always use functions** → repeatable, testable pipeline steps
- ✅ **Join strategy** → always LEFT JOIN primary table + lookup tables; use `indicator=True` for audits
- ✅ **RFM model** → Recency, Frequency, Monetary — the classic customer segmentation framework

---

🔖 EDA Checklist

```
📐 Shape & Structure
  ☐ df.shape, df.dtypes, df.info()
  ☐ df.head(), df.tail(), df.sample(5)

❓ Data Quality
  ☐ df.isnull().sum() → missing counts
  ☐ df.duplicated().sum() → duplicate rows
  ☐ df.describe() → outlier check (min/max)

📊 Distributions
  ☐ value_counts() for categoricals
  ☐ mean/median/std/skew for numerics
  ☐ pd.cut() bins for histograms

🔗 Relationships
  ☐ df.corr() → correlation matrix
  ☐ groupby agg → segment differences
```

🔖 Feature Engineering Checklist

```
💰 Revenue/Profit
  ☐ gross = qty × price
  ☐ net   = gross × (1 - discount%)
  ☐ profit= net - cost
  ☐ margin= profit / net × 100

📅 Date Features
  ☐ .dt.year, .dt.month, .dt.quarter
  ☐ .dt.day_name(), .dt.dayofweek
  ☐ is_weekend, is_month_end

🏷️ Category Features
  ☐ pd.cut() → fixed bins
  ☐ pd.qcut() → equal-size bins
  ☐ map() → encode labels

📈 Time Series Features
  ☐ shift(1), shift(7), shift(30) → lags
  ☐ rolling(7).mean() → trend
  ☐ pct_change() → growth rates

👥 Aggregation Features
  ☐ groupby customer → total, avg, count
  ☐ RFM → Recency, Frequency, Monetary
```

🔥 MEGA PRACTICE PHASE — 100+ Real-World Questions

> **Instructions:** Set up the dataset below, then attempt each question yourself before checking `Mega_Practice_Answers.md`.
> Work through ALL questions systematically — this is your final consolidation of everything learned in Phases 1–12.

📦 MASTER DATASET SETUP

> Run this ONCE before attempting any question. All questions use these DataFrames.

```python
import pandas as pd
import numpy as np

np.random.seed(42)

# ── ORDERS ──────────────────────────────────────────────────────────────
orders = pd.DataFrame({
    "order_id"    : range(1, 1001),
    "customer_id" : np.random.choice(range(1, 201), 1000),
    "product_id"  : np.random.choice(range(1, 51), 1000),
```

```
    "order_date"  : pd.date_range("2022-01-01", periods=1000, freq="8h"),
    "quantity"    : np.random.randint(1, 15, 1000),
    "unit_price"  : np.random.uniform(50, 5000, 1000).round(2),
    "discount"    : np.random.choice([0, 5, 10, 15, 20, 25], 1000),
    "status"      : np.random.choice(["Delivered","Pending","Cancelled","Returned"], 1
000,
                                     p=[0.60, 0.20, 0.12, 0.08]),
    "payment"     : np.random.choice(["Credit Card","UPI","Net Banking","COD","Walle
t"], 1000),
    "ship_days"   : np.random.randint(1, 15, 1000)
})
# Introduce real-world messiness
orders.loc[np.random.choice(1000, 50, replace=False), "unit_price"] = np.nan
orders.loc[np.random.choice(1000, 30, replace=False), "discount"]   = np.nan
orders = pd.concat([orders, orders.sample(20, random_state=7)],
                   ignore_index=True)  # 20 duplicate rows

# ── CUSTOMERS ──────────────────────────────────────────────────────────
customers = pd.DataFrame({
    "customer_id"  : range(1, 201),
    "name"         : [f"Customer_{i:03d}" for i in range(1, 201)],
    "age"          : np.random.randint(18, 70, 200),
    "gender"       : np.random.choice(["Male","Female","Other"], 200, p=[0.48,0.48,0.0
4]),
    "city"         : np.random.choice(["Mumbai","Delhi","Bangalore","Chennai",
                                       "Hyderabad","Pune","Kolkata"], 200),
    "segment"      : np.random.choice(["Premium","Standard","Budget"], 200, p=[0.2,0.
5,0.3]),
    "join_year"    : np.random.choice([2019,2020,2021,2022,2023], 200)
})

# ── PRODUCTS ───────────────────────────────────────────────────────────
products = pd.DataFrame({
    "product_id"   : range(1, 51),
    "product_name" : [f"Product_{i:02d}" for i in range(1, 51)],
    "category"     : np.random.choice(["Electronics","Clothing","Books",
                                       "Home & Kitchen","Sports","Beauty"], 50),
    "brand"        : np.random.choice(["BrandA","BrandB","BrandC","BrandD","BrandE"],
50),
    "cost_price"   : np.random.uniform(30, 3000, 50).round(2),
    "weight_kg"    : np.random.uniform(0.1, 20.0, 50).round(2)
})

# ── RETURNS ────────────────────────────────────────────────────────────
returned_ids = orders[orders["status"] == "Returned"]["order_id"].values
returns = pd.DataFrame({
    "return_id"   : range(1, len(returned_ids) + 1),
    "order_id"    : returned_ids,
    "return_date" : pd.date_range("2022-06-01", periods=len(returned_ids), freq="2D"),
    "reason"      : np.random.choice(["Defective","Wrong Item","Changed Mind",
                                      "Late Delivery"], len(returned_ids))
})

print("orders    :", orders.shape)
print("customers :", customers.shape)
print("products  :", products.shape)
print("returns   :", returns.shape)
```

🟢 EASY QUESTIONS (1–30)

🔹 Basic Inspection

Q1. Display the first 7 rows and last 5 rows of the `orders` DataFrame.

Q2. How many rows and columns does `orders` have? Print the shape.

Q3. What are the data types of each column in `orders`? Which columns need type conversion?

Q4. How many missing values are there in each column of `orders`? Show as count AND percentage.

Q5. How many duplicate rows exist in `orders`? Remove them and reset the index.

Q6. Print summary statistics for all numeric columns in `orders`. What is the average `unit_price`?

Q7. How many unique customers placed orders? How many unique products were ordered?

Q8. What is the distribution of `status`? Show counts and percentage of each status.

Q9. What are the unique payment methods used? Show count per method.

Q10. Display all columns and their value ranges (min and max) in one table.

🔹 Filtering & Selection

Q11. Select all orders where `status` is `"Delivered"`.

Q12. Find all orders with `unit_price` greater than ₹3,000.

Q13. Show all orders placed by `customer_id = 42`.

Q14. Filter orders where `discount` is 20% or more AND `status` is `"Delivered"`.

Q15. Find orders where `quantity` is between 5 and 10 (inclusive) using `between()`.

Q16. Select only the columns: `order_id`, `customer_id`, `order_date`, `status`, `unit_price`.

Q17. Find orders where payment method is either `"UPI"` or `"Wallet"` using `isin()`.

Q18. Get all orders from the first 100 rows using `iloc[]`.

Q19. Find the order with the highest `unit_price`.

Q20. Show all orders where `ship_days` is greater than 10 (late deliveries).

🔹 Basic Calculations

Q21. Add a new column `gross_revenue = quantity × unit_price`. Fill missing `unit_price` with the median before calculating.

Q22. Add column `net_revenue = gross_revenue × (1 − discount/100)`. Handle missing `discount` by treating it as 0.

Q23. How many orders were delivered vs cancelled vs returned vs pending? Show as a bar-style value count.

Q24. What is the total quantity of items ordered across all orders?

Q25. What percentage of orders used COD (Cash on Delivery) as payment?

Q26. Sort `orders` by `unit_price` descending. Show top 10.

Q27. Sort `orders` by `order_date` ascending and `unit_price` descending simultaneously.

Q28. Rename columns: `ship_days` → `shipping_days` and `payment` → `payment_method`.

Q29. Drop the `weight_kg` column from `products`.

Q30. Reset the index of `orders` after dropping duplicates.

🟡 MEDIUM QUESTIONS (31–70)

🔸 Groupby & Aggregation

Q31. What is the total revenue (net_revenue) per `status`?

Q32. Find the average `unit_price` per product `category` (after joining products table).

Q33. Which city has the highest number of customers? (Use customers table)

Q34. How many orders did each customer place? Show top 10 customers by order count.

Q35. What is the average `ship_days` per `payment` method?

Q36. Find the total revenue, total orders, and average order value per customer `segment` (requires join).

Q37. Which `status` has the highest average `discount`?

Q38. Find the top 5 most ordered `product_id`s by total quantity.

Q39. What is the median `unit_price` per `category`? Sort by median descending.

Q40. For each payment method, compute: count, total revenue, avg revenue per order.

🔸 Data Cleaning

Q41. Fill missing `unit_price` with the median price of that product's category (group-wise fill).

Q42. Fill missing `discount` with 0 (assume no discount when missing).

Q43. After removing duplicates, verify the order_id column has no duplicates.

Q44. Standardize the `status` column: strip whitespace and apply `.str.title()`.

Q45. Check if any `order_date` is in the future (relative to the latest date in data). Remove them.

Q46. Convert `discount` column to `float` dtype and `customer_id` to `int` dtype.

Q47. Find all customers in the `customers` table who have NEVER placed an order (anti-join).

Q48. Find products that appear in `orders` but do NOT exist in the `products` table.

Q49. Check and fix: `quantity` should be between 1 and 50. Flag invalid rows.

Q50. Create a clean copy of `orders` with: no nulls, no duplicates, correct dtypes, and status standardized.

🔸 Merging & Joining

Q51. Join `orders` with `customers` on `customer_id`. Keep all orders (even if customer info missing).

Q52. Join the result from Q51 with `products` on `product_id`. This is the master table.

Q53. How many orders don't have a matching customer in the customers table?

Q54. Using the master table, find total revenue per customer `segment` and `city`.

Q55. Join `orders` with `returns` to mark which orders were returned. Add a boolean column `is_returned`.

Q56. What is the total `gross_revenue` per product `category` and `brand`? (pivot_table)

Q57. Find the most common `reason` for returns per product `category`.

Q58. Which customers appear in both `orders` AND `returns` (as returners)?

Q59. Combine `orders` from 2022 and 2023 into one DataFrame after filtering by year. Use `concat`.

Q60. After joining all tables, compute: profit = net_revenue − (quantity × cost_price). Find negative profit orders.

🔸 Time Series

Q61. Convert `order_date` to datetime and set it as the index.

Q62. Calculate total daily revenue. Show the top 5 highest revenue days.

Q63. Resample orders to monthly frequency. Compute total revenue and order count per month.

Q64. What is the week-over-week (WoW) revenue growth rate? Compute for each week.

Q65. Find the month with the highest number of "Cancelled" orders.

Q66. Add a column for the 7-day rolling average of daily revenue.

Q67. Extract the day of week from `order_date`. Which weekday has the highest average order value?

Q68. How many orders were placed in Q1 (Jan–Mar) vs Q2 (Apr–Jun) vs Q3 vs Q4?

Q69. Find the top 3 busiest hours for order placement (by count).

Q70. Compute the month-over-month (MoM) percentage change in total revenue.

🔴 HARD QUESTIONS (71–105)

🔺 Advanced Groupby & Window Functions

Q71. Rank customers by total net_revenue within each city using `groupby` + `rank()`.

Q72. For each customer, compute the cumulative revenue over time (sorted by order_date).

Q73. Find the top 2 products (by revenue) within each category using `groupby` + `nlargest`.

Q74. Compute the 30-day rolling average revenue per day. Find days where actual revenue was 20%+ above the 30-day rolling average (spikes).

Q75. For each product, find the month in which it generated the highest total revenue.

🔺 Feature Engineering

Q76. Create a `profit_margin_%` column: `(net_revenue - total_cost) / net_revenue × 100`.

Q77. Label each order as `"High Value"` (net_revenue > ₹20,000), `"Mid Value"` (₹5,000–₹20,000), or `"Low Value"` (< ₹5,000) using `pd.cut()`.

Q78. Create a `customer_loyalty` column using `join_year`: 2019 = "Veteran", 2020–2021 = "Regular", 2022–2023 = "New".

Q79. Compute the `discount_impact`: how much additional revenue would have been earned with 0% discount across all delivered orders?

Q80. Create an `is_peak_day` flag: 1 if the order was on a day where total orders > 90th percentile, else 0.

🔺 RFM & Customer Analysis

Q81. Build a full RFM table: Recency (days since last order), Frequency (order count), Monetary (total spend) per customer.

Q82. Score each RFM dimension 1–4 and create an `rfm_segment`: Champions, Loyal, At Risk, Lost.

Q83. Which customer segment (Premium/Standard/Budget) has the highest average RFM score?

Q84. Find the "Best Customers" — top 10% by total spend. What % of total revenue do they contribute?

Q85. Compute repeat purchase rate: % of customers who placed 2 or more orders.

🔺 Multi-Table Business Problems

Q86. Which product has the worst return rate (returns / total orders)? Show top 5.

Q87. What is the average time between order_date and return_date for returned orders? (join with returns)

Q88. Flag customers who have: (a) more than 3 cancelled orders OR (b) a cancellation rate > 30%.

Q89. Build a product performance scorecard: for each product show revenue, units sold, return rate, avg rating (use a proxy: avg discount as rating proxy), profit margin.

Q90. Which city × segment combination generates the most revenue? Create a pivot table.

🔺 Advanced Reshaping

Q91. Create a pivot table: rows = customer `segment`, columns = product `category`, values = total `net_revenue`. Add grand totals.

Q92. Melt the above pivot table back to long format.

Q93. Show quarterly revenue by category as a crosstab (pivot_table with quarter and category).

Q94. Use `stack()` on the pivot from Q91 to convert it to a Series with MultiIndex.

Q95. Unstack one level of the MultiIndex from Q94 to get a different wide view.

🔺 Performance & Optimization

Q96. Profile memory usage of the master table. Apply dtype optimization to reduce memory by at least 50%.

Q97. Rewrite this slow operation as a vectorized expression:

```
# SLOW — fix this
results = []
for _, row in orders.iterrows():
    r = row["unit_price"] * row["quantity"] * (1 - row["discount"]/100)
    results.append(r)
```

Q98. Convert `category`, `brand`, `segment`, `city`, `gender`, `status`, `payment` to `category` dtype. Measure before/after memory.

Q99. Use `query()` to find: all Premium customers from Mumbai with net_revenue > ₹10,000 and status = Delivered.

Q100. Use `eval()` to create 3 new columns in one statement: `gross`, `discount_amount`, `net`.

🔺 Final Boss Questions

Q101. Build a monthly cohort analysis: group customers by the month they placed their FIRST order. For each cohort, count how many customers ordered in subsequent months.

Q102. Find the "Golden Hours" — the 3 hours of the day that consistently generate the highest revenue across all days. Use groupby on hour + mean.

Q103. Detect revenue outlier days: days where revenue is more than 2 standard deviations from the 30-day rolling mean. List them with their revenue and z-score.

Q104. Create a full automated Data Quality Report function that accepts any DataFrame and outputs: shape, dtypes, nulls (count+%), duplicates, numeric distributions, and cardinality for string columns.

Q105. End-to-End Pipeline: Write a single function `run_analysis(orders, customers, products, returns)` that: cleans all tables → joins them → engineers features → computes RFM → outputs top 10 customers, top 5 products, revenue by segment, and exports to CSV.

📊 Self-Assessment Scorecard

Level	Questions	Target Score
🟢 Easy	Q1–Q30	28/30 = Pass
🟡 Medium	Q31–Q70	34/40 = Pass
🔴 Hard	Q71–Q105	25/35 = Pass
Total	**105 Questions**	**87/105 = Expert**

> 💡 **Pro Tips:**
>
> - Attempt questions WITHOUT looking at answers first
> - Time yourself: Easy < 2min, Medium < 5min, Hard < 10min
> - If stuck after 10 minutes → check the answer, understand, then redo from scratch
> - After finishing all 105, write your OWN 10 questions for extra reinforcement

---

🔥 MEGA PRACTICE PHASE — FULL ANSWERS & EXPLANATIONS

> **Note:** Here are the full, step-by-step solutions for all 105 real-world questions. Work through these carefully to solidify your practical Pandas skills!

🟢 EASY QUESTIONS (1–30)

🔹 Basic Inspection

Q1.

```
print(orders.head(7))
print(orders.tail(5))
```

Q2.

```
print("Rows, Columns:", orders.shape)
```

Q3.

```
print(orders.dtypes)
# 'order_date' should be datetime, 'unit_price'/'discount' numeric
```

Q4.

```
null_counts = orders.isnull().sum()
null_pct = (orders.isnull().mean() * 100).round(2)
print(pd.DataFrame({'Count': null_counts, 'Pct': null_pct}))
```

Q5.

```
print("Duplicates:", orders.duplicated().sum())
orders = orders.drop_duplicates().reset_index(drop=True)
```

Q6.

```
print(orders.describe())
print("Avg unit_price:", orders['unit_price'].mean())
```

Q7.

```
print("Unique Customers:", orders['customer_id'].nunique())
print("Unique Products:", orders['product_id'].nunique())
```

Q8.

```
print(orders['status'].value_counts())
print(orders['status'].value_counts(normalize=True) * 100)
```

Q9.

```
print(orders['payment'].value_counts())
```

Q10.

```
print(orders.select_dtypes(include='number').agg(['min', 'max']).T)
```

🔹 Filtering & Selection

Q11.

```
delivered = orders[orders['status'] == 'Delivered']
```

Q12.

```
high_value = orders[orders['unit_price'] > 3000]
```

Q13.

```
cust_42 = orders[orders['customer_id'] == 42]
```

Q14.

```
discount_delivered = orders[(orders['discount'] >= 20) & (orders['status'] == 'Deliver
ed')]
```

Q15.

```
qty_5_10 = orders[orders['quantity'].between(5, 10)]
```

Q16.

```
subset = orders[['order_id', 'customer_id', 'order_date', 'status', 'unit_price']]
```

Q17.

```
upi_wallet = orders[orders['payment'].isin(['UPI', 'Wallet'])]
```

Q18.

```
first_100 = orders.iloc[:100]
```

Q19.

```
highest_price_order = orders.loc[orders['unit_price'].idxmax()]
```

Q20.

```
late_del = orders[orders['ship_days'] > 10]
```

🔹 Basic Calculations

Q21.

```
orders['unit_price'] = orders['unit_price'].fillna(orders['unit_price'].median())
orders['gross_revenue'] = orders['quantity'] * orders['unit_price']
```

Q22.

```
orders['discount'] = orders['discount'].fillna(0)
orders['net_revenue'] = orders['gross_revenue'] * (1 - orders['discount'] / 100)
```

Q23.

```
orders['status'].value_counts().plot(kind='bar')
```

Q24.

```
print("Total Quantity:", orders['quantity'].sum())
```

Q25.

```
cod_pct = (orders['payment'] == 'COD').mean() * 100
print(f"COD: {cod_pct:.2f}%")
```

Q26.

```
top_10 = orders.sort_values(by='unit_price', ascending=False).head(10)
```

Q27.

```
sorted_orders = orders.sort_values(by=['order_date', 'unit_price'], ascending=[True, F
alse])
```

Q28.

```
orders = orders.rename(columns={'ship_days': 'shipping_days', 'payment': 'payment_meth
od'})
```

Q29.

```
products = products.drop(columns=['weight_kg'])
```

Q30.

```
orders = orders.reset_index(drop=True)
```

🟡 MEDIUM QUESTIONS (31–70)

🔸 Groupby & Aggregation

Q31.

```
print(orders.groupby('status')['net_revenue'].sum())
```

Q32.

```
orders_products = orders.merge(products, on='product_id')
print(orders_products.groupby('category')['unit_price'].mean())
```

Q33.

```
print(customers['city'].value_counts().idxmax())
```

Q34.

```
print(orders['customer_id'].value_counts().head(10))
```

Q35.

```
print(orders.groupby('payment_method')['shipping_days'].mean())
```

Q36.

```
orders_cust = orders.merge(customers, on='customer_id')
print(orders_cust.groupby('segment').agg(
    total_rev=('net_revenue', 'sum'),
    orders=('order_id', 'count'),
    avg_val=('net_revenue', 'mean')
))
```

Q37.

```
print(orders.groupby('status')['discount'].mean().idxmax())
```

Q38.

```
print(orders.groupby('product_id')['quantity'].sum().nlargest(5))
```

Q39.

```
print(orders_products.groupby('category')['unit_price'].median().sort_values(ascending
=False))
```

Q40.

```
print(orders.groupby('payment_method').agg(
    count=('order_id', 'count'), rev=('net_revenue', 'sum'), avg=('net_revenue', 'mea
n')
))
```

🔸 Data Cleaning

Q41.

```
orders_products['unit_price'] = orders_products.groupby('category')['unit_price'].tran
sform(lambda x: x.fillna(x.median()))
```

Q42.

```
orders['discount'] = orders['discount'].fillna(0)
```

Q43.

```
print("Is duplicated?", orders['order_id'].duplicated().any())
```

Q44.

```
orders['status'] = orders['status'].str.strip().str.title()
```

Q45.

```
max_date = orders['order_date'].max()
orders = orders[orders['order_date'] <= pd.Timestamp.now()] # conceptually
```

Q46.

```
orders['discount'] = orders['discount'].astype(float)
orders['customer_id'] = orders['customer_id'].astype(int)
```

Q47.

```
no_orders = customers[~customers['customer_id'].isin(orders['customer_id'])]
```

Q48.

```
missing_prod = orders[~orders['product_id'].isin(products['product_id'])]
```

Q49.

```
orders['invalid_qty'] = ~orders['quantity'].between(1, 50)
print("Invalid:", orders['invalid_qty'].sum())
```

Q50.

```
clean_orders = orders.dropna().drop_duplicates().copy()
clean_orders['status'] = clean_orders['status'].str.strip().str.title()
```

🔸 Merging & Joining

Q51.

```
m1 = orders.merge(customers, on='customer_id', how='left')
```

Q52.

```
master = m1.merge(products, on='product_id', how='left')
```

Q53.

```
print(master['name'].isnull().sum())
```

Q54.

```
print(master.groupby(['segment', 'city'])['net_revenue'].sum())
```

Q55.

```
master = master.merge(returns, on='order_id', how='left')
master['is_returned'] = master['return_id'].notna()
```

Q56.

```
print(master.pivot_table(index='category', columns='brand', values='gross_revenue', ag
gfunc='sum'))
```

Q57.

```
print(master.dropna(subset=['reason']).groupby('category')['reason'].agg(lambda x: x.m
ode()[0]))
```

Q58.

```
returners = master[master['is_returned']]['customer_id'].unique()
print(customers[customers['customer_id'].isin(returners)])
```

Q59.

```
orders_22 = orders[orders['order_date'].dt.year == 2022]
orders_23 = orders[orders['order_date'].dt.year == 2023]
combined = pd.concat([orders_22, orders_23], ignore_index=True)
```

Q60.

```
master['profit'] = master['net_revenue'] - (master['quantity'] * master['cost_price'])
neg_profit = master[master['profit'] < 0]
```

🔸 Time Series

Q61.

```
ts = master.set_index('order_date')
```

Q62.

```
print(ts['net_revenue'].resample('D').sum().nlargest(5))
```

Q63.

```
monthly = ts.resample('M').agg({'net_revenue': 'sum', 'order_id': 'count'})
```

Q64.

```
weekly = ts['net_revenue'].resample('W').sum()
wow = weekly.pct_change() * 100
```

Q65.

```
print(ts[ts['status'] == 'Cancelled'].resample('M').size().idxmax())
```

Q66.

```
daily_rev = ts['net_revenue'].resample('D').sum().to_frame()
daily_rev['7D_MA'] = daily_rev['net_revenue'].rolling(7).mean()
```

Q67.

```
master['weekday'] = master['order_date'].dt.day_name()
print(master.groupby('weekday')['net_revenue'].mean().idxmax())
```

Q68.

```
print(master['order_date'].dt.quarter.value_counts())
```

Q69.

```
print(master['order_date'].dt.hour.value_counts().head(3))
```

Q70.

```
mom = ts['net_revenue'].resample('M').sum().pct_change() * 100
```

🔴 HARD QUESTIONS (71–105)

🔺 Advanced Groupby & Window Functions

Q71.

```
master['city_rank'] = master.groupby('city')['net_revenue'].rank(ascending=False, meth
od='min')
```

Q72.

```
master = master.sort_values(['customer_id', 'order_date'])
master['cum_rev'] = master.groupby('customer_id')['net_revenue'].cumsum()
```

Q73.

```
top_2 = master.groupby('category').apply(lambda x: x.groupby('product_name')['net_reve
nue'].sum().nlargest(2))
```

Q74.

```
daily = ts['net_revenue'].resample('D').sum().to_frame()
daily['30D_MA'] = daily['net_revenue'].rolling(30).mean()
spikes = daily[daily['net_revenue'] > 1.2 * daily['30D_MA']]
```

Q75.

```
prod_mth = ts.groupby('product_id')['net_revenue'].resample('M').sum().reset_index()
best_month = prod_mth.loc[prod_mth.groupby('product_id')['net_revenue'].idxmax()]
```

🔺 Feature Engineering

Q76.

```
master['total_cost'] = master['quantity'] * master['cost_price']
master['profit_margin_%'] = (master['profit'] / master['net_revenue']) * 100
```

Q77.

```
master['value_segment'] = pd.cut(master['net_revenue'], bins=[-np.inf, 5000, 20000, n
p.inf], labels=['Low', 'Mid', 'High'])
```

Q78.

```
master['loyalty'] = np.where(master['join_year'] == 2019, 'Veteran', np.where(master
['join_year'] >= 2022, 'New', 'Regular'))
```

Q79.

```
deliv = master[master['status'] == 'Delivered']
impact = deliv['gross_revenue'].sum() - deliv['net_revenue'].sum()
print("Discount Impact:", impact)
```

Q80.

```
daily_counts = master['order_date'].dt.date.value_counts()
p90 = daily_counts.quantile(0.9)
master['is_peak'] = master['order_date'].dt.date.map(lambda x: 1 if daily_counts[x] >
p90 else 0)
```

🔺 RFM & Customer Analysis

Q81.

```
ref_date = master['order_date'].max() + pd.Timedelta(days=1)
rfm = master.groupby('customer_id').agg(
    Recency=('order_date', lambda x: (ref_date - x.max()).days),
    Frequency=('order_id', 'count'),
    Monetary=('net_revenue', 'sum')
)
```

Q82.

```
rfm['R'] = pd.qcut(rfm['Recency'], 4, labels=[4,3,2,1])
rfm['F'] = pd.qcut(rfm['Frequency'].rank(method='first'), 4, labels=[1,2,3,4])
rfm['M'] = pd.qcut(rfm['Monetary'], 4, labels=[1,2,3,4])
rfm['RFM_Score'] = rfm['R'].astype(str) + rfm['F'].astype(str) + rfm['M'].astype(str)
```

Q83.

```
rfm['Total_Score'] = rfm['R'].astype(int) + rfm['F'].astype(int) + rfm['M'].astype(in
t)
merged_rfm = rfm.merge(customers, on='customer_id')
print(merged_rfm.groupby('segment')['Total_Score'].mean().idxmax())
```

Q84.

```
top_10pct = rfm.nlargest(int(len(rfm)*0.1), 'Monetary')
print("Rev %:", top_10pct['Monetary'].sum() / rfm['Monetary'].sum() * 100)
```

Q85.

```
print("Repeat Rate:", (rfm['Frequency'] > 1).mean() * 100)
```

🔺 Multi-Table Business Problems

Q86.

```
ret_rate = (master[master['is_returned']].groupby('product_name').size() / master.grou
pby('product_name').size()).fillna(0)
print(ret_rate.nlargest(5))
```

Q87.

```
print((master['return_date'] - master['order_date']).dt.days.mean())
```

Q88.

```
cust_cancel = master[master['status'] == 'Cancelled'].groupby('customer_id').size()
cust_orders = master.groupby('customer_id').size()
bad_cust = cust_cancel[(cust_cancel > 3) | ((cust_cancel / cust_orders) > 0.3)]
```

Q89.

```
scorecard = master.groupby('product_name').agg(
    rev=('net_revenue', 'sum'), units=('quantity', 'sum'), profit=('profit', 'sum')
)
scorecard['margin'] = scorecard['profit'] / scorecard['rev']
```

Q90.

```
print(master.pivot_table(index='city', columns='segment', values='net_revenue', aggfun
c='sum'))
```

🔺 Advanced Reshaping

Q91.

```
piv = master.pivot_table(index='segment', columns='category', values='net_revenue', ag
gfunc='sum', margins=True)
```

Q92.

```
melted = piv.drop('All', axis=0).drop('All', axis=1).reset_index().melt(id_vars='segme
nt')
```

Q93.

```
print(pd.crosstab(master['order_date'].dt.to_period('Q'), master['category'], values=m
aster['net_revenue'], aggfunc='sum'))
```

Q94.

```
stacked = piv.stack()
```

Q95.

```
print(stacked.unstack(level=0))
```

🔺 Performance & Optimization

Q96.

```
master.info(memory_usage='deep')
master['category'] = master['category'].astype('category')
```

Q97.

```
orders['faster_result'] = orders['unit_price'] * orders['quantity'] * (1 - orders['dis
count']/100)
```

Q98.

```
cols = ['status', 'payment_method', 'city', 'segment']
master[cols] = master[cols].astype('category')
```

Q99.

```
res = master.query("segment == 'Premium' & city == 'Mumbai' & net_revenue > 10000 & st
atus == 'Delivered'")
```

Q100.

```
orders = orders.eval('''
    gross = quantity * unit_price
    discount_amount = gross * (discount / 100)
    net = gross - discount_amount
''')
```

🔺 Final Boss Questions

Q101.

```
master['cohort_M'] = master.groupby('customer_id')['order_date'].transform('min').dt.t
o_period('M')
master['order_M'] = master['order_date'].dt.to_period('M')
print(master.groupby(['cohort_M', 'order_M'])['customer_id'].nunique().unstack())
```

Q102.

```
print(master.groupby(master['order_date'].dt.hour)['net_revenue'].mean().nlargest(3))
```

Q103.

```
daily = ts['net_revenue'].resample('D').sum().to_frame()
daily['z'] = (daily['net_revenue'] - daily['net_revenue'].rolling(30).mean()) / daily
['net_revenue'].rolling(30).std()
print(daily[daily['z'].abs() > 2])
```

Q104.

```
def dq_report(df):
    return pd.DataFrame({
        'dtype': df.dtypes,
        'nulls': df.isnull().sum(),
        'nunique': df.nunique()
    })
```

Q105.

```python
def run_analysis(o, c, p, r):
    o['unit_price'] = o['unit_price'].fillna(o['unit_price'].median())
    df = o.merge(c, on='customer_id').merge(p, on='product_id')
    df['net'] = df['quantity'] * df['unit_price'] * (1 - df['discount'].fillna(0)/100)
    return df.groupby('name')['net'].sum().nlargest(10), df.groupby('product_name')['n
et'].sum().nlargest(5)
```